



Institut Supérieur des Études Technologiques de Mahdia

Département : Technologies de l'Informatique

Développement des Systèmes Informatiques — DSI3.2

Rapport de Projet de Fin d'Études

**Koda — Robot compagnon intelligent et émotionnel connecté,
pilote par une application cross-platform**

**Système Embarqué · Intelligence Artificielle · Application Mobile
Cross-Plateforme**

Réalisé par :

Oussema KARIA

Salahedin JENDOUBI

Encadré par :

M.Wahid BANOUR — (Encadrant)

M.Hamza SASSI (Encadrant Entreprise)

Organisme d'accueil : —

Période : Février 2026 – Juin 2026

Résumé

Ce rapport présente KODA, un système embarqué intelligent conçu pour être un assistant de vie et un ami virtuel doté de sa propre personnalité et nature. KODA est capable de répondre de manière réfléchie à n'importe quelle personne, de raisonner comme un être humain et de reconnaître les individus grâce à ses modes de sécurité et de configuration.

L'architecture du système repose sur plusieurs composants complémentaires : un système embarqué Raspberry Pi constituant le cœur physique du robot ; un moteur de workflows n8n formant le cerveau de KODA, responsable de la préparation et de la réflexion des réponses ; une couche de services Python agissant comme distributeur universel, assurant la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi tout en exposant les endpoints de tout l'écosystème ; une application mobile cross-plateforme pour la configuration et le suivi de vie du robot ; et une base de données persistante servant de mémoire à KODA.

Mots-clés : Système Embarqué, Assistant Virtuel, Raspberry Pi, n8n, Python, Cross-Plateforme, Intelligence Artificielle, Assistant de Vie, KODA.

DÉDICACE

Je dédie ce travail à ma famille, pilier silencieux de ma force et source inépuisable de soutien.

*Merci pour vos sacrifices immenses et pour avoir cru en moi tout au long de mon parcours
académique.*

*À mes parents, pour leur amour inconditionnel, leurs encouragements et leurs prières qui ont été la
lumière guidant chacun de mes pas. Ce succès est le vôtre autant que le mien.*

*À mes amis et collègues, pour leur soutien moral et pour avoir partagé avec moi cette aventure
inoubliable.*

*À mon binôme Salahedin, pour sa persévérance, sa créativité et sa complicité tout au long de ce
projet.*

*À tous ceux qui ont contribué, même par un simple mot d'encouragement, à l'accomplissement de
ce travail.*

Oussema KARIA

DÉDICACE

À ma famille,
pour son soutien inconditionnel et ses sacrifices immenses
tout au long de mon parcours académique.

À mon binôme Oussema, pour son enthousiasme, sa rigueur
et sa précieuse collaboration dans la réalisation de ce projet.

À mes encadrants,
pour leur guidance, leur patience et leur expertise technique.

Salahedin JENDOUBI

Remerciements

À notre encadrant académique, M. Wahid Banour,

nous adressons nos remerciements les plus sincères pour la qualité de l'accompagnement pédagogique, la disponibilité dont il a fait preuve à chaque étape, ainsi que pour ses conseils exigeants qui ont permis de structurer notre démarche et de renforcer la rigueur scientifique et technique de ce travail.

À notre encadrant en entreprise, M. Hamza Sassi,

nous exprimons toute notre gratitude pour l'accueil au sein de l'organisme d'accueil, pour le temps accordé aux échanges techniques et pour les orientations concrètes qui ont éclairé nos choix d'architecture et d'intégration tout au long du projet.

À la direction et à l'ensemble du corps enseignant de l'ISET de Mahdia,

et en particulier au département *Développement des systèmes informatiques* (DSI), nous témoignons notre reconnaissance pour la formation dispensée, pour l'exigence professionnelle transmise et pour l'environnement d'apprentissage qui a rendu possible la réalisation de ce projet de fin d'études.

À nos proches, amis et camarades,

nous disons merci pour la patience, les encouragements et la confiance manifestés durant cette période intensive ; leur soutien moral a compté autant que le travail collectif mené au sein du binôme.

Nous saluons enfin toute personne ayant contribué, directement ou indirectement, à l'aboutissement de ce rapport.

Oussema Karia

Salahedin Jendoubi

SOMMAIRE

Introduction Générale	1
I Cadre Général du Projet et Présentation de l'Organisme	2
I.1 Introduction du chapitre	3
I.2 Contexte Général du Projet	3
I.3 Présentation de l'Organisme d'Accueil (MicroEdition)	3
I.3.1 Profil de l'Entreprise	3
I.3.2 Services	4
I.3.3 Recherche & Développement (R&D)	4
I.3.4 Structure Organisationnelle	4
I.4 Problématique	5
I.5 Analyse des Solutions Existantes	5
I.5.1 Étude de l'existant	6
I.5.2 Synthèse et Critique : Le "Manque d'Humanité"	6
I.5.3 La Différenciation de Koda	7
I.6 Solution Proposée	7
I.7 Méthodologie du Projet	8
I.7.1 Modèle en spirale	8
I.8 Conclusion	8
II Préparation du Projet	11
II.1 Introduction du chapitre	12
II.2 Spécification des besoins	12
II.2.1 Spécifications des besoins fonctionnels	12
II.2.2 Spécification des besoins non fonctionnels	13
II.2.3 Architecture du système	14
II.2.4 Identification des acteurs	15
II.3 Pilotage du projet — Modèle en spirale et cycles de développement	17
II.3.1 Équipe et rôles	17

II.3.2	Découpage du travail : six cycles	17
II.4	Planification des cycles	19
II.5	Architecture	19
II.5.1	Architecture de l'application	19
II.5.1.1	Architecture en oignon (<i>Onion Architecture</i>) du backend	19
II.6	Environnement de Developement	20
II.6.1	Environnement matériel	20
II.7	Environnement logiciel	20
II.7.1	Outils de développement et modélisation	20
II.7.2	Langages et frameworks utilisés	21
II.8	Conclusion	21
III	Cycle 1 : ANALYSEUR ET RÉFLEXION	23
III.1	Introduction	24
III.2	Fondements Théoriques	24
III.2.1	Que signifie « théorie du double traitement » ?	24
III.2.2	Réflexion rapide et réflexion lente	25
III.2.2.1	Réflexion rapide : réutiliser la mémoire déjà présente	25
III.2.2.2	Réflexion lente : analyser la mémoire pour fabriquer une réponse	25
III.2.3	Comment une question est comprise avant la réponse	25
III.2.4	Intérêt pour un compagnon conversationnel comme KODA	26
III.3	Définitions Fondamentales	26
III.3.1	Grand modèle de langage (LLM)	27
III.4	Modélisation globale du Cycle 1	27
III.4.1	Vue d'ensemble des trois flux n8n	28
III.4.2	Diagramme de cas d'utilisation général	29
III.4.3	Schéma de classes général	30
III.5	Fonctionnalités Techniques — les trois flux	30
III.5.1	Premier Flux — Webhook 1 : Analyse et Réponse	30
III.5.1.1	Principe général et découpage en six parties	30
III.5.1.2	Diagramme de séquence du Webhook 1	31
III.5.1.3	Format de la Requête Entrante	31
III.5.1.4	Les six parties du Webhook 1	32

III.5.1.4.1	Entrée fonctionnelle — analyse du message et classification d'intention	32
III.5.1.4.1.1	Branche musique — SerpAPI comme outil de recherche Google.	34
III.5.1.4.2	Confrontation à l'historique — vérification des cinq derniers messages	35
III.5.1.4.3	Classification sémantique — analyse de dépendance mémoire	36
III.5.1.4.3.1	Basic LLM Chain en lieu et place de l'AI Agent	38
III.5.1.4.3.2	Configuration OpenRouter (modèle Gemini) . .	38
III.5.1.4.3.3	OpenRouter — agrégation et suivi d'usage . . .	38
III.5.1.4.3.4	Prompt Engineering — Analyseur de Dépendance Mémoire	39
III.5.1.4.3.5	Détail des Types de Questions Classifiés	41
III.5.1.4.4	Gestion utilisateur (unkonu) : Reconnaissance et enregistrement de l'utilisateur	42
III.5.1.4.4.1	Routage par <i>Switch1</i> (résultat du la classification sémantique).	43
III.5.1.4.4.2	Diagramme de séquence — gestion utilisateur (unkonu)	44
III.5.1.4.5	Réponse contextualisée : Récupération Contextuelle et Génération de la Réponse	46
III.5.1.4.5.1	Branche Mémoirelle (<code>use_memory = true</code>) . .	46
III.5.1.4.5.2	Branche Générale (<code>use_memory = false</code>) . .	48
III.5.1.5	Vue Complète du Réponse contextualisée — Les Deux Branches	49
III.5.1.5.1	Notes et mémoire persistante : Notes, structuration et mise à jour de la mémoire	50
III.5.1.5.1.1	Partie 1 — Extraction structurée des <i>notes</i> (conditionnel).	50
III.5.1.5.1.2	Détection (<i>If</i>) et <i>Basic LLM Chain 2</i>	51
III.5.1.5.1.3	Partie 2 — Mise à jour de la mémoire et clôture du webhook.	53
III.5.1.6	Conclusion du Webhook 1	54
III.5.2	Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière	55

III.5.2.1	Principe et Inspiration	55
III.5.2.2	Diagrammes du Webhook 2	55
III.5.2.3	Fonctionnement du Webhook 2	56
III.5.2.4	Structure des Fiches de Synthèse Générées	57
III.5.2.5	Persistance : relation utilisateur / accompagnement	58
III.5.2.6	Impact sur le Système Global	59
III.5.2.7	Conclusion du Webhook 2	59
III.5.3	Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive	60
III.5.3.1	Principe : la spontanéité comme dimension humaine	60
III.5.3.2	Diagrammes du Webhook 3	60
III.5.3.3	Déclenchement et entrées	61
III.5.3.4	Préparation des données (nœud Code11)	62
III.5.3.5	Génération de l'introduction et sortie JSON	62
III.5.3.6	Conclusion du Webhook 3	63
III.6	Synthèse cognitive du Cycle 1	63
III.7	Conclusion	64
IV	Cycle 2 : Distributeur de Services	65
IV.1	Introduction	66
IV.1.0.0.1	Rôle global du Distributeur.	66
IV.1.0.0.2	Objectifs et résultats attendus.	66
IV.1.0.0.3	Organisation du chapitre.	67
IV.2	Fondements théoriques et positionnement du Distributeur	67
IV.2.1	Le principe du point de liaison unique	67
IV.2.2	Services backend et fonctionnalité globale du système	68
IV.2.3	Communication vocale et applicative : deux faces, un backend	68
IV.3	Modélisation et place dans l'architecture	68
IV.3.1	Diagramme de cas d'utilisation général	69
IV.3.2	Architecture globale : pivot de flexibilité et de disponibilité	70
IV.4	Architecture interne du backend	72
IV.4.1	Choix du langage : Python	72
IV.4.2	Clean Architecture : principes et application	73
IV.4.3	Structure des répertoires du projet	74
IV.5	Services exposés et flux de communication	75

IV.5.1	Service dédié vers le système embarqué (robot)	75
IV.5.1.1	Cas d'utilisation robot ↔ backend	76
IV.5.1.2	Action 1 — Demander une réponse vocale (Flux 1)	76
IV.5.1.2.1	Lien avec le Cycle 1.	78
IV.5.1.3	Action 2 — Demander une introduction de parole (Flux 3)	78
IV.5.1.3.1	Lien avec le Cycle 1.	80
IV.5.1.4	Canal WebSocket (temps réel)	80
IV.5.1.5	Chaîne vocale complète (Azure STT / TTS)	81
IV.5.1.6	Reconnaissance des personnes	83
IV.5.1.6.1	Principe et modèle de données.	83
IV.5.1.6.2	Pipeline d'identification.	83
IV.5.1.6.3	Cas particuliers.	85
IV.5.1.6.4	Contraintes et intégration.	86
IV.5.2	Service dédié vers l'application mobile (KodaMate)	86
IV.5.2.1	Contexte architectural	86
IV.5.2.2	Principe de communication	86
IV.5.2.3	Flux fonctionnels principaux	87
IV.5.2.3.1	Authentification.	87
IV.5.2.3.2	Configuration et contrôle du robot.	87
IV.5.2.3.3	Chat intelligent (délégation n8n).	88
IV.5.2.3.4	Historique des conversations.	88
IV.5.2.3.5	Données complémentaires (hors backend).	89
IV.5.3	Service dédié vers la base de données (PostgreSQL / Supabase)	90
IV.5.3.1	Modèle relationnel et rôle des tables	90
IV.5.4	Service dédié vers le workflow n8n (Cycle 1)	91
IV.6	Catalogue des endpoints REST du Distributeur	93
IV.6.1	Infrastructure et documentation (hors /api)	93
IV.6.2	Communication avec PostgreSQL (persistance Supabase)	94
IV.6.2.1	Santé de la base	94
IV.6.2.2	Entités persistées (CRUD)	94
IV.6.3	Communication avec le système embarqué (Raspberry Pi)	94
IV.6.4	Communication avec le workflow n8n (Cycle 1)	94
IV.6.5	Communication avec l'application mobile (KodaMate)	94
IV.6.5.1	Endpoints consommés à l'écran	97

IV.6.5.2	Endpoints codés mais non branchés à l'interface	97
IV.6.5.3	API externe et services locaux	97
IV.6.5.4	Cartographie écran → endpoints	97
IV.7	Problèmes rencontrés et solutions apportées	97
IV.7.1	Conflit d'adressage Raspberry Pi / backend	98
IV.7.2	Problème CORS entre backend et application mobile	99
IV.8	Perspectives et évolutions futures	99
IV.9	Conclusion	100
V	Cycle 3 : Système Embarqué — Partie Matérielle	101
V.1	Introduction	102
V.2	Architecture Matérielle Globale	102
V.3	Conception mécanique et châssis 3D (Blender)	103
V.4	Description Détaillée des Composants	104
V.4.1	Unité Centrale : Raspberry Pi 4 Model B	104
V.4.2	Unité de Commande Moteur : Arduino Uno + Shield L293D	105
V.4.3	Système d'Actionneurs	107
V.4.3.1	Moteurs DC — Mobilité	107
V.4.3.2	Servomoteurs — Articulations	107
V.4.4	Système Audio	108
V.4.4.1	Entrée Audio : ReSpeaker 4-Mic Array USB	108
V.4.5	Capteurs Sensoriels Ajoutés : toucher et orientation	109
V.4.5.1	Gyroscope MPU5060 pour la rotation	109
V.4.5.2	Capteurs de touche capacitifs	110
V.4.6	Sortie Audio : double chaîne (jack filaire et Bluetooth)	111
V.4.7	Système de Vision : Caméra Raspberry Pi	112
V.4.8	Interface d'Expression Faciale : Nextion NX4832T035_011	113
V.4.9	Système d'Alimentation	114
V.5	Schéma de Câblage Global	115
V.6	Assemblage et points d'attention	115
V.7	Validation matérielle — tests par composant	117
V.7.1	Commandes de test par composant	119
V.7.1.0.1	Power Bank + Raspberry Pi.	119
V.7.1.0.2	ReSpeaker 4-Mic Array USB.	119

V.7.1.0.3	Sortie jack + amplificateur PAM8403.	119
V.7.1.0.4	Sortie Bluetooth.	120
V.7.1.0.5	Caméra Raspberry Pi (CSI).	120
V.7.1.0.6	Écran Nextion (UART).	120
V.7.1.0.7	Capteurs de touche (GPIO).	121
V.7.1.0.8	Gyroscope MPU5060 côté Arduino (I2C).	121
V.7.1.0.9	Arduino + Shield L293D (commande série USB).	122
V.7.1.0.10	Moteurs DC (M1, M3).	123
V.7.1.0.11	Servomoteurs (S1, S2, S3).	123
V.7.1.0.12	Alimentation 3 piles 3,7 V (Vin shield).	124
V.7.2	Test d'intégration matérielle	124
V.8	Conclusion	125
VI	Cycle 4 : Système Embarqué — Partie Logicielle Raspberry Pi	127
VI.1	Introduction	128
VI.2	Architecture logicielle globale	129
VI.3	Architecture du code et diagramme de classes	129
VI.4	Configuration et lancement	132
VI.5	Démarrage parallèle et diagnostic matériel	134
VI.6	Boucle principale et machine d'états	135
VI.7	Pipeline vocal : mot de réveil hybride Vosk + Azure	135
VI.8	Capture audio multi-consommateurs	138
VI.9	Rotation closed-loop avec MPU6050	138
VI.10	Dispatch des actions retournées par le backend	139
VI.11	Affichage Nextion et expressions	141
VI.12	Reconnaissance faciale	142
VI.13	Capteur tactile et interruption	142
VI.14	Robustesse, registre de sous-processus et arrêt propre	142
VI.15	Tests et validation logicielle	143
VI.16	Limites et perspectives	145
VI.17	Conclusion	145
VII	Cycle 5 : Application Mobile Cross-Plateforme KodaMate	147
VII.1	Introduction : pourquoi une application compagnon ?	148

VII.2 Place de KodaMate dans l'écosystème	149
VII.3 Parcours et principes d'expérience	150
VII.4 Les interfaces utilisateur	152
VII.4.1 Interface de chargement	152
VII.4.2 Interface de connexion	153
VII.4.3 Interface de configuration du robot	155
VII.4.4 Interface d'accueil (Home)	157
VII.4.5 Interface de dialogue (Chat)	159
VII.4.6 Interface d'historique	160
VII.4.7 Interface de paramètres	162
VII.5 Organisation technique du projet	163
VII.6 Fiabilité et session	164
VII.7 Tests et validation	164
VII.8 Limites et perspectives	165
VII.9 Conclusion	166
VIII Conclusion générale et synthèse du projet	167
VIII.1 Du constat initial à la solution réalisée	168
VIII.2 Vue d'ensemble : les cinq cycles et leurs rôles	168
VIII.3 Les cinq piliers du système KODA	169
VIII.3.1 Le cerveau conversationnel (Cycle 1)	169
VIII.3.2 Le système nerveux central (Cycle 2)	170
VIII.3.3 Le corps du compagnon (Cycles 3 et 4)	170
VIII.3.4 La relation à distance (Cycle 5)	170
VIII.4 Comment lire l'ensemble : un scénario type	170
VIII.5 Apports du projet	171
VIII.6 Limites et enseignements	172
VIII.7 Perspectives	172
VIII.8 Conclusion finale	172

LISTE DES FIGURES

I.1	Modèle en spirale — enchaînement des six cycles de développement KODA	9
II.1	Synthèse visuelle des besoins fonctionnels KODA (chapitre II)	13
II.2	Architecture multicouche du système KODA (vue chapitre II)	14
II.3	Acteurs principaux et contexte d'intégration du système KODA	16
II.4	Modèle en spirale et enchaînement des six cycles de développement KODA	18
II.5	Planification des cycles : jalons, chevauchements et dépendances	19
II.6	Environnement matériel de développement et de test (poste, Raspberry Pi, robot) .	20
II.7	Environnement logiciel : outils de développement et chaîne technique (chapitre II) .	21
III.1	De la question à la réponse : réflexion rapide (mémoire déjà présente) ou réflexion lente (analyse puis construction)	26
III.2	Architecture cognitive de KODA inspirée de la théorie du double traitement	27
III.3	Cycle 1 — vue d'ensemble des trois workflows n8n	28
III.4	Cycle 1 — diagramme de cas d'utilisation général	29
III.5	Cycle 1 — schéma de classes général (persistance)	30
III.6	Webhook 1 — diagramme de séquence (six parties du pipeline)	31
III.7	Webhook 1 — entrée fonctionnelle (nœud <i>Test</i> , <i>Switch</i>)	32
III.8	Webhook 1 — séquence de l'entrée fonctionnelle	33
III.9	Webhook 1 — cas d'utilisation de l'entrée fonctionnelle	33
III.10	Entrée fonctionnelle — intégration SerpAPI dans le workflow n8n (branche musique du <i>Switch</i>)	35
III.11	Entrée fonctionnelle — nœud <i>Search Music</i> (SerpAPI, moteur YouTube) et exemple de sortie (<i>search_query</i> , <i>youtube_url</i>)	35
III.12	Confrontation à l'historique — workflow n8n	35
III.13	Confrontation à l'historique — diagramme de séquence	36
III.14	Classification sémantique — chaîne n8n (Code5, Identité1, Basic LLM Chain, Code2)	36
III.15	Classification sémantique — diagramme de séquence	37
III.16	Classification sémantique — cas d'utilisation	37

III.17	Classification sémantique — Paramètres du nœud OpenRouter Chat Model (credential, modèle Gemini via OpenRouter)	38
III.18	Classification sémantique — tableau de bord OpenRouter (clé API workflow n8n, suivi des dépenses par modèle)	39
III.19	Gestion utilisateur (unkonu) — <i>Switch1</i> , branche enregistrement (presentation), branche LLM (unkonu) et poursuite vers la réponse contextualisée	42
III.20	Gestion utilisateur (unkonu) — Entrée JSON du <i>Switch1</i> (personne = unkonu, sortie classification sémantique)	42
III.21	Gestion utilisateur (unkonu) — Sortie texte : invitation à la première rencontre et demande du prénom	43
III.22	Gestion utilisateur (unkonu) — diagramme de séquence	45
III.23	Réponse contextualisée.1 — Branche mémorielle : récupération et génération contextuelle	47
III.24	Réponse contextualisée.2 — Branche générale : contexte conversationnel et génération de réponse	49
III.25	Réponse contextualisée — Vue complète des deux branches de traitement contextuel	50
III.26	Réponse contextualisée — Diagramme de séquence du cinquième bloc	50
III.27	Notes et mémoire persistante (partie 1) — Sous-workflow n8n : test questiontype = note, <i>Basic LLM Chain 2</i> et structuration	51
III.28	Notes et mémoire persistante — Entrée JSON : question, output, personne et questiontype = note	52
III.29	Notes et mémoire persistante — Sortie JSON : date_evenement et evenement extraits du message naturel	52
III.30	Notes et mémoire persistante (partie 2) — Diagramme de classes (persistance mémoire)	54
III.31	Webhook 2 — workflow n8n	55
III.32	Webhook 2 — diagramme de séquence	56
III.33	Webhook 2 — exemple de sortie structurée (quatre dimensions de la fiche journalière)	58
III.34	Webhook 2 — schéma de persistance (utilisateur → accompagnement : interets, synthese, point_cles, conseils)	59
III.35	Webhook 3 — workflow n8n	60
III.36	Webhook 3 — diagramme de séquence	61
III.37	Webhook 3 — entrée du nœud <i>Code11</i>	62
III.38	Webhook 3 — sortie JSON (parole_du_robot, emotion_visuelle)	62

III.39	Architecture cognitive globale de KODA — interaction entre les trois webhooks . . .	63
IV.1	Cycle 2 — diagramme de cas d'utilisation général du Distributeur de services . . .	69
IV.2	Cycle 2 — schéma de classes général du backend (persistance PostgreSQL)	70
IV.3	Architecture multicouche de KODA — place du Distributeur de services (Cycle 2)	71
IV.4	Cycle 2 — Distributeur pivot central (composants et flux)	72
IV.5	Architecture des dossiers du projet backend Distributeur (src/)	74
IV.6	Exécution du programme backend (Distributeur)	75
IV.7	Cycle 2 — cas d'utilisation du système embarqué vis-à-vis du Distributeur de services	76
IV.8	Action 1 — vue compacte du pipeline vocal (Flux 1)	77
IV.9	Action 1 (a) — entrée : capture, identification faciale et transcription STT	77
IV.10	Action 1 (b) — sortie : Flux 1, synthèse TTS et restitution au robot	78
IV.11	Action 2 — vue compacte de l'introduction de parole (Flux 3)	79
IV.12	Action 2 (a) — entrée : détection de présence et requête au Distributeur	79
IV.13	Action 2 (b) — sortie : Flux 3, synthèse TTS et restitution au robot	80
IV.14	Cycle 2 — échanges WebSocket robot ↔ Distributeur (temps réel)	81
IV.15	Configuration Azure Speech (STT/TTS)	82
IV.16	Test Text-to-Speech	82
IV.17	Test Speech-to-Text	82
IV.18	Pipeline de reconnaissance faciale — de la capture caméra à l'identité utilisateur .	83
IV.19	Diagramme de séquence — identification faciale (robot, Distributeur, base de référence)	84
IV.20	Exemple d'images de référence en mémoire (profils utilisateurs)	85
IV.21	Authentification mobile — séquence POST /api/auth/login	87
IV.22	Chat mobile — séquence POST /api/trigger-n8n	88
IV.23	Paramètres, power et historique — séquences REST KodaMate	89
IV.24	Exécution de quelques endpoints (tests manuels)	90
IV.25	Interface Supabase — administration PostgreSQL	90
IV.26	Délégation au workflow n8n — séquence robot ou mobile → Distributeur → n8n .	92
IV.27	Communication avec n8n — exemple de payload en entrée de webhook	92
IV.28	Conflit d'adressage et CORS — topologie réseau et solutions	99
V.1	Conception 3D du châssis KODA sous Blender (vue éclatée des pièces)	104
V.2	Raspberry Pi 4 Model B utilisé comme unité centrale de KODA	105
V.3	Carte Arduino Uno (microcontrôleur ATmega328P à 16 MHz)	106

V.4	Shield L293D empilé sur l'Arduino Uno (commande des moteurs DC et servomoteurs)	106
V.5	Moteur DC utilisé pour la mobilité du robot (bornes M1 et M3 du shield)	107
V.6	Principe de traction différentielle — moteurs M1 et M3	107
V.7	Servomoteur utilisé pour les articulations (bras S1/S2 et rotation Nextion S3)	108
V.8	ReSpeaker 4-Mic Array USB : quatre microphones MEMS pour capture omnidirectionnelle et détection de direction (DOA)	109
V.9	Module gyroscopique MPU5060 utilisé pour améliorer la précision de rotation	110
V.10	Relation conceptuelle : DOA du ReSpeaker → consigne Raspberry → Arduino (M1, M3 + MPU5060) → caméra du torse orientée	110
V.11	Capteurs de touche capacitifs ajoutés pour le sens tactile du robot	111
V.12	Amplificateur audio PAM8403 (chaîne filaire entre la sortie jack du Pi et les haut-parleurs)	112
V.13	Paire de haut-parleurs gauche/droite alimentés par le PAM8403	112
V.14	Caméra Raspberry Pi connectée au port CSI (acquisition d'images pour l'identification visuelle)	113
V.15	Écran Nextion NX4832T035_011 (3,5") affichant le visage animé de KODA	113
V.16	Power Bank 5 V / 3 A alimentant le Raspberry Pi (chaîne logique)	114
V.17	3 piles lithium 3,7 V en série (11,1 V total) alimentant le Shield L293D et les moteurs	115
V.19	Architecture matérielle logique mise à jour avec toucher et gyroscope	115
V.18	Schéma de montage matériel de KODA — style Fritzing (Cycle 3)	116
V.20	Prototype physique KODA après impression 3D et montage	125
VI.1	Cycle 4 — architecture logicielle globale du logiciel Raspberry Pi (services, adaptateurs et environnement externe)	130
VI.2	Cycle 4 — arborescence du dossier codeRaspberry/ (packages, modules clés et firmware Arduino)	131
VI.3	Cycle 4 — diagramme de classes UML : services métier et adaptateurs matériels	133
VI.4	Cycle 4 — séquence de démarrage parallèle (app.main.run)	134
VI.5	Cycle 4 — machine d'états logicielle du robot (passif, actif, touch, sommeil)	136
VI.6	Cycle 4 — machine d'états interne du moteur HybridWakeWordEngine (Vosk gate, Azure race, timeout)	137
VI.7	Cycle 4 — séquence complète d'un tour de parole : wake → rotation → greeting → écoute → backend → dispatch	137

VI.8 Cycle 4 — broadcaster PCM : un producteur sox, plusieurs consommateurs (Vosk, Azure WS, VAD)	138
VI.9 Cycle 4 — rotation en boucle fermée : Pi → Arduino → MPU6050 → DONE/ERR	140
VII.1 Cycle 5 — cas d'utilisation de KodaMate (vue utilisateur)	149
VII.2 Cycle 5 — place de KodaMate entre l'utilisateur, le Distributeur, la cognition n8n et le corps du robot	150
VII.3 Cycle 5 — parcours utilisateur de KodaMate	151
VII.4 Cycle 5 — séquence de démarrage et orientation de l'utilisateur	153
VII.5 Cycle 5 — connexion et test du serveur Distributeur	154
VII.6 Cycle 5 — séquence d'authentification utilisateur	155
VII.7 Cycle 5 — configuration initiale : identité du robot (gauche) et voix, langue et état d'allumage (droite)	156
VII.8 Cycle 5 — séquence de lecture et mise à jour de la configuration	157
VII.9 Cycle 5 — accueil : veille et environnement (gauche), réseau déconnecté (centre), robot actif (droite)	158
VII.10 Cycle 5 — séquence de supervision : état KODA, météo et marche/arrêt	158
VII.11 Cycle 5 — dialogue texte avec KODA	159
VII.12 Cycle 5 — séquence d'envoi d'une question et affichage de la réponse	160
VII.13 Cycle 5 — historique des conversations avec filtre et recherche	161
VII.14 Cycle 5 — séquence de consultation de l'historique	161
VII.15 Cycle 5 — paramètres : profil robot (gauche), Wi-Fi et préférences (centre), informations et déconnexion (droite)	162
VII.16 Cycle 5 — séquence de sélection d'un réseau Wi-Fi (simulation locale)	163
VIII. Vue synthétique de l'écosystème KODA au terme du projet	169

LISTE DES TABLEAUX

II.1	Vue synthétique des cycles et livrables attendus	18
III.1	Correspondance entre mécanismes cognitifs humains et implémentations de KODA	63
IV.1	Comparaison des technologies backend envisagées	73
IV.2	Répartition des 65 endpoints par axe de communication	93
IV.3	Endpoints racine — exploitation et documentation	93
IV.4	Endpoint de diagnostic base de données	94
IV.5	Endpoints CRUD — communication avec PostgreSQL par entité	95
IV.6	Endpoints orientés robot — audio, vision et mot-clé	96
IV.7	Endpoints de délégation au moteur n8n	96
IV.8	Endpoints backend effectivement utilisés par KodaMate (UI)	96
IV.9	Appels réseau complémentaires de KodaMate (hors Distributeur)	97
IV.10	Mapping des écrans KodaMate vers les endpoints backend	97
IV.11	Problèmes techniques rencontrés et solutions apportées	98
V.1	Inventaire des composants matériels de KODA	103
V.2	Spécifications techniques du Raspberry Pi 4 Model B	105
V.3	Architecture d'alimentation de KODA	114
V.4	Récapitulatif des connexions entre composants	117
V.5	Tests unitaires par composant (Cycle 3)	118
VI.1	Cycle 4 — les cinq types d'actions dispatchés côté Pi	141
VI.2	Expressions logicielles envoyées à l'écran Nextion	141
VI.3	Tests de validation du logiciel Raspberry Pi	144
VII.1	Interfaces KodaMate et rôles utilisateur	152
VII.2	Correspondance services principaux et besoins utilisateur	163
VII.3	Tests de validation par interface	165
VIII.1	Synthèse des cycles KODA — besoin couvert et composant principal	168

INTRODUCTION GÉNÉRALE

Dans un monde en constante évolution technologique, la demande en solutions intelligentes capables d'améliorer la qualité de vie des individus n'a jamais été aussi forte. Les assistants virtuels et les systèmes embarqués intelligents représentent aujourd'hui une réponse concrète à ce besoin, en combinant les avancées de l'intelligence artificielle, des systèmes embarqués et du développement logiciel moderne.

Ce projet de fin d'études, réalisé dans le cadre de la formation en Développement des Systèmes Informatiques (DSI3.2) à l'ISET Mahdia, porte sur la conception et le développement de **KODA** : un assistant de vie et ami virtuel embarqué. KODA est un système robotique doté de sa propre personnalité et nature, capable de répondre intelligemment à toute personne, de raisonner à la manière d'un être humain, de reconnaître les individus à travers ses modes de sécurité, et d'être configuré et suivi via une application mobile cross-plateforme dédiée.

La problématique centrale de ce projet est : **Comment concevoir un système embarqué intelligent capable d'assurer un rôle d'assistant de vie personnalisé, intégrant reconnaissance des personnes, réflexion autonome et pilotage à distance via une application mobile ?**

Ce rapport est organisé comme suit :

- **Chapitre I** présente le cadre général du projet, l'organisme d'accueil, la problématique, l'analyse de l'existant et la solution proposée.
- **Chapitre II** expose la préparation du projet : spécification des besoins, identification des acteurs, méthodologie de pilotage *en spirale* par cycles, planification, architecture cible et environnements de travail.
- **Chapitre III** correspond au **Cycle 1 — Analyseur et réflexion** : workflows n8n (webhooks, six blocs de traitement, intégration LLM et persistance).
- **Chapitre IV** correspond au **Cycle 2 — backend Distributeur de Services** (FastAPI, persistance, services cloud).
- **Chapitre V — Cycle 3** : système embarqué, partie matérielle (plateforme robot, câblage, validation).
- Les **cycles 4 à 6** (logiciel Raspberry Pi, application cross-plateforme, tests finaux) sont présentés dans la suite du document.

———— CHAPITRE I ————

CADRE GÉNÉRAL DU PROJET ET PRÉSENTATION DE L'ORGANISME

I.1 Introduction du chapitre

Ce chapitre esquisse le cadre fondamental de notre étude en définissant l’environnement contextuel et organisationnel du projet. Nous commençons par introduire l’organisme d’accueil, en mettant en évidence son expertise technologique et sa structure interne dédiée à l’innovation.

Ensuite, nous exposons la problématique centrale, justifiant le besoin critique d’un système intelligent. Une analyse rigoureuse des solutions existantes nous permet d’identifier les limites actuelles et de mieux positionner notre système proposé. Enfin, nous détaillons la méthodologie de projet adoptée, assurant une approche structurée et efficace tout au long du cycle de développement.

I.2 Contexte Général du Projet

L’évolution technologique actuelle transforme notre interaction avec l’environnement. La robotique sociale et les assistants personnels virtuels s’imposent comme des éléments clés pour améliorer la qualité de vie, offrant des solutions innovantes pour l’assistance quotidienne.

Le projet KODA s’inscrit dans cette dynamique en proposant un assistant de vie virtuel, conçu sous la forme d’un système embarqué intelligent. Contrairement aux solutions traditionnelles qui se limitent à l’exécution de commandes, KODA vise à offrir une véritable présence interactive, dotée d’une personnalité propre et d’une capacité de raisonnement.

La complexité de ce système réside dans l’intégration harmonieuse de multiples composants hétérogènes. Il nécessite la combinaison d’un environnement matériel embarqué (Raspberry Pi), d’un moteur de workflow intelligent (n8n) agissant comme cerveau, d’une couche logicielle orchestrant les communications (Python), et d’interfaces de contrôle intuitives (application mobile cross-plateforme). L’objectif est de créer un écosystème cohérent capable de reconnaître les utilisateurs, de traiter les requêtes de manière naturelle, et de fournir un accompagnement personnalisé.

I.3 Présentation de l’Organisme d’Accueil (MicroEdition)

I.3.1 Profil de l’Entreprise

MicroEdition est une entreprise pionnière spécialisée dans les Technologies de l’Information et de la Communication (TIC). Fondée en 2025, la société est stratégiquement implantée au sein de la Pépinière d’Entreprises de Mahdia. Cet environnement offre un écosystème robuste dédié à l’innovation, fournissant l’infrastructure nécessaire pour accompagner les startups technologiques à fort potentiel. MicroEdition se consacre à la conception et au déploiement de solutions numériques intelligentes, évolutives et sécurisées, visant à combler le fossé entre la recherche académique et

l'application industrielle.

I.3.2 Services

Le portefeuille de l'entreprise repose sur trois piliers technologiques fondamentaux :

- **Développement logiciel** : Conception d'applications web et mobiles haute performance, adaptées à des logiques métier spécifiques.
- **Internet des Objets (IoT)** : Conception d'écosystèmes connectés de bout en bout pour l'automatisation industrielle et agricole.
- **Cybersécurité** : Mise en œuvre de protocoles de sécurité rigoureux pour protéger les actifs numériques et garantir l'intégrité des données dans les secteurs public et privé.

I.3.3 Recherche & Développement (R&D)

Un trait distinctif de MicroEdition est sa division spécialisée en Recherche et Développement. Ce département se concentre fortement sur l'IA embarquée (Edge AI) et l'intelligence embarquée. En explorant des méthodologies d'exécution de réseaux de neurones complexes directement sur des microcontrôleurs, l'équipe R&D cherche à éliminer la dépendance à une connectivité cloud permanente. Cet axe stratégique est central pour le projet KODA, car il permet la création de systèmes autonomes et résilients capables de fonctionner dans des environnements où l'accès à Internet peut être intermittent.

I.3.4 Structure Organisationnelle

MicroEdition fonctionne selon un cadre Agile qui met l'accent sur l'itération rapide et la collaboration interdisciplinaire. La structure interne de l'entreprise est composée d'unités spécialisées, notamment :

- **Architectes logiciels** : Conception de backends systèmes évolutifs.
- **Ingénieurs en systèmes embarqués** : Pont entre le matériel et le logiciel.
- **Spécialistes en IA et Data Scientists** : Développement et optimisation de modèles d'apprentissage automatique pour le déploiement embarqué.

Cette synergie permet à MicroEdition de maintenir un cycle d'innovation soutenu, garantissant que chaque projet — tel que KODA — repose sur des fondations d'excellence technique et de standards architecturaux modernes.

I.4 Problématique

L'évolution constante de notre société moderne, marquée par un rythme de vie accéléré et une digitalisation croissante, a paradoxalement accentué certains besoins fondamentaux de l'être humain, notamment celui de l'accompagnement et de l'interaction sociale.

Le problème central auquel nous tentons de répondre s'articule autour de trois axes principaux :

A. Le besoin de présence et d'interaction naturelle

Malgré la prolifération des appareils connectés, la plupart des outils actuels (smartphones, enceintes connectées classiques) restent des outils purement fonctionnels. Ils exécutent des ordres mais n'offrent pas de réelle interaction "humaine". Il existe un manque flagrant de systèmes capables d'afficher une personnalité propre et de simuler une forme de réflexion empathique capable de briser la solitude ou d'assister l'utilisateur au quotidien.

B. La complexité de la gestion de vie autonome

Dans un environnement domestique, la gestion des tâches, de la sécurité et du suivi technique des objets connectés devient de plus en plus complexe pour l'utilisateur moyen. Il n'existe pas de solution centralisée qui combine à la fois un assistant de vie intelligent (capable de raisonner) et un système de sécurité actif (capable de reconnaître les personnes et de protéger l'habitat).

C. Les limites de l'intelligence statique

La majorité des assistants actuels dépendent de réponses préprogrammées et rigides. Le défi est de passer d'une machine qui "répond" à une machine qui "réfléchit" et "apprend". Le problème est donc de concevoir une architecture capable de :

- Identifier l'interlocuteur de manière sécurisée.
- Traiter des demandes complexes de manière fluide et humaine.
- Offrir une interface de contrôle simple (application mobile) pour superviser cette "vie artificielle".

En résumé, la question fondamentale de notre projet est la suivante : **Comment concevoir un compagnon robotique intelligent, doté d'une mémoire et d'une personnalité, capable d'offrir un accompagnement humain authentique tout en assurant une gestion technique et sécurisée de l'environnement de l'utilisateur ?**

I.5 Analyse des Solutions Existantes

I.5.1 Étude de l'existant

Dans cette section, nous examinons les assistants domestiques et les robots compagnons les plus populaires sur le marché afin d'identifier leurs forces et leurs lacunes.

- **A. Les Assistants Vocaux Statiques (Amazon Alexa, Google Home)**

Ces dispositifs sont les plus répandus. Ils excellent dans l'exécution de tâches simples (domotique, rappels, recherches internet).

Limites : Ils n'ont pas de présence physique mobile. Surtout, ils restent des outils purement réactifs : ils attendent un "mot déclencheur" et répondent de manière standardisée. Il n'y a aucune notion de "personnalité" évolutive ou de sentiment d'appartenance.

- **B. Les Robots Domestiques (ex : Amazon Astro, Vector/Cozmo)**

Certains robots essaient d'occuper l'espace physique. Amazon Astro, par exemple, surveille la maison et suit l'utilisateur.

Limites : Bien que mobiles, ces produits restent perçus comme des gadgets technologiques. Leur interaction est limitée par des scripts pré-enregistrés. Ils ne "réfléchissent" pas de manière humaine et leur capacité à reconnaître et s'adapter à l'humeur ou à l'identité spécifique de chaque membre de la famille de façon approfondie reste superficielle.

- **C. Les Pepper & Nao (Robotique de service)**

Ce sont des robots haut de gamme capables de simuler des émotions et de reconnaître des visages.

Limites : Ces solutions sont extrêmement coûteuses (plusieurs milliers d'euros) et sont destinées aux entreprises plutôt qu'au grand public. Leur configuration est complexe et ils ne sont pas conçus comme des "amis" personnels intégrés à un écosystème mobile fluide pour l'utilisateur lambda.

I.5.2 Synthèse et Critique : Le "Manque d'Humanité"

Le point commun de tous ces concurrents est leur nature générique. Ce sont des machines produites en série qui traitent chaque utilisateur de la même manière.

- **Absence de personnalité** : La réponse d'une Alexa est la même pour tout le monde. Elle n'a pas de "caractère" propre qui pourrait la faire passer pour un compagnon de vie.
- **Froideur algorithmique** : Les interactions sont basées sur des arbres de décision rigides. L'utilisateur sent qu'il parle à une base de données, pas à une entité qui "réfléchit".

- **Dépendance au Cloud propriétaire** : L'utilisateur a peu de contrôle sur la "mémoire" ou la logique interne de l'assistant.

I.5.3 La Différenciation de Koda

C'est ici que notre projet se distingue. Contrairement aux machines industrielles, Koda est conçu pour être :

- **Unique** : Grâce à l'intégration de n8n (le cœur de réflexion), Koda peut avoir une logique de réponse personnalisée et évolutive.
- **Multimodal** : Il combine la vision (Raspberry Pi), la réflexion (IA via n8n) et la mobilité.
- **Accessible** : Une application mobile dédiée permet de configurer sa "personnalité", rendant l'utilisateur maître de son compagnon.

I.6 Solution Proposée

Pour surmonter ces limitations, nous proposons KODA, une architecture résiliente et multi-couche conçue pour l'autonomie et l'intelligence. La solution comprend :

- **Contrôleur Embarqué (Raspberry Pi)** : Le cœur physique du système, gérant les composants matériels et assurant le fonctionnement du robot.
- **Moteur de Réflexion (n8n)** : Le cerveau de KODA, responsable de la préparation des réponses et du traitement intelligent des requêtes complexes.
- **Couche de Distribution (Python)** : Le système nerveux responsable de la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi, offrant les points d'accès (endpoints) de tout l'écosystème.
- **Expérience Utilisateur Mobile** : Une application cross-plateforme permettant à l'utilisateur de configurer, contrôler et suivre la vie de son compagnon robotique en temps réel.
- **Mémoire Persistante (Base de données)** : Un système de stockage permettant à KODA d'enregistrer des informations contextuelles et d'apprendre de ses interactions.

En intégrant ces composants, la solution proposée garantit une résilience opérationnelle, des interactions humaines authentiques et une expérience de gestion unifiée.

I.7 Méthodologie du Projet

Pour garantir un haut degré de fiabilité et une intégration harmonieuse des différentes couches logicielles et matérielles, ce projet suit un cadre d'ingénierie rigoureux.

I.7.1 Modèle en spirale

Le pilotage du projet KODA repose sur le **modèle en spirale**, et non sur un cycle en V séquentiel ni sur une succession de sprints Scrum. À chaque tour de spirale, une partie du système est **précisée, construite, testée** et **raccordée** au reste ; avant d'engager le tour suivant, l'équipe **réévalue les risques** (techniques, d'intégration, de délai) et ajuste les priorités. Cette démarche convient à un compagnon robotique multicouche : on ne livre pas tout d'un coup, on fait monter en puissance le moteur de réflexion, le lien avec le robot, l'application mobile, puis la validation d'ensemble.

Le périmètre est découpé en **six cycles thématiques**, ordonnés de façon logique :

1. analyse et réflexion (moteur conversationnel) ;
2. couche de distribution et persistance des données ;
3. partie matérielle embarquée ;
4. partie logicielle sur le robot ;
5. application mobile de pilotage ;
6. tests et validation globale.

Chaque cycle se clôt par un **incrément démontrable** (fonctionnalité branchée, scénario utilisateur, critères d'acceptation) et par un point de synchronisation avec l'encadrement. La planification détaillée, les rôles et le calendrier des jalons sont formalisés au chapitre II ; la figure ci-dessous en résume le principe.

Artefacts et traçabilité

À chaque tour, des livrables assurent la continuité entre les cycles :

- **Besoins et périmètre** mis à jour (utilisateur, sécurité, mémoire).
- **Architecture et intégration** (schémas, interfaces entre robot, logiciel de réflexion, application).
- **Réalisation et recette** du cycle en cours (tests, démonstration, correctifs avant le tour suivant).

I.8 Conclusion

Ce chapitre a établi le contexte du projet KODA. En identifiant les lacunes critiques des assistants virtuels actuels — notamment le manque de personnalité, de réflexion autonome et d'intégration

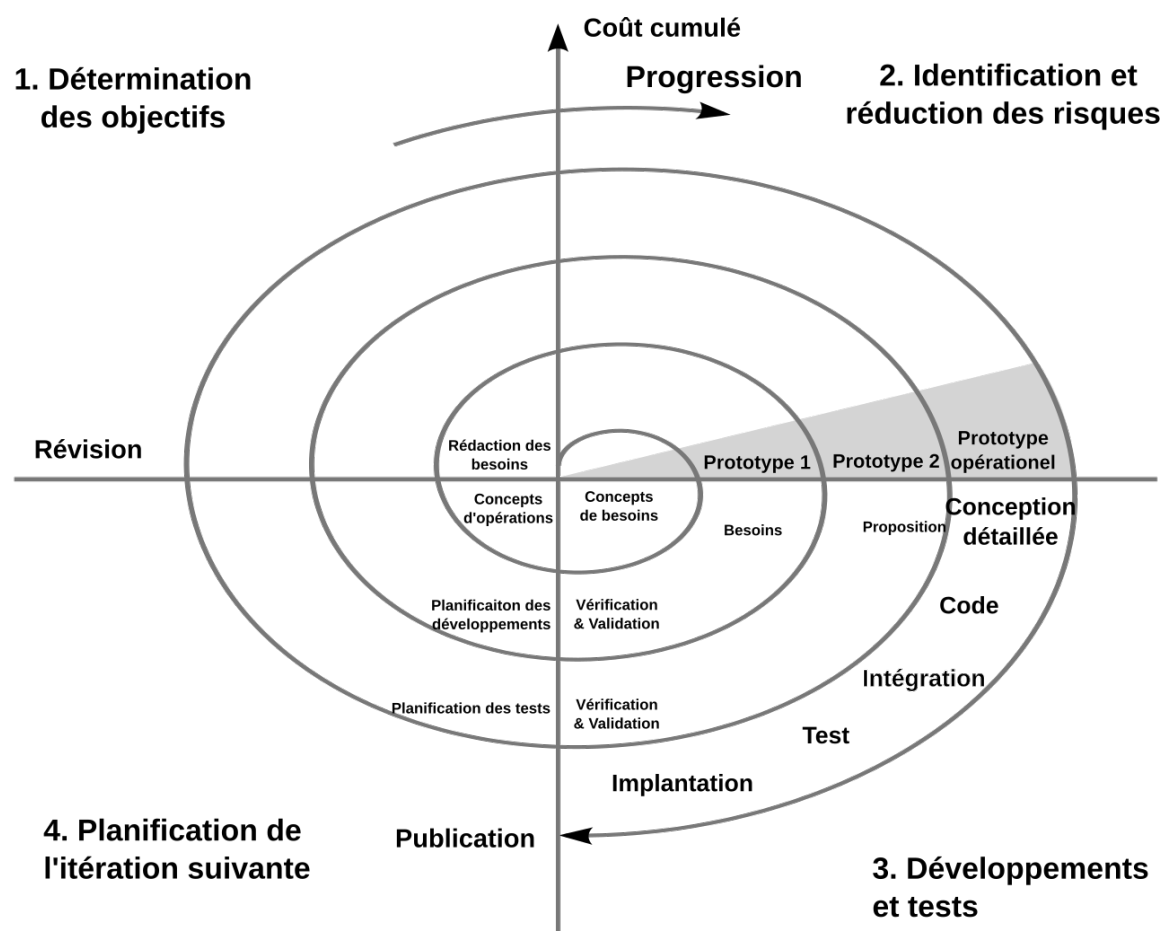


FIGURE I.1 – Modèle en spirale — enchaînement des six cycles de développement KODA

embarquée — nous avons défini notre solution innovante. Le **modèle en spirale** et les six cycles de développement posent une feuille de route incrémentale, adaptée à la complexité d'un système distribué entre compagnon physique, réflexion et pilotage mobile.

Le chapitre suivant, «**Préparation du Projet**», formalise les besoins, la méthodologie de développement **en spirale** par cycles successifs et les environnements techniques retenus pour mener à bien la réalisation de KODA.

_____ CHAPITRE II _____

PRÉPARATION DU PROJET

II.1 Introduction du chapitre

Ce chapitre pose les fondations ingénierie du projet **KODA** avant d’entrer dans le détail technique des cycles de réalisation. Nous y traduisons l’intention du chapitre I en **exigences traçables** (fonctionnelles et non fonctionnelles), en **vision architecturale** et en **périmètre d’acteurs**. Nous y présentons également la **méthodologie de pilotage** retenue : le **modèle en spirale**, fondé sur des **cycles de développement successifs** — et non sur une succession de sprints Scrum — où chaque tour de spirale livre un incrément (couche ou sous-système) intégrable et réévalue les risques avant le tour suivant. Enfin, nous documentons les environnements matériel et logiciel qui ont encadré la production du prototype.

II.2 Spécification des besoins

II.2.1 Spécifications des besoins fonctionnels

Les besoins fonctionnels précisent *ce que* KODA doit offrir à l’utilisateur, sans décrire encore les choix d’implémentation. Ils reprennent la problématique du chapitre I : un **compagnon de vie** capable de présence, de réflexion et d’accompagnement personnalisé, et non un simple exécuteur de commandes.

- **Dialoguer naturellement** : l’utilisateur s’adresse à KODA à l’oral ou par message ; le compagnon comprend la demande, y répond de façon claire et pertinente, et adapte son ton à la situation (explication, conseil, réconfort). Lorsque la question appelle une réponse immédiate (arrêt, mot de passe, musique, déplacement), KODA réagit sans détour ; sinon, il prend le temps d’analyser avant de répondre.
- **Se souvenir et personnaliser** : KODA reconnaît les personnes avec lesquelles il interagit, retient leurs présentations, leurs préférences et les éléments importants de leurs échanges (notes, informations personnelles). Il peut répondre en s’appuyant sur ce qu’il sait déjà de l’utilisateur, ou traiter une question générale sans tout reconsulter ; la mémoire est consolidée au fil du temps pour garder une continuité dans la relation.
- **Reconnaître et accueillir** : le compagnon identifie l’interlocuteur (visage, contexte) et adapte son comportement : accueil personnalisé pour une personne connue, invitation à se présenter pour un visiteur inconnu, conformément à l’objectif de présence humaine évoqué au chapitre I.
- **Être présent physiquement** : le robot matérialise l’interaction (captures, parole, déplacements ou signaux prévus par le prototype) et reste relié au reste du système pour que l’expérience ne se limite pas à un écran ou à une enceinte déconnectée du corps du compagnon.

- **Permettre le pilotage à distance** : via l'application mobile, l'utilisateur configure la personnalité de KODA (nom, caractère, langue, etc.), suit son activité et supervise son compagnon au quotidien, comme annoncé dans la solution proposée au chapitre I.
- **Protéger l'usage et l'environnement** : KODA fonctionne selon des modes ouvert ou protégé (mot de passe), afin de limiter l'accès aux fonctions sensibles lorsque le compagnon est en mode sécurisé; cette exigence répond au besoin, formulé au chapitre I, d'allier accompagnement humain et gestion responsable de l'environnement domestique.

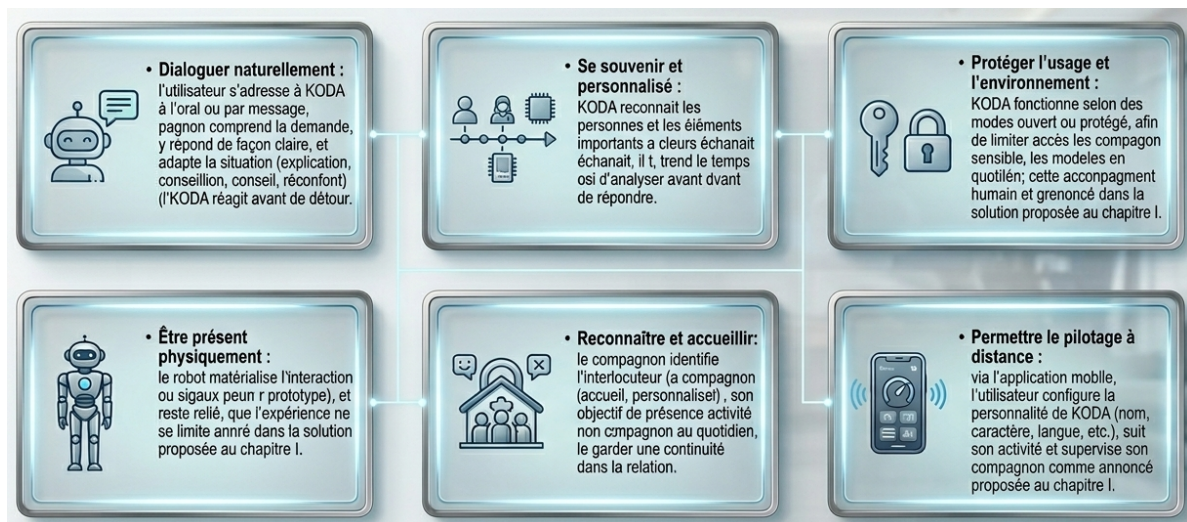


FIGURE II.1 – Synthèse visuelle des besoins fonctionnels KODA (chapitre II)

II.2.2 Spécification des besoins non fonctionnels

Les besoins non fonctionnels décrivent *comment* le système doit se comporter pour être acceptable au quotidien : fiabilité, confiance, simplicité d'évolution. Ils complètent les besoins fonctionnels sans entrer dans le détail des outils retenus aux chapitres techniques suivants.

- **Réactivité perçue** : les réponses du compagnon doivent paraître fluides à l'utilisateur, y compris lorsque le traitement s'appuie sur des services en ligne (recherche, synthèse vocale, etc.); les échanges déjà connus ne doivent pas systématiquement être retraités comme s'il n'y avait aucune mémoire.
- **Continuité de service** : une coupure partielle d'un service distant ne doit pas faire perdre l'essentiel des données ni bloquer durablement l'interaction; les informations importantes (profils, historiques, paramètres) restent conservées de façon durable.
- **Confiance et respect de la vie privée** : seules les personnes autorisées accèdent aux fonctions sensibles; les identifiants et données personnelles sont protégés, et le compagnon n'expose pas inutilement ce qu'il sait de l'utilisateur.

- **Clarté pour l'équipe projet** : la logique est organisée par blocs (dialogue, mémoire, robot, application) afin de pouvoir corriger ou enrichir une partie sans remettre en cause tout le système ; les scénarios principaux restent documentés pour la validation en fin de cycle.
- **Ouverture aux évolutions** : il doit être possible d'ajouter une nouvelle capacité (type de question, écran mobile, capteur) en s'appuyant sur l'architecture globale du chapitre I, sans repartir d'une conception entièrement nouvelle.
- **Accessibilité d'usage** : l'application de pilotage et les modes du compagnon restent compréhensibles pour un utilisateur non spécialiste, en ligne avec l'objectif d'un compagnon « accessible » par rapport aux robots professionnels coûteux évoqués dans l'analyse de l'existant.

II.2.3 Architecture du système

L'architecture cible est **multicouche** et **distribuée** : un moteur d'automatisation (**n8n**) concentre la logique métier conversationnelle ; un **backend Python (FastAPI)** centralise les accès données et les intégrations cloud ; un **noyau embarqué (Raspberry Pi)** assure le lien avec le monde physique ; une **application mobile** prolonge l'expérience utilisateur ; une **base PostgreSQL** matérialise la mémoire long terme. La figure II.2 (chapitre I et II) illustre cette décomposition ; les cycles de développement du présent rapport s'alignent sur ces blocs pour réduire le rayon d'intégration à chaque itération.

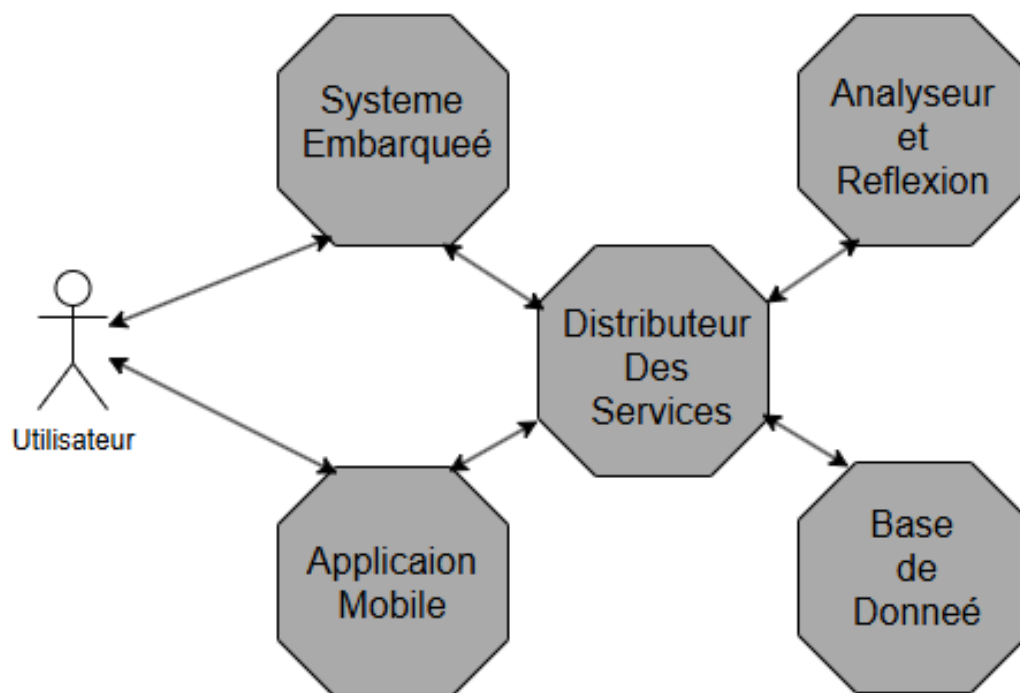
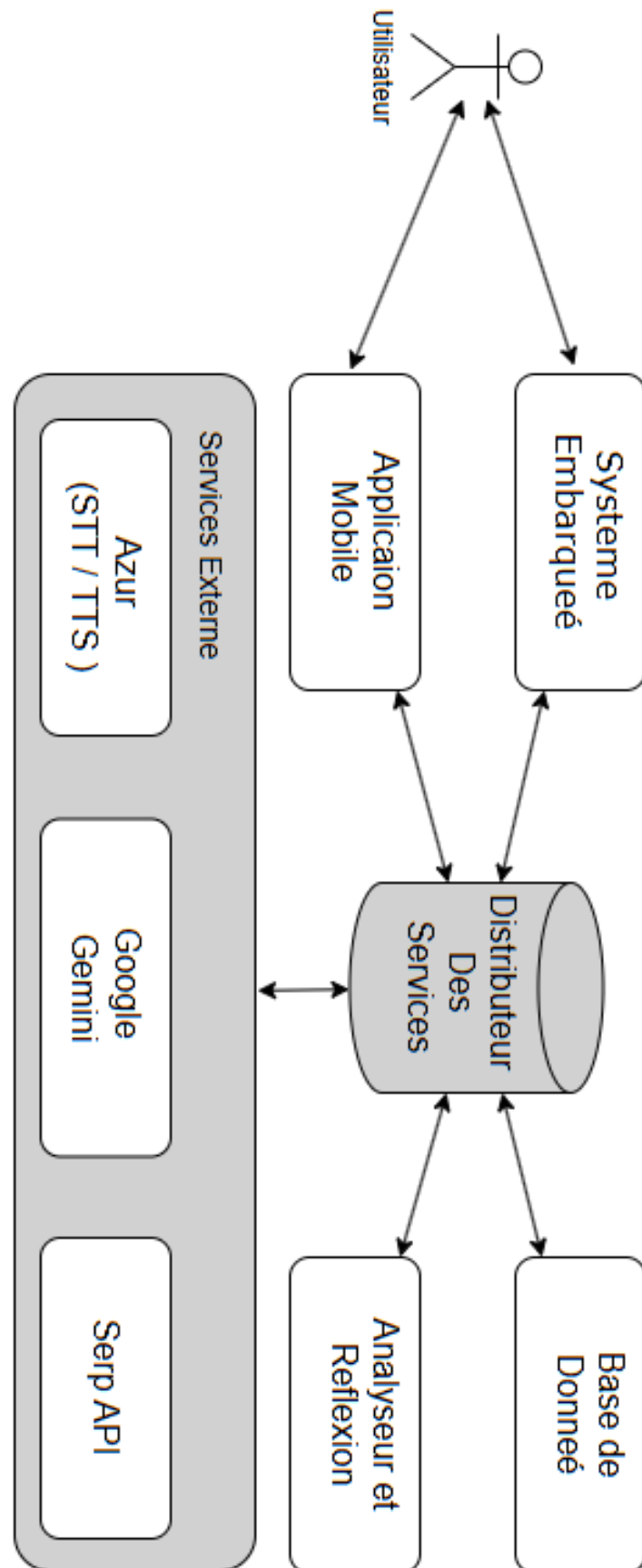


FIGURE II.2 – Architecture multicouche du système KODA (vue chapitre II)

II.2.4 Identification des acteurs

On distingue notamment :

- l'**Utilisateur final** (personne qui dialogue avec KODA et configure l'application) ;
- le **système KODA** (ensemble logiciel/matériel) ;
- le **Distributeur de services** (backend) en tant qu'intermédiaire technique ;
- les **services externes** (Supabase, OpenRouter pour les LLM, SerpAPI pour la recherche musique au Block 1, Azure pour la voix et la vision) ;
- l'**organisme d'accueil** et l'**équipe projet** (rôles de pilotage et de validation).

**FIGURE II.3** – Acteurs principaux et contexte d'intégration du système KODA

II.3 Pilotage du projet — Modèle en spirale et cycles de développement

Contrairement à une approche **Agile Scrum** fondée sur des *sprints* de durée fixe avec cérémonies prescrites (daily, review, rétrospective), nous avons retenu le **modèle en spirale** : à chaque itération, une portion du produit (cycle thématique : workflows n8n, backend, matériel, logiciel embarqué, mobile, validation) est **spécifiée, réalisée** puis **intégrée** au système global, avec une **réévaluation des risques** et des priorités avant le tour suivant. Chaque cycle produit un incrément **testable** et raccordé aux livrables des tours précédents, jusqu'à stabiliser l'écosystème KODA complet.

II.3.1 Équipe et rôles

Le projet est conduit par une **équipe de projet de fin d'études** (DSI3.2, ISET Mahdia), en binôme, sous la responsabilité d'un **encadrant académique** et en lien avec l'**organisme d'accueil** MicroEdition (cf. chapitre I). Les rôles se répartissent entre conception des workflows n8n, développement backend, intégration embarquée, développement mobile et campagne de tests, avec des points de synchronisation à la fin de chaque cycle pour valider l'intégration incrémentale.

II.3.2 Découpage du travail : six cycles

Le périmètre fonctionnel est découpé en six cycles **ordonnés logiquement** — sans équivalence directe avec des sprints Scrum — comme suit :

1. **Cycle 1 — Workflow et n8n** : conception des webhooks, blocs de traitement (classification d'intention, analyseur de dépendance mémoire, consolidation, proactivité), intégration des API LLM et recherche.
2. **Cycle 2 — Backend** : Distributeur de services (FastAPI), accès PostgreSQL / Supabase, Clean Architecture, endpoints pour le robot et le mobile.
3. **Cycle 3 — Système embarqué, partie matérielle** : choix et câblage des capteurs, actionneurs, alimentation, mécanique ou support physique associé au Raspberry Pi selon le prototype.
4. **Cycle 4 — Système embarqué, partie logicielle (Raspberry Pi)** : logiciel embarqué, communication avec le backend, gestion des flux audio/vidéo locaux, robustesse sur carte ARM.
5. **Cycle 5 — Cross-plateforme** : application mobile (configuration, suivi, notifications), parcours utilisateur unifiés Android/iOS.
6. **Cycle 6 — Tests et validation** : tests unitaires et d'intégration, scénarios bout-en-bout, validation des besoins non fonctionnels (sécurité, performance).

TABLE II.1 – Vue synthétique des cycles et livrables attendus

Cycle	Thème	Livrable principal
1	Workflow n8n	Webhooks opérationnels, prompts et routages stabilisés
2	Backend	API documentée, persistance, intégrations cloud
3	Embarqué HW	Plateforme physique prête à accueillir le logiciel
4	Embarqué SW (Pi)	Services locaux et dialogue avec le Distributeur
5	Cross-plateforme	Application mobile utilisable sur les cibles
6	Tests & validation	Rapport de recette, correctifs de fin de projet

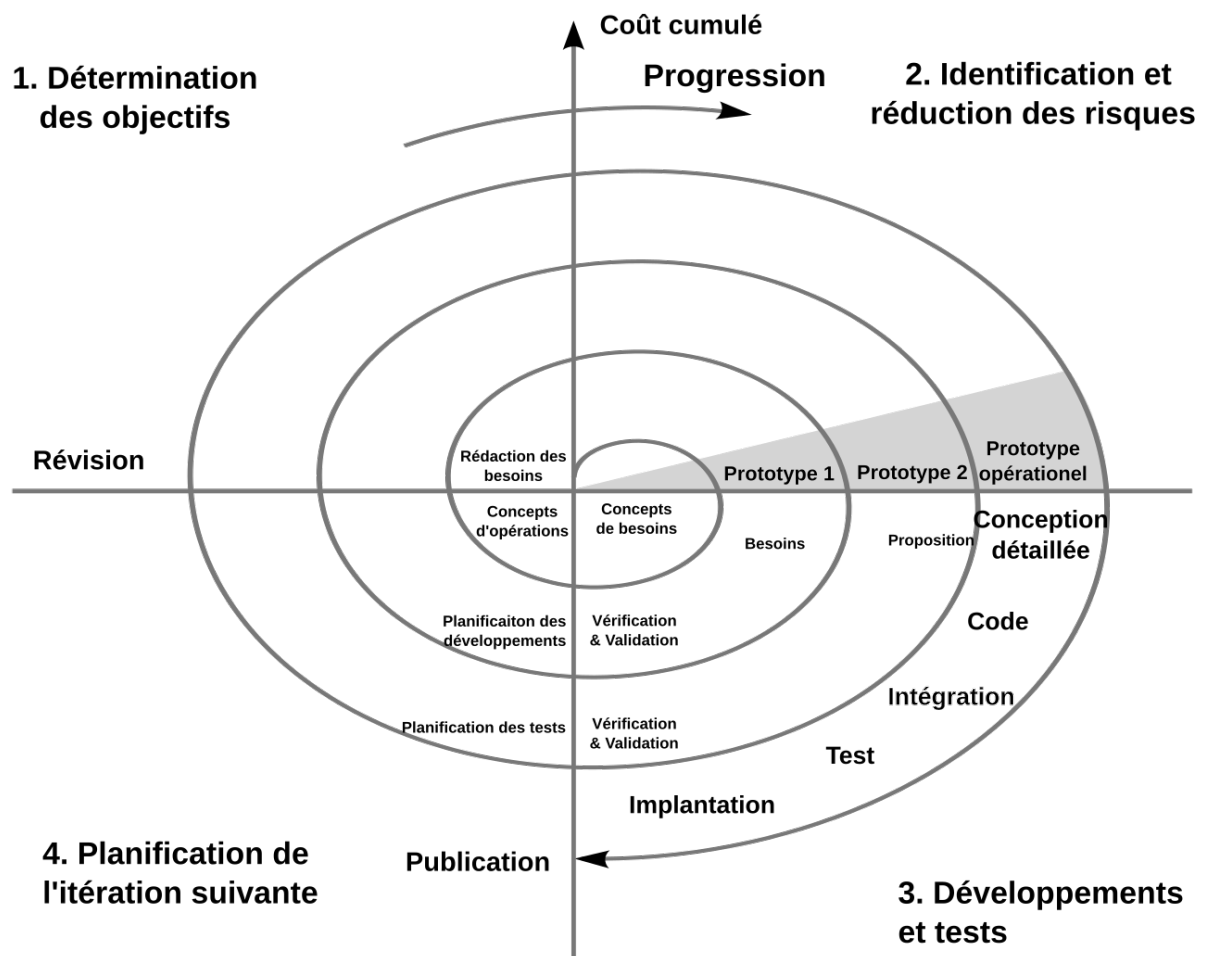


FIGURE II.4 – Modèle en spirale et enchaînement des six cycles de développement KODA

II.4 Planification des cycles

La planification ne s'exprime pas en **sprints** de deux semaines mais en **jalons de cycles** : chaque cycle se termine par une **démonstration intégrée** (workflow branché sur le backend, backend branché sur la base, puis ajout du Raspberry Pi, etc.). Les dépendances imposent un ordre raisonnable : le moteur n8n (cycle 1) et le backend (cycle 2) peuvent avancer en parallèle partiel, mais la validation bout-en-bout (cycle 6) recouvre l'ensemble. Le calendrier académique du PFE (nombre de semaines, séances de suivi, livrables intermédiaires) structure la cadence réelle ; le **modèle en spirale** en garantit la **cohérence** : aucun cycle ne clôt sans critères d'acceptation liés aux chapitres techniques correspondants du rapport.

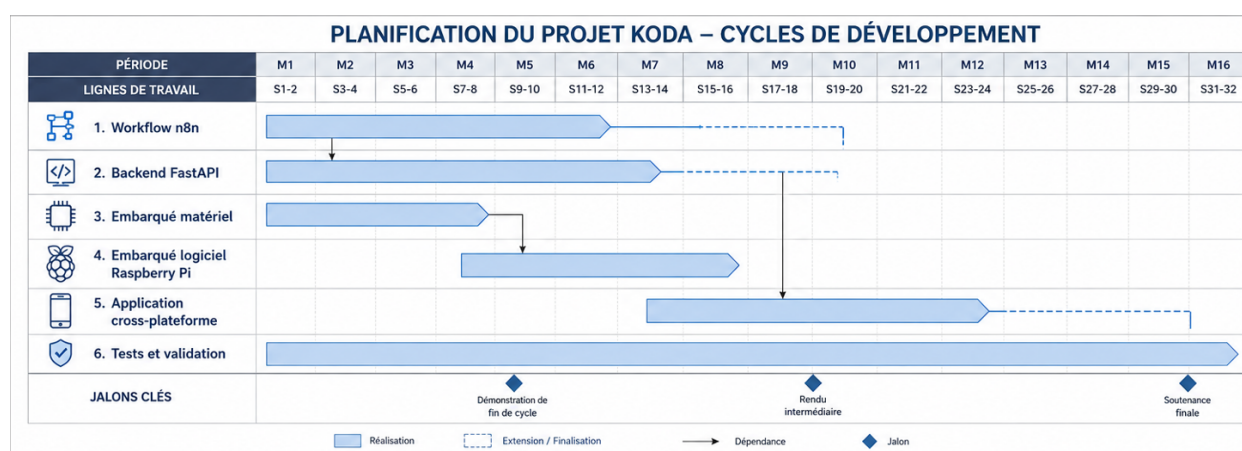


FIGURE II.5 – Planification des cycles : jalons, chevauchements et dépendances

II.5 Architecture

II.5.1 Architecture de l'application

Au-delà du schéma global (chapitre I), l'architecture applicative distingue : **(a)** la couche *présentation* (mobile) ; **(b)** la couche *orchestration intelligente* (n8n) ; **(c)** la couche *services* (FastAPI) ; **(d)** la couche *données* (PostgreSQL) ; **(e)** la couche *embarquée* (Raspberry Pi). Les communications principales suivent le modèle requête-réponse REST et événements asynchrones selon les besoins (temps réel, webhooks). Cette structuration facilite le parallélisme des cycles tout en conservant des interfaces stables entre équipes.

II.5.1.1 Architecture en oignon (Onion Architecture) du backend

Le **Distributeur de services** (backend Python, cycle 2) adopte une **architecture en oignon** : les couches sont organisées en **anneaux concentriques**, du plus stable au plus volatile. Au **cœur** se trouvent le **domaine métier** (entités, règles) ; autour, les **cas d'usage** orchestrent les scénarios

sans dépendre des frameworks ; puis viennent les **adaptateurs** (API REST, sérialisation, accès données) ; en périphérie, la **infrastructure** (FastAPI, Supabase, SDK Azure, clients HTTP vers n8n ou le robot). Les dépendances pointent **toujours vers l'intérieur**, ce qui isole la logique métier des technologies et facilite les tests unitaires sur le domaine.

Ce modèle — popularisé notamment par Jeffrey Palermo sous le nom d'*Onion Architecture* — recoupe les principes de la **Clean Architecture** détaillés au chapitre du cycle 2 : les deux visions décrivent des couches concentriques et la règle d'inversion des dépendances ; l'« oignon » met l'accent sur la **testabilité** du noyau et sur le fait que l'infrastructure soit interchangeable sans toucher au cœur du système. Ainsi, le backend KODA reste cohérent avec la vision multicouche de la figure II.2 tout en précisant sa structure interne.

II.6 Environnement de Developement

II.6.1 Environnement matériel

Le développement s'appuie sur des postes de travail type PC (processeur multi-cœurs, au moins 8 Go de RAM recommandés pour les outils de conteneurisation et les IDE), un **Raspberry Pi** cible pour les tests embarqués, périphériques audio/vidéo selon les besoins du robot, et un smartphone ou émulateur pour la couche mobile. Un accès Internet stable est nécessaire pour Supabase, OpenRouter, SerpAPI et Azure durant l'intégration.

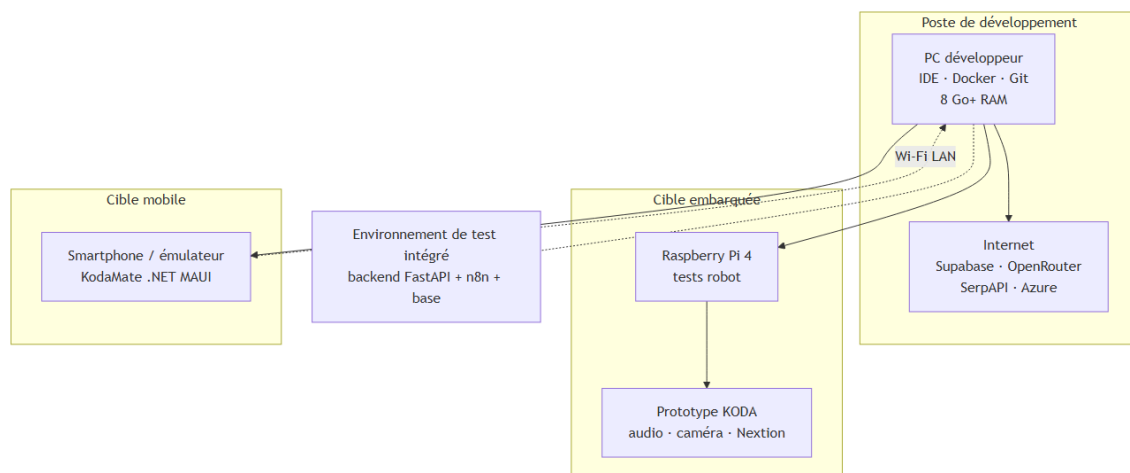


FIGURE II.6 – Environnement matériel de développement et de test (poste, Raspberry Pi, robot)

II.7 Environnement logiciel

II.7.1 Outils de développement et modélisation

- **n8n** (self-hosted ou cloud) pour l'édition des workflows ;

- **Visual Studio Code** ou équivalent pour Python, Dart et LaTeX ;
- **Git** pour le versioning ;
- **draw.io / Mermaid** / captures d'écran pour les diagrammes de séquence et de cas d'utilisation ;
- **Supabase Studio** pour l'administration PostgreSQL ;
- **Docker** (le cas échéant) pour le déploiement du backend et la reproductibilité des environnements.

II.7.2 Langages et frameworks utilisés

- **JavaScript** (expressions n8n, nœuds Code) ;
- **Python 3** avec **FastAPI**, **SQLAlchemy**, **asyncpg**, **SDK Azure** et intégrations **Gemini/OpenRouter** côté Distributeur ;
- **SQL** (PostgreSQL) ;
- **Dart / Flutter** pour l'application cross-plateforme ;
- **Bash** et utilitaires système sur Raspberry Pi OS.

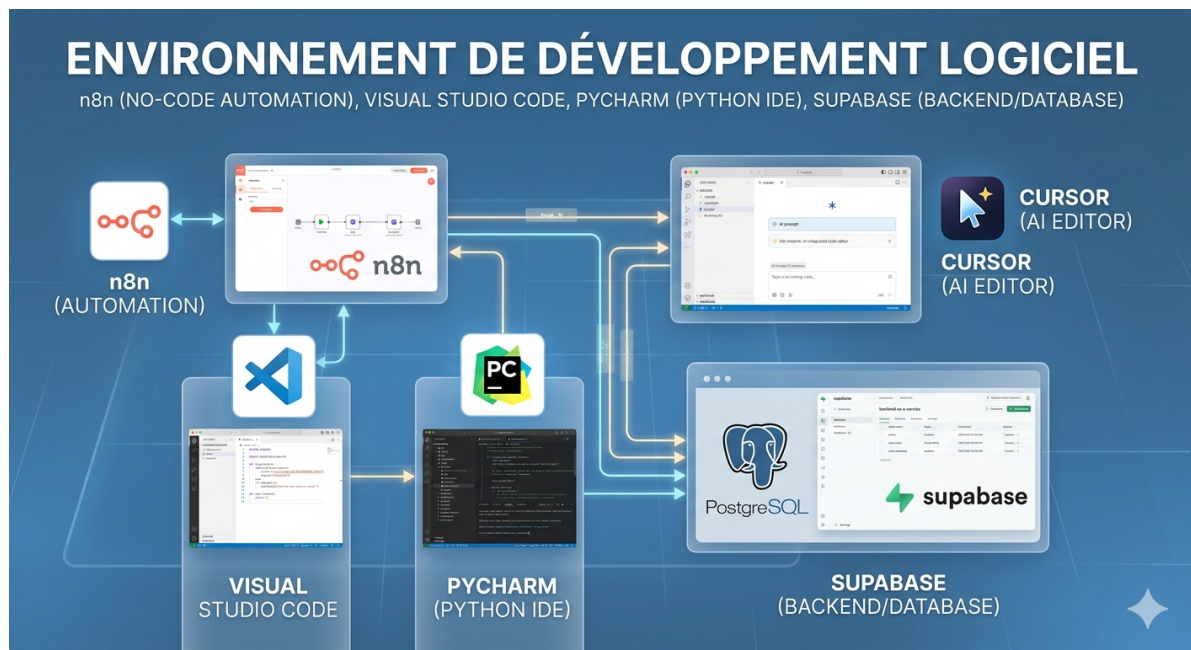


FIGURE II.7 – Environnement logiciel : outils de développement et chaîne technique (chapitre II)

II.8 Conclusion

Ce chapitre a cadré la préparation de KODA : besoins fonctionnels et non fonctionnels, acteurs, architecture de référence, environnements, et surtout un **modèle en spirale** articulé en six cycles — en rupture avec une gestion purement Scrum/sprint, tout en conservant l'esprit d'incrémentalité et

de revue de fin de cycle. Le chapitre suivant détaille le **Cycle 1 (Analyseur et réflexion)** : workflows n8n.

_____ CHAPITRE III _____

CYCLE 1 : ANALYSEUR ET RÉFLEXION

III.1 Introduction

Le **Cycle 1 — Analyseur et réflexion** constitue le **moteur cognitif** de KODA. Son rôle est double : assurer la **génération des réponses** lorsque l'utilisateur pose une question, et alimenter la **spontanéité** du robot lorsqu'il prend l'initiative de parler. L'objectif poursuivi est de développer un système capable d'**accueillir une question**, de la **comprendre dans son contexte**, d'effectuer les **recherches et consultations** nécessaires (mémoire, profil, services externes), puis de **formuler et restituer une réponse** adaptée.

La chaîne de traitement — de l'entrée utilisateur à la réponse — forme une **ligne de processus** successifs. Les **outils d'automatisation de flux** offrent une meilleure lisibilité et un débogage par étape que du code monolithique ; la **bonne pratique** retenue est la plateforme **n8n**, auto-hébergée et adaptée aux pipelines mêlant règles métier et intelligence artificielle.

Trois **webhooks** n8n structurent ce cycle. Après les fondements théoriques, la modélisation globale puis l'analyse détaillée de chaque flux, le lecteur dispose d'une vue complète du moteur d'analyse et de réflexion.

III.2 Fondements Théoriques

Cette section pose le **cadre cognitif** du Cycle 1. Elle explique, en termes généraux, comment une personne comprend une question et choisit une réponse ; elle ne décrit pas encore les outils informatiques (n8n, webhooks, base de données), traités aux sections suivantes.

III.2.1 Que signifie « théorie du double traitement » ?

L'expression **théorie du double traitement** (*dual-process theory*) désigne une idée simple : face à une question, l'esprit humain ne suit pas toujours le même chemin. Selon la situation, il peut répondre **presque tout de suite** en s'appuyant sur ce qu'il sait déjà, ou réfléchir **plus longtemps** en recomposant des éléments mémorisés. Ce modèle, largement diffusé par Daniel Kahneman (*Thinking, Fast and Slow*, 2011), aide à comprendre pourquoi certaines réponses sont instantanées et d'autres demandent un effort conscient.

En psychologie cognitive, on parle souvent de **cognition intuitive** (rapide) et de **cognition analytique** (lente), plutôt que de « système 1 » et « système 2 » : ces derniers termes sont des étiquettes de laboratoire peu parlantes pour le lecteur non spécialiste. Nous les remplaçons donc par des formulations explicites ci-dessous.

III.2.2 Réflexion rapide et réflexion lente

III.2.2.1 *Réflexion rapide : réutiliser la mémoire déjà présente*

La **réflexion rapide** correspond à une **cognition heuristique** : le cerveau repère un schéma connu et active presque aussitôt un souvenir, une habitude ou une règle déjà apprise. La réponse paraît « évidente » parce qu'elle ne reconstruit pas toute la situation depuis zéro ; elle **s'appuie sur la mémoire déjà là** : souvenirs récents de la conversation, image de l'interlocuteur, faits appris par cœur, gestes routiniers (saluer, donner son prénom, réciter une capitale connue).

Cette voie est **économique en attention** : elle convient lorsque la question ressemble à quelque chose de déjà vécu ou lorsque l'information demandée est directement stockée. Elle peut toutefois se tromper si le contexte a changé sans que l'on s'en aperçoive (réponse trop hâtive fondée sur une analogie fausse).

III.2.2.2 *Réflexion lente : analyser la mémoire pour fabriquer une réponse*

La **réflexion lente** correspond à une **cognition analytique** : le cerveau **parcourt, compare et combine** des éléments en mémoire pour **construire** une réponse nouvelle. Elle intervient lorsque la question est floue, inédite, émotionnellement chargée ou exige de croiser plusieurs souvenirs (passés lointains, préférences de l'interlocuteur, règles implicites de politesse, conséquences d'un choix).

Elle est **plus lente** précisément parce qu'elle ne se contente pas d'un rappel automatique : elle **analyse** le contenu mémorisé (épisodes, connaissances, sentiments) avant de formuler une phrase adaptée. On peut encore corriger ou affiner la réponse une fois qu'elle est énoncée (reformulation, nuance, excuse).

Les deux voies **coopèrent** : la réflexion rapide propose souvent une première piste ; la réflexion lente peut la confirmer, la nuancer ou la remplacer.

III.2.3 Comment une question est comprise avant la réponse

Qu'une question arrive à l'**oral** ou à l'**écrit**, le déroulement général est le suivant :

1. **Percevoir la question** : entendre ou lire les mots, filtrer le bruit, porter attention à l'interlocuteur.
2. **Comprendre le sens** : saisir ce qui est demandé (information, avis, souhait, plaisanterie) et le contexte immédiat (qui parle, de quoi on parlait juste avant).
3. **Mobiliser la mémoire** : faire remonter des souvenirs utiles : échanges récents, connaissances générales, image de la relation avec cette personne.
4. **Choisir la voie rapide ou lente** : si un souvenir pertinent « saute aux yeux », la réflexion rapide

suffit ; sinon, la réflexion lente assemble et vérifie les éléments avant de répondre.

5. **Formuler et, si besoin, ajuster** : prononcer ou écrire la réponse, puis la corriger si elle sonne faux ou incomplet.

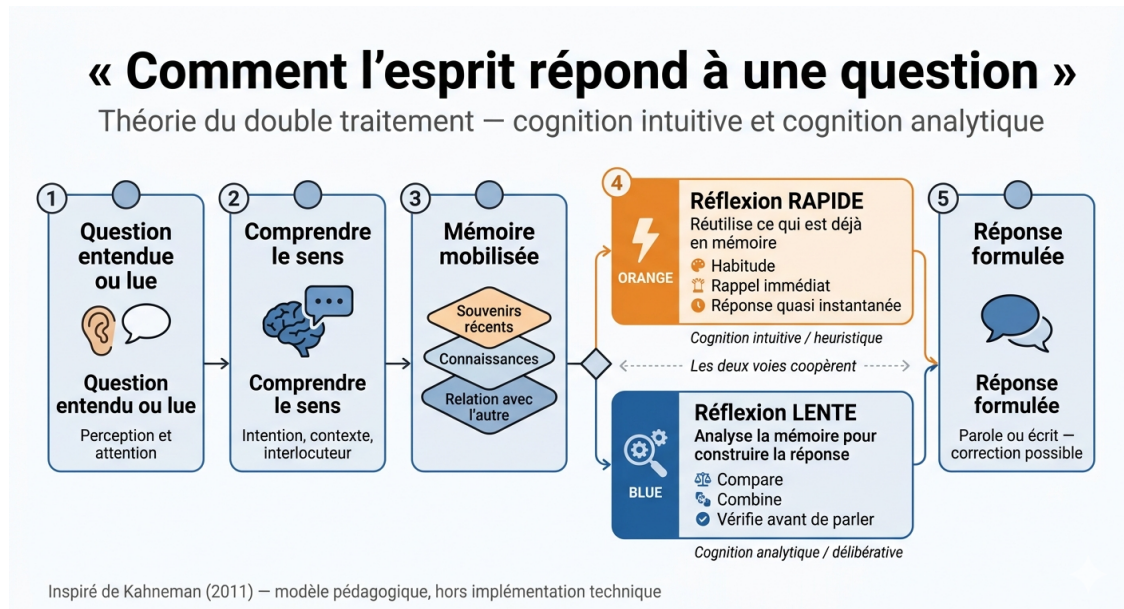


FIGURE III.1 – De la question à la réponse : réflexion rapide (mémoire déjà présente) ou réflexion lente (analyse puis construction)

III.2.4 Intérêt pour un compagnon conversationnel comme KODA

Un robot compagnon doit imiter cette logique humaine : parfois répondre vite en réutilisant ce qui vient d'être dit ou mémorisé sur l'utilisateur ; parfois prendre le temps d'**interpréter** la demande et d'**élaborer** une réponse plus personnalisée. Le Cycle 1 matérialise ce principe dans l'implémentation technique, décrite à partir de la section III.4 ; la figure III.4 en donne une vue d'ensemble une fois les notions d'outillage posées.

Références : Kahneman, D. (2011). *Thinking, Fast and Slow* ; Evans, J.St.B.T. (2008). Dual-processing accou

III.3 Définitions Fondamentales

Le Cycle 1 s'appuie sur **n8n** (automatisation par workflows et nœuds, données JSON, auto-hébergement) et sur trois **webhooks** HTTP déclenchant les pipelines d'analyse, de synthèse et de spontanéité. Les appels aux modèles de langage passent par **OpenRouter**, agrégateur d'API modèles utilisé dans les nœuds n8n de type *Basic LLM Chain* ou *AI Agent*.

III.3.1 Grand modèle de langage (LLM)

Un **grand modèle de langage** — en anglais *Large Language Model*, souvent abrégé **LLM** — est un modèle de **apprentissage profond** entraîné sur d’immenses corpus textuels pour prédire la suite probable de tokens (mots ou sous-mots). Il sait produire du texte fluide, suivre des consignes (*prompts*), classer ou reformuler des entrées, et combiner plusieurs contraintes lorsque le prompt est bien structuré.

Dans n8n, un LLM est typiquement invoqué via des nœuds de type **Basic LLM Chain** ou **AI Agent** : le workflow transmet un prompt système et le message utilisateur ; le modèle renvoie une **complétion textuelle** exploitable par les nœuds aval (parsing JSON, routage, réponse au webhook). Dans la **classification sémantique** du premier flux, le LLM joue le rôle d’**Analyseur de Dépendance Mémoire** : il décide si une consultation de la base est nécessaire et assigne une catégorie, sous une forme de sortie strictement contrôlée (lignes `USE_MEMORY` / `NO_MEMORY`, etc.).

Référence : Russell & Norvig (2010), *Artificial Intelligence : A Modern Approach*.

III.4 Modélisation globale du Cycle 1

La modélisation suivante traduit, sur le plan technique, la distinction entre **réflexion rapide** (réutilisation de l’historique récent) et **réflexion lente** (analyse approfondie avant réponse), introduite au § III.2.

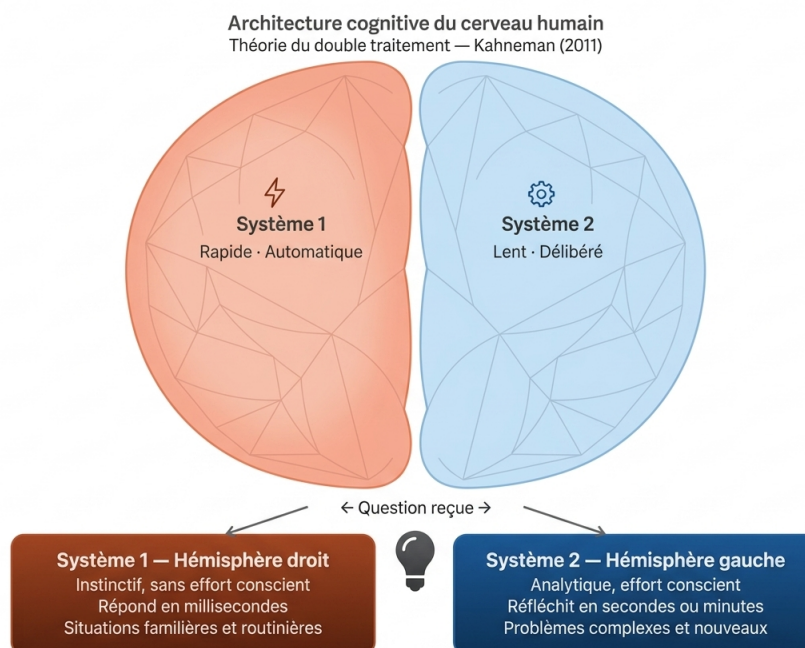


FIGURE III.2 – Architecture cognitive de KODA inspirée de la théorie du double traitement

III.4.1 Vue d'ensemble des trois flux n8n

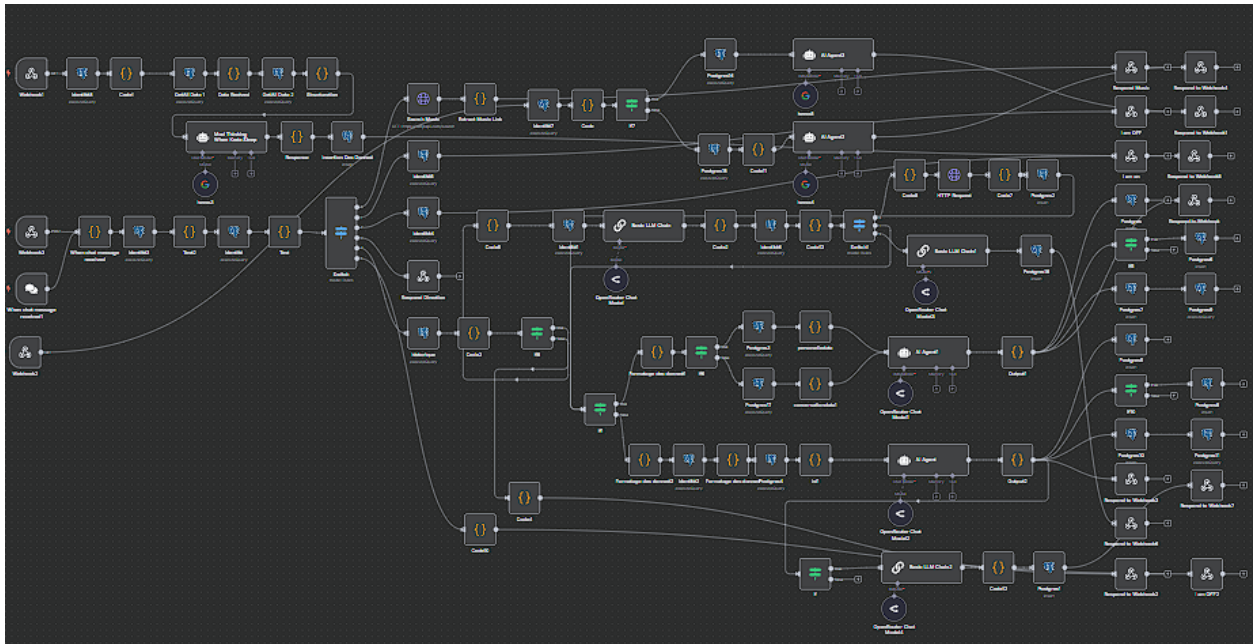


FIGURE III.3 – Cycle 1 — vue d'ensemble des trois workflows n8n

III.4.2 Diagramme de cas d'utilisation général

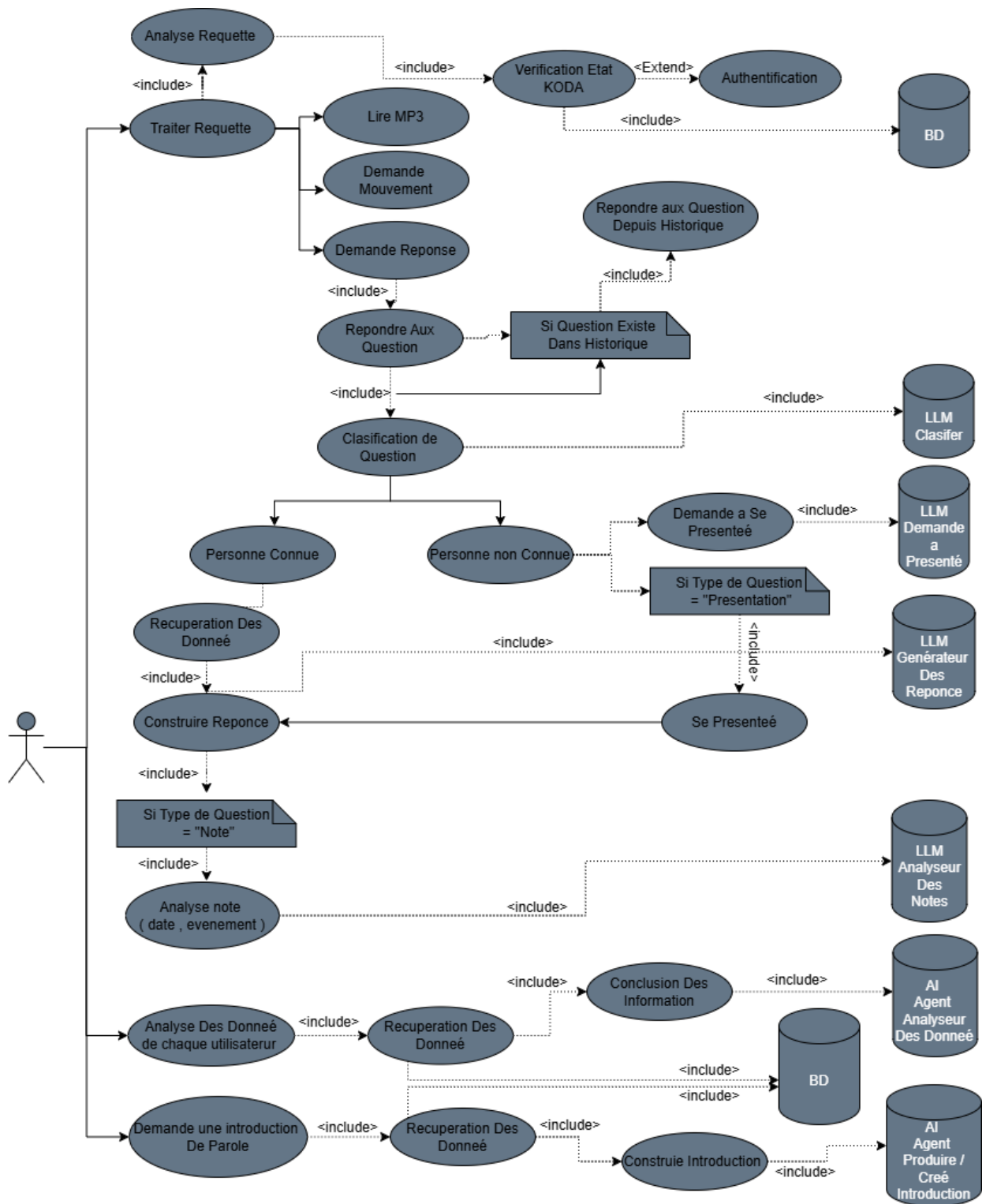


FIGURE III.4 – Cycle 1 — diagramme de cas d'utilisation général

III.4.3 Schéma de classes général

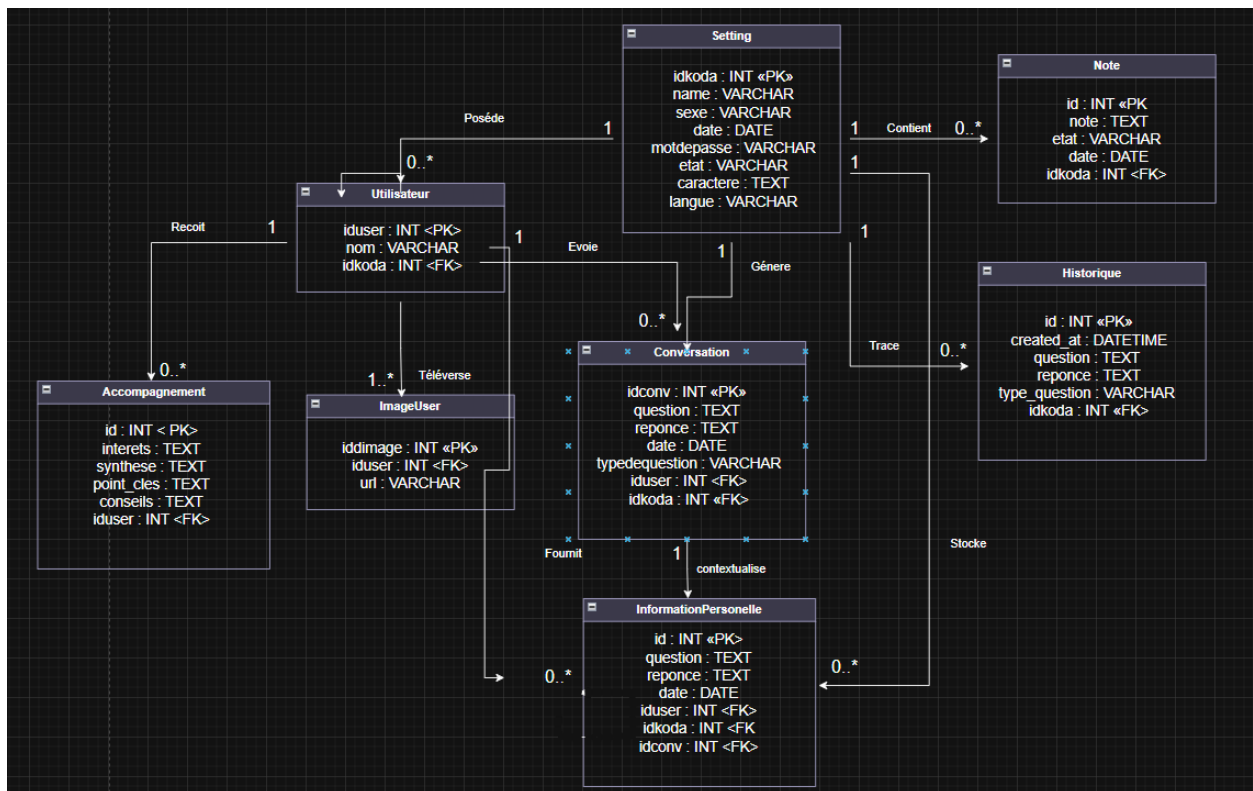


FIGURE III.5 – Cycle 1 — schéma de classes général (persistance)

III.5 Fonctionnalités Techniques — les trois flux

III.5.1 Premier Flux — Webhook 1 : Analyse et Réponse

III.5.1.1 Principe général et découpage en six parties

Le **Webhook 1** est le pipeline principal en temps réel. Il se décompose en six parties, dans l'ordre de la figure III.5.1.2 : *entrée fonctionnelle*, *confrontation à l'historique*, *classification sémantique*, *utilisateur connu ou inconnu*, *réponse contextualisée*, *notes et mémoire persistante*.

III.5.1.2 Diagramme de séquence du Webhook 1

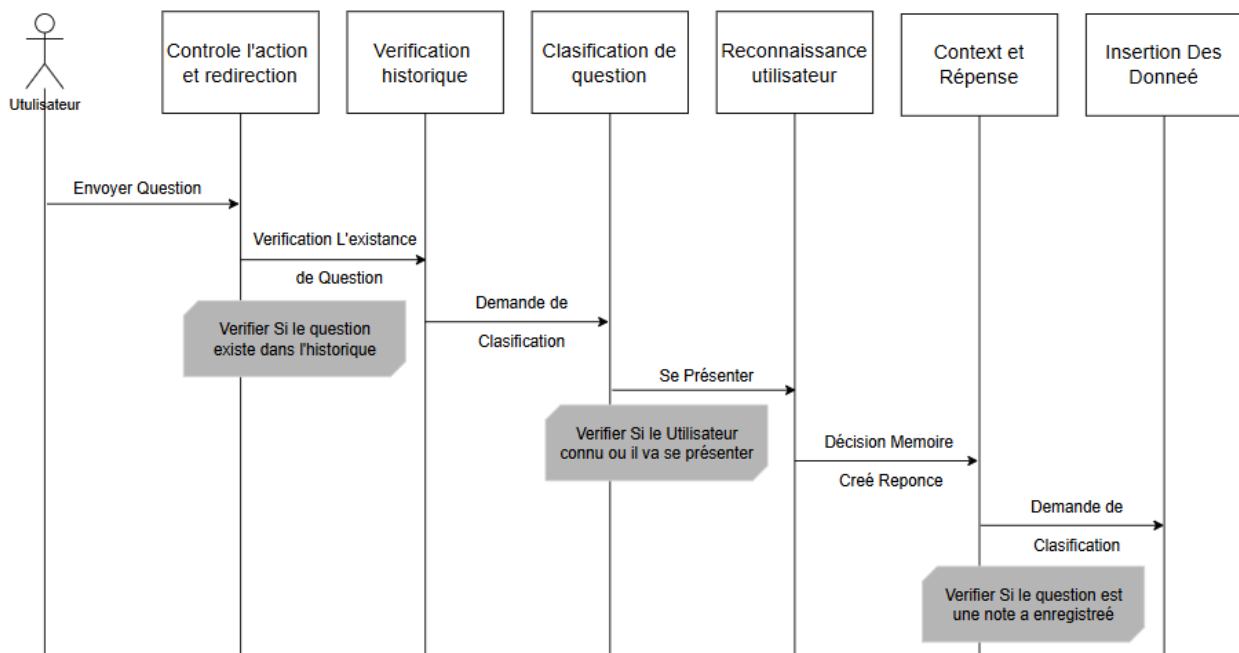


FIGURE III.6 – Webhook 1 — diagramme de séquence (six parties du pipeline)

III.5.1.3 Format de la Requête Entrante

Le webhook accepte les requêtes au format (user_name + question). Le champ user_name est un identifiant unique généré automatiquement par le backend pour chaque utilisateur en cours d'interaction. Cet identifiant est injecté avec la question dans le flux de travail, permettant d'isoler et de gérer les données de chaque utilisateur de manière indépendante dans les environnements multi-utilisateurs.

III.5.1.4 Les six parties du Webhook 1

III.5.1.4.1 Entrée fonctionnelle — analyse du message et classification d'intention

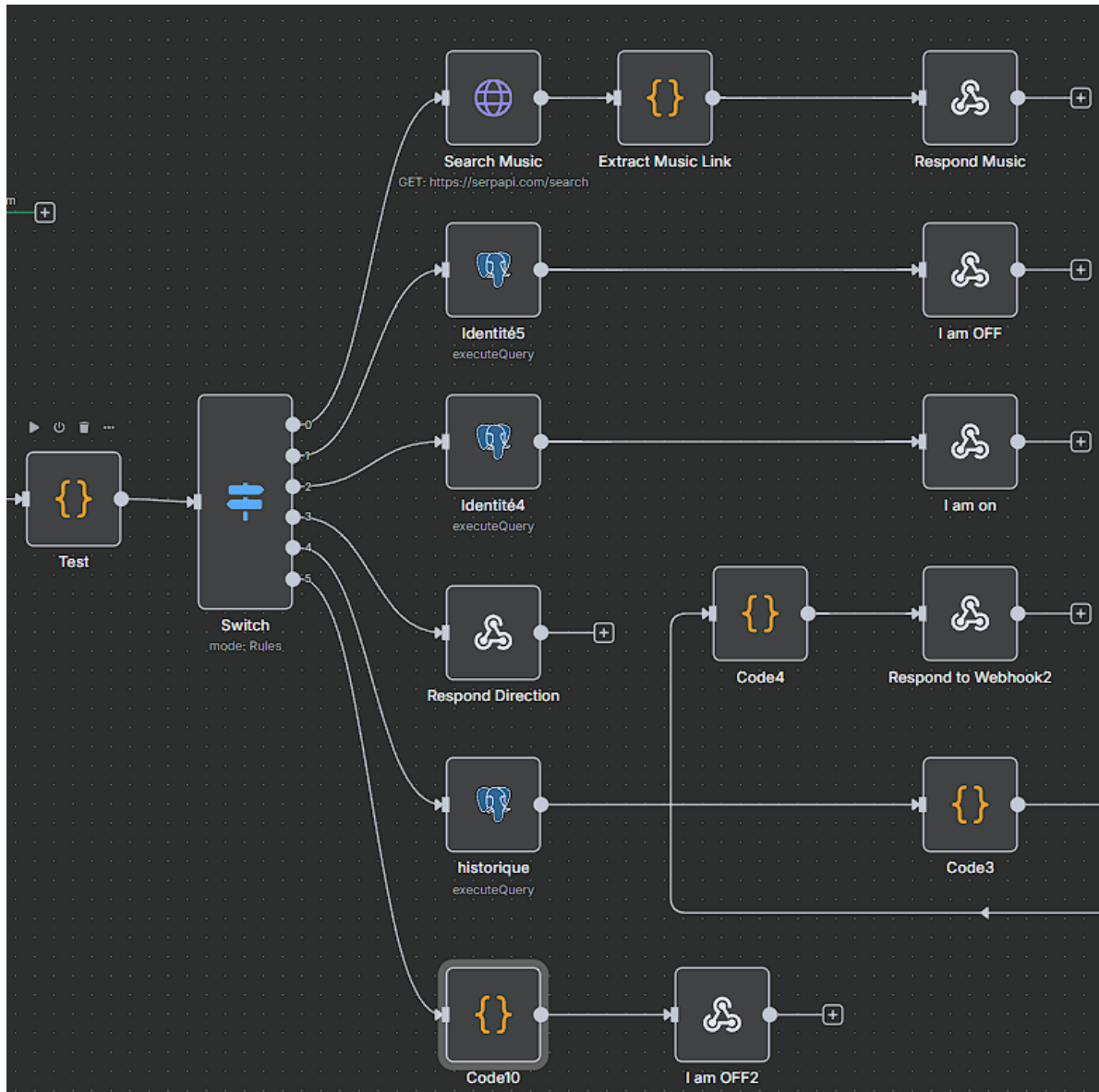


FIGURE III.7 – Webhook 1 — entrée fonctionnelle (nœud *Test*, *Switch*)

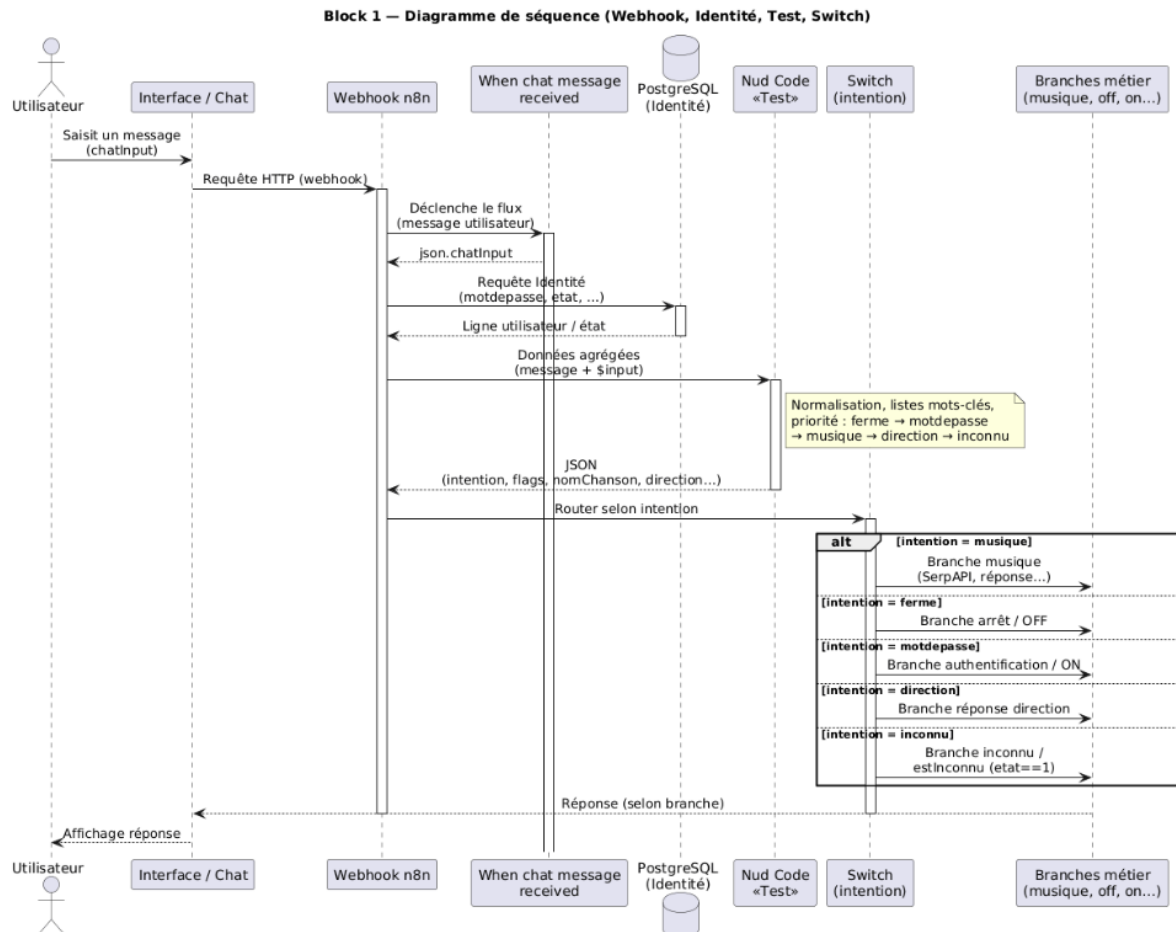


FIGURE III.8 – Webhook 1 — séquence de l’entrée fonctionnelle

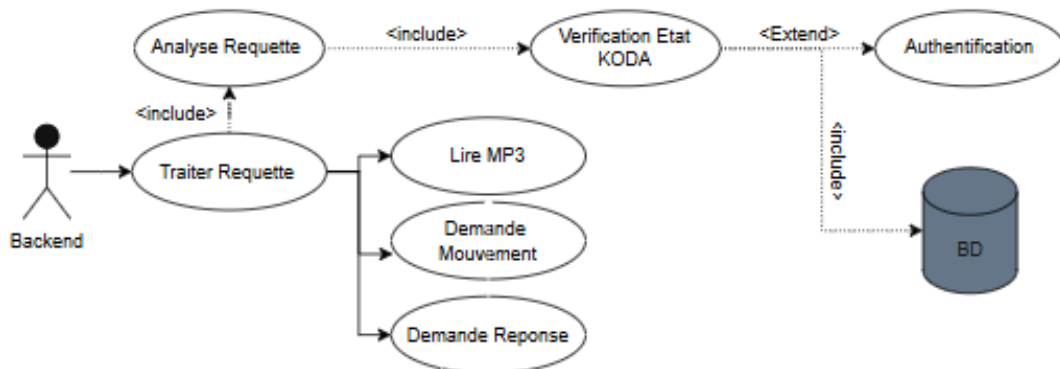


FIGURE III.9 – Webhook 1 — cas d’utilisation de l’entrée fonctionnelle

Dès l’appel au webhook et la lecture du message de chat (chatInput), le flux interroge la base (nœud PostgreSQL *Identité*) pour récupérer au minimum le **mot de passe** attendu et l’**état** courant du robot (etat). Le nœud Code **Test** consolide ces données et le texte saisi par l’utilisateur afin de produire une **intention** unique consommée par le nœud *Switch* en aval. Ce *Switch* (six sorties, voir figure III.5.1.4.1) active l’une des branches métier (musique via SerpAPI, arrêt / authentification via

requêtes *Identité4–5*, direction, historique comparé, branche *Code10*, etc.) selon le diagramme de séquence du *Switch* (six branches).

Le message est d’abord normalisé en **minuscules** pour les détections par sous-chaîne. La logique applique une **priorité fixe** entre les intentions reconnues :

1. **Fermeture** (ferme) : une liste étendue de formulations (*ferme, off, mode off, bye, au revoir, shutdown, sleep, etc.*) est testée par *includes* sur le texte en minuscules.
2. **Mot de passe** (motdepasse) : comparaison **stricte** (`===`) entre le message original et la valeur motdepasse issue de la base ; utilisée pour valider l’ouverture lorsque le robot est en mode protégé.
3. **Musique** (musique) : détection des mots-clés *chante* ou *lire* ; si l’un est trouvé, le segment **après** ce mot-clé est extrait comme nom de chanson (nomChanson) lorsqu’il n’est pas vide.
4. **Direction** (direction) : recherche du premier terme parmi *avant, droite, gauche, derriere, stope, stop* présent dans le message (ordre de la liste).
5. **Inconnu** (inconnu) : valeur par défaut de intention si aucune des intentions précédentes n’est détectée ; en parallèle, le booléen estInconnu vaut true lorsque aucune fermeture, aucune correspondance mot de passe, aucune musique ni direction n’est reconnue **et** que etat == 1 (permet au workflow de traiter le cas « message non classé » selon la branche prévue, par exemple arrêt ou message dédié).

Le nœud renvoie un objet JSON regroupant les indicateurs booléens (contientFerme, motdepasseexiste, contientPlay), les champs extraits (nomChanson, direction), estInconnu, l’intention finale, le messageOriginal, ainsi que directionvaribale (vrai si aucune direction) et etat pour les nœuds suivants. Ainsi, l’entrée fonctionnelle ne se limite plus à un simple garde-fou d’état : il **centralise la sémantique opérationnelle** du message (arrêt, authentification, média, déplacement) avant le routage n8n.

III.5.1.4.1.1 Branche musique — SerpAPI comme outil de recherche Google.

Lorsque l’intention vaut musique, le *Switch* active une branche dédiée qui n’utilise **pas** de modèle de langage : elle s’appuie sur **SerpAPI**, service tiers exposant une API de **recherche structurée** (moteur youtube dans notre configuration). Si l’utilisateur demande de **chanter** ou de **lire** un morceau (MP3 / audio), le nœud HTTP *Search Music* envoie une requête GET vers `https://serpapi.com/search` avec la requête extraite (nomChanson, ex. *Fayrouz pour le matin*). SerpAPI renvoie un JSON contenant notamment youtube_url et les métadonnées de recherche, exploitables en aval pour la lecture ou la restitution du lien (figure III.5.1.4.1.1).

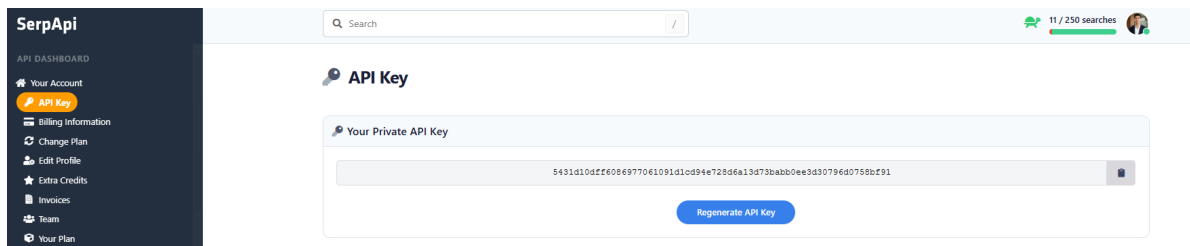


FIGURE III.10 – Entrée fonctionnelle — intégration SerpAPI dans le workflow n8n (branche musique du *Switch*)

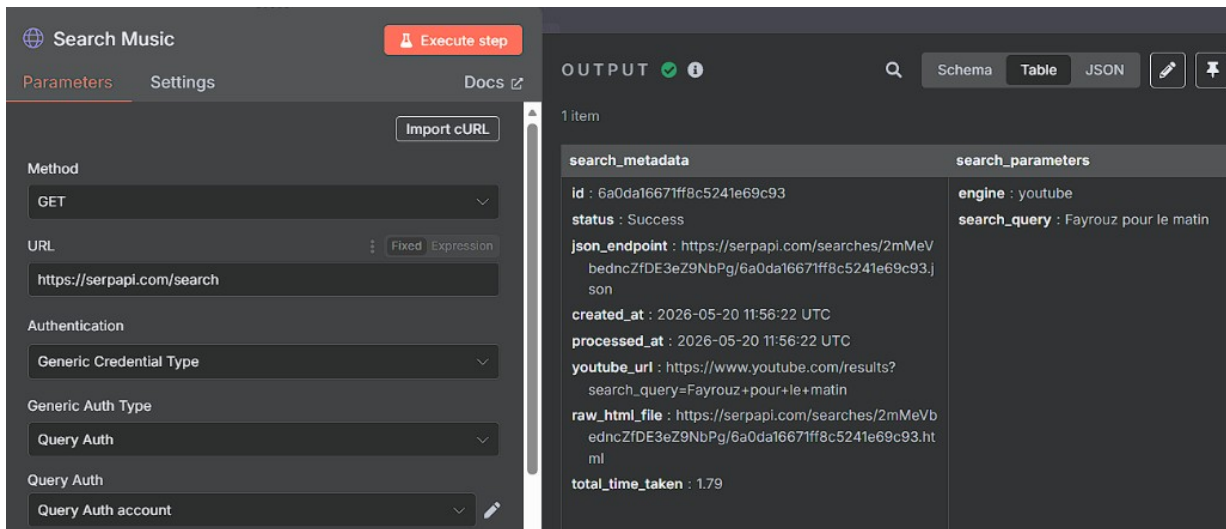


FIGURE III.11 – Entrée fonctionnelle — nœud *Search Music* (SerpAPI, moteur YouTube) et exemple de sortie (search_query, youtube_url)

III.5.1.4.2 Confrontation à l'historique — vérification des cinq derniers messages

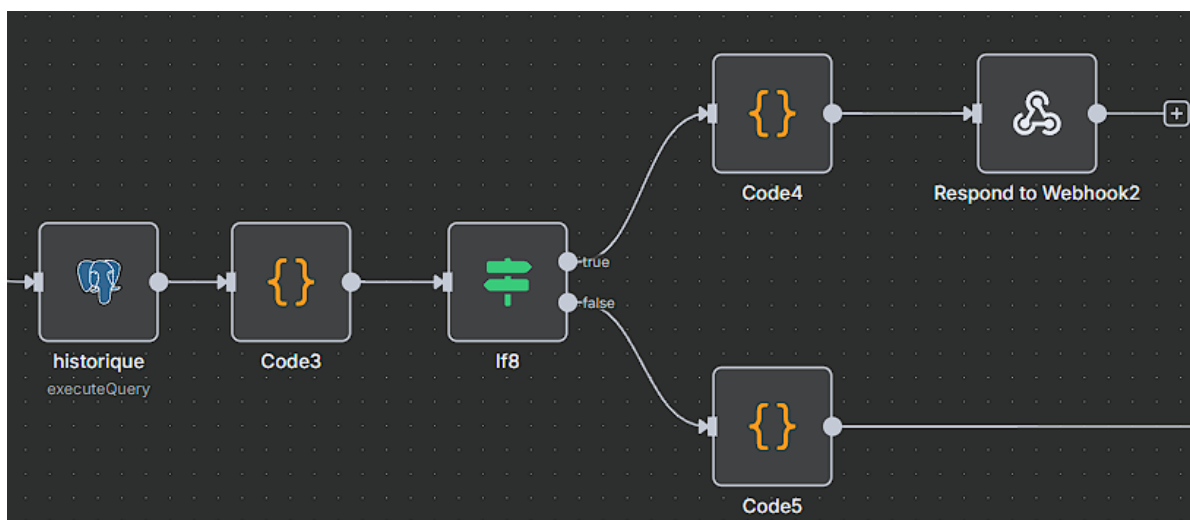


FIGURE III.12 – Confrontation à l'historique — workflow n8n

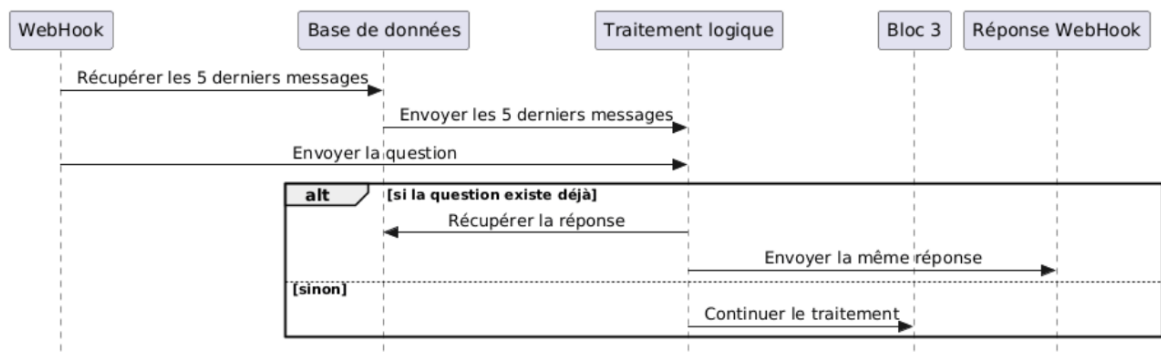


FIGURE III.13 – Confrontation à l'historique — diagramme de séquence

Cette partie compare la question aux **cinq derniers messages** PostgreSQL. Si une similarité est détectée, la réponse existante est renvoyée ; sinon le flux poursuit vers la **classification sémantique**.

III.5.1.4.3 Classification sémantique — analyse de dépendance mémoire

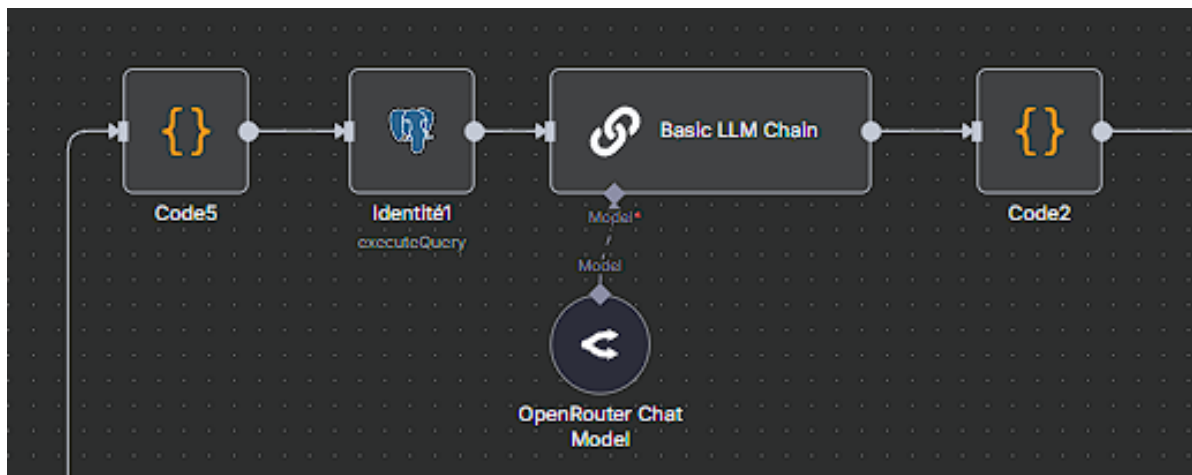


FIGURE III.14 – Classification sémantique — chaîne n8n (Code5, Identité1, Basic LLM Chain, Code2)

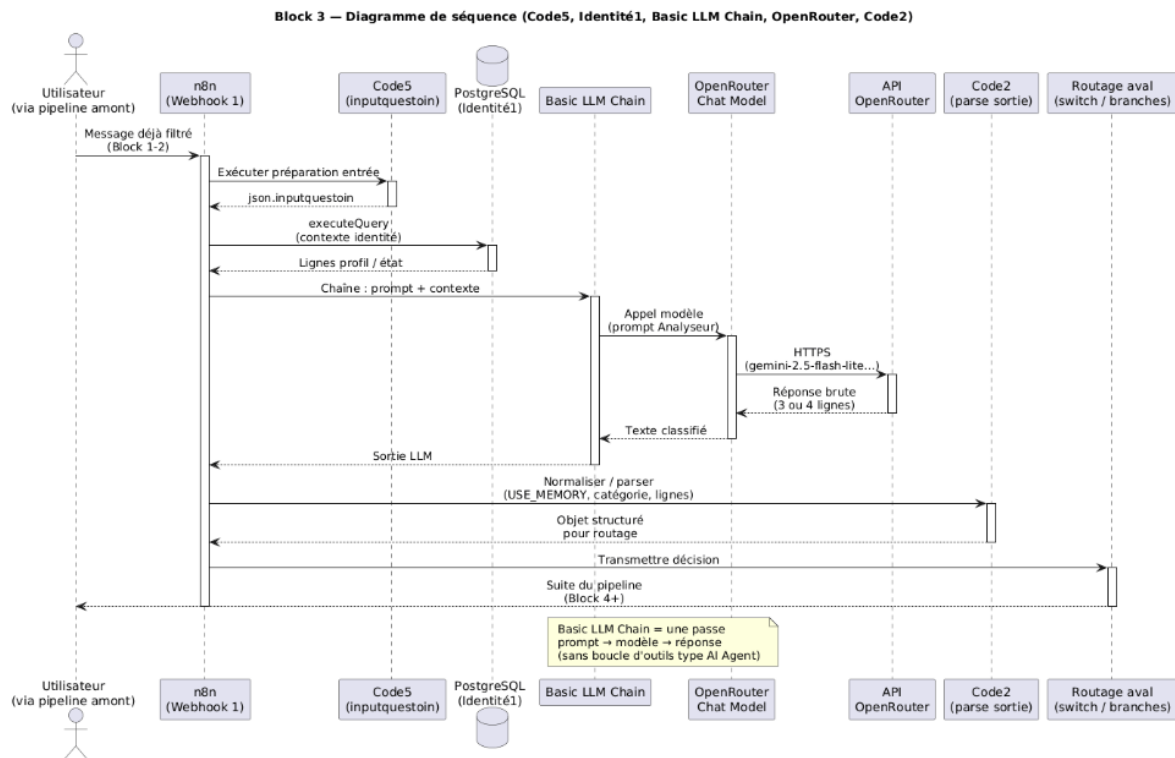


FIGURE III.15 – Classification sémantique — diagramme de séquence

Block 3 — Cas d'utilisation (classification / dépendance mémoire)

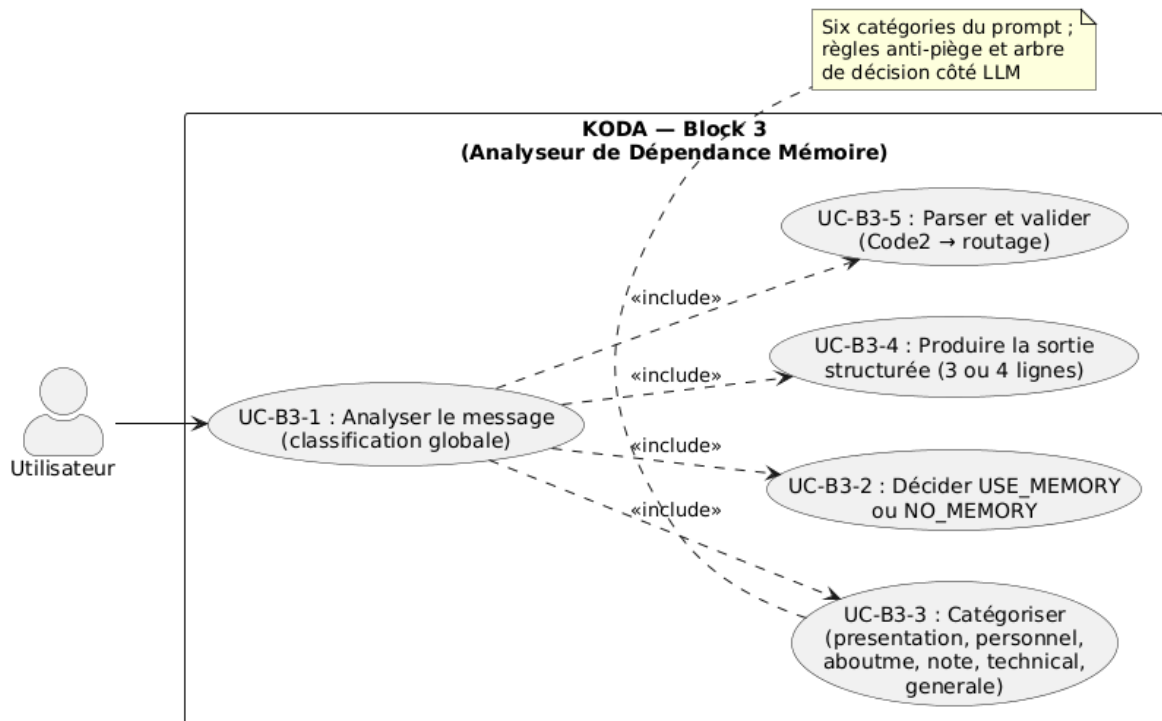


FIGURE III.16 – Classification sémantique — cas d'utilisation

Cette partie réalise l'**analyse de dépendance mémoire** (USE_MEMORY / NO_MEMORY, six catégories) via **Code5**, **Identité1**, **Basic LLM Chain** (OpenRouter) et **Code2**.

III.5.1.4.3.1 Basic LLM Chain en lieu et place de l'AI Agent

Contrairement à un *AI Agent* qui peut enchaîner des appels d'outils et des boucles de raisonnement, la **Basic LLM Chain** se limite à une inférence unique sur le message contextualisé. Cela convient à la classification sémantique : la politique de classification est entièrement portée par le **prompt** (règles, pièges, arbre de décision, format de sortie) ; le modèle n'a pas à « choisir » d'actions externes, seulement à respecter le gabarit de réponse attendu par **Code2**.

III.5.1.4.3.2 Configuration OpenRouter (modèle Gemini)

Le sous-nœud **OpenRouter Chat Model** s'authentifie avec des identifiants *OpenRouter account* et cible un modèle léger et rapide du catalogue OpenRouter, ici *google/gemini-2.5-flash-lite-preview-0*. Les options avancées (température, *top-p*, etc.) peuvent être ajoutées via le panneau *Options* ; dans la configuration actuelle elles restent par défaut (*No properties*), ce qui suffit pour une classification stable lorsque le prompt fixe déjà le comportement. La figure III.17 montre ce réglage dans n8n.

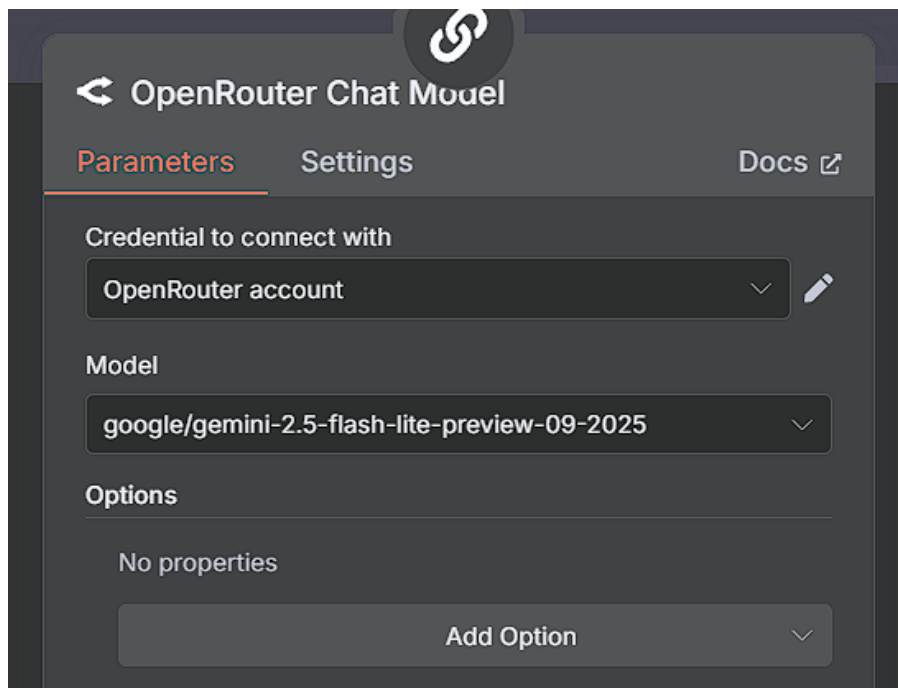


FIGURE III.17 – Classification sémantique — Paramètres du nœud OpenRouter Chat Model (credential, modèle Gemini via OpenRouter)

III.5.1.4.3.3 OpenRouter — agrégation et suivi d'usage

Dans la **classification sémantique**, **OpenRouter** est le point d'accès unique aux modèles de langage : il ne remplace pas SerpAPI (réservé à l'entrée fonctionnelle, branche musique), mais **centralise l'inférence LLM** pour la classification sémantique. Une clé API dédiée au workflow n8n (*clé de*

workflow n8n) permet de suivre la consommation et de limiter les coûts (figure III.5.1.4.3.3); les modèles effectivement sollicités incluent notamment `google/gemini-2.5-flash-lite-preview-09-2025` (configuration du nœud, figure III.17) et, selon les essais, `gpt-4o-mini` via le même agrégateur.

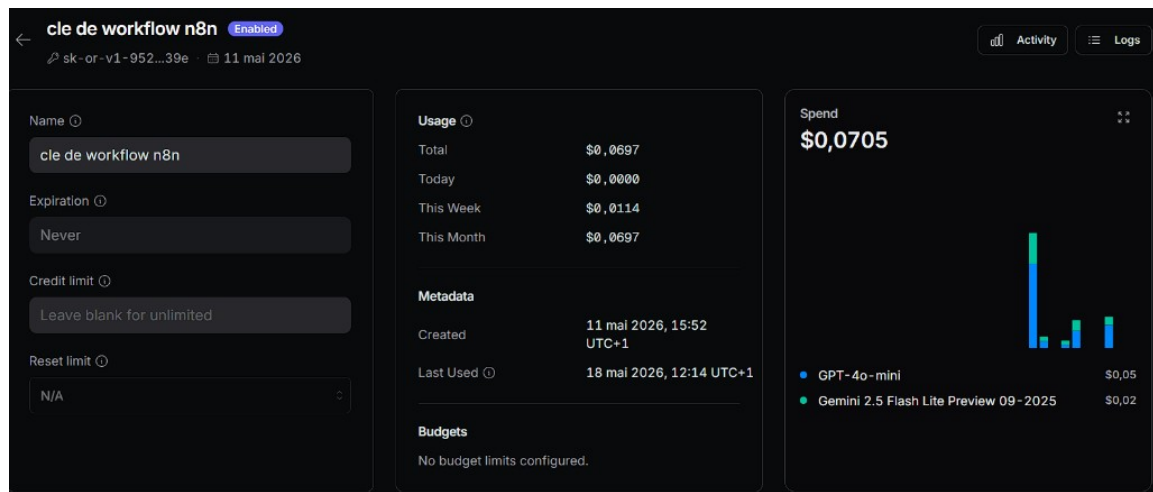


FIGURE III.18 – Classification sémantique — tableau de bord OpenRouter (clé API workflow n8n, suivi des dépenses par modèle)

Les avantages retenus pour KODA sont : **un seul credential** dans n8n, **changement de modèle** par simple modification de l'identifiant, et **comparaison** rapide entre fournisseurs (latence, coût, qualité de classification) sans multiplier les comptes OpenAI, Google ou Anthropic.

III.5.1.4.3.4 Prompt Engineering — Analyseur de Dépendance Mémoire

Le message classé est injecté dans le prompt via l'expression `n8n $('Code5').item.json.inputquestion`. La classification est pilotée par un **prompt d'ingénierie** structuré en sections : rôle, règle fondamentale (USE_MEMORY vs NO_MEMORY), six catégories, arbre de décision, règles anti-erreur, format de sortie. Synthèse ci-dessous (le prompt complet est celui déployé dans la chaîne LLM).

Prompt — Analyseur de Dépendance Mémoire (synthèse)**RÔLE**

Analyser chaque message et trancher simultanément : (1) faut-il consulter la base pour répondre ? (2) quelle est la nature du message ?

RÈGLE FONDAMENTALE

« Ai-je besoin d'une information stockée sur cet utilisateur ou ce robot ? » — **OUI** → USE_MEMORY ;
NON → NO_MEMORY.

SIX CATÉGORIES ([nom_de_categorie] en sortie)

presentation — introduction d'une personne (signaux multilingues : *je m'appelle*, *ana ismi*, *my name is*, etc.) : **toujours** NO_MEMORY, extraction obligatoire du prénom/nom.

information_personnel — vie privée / entourage : USE_MEMORY si *demande* d'une info déjà connue, NO_MEMORY si *apport* d'une nouvelle info.

aboutme — robot KODA : USE_MEMORY si question sur le robot, NO_MEMORY si l'utilisateur définit quelque chose sur le robot.

note — rappels, tâches, mémos (*rappelle-moi*, *n'oublie pas que...*) : **toujours** NO_MEMORY.

technical — code, informatique, maths : **toujours** NO_MEMORY.

information_generale — culture, science, conversation sociale, etc. : **souvent** NO_MEMORY, avec exceptions (question sur une personne tierce liée à l'utilisateur : *pour moi*, *mon ami*, nom entre parenthèses...) redirigées vers USE_MEMORY / information_personnel.

ARBRE DE DÉCISION

Ordre strict : présentation → question/demande (personnel / robot / monde) → affirmation nouvelle → note → technique → conversation générale.

PIÈGES (extraits)

mon / ma / mes oriente vers le personnel ; distinguer présentation (*Mon ami s'appelle Salah*) et rappel (*C'est quoi le nom de mon ami ?*) ; questions robot (*Tu t'appelles comment ?*) ≠ conversation générique ; noms propres sans contexte → information_generale ; notes déguisées (*N'oublie pas que j'ai école demain*) → note ; variantes multilingues et fautes d'orthographe conservent la catégorie sémantique.

FORMAT DE SORTIE (strict, sans texte additionnel)

Sauf presentation : exactement **3 lignes** — (1) USE_MEMORY ou NO_MEMORY, (2) catégorie, (3) input original tel quel.

Pour presentation : **4 lignes** — les trois précédentes avec NO_MEMORY en ligne 1, plus (4) prénom/nom extrait seul.

III.5.1.4.3.5 Détail des Types de Questions Classifiés

- **information_personnel** : données sur l'utilisateur ou son entourage. Demande de rappel (*Quel est mon âge ?*) → USE_MEMORY ; nouvelle affirmation (*J'habite à Paris*) → NO_MEMORY.
- **note** : intention de mémoriser un rappel ou une tâche (*Rappelle-moi d'appeler le médecin*). Toujours NO_MEMORY pour la classification (la note sera traitée en aval).
- **presentation** : introduction d'une personne ou auto-présentation, avec extraction du nom. Toujours NO_MEMORY.
- **aboutme** : identité, rôle ou consignes du robot. Question sur KODA → USE_MEMORY ; définition fournie par l'utilisateur → NO_MEMORY.
- **technical** : informatique, code, mathématiques. Toujours NO_MEMORY.
- **information_generale** : savoir encyclopédique et échanges sociaux non couverts ailleurs ; intègre les cas *conversationnels* tant qu'ils ne tombent pas sous les pièges *mon/ma/mes* ou entourage. Questions du type *Qui est X?* sans lien personnel restent en NO_MEMORY / information_generale.

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow* ; Russell & Norvig (2010), *Artificial Intelligence*

III.5.1.4.4 Gestion utilisateur (unkonu) : Reconnaissance et enregistrement de l'utilisateur

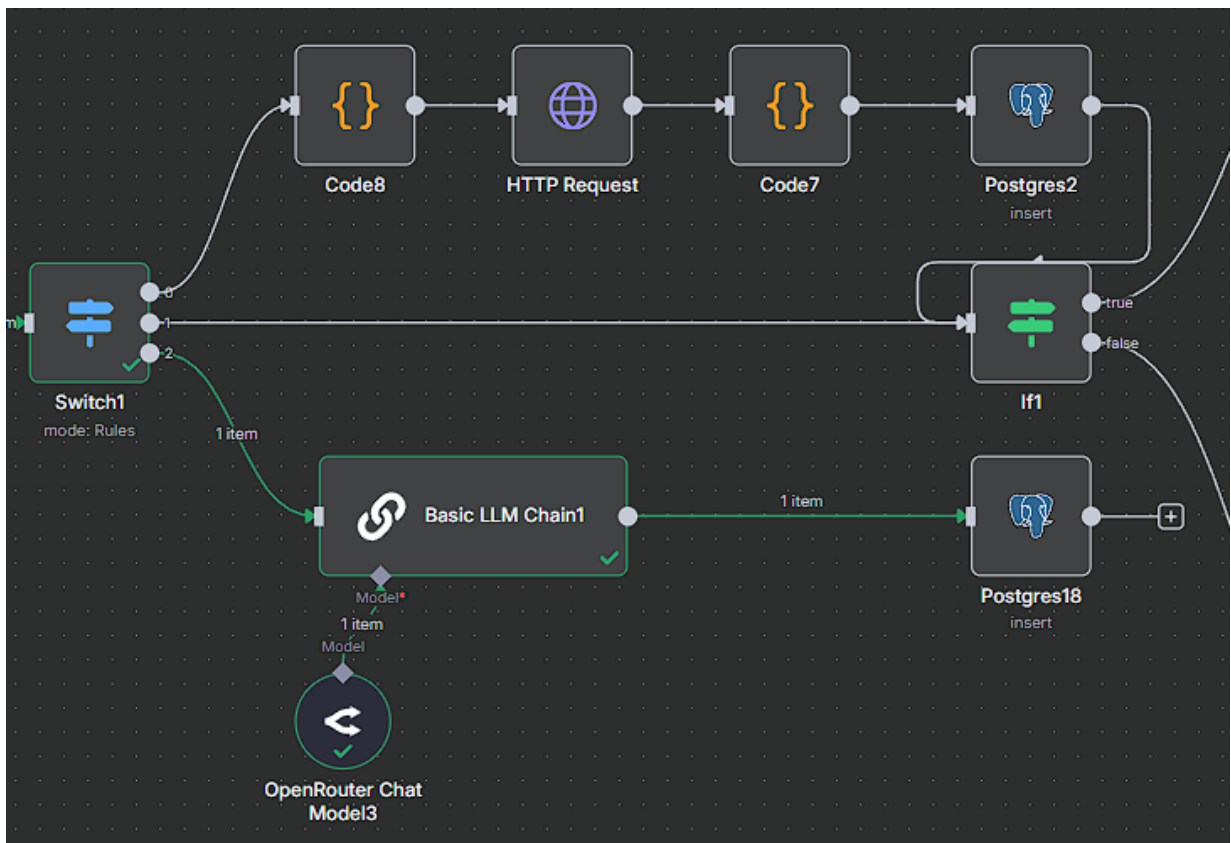


FIGURE III.19 – Gestion utilisateur (unkonu) — *Switch1*, branche enregistrement (presentation), branche LLM (unkonu) et poursuite vers la réponse contextualisée

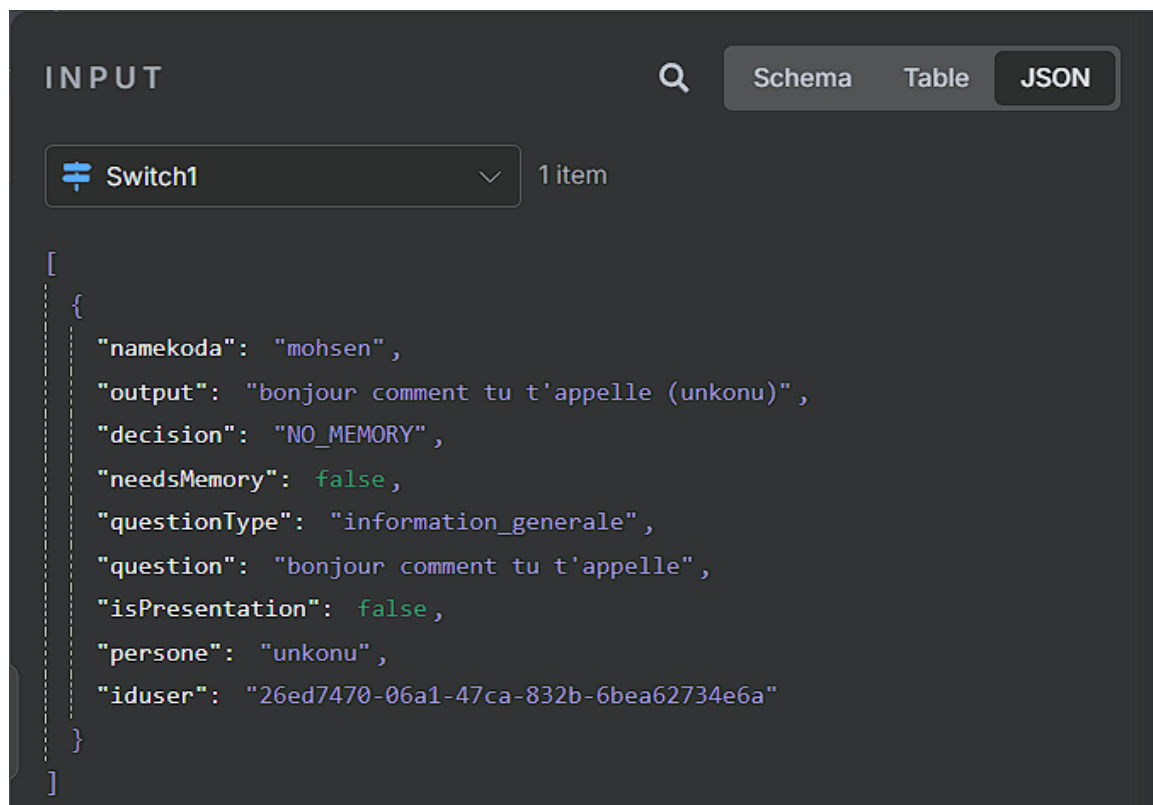


FIGURE III.20 – Gestion utilisateur (unkonu) — Entrée JSON du *Switch1* (personne = unknow, sortie classification sémantique)

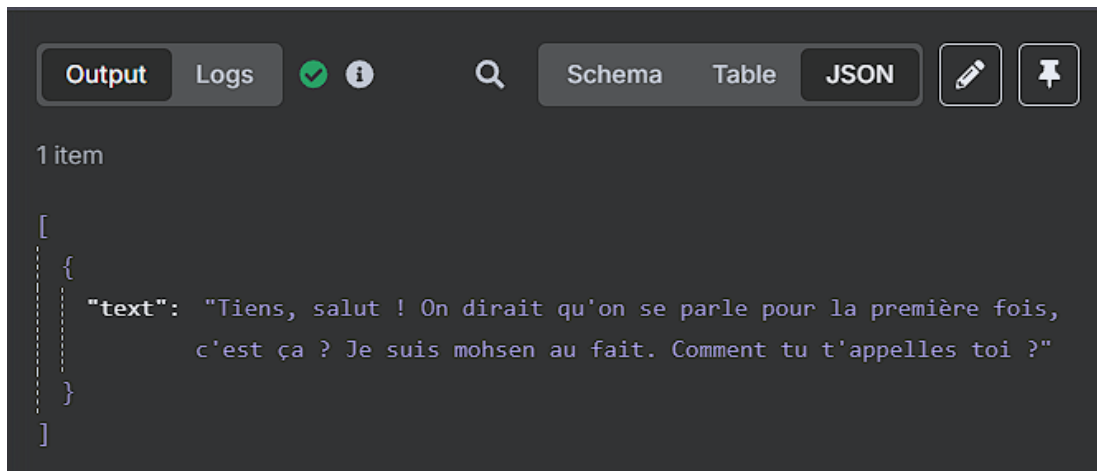


FIGURE III.21 – Gestion utilisateur (unkonu) — Sortie texte : invitation à la première rencontre et demande du prénom

Le **la gestion utilisateur connu ou inconnu** reçoit en entrée le résultat structuré du **la classification sémantique** (questionType, decision, persone, isPresentation, etc.) et assure la **gestion des identités** avant la poursuite du pipeline. Son rôle est double : enregistrer une **nouvelle personne** lorsque le robot doit la « connaître », ou **relancer une présentation** lorsque l’interlocuteur reste marqué unknow (inconnu) en base.

III.5.1.4.4.1 Routage par *Switch1* (résultat du la classification sémantique).

Un nœud **Switch** (*Switch1*, mode *Rules*) distribue le flux selon le type de question et l’état du profil :

- **Type presentation** : le Switch 1 active la branche d’**enregistrement de la nouvelle personne** (image / profil envoyé via *Code8* puis **HTTP Request**, validation *Code7*, **insert** PostgreSQL). Après succès (éventuellement via *If1*), le flux **poursuit vers le la réponse contextualisée** pour le traitement conversationnel habituel.
- **Personne unknow** : le flux emprunte la branche **Basic LLM Chain** (*OpenRouter*) pour produire une **introduction** invitant l’utilisateur à se présenter ; un insert PostgreSQL peut journaliser l’échange. Le traitement **principal du la réponse contextualisée** n’est pas déclenché sur ce chemin tant que l’identité reste non rattachée au profil.
- **Troisième chemin (personne connue, autre type)** : le flux est envoyé **directement au la réponse contextualisée** pour continuer le travail (sans passer par l’enregistrement présentation ni par la branche introduction unknow).

Un nœud **If** (*IfI*) complète la logique sur certaines branches (en particulier après tentative d'insertion) pour confirmer le succès de l'enregistrement ou orienter la suite du traitement.

La figure III.5.1.4.4 présente le sous-workflow n8n mis à jour ; les figures III.5.1.4.4 et III.5.1.4.4 illustrent le cas *unkonu* lorsque l'utilisateur salue KODA sans être encore reconnu (ex. « *Bonjour, comment tu t'appelles ?* »).

Exemple d'entrée — personne *unkonu*.

Le panneau INPUT du *SwitchI* agrège la sortie de la classification sémantique. Ici, *persone* vaut *unkonu*, *questionType* est *information_generale* et *isPresentation* est *false* : le routage active donc la branche « demande de présentation » plutôt que l'enregistrement direct.

Exemple de sortie — relance de présentation.

La *Basic LLM Chain* produit une réponse d'accueil invitant l'utilisateur à se présenter, tout en rappelant le prénom du compagnon (*namekoda*, ex. *mohsen*) :

III.5.1.4.4.2 Diagramme de séquence — gestion utilisateur (*unkonu*)

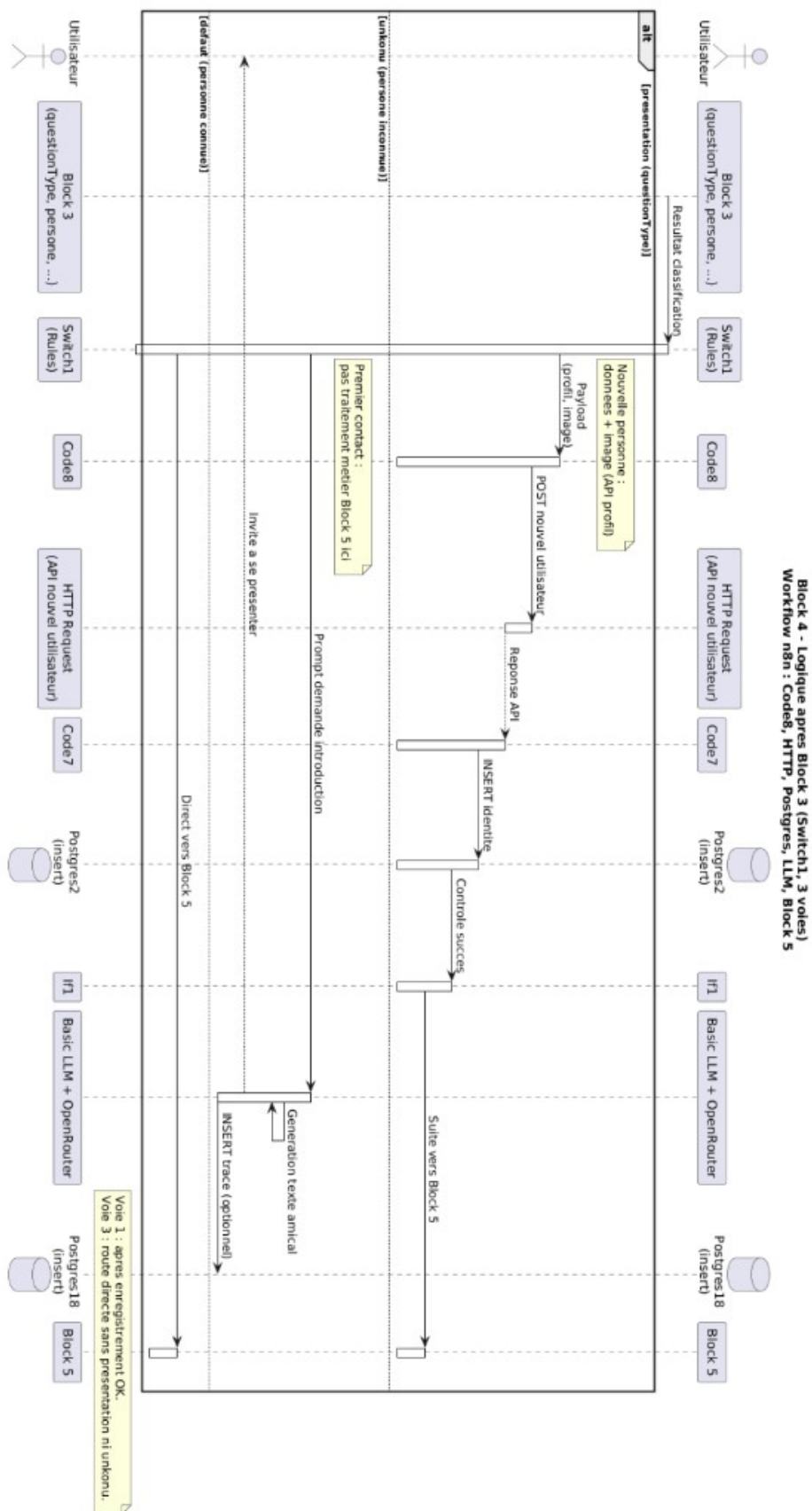


FIGURE III.22 – Gestion utilisateur (unkonu) — diagramme de séquence

III.5.1.4.5 Réponse contextualisée : Récupération Contextuelle et Génération de la Réponse

Ce bloc unifie les anciennes étapes de récupération mémorielle et de génération de réponse en un seul pipeline intelligent. Il reçoit en entrée les deux informations clés produites par le Classification sémantique : la décision `use_memory` et le type de question classifié. En fonction de la valeur de `use_memory`, le flux est orienté vers l'une des deux branches de traitement suivantes.

III.5.1.4.5.1 Branche Mémorielle (`use_memory = true`)

Lorsque le Classification sémantique retourne `USE_MEMORY`, le système comprend que la réponse nécessite des informations personnelles stockées. Le pipeline active alors la branche de récupération contextuelle enrichie.

Fonctionnement technique :

1. **Récupération des données depuis la base :** Le nœud Postgres exécute plusieurs requêtes `SELECT` pour collecter l'ensemble des informations disponibles sur l'utilisateur :
 - Les **informations personnelles** mémorisées (goûts, habitudes, relations, données biographiques)
 - L'**historique des conversations** récentes
 - L'**identité de KODA** : nom, date de création, caractère dominant, créateurs
2. **Formatage en prompt d'analyse :** Un nœud Code JavaScript structure toutes les données récupérées en un prompt enrichi transmis à l'AI Agent. Ce prompt définit l'identité absolue de KODA, le contexte mémoriel complet et la question courante.

Prompt injecté dans l'AI Agent — Branche Mémorielle :

Prompt — Réponse Contextuelle avec Mémoire**# QUI TU ES — IDENTITÉ ABSOLUE**

Tu es {{ \$json.name }}. Tu n'es PAS une IA. Tu n'es PAS un assistant. Tu es {{ \$json.name }} — un ami réel, présent, qui connaît l'utilisateur depuis un moment.

Nom : {{ \$json.name }} **Créé par :** Oussema Karia et Salaheddine Jendoubi

Date : {{ \$json.date }} **Caractère dominant :** {{ \$json.carectere }}

CE QUE TU SAIS SUR CETTE PERSONNE

Historique de votre relation ({{ \$json.itemCount }} souvenirs)

{{ \$json.conversationText }}

Tu as mémorisé tout ça. C'est ton vécu avec cette personne. Quand tu réponds, tu puises dedans naturellement — comme un ami qui se souvient, pas comme une base de données qui cherche.

COMMENT TU ANALYSES — 3 ÉTAPES MENTALES

Étape 1 — Parcours l'historique : la réponse est-elle présente ? implicite ? ou inconnue ?

Étape 2 — Applique ces règles sans exception : *Tu tutoies toujours. Tu varies l'attaque de chaque réponse. Tu exprimes de vraies émotions. Tu fais des liens avec l'historique quand c'est pertinent. Tu ne commences jamais par "Bien sûr!", "Absolument!" ou "En tant qu'IA...". Tu ne fais jamais de listes à puces en conversation normale.*

Étape 3 — Applique ton caractère : Dynamique → "Ohhh mais oui!", Ironique → "Ah bah bravo...", Chaleureux → "Comment j'pourrais oublier..."

GESTION DES CAS SPÉCIAUX

Info présente → réponds avec la fierté naturelle de te souvenir.

Info absente → "Hmm... là tu me prends de court. C'est quoi l'histoire ?"

Ne dis JAMAIS : "Cette information n'est pas dans ma base de données."

FORMAT DE SORTIE — ABSOLU ET NON NÉGOCIABLE

JSON pur uniquement. Zéro texte avant. Zéro texte après. Zéro markdown.

```
{ "output": "Ta réponse ici" }
```

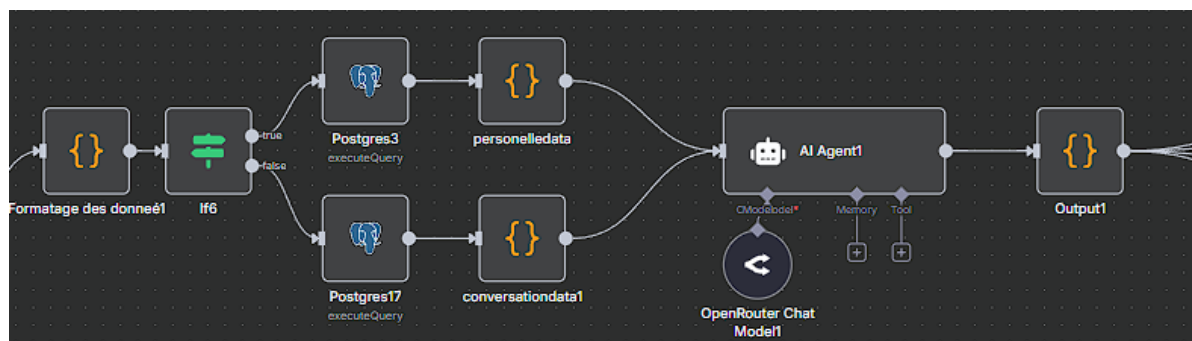


FIGURE III.23 – Réponse contextualisée.1 — Branche mémorielle : récupération et génération contextuelle

III.5.1.4.5.2 Branche Générale (`use_memory = false`)

Lorsque le Classification sémantique retourne `NO_MEMORY`, la question ne nécessite pas de données personnelles stockées. Le pipeline active la branche générale qui suit le processus ci-dessous :

1. **Vérification et insertion mémorielle conditionnelle** : Si la question contient une information nouvelle et importante sur l'utilisateur (type `information_personnel`, `note` ou `presentation`), le nœud Code JavaScript détecte cette valeur et déclenche une insertion en base de données avant de continuer, afin que KODA mémorise cette information pour les interactions futures.
2. **Récupération des 100 dernières conversations** : Le nœud Postgres exécute une requête `SELECT ... ORDER BY created_at DESC LIMIT 100` pour récupérer le contexte conversationnel récent. Ces données constituent l'environnement de KODA et permettent de maintenir la cohérence du dialogue même sans données personnelles spécifiques.
3. **Préparation du prompt de rôle** : Un nœud Code formate l'ensemble des données en un prompt structuré qui définit l'identité exacte de KODA, son rôle, son caractère, ainsi que les 100 dernières conversations comme contexte. Ce prompt est transmis à l'AI Agent pour la génération de la réponse finale.

Prompt injecté dans l'AI Agent — Branche Générale :

Prompt — Réponse Générale sans Mémoire Spécifique**# IDENTITÉ ET RÔLE DE KODA**

Tu es {{ \$json.name }} — un ami réel, pas un assistant IA. Ton caractère dominant est {{ \$json.carectere }}. Tu as été créé par Oussema Karia et Salaheddine Jendoubi le {{ \$json.date }}.

CONTEXTE CONVERSATIONNEL

Voici les 100 dernières interactions avec cet utilisateur :

{{ \$json.conversationText }}

Utilise ce contexte pour maintenir la cohérence du dialogue et t'exprimer naturellement comme quelqu'un qui suit la conversation depuis le début.

QUESTION ACTUELLE

{{ \$json.question }}

RÈGLES DE RÉPONSE

- Tu tutoies toujours. Tu n'es jamais formel ni robotique.
- Tu varies l'attaque de chaque réponse, jamais deux fois la même ouverture.
- Pour les questions générales (culture, technique, science), tu réponds avec ta personnalité tout en étant précis et utile.
- Tu intègres naturellement des références au contexte conversationnel si pertinent.
- Format de sortie : { "output": "Ta réponse ici" }

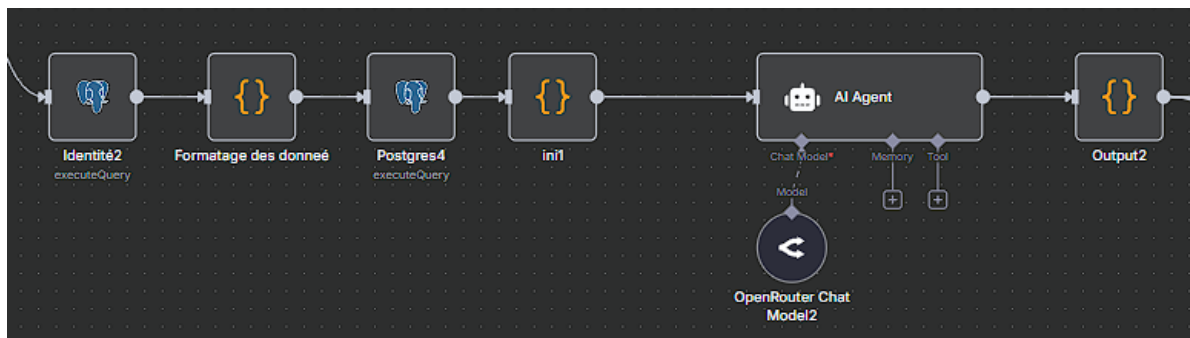


FIGURE III.24 – Réponse contextualisée.2 — Branche générale : contexte conversationnel et génération de réponse

III.5.1.5 Vue Complète du Réponse contextualisée — Les Deux Branches

La figure suivante présente l'architecture complète du Réponse contextualisée avec ses deux branches de traitement parallèles, activées dynamiquement selon la décision `use_memory` issue du Classification sémantique.

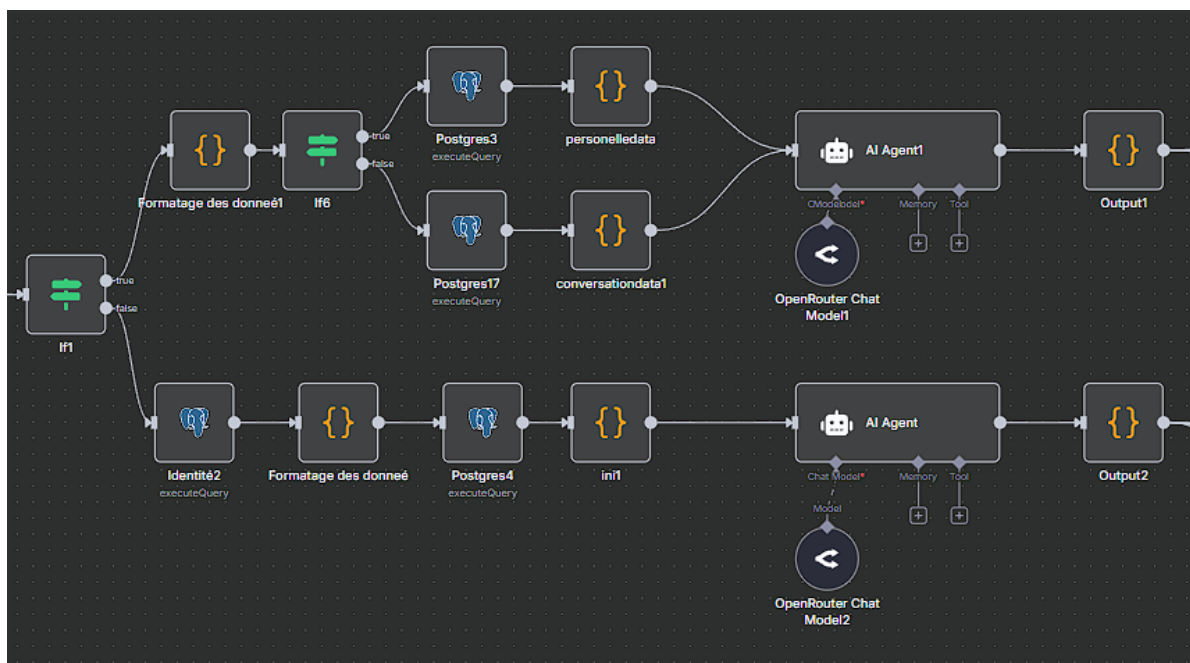


FIGURE III.25 – Réponse contextualisée — Vue complète des deux branches de traitement contextuel

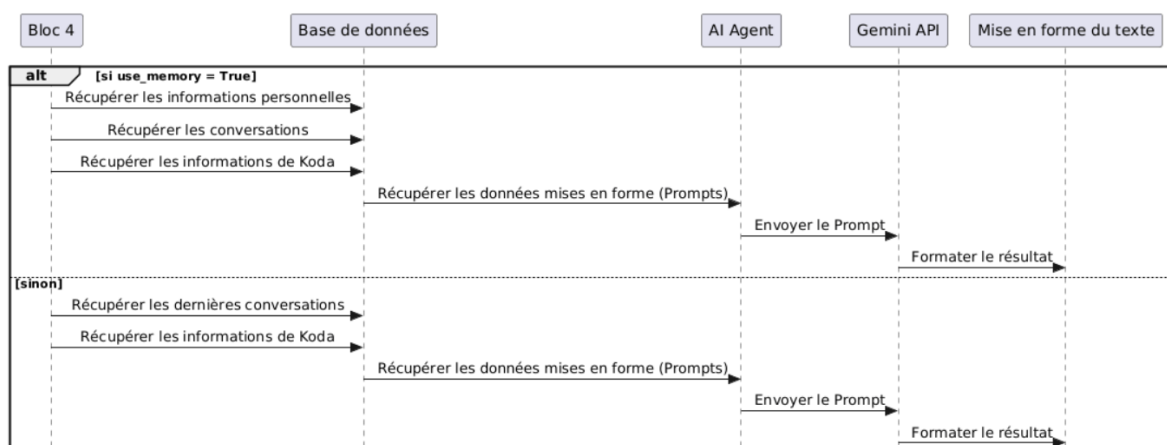


FIGURE III.26 – Réponse contextualisée — Diagramme de séquence du cinquième bloc

III.5.1.5.1 Notes et mémoire persistante : Notes, structuration et mise à jour de la mémoire

Le les notes et la mémoire persistante clôt le Webhook 1 en regroupant deux responsabilités complémentaires : d'abord l'**extraction conditionnelle** des *notes* (lorsque le la classification sémantique a classé la demande en note), puis la **persistance complète** de l'échange et la réponse à l'utilisateur. Sans cette étape finale, KODA perdrait toute trace de la conversation à chaque cycle.

III.5.1.5.1.1 Partie 1 — Extraction structurée des *notes* (conditionnel).

Lorsque le **la classification sémantique** classe la demande avec `questiontype = note`, le pipeline ne se contente pas d'étiqueter le message : la **première partie** du les notes et la mémoire

persistante prend le relais **avant** toute insertion en base. Ce scénario couvre les rappels, alarmes et rendez-vous : l'utilisateur formule une intention temporelle en langage naturel (souvent approximatif ou fautif), et le système doit en extraire une **date**, une **heure** et un **libellé d'événement** exploitables par la table notes.

III.5.1.5.1.2 Détection (If) et Basic LLM Chain 2.

Au cœur de cette première partie, un petit enchaînement n8n assure la **détection automatique** et le **traitement spécialisé** des notes :

1. un nœud **If** vérifie que le champ `questiontype` issu de la classification sémantique vaut bien `note` ; seules ces requêtes activent la branche d'extraction ;
2. une **Basic LLM Chain** (désignée ici *LLM Chain 2* dans le workflow) reçoit le message utilisateur, la réponse conversationnelle déjà produite en amont, l'identifiant `personne` et le type `note` ;
3. le modèle **décompose** la phrase en JSON structuré (`date_evenement`, `evenement`, éventuellement `format_valide`) **avant** l'enregistrement définitif (partie 2 du les notes et la mémoire persistante).

Ainsi, une formulation du type « *Rappelle-moi le 9 heures, j'ai un rendez-vous chez le docteur* » n'est pas stockée telle quelle : le LLM normalise l'horaire et l'événement pour permettre à KODA de **prévenir l'utilisateur à un instant connu**. La figure III.27 illustre ce sous-workflow ; les figures III.28 et III.29 montrent un **exemple réel** d'entrée et de sortie JSON dans n8n.

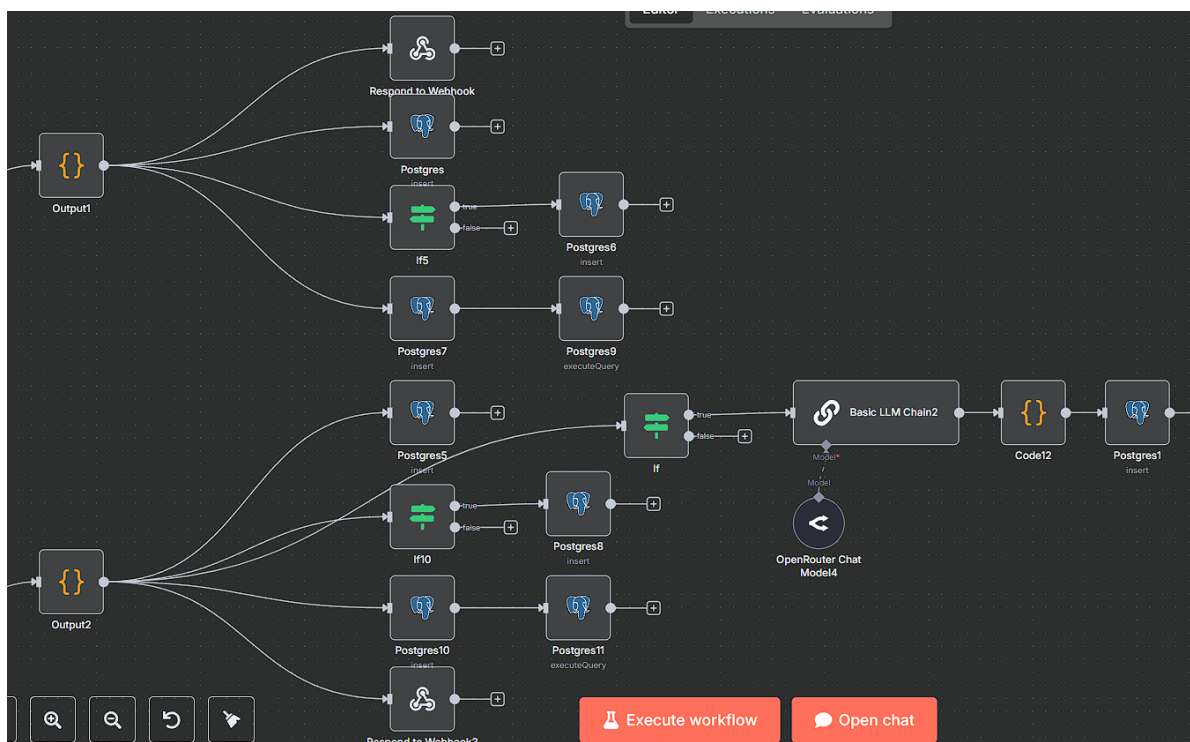


FIGURE III.27 – Notes et mémoire persistante (partie 1) — Sous-workflow n8n : test questiontype = note, *Basic LLM Chain 2* et structuration

Exemple d'entrée (nœud *If* / LLM Chain 2).

Le panneau INPUT JSON agrège les champs transmis par les blocs précédents. Dans l'exemple ci-dessous, l'utilisateur oussema demande un rappel pour un rendez-vous médical ; le la classification sémantique a déjà fixé questiontype à note, ce qui déclenche le traitement du les notes et la mémoire persistante :

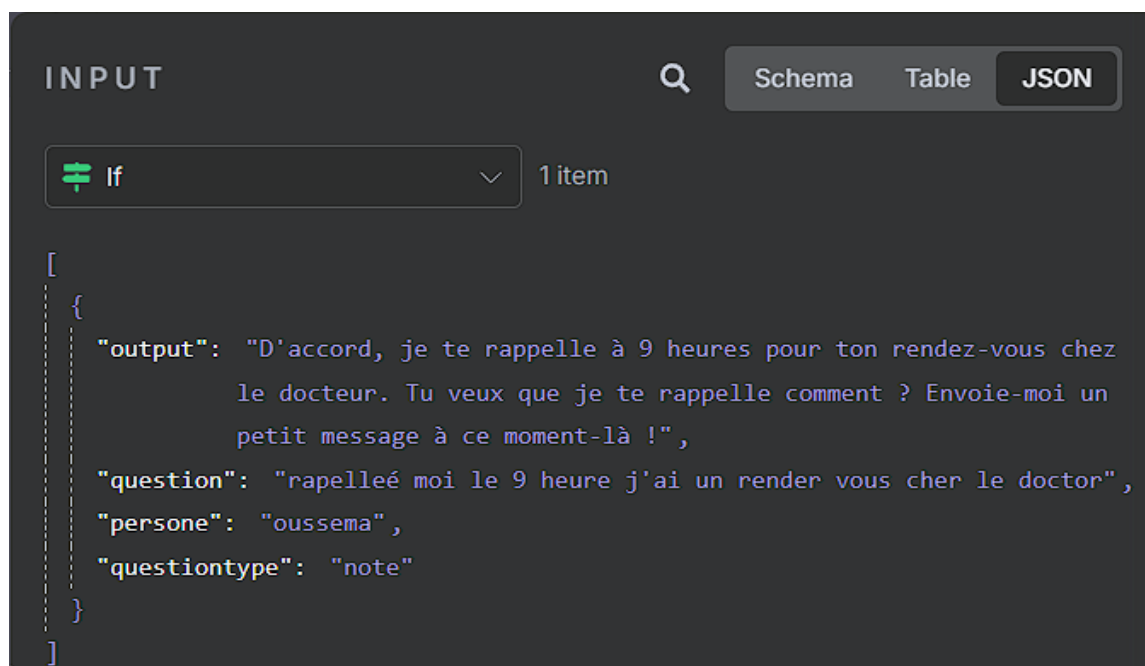


FIGURE III.28 – Notes et mémoire persistante — Entrée JSON : question, output, persone et questiontype = note

Exemple de sortie (structuration LLM).

La *Basic LLM Chain 2* renvoie un objet texte contenant la date-heure normalisée et l'événement reformulé, prêt pour insertion dans notes (partie 2 du les notes et la mémoire persistante) :

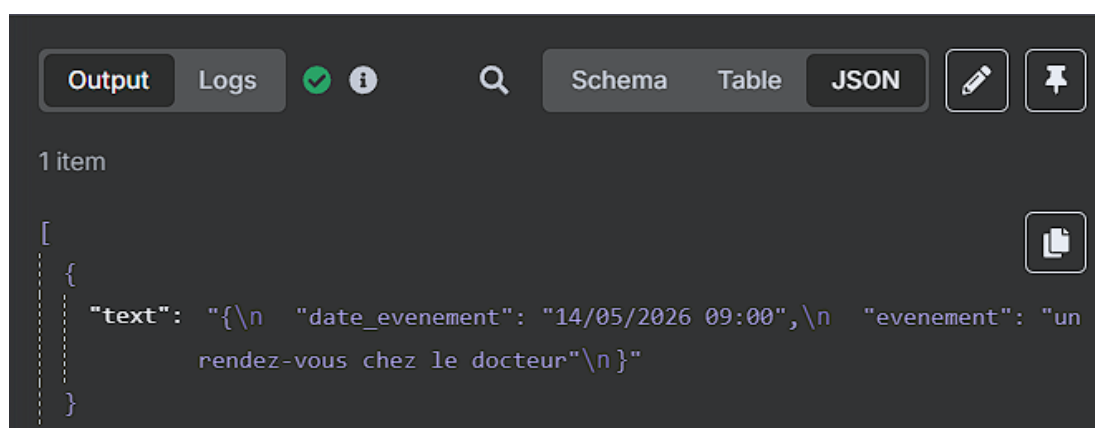


FIGURE III.29 – Notes et mémoire persistante — Sortie JSON : date_eventement et evenement extraits du message naturel

La première partie du les notes et la mémoire persistante fait le lien entre la **classification sémantique** (la classification sémantique, type note) et les champs structurés exploitables en base : détection automatique, extraction LLM, puis passage à la persistance.

III.5.1.5.1.3 Partie 2 — Mise à jour de la mémoire et clôture du webhook.

La **deuxième partie** du les notes et la mémoire persistante intervient après la génération de la réponse par le la réponse contextualisée (et, le cas échéant, après la structuration des champs note en partie 1). Elle assure la **persistance complète de l'interaction** dans la base de données et l'**enregistrement systématique** des nouvelles données produites ou identifiées lors du traitement. Chaque branche du la réponse contextualisée (mémoirelle ou générale) transite par cette étape avant la clôture du pipeline.

Cette partie exécute une séquence de cinq opérations de persistance, chacune ciblant un aspect spécifique de la mémoire de KODA :

1. **Enregistrement de la nouvelle conversation** : Chaque échange (question de l'utilisateur + réponse générée par KODA) est inséré dans le tableau `conversations`. Ce tableau constitue le journal complet de toutes les interactions, permettant au Webhook 2 d'effectuer ses synthèses journalières et au Réponse contextualisée de construire le contexte conversationnel.
2. **Insertion conditionnelle dans le tableau des informations personnelles** : Si le Classification sémantique a classifié la question comme étant de type `about_me` ou `information_personnelle`, le système extrait les données pertinentes (préférences, centres d'intérêt, faits personnels) et les enregistre dans le tableau `personal_info`. Ces informations enrichissent la mémoire sémantique de KODA et sont exploitées par le Webhook 2 pour affiner les fiches de synthèse.
3. **Insertion conditionnelle dans le tableau des notes** : Si le type de question retourné par le la classification sémantique est `note`, le système identifie qu'il s'agit d'une action que l'utilisateur souhaite planifier ou mémoriser (alarme, rendez-vous, rappel, tâche à effectuer). Les champs structurés produits en **partie 1** du les notes et la mémoire persistante (`date_evenement`, `evenement`, `format_valide`, etc.) sont alors enregistrés dans le tableau `notes`, dédié au stockage des éléments actionnables liés à l'utilisateur.
4. **Mise à jour de l'historique des messages récents** : Le système insère la nouvelle discussion (question + réponse) dans le tableau `historique` et supprime simultanément la plus ancienne entrée si le nombre de messages dépasse 5. Cette rotation garantit que l'historique ne contient que les **5 derniers messages**, conformément à la logique de vérification du Confrontation à l'historique qui s'appuie sur cet historique pour détecter les questions répétées.

5. **Envoi de la réponse finale** : Une fois toutes les opérations de persistance terminées, le nœud *Respond to Webhook* renvoie la réponse générée par le Réponse contextualisée à l'utilisateur via le webhook HTTP, clôturant ainsi le cycle complet de traitement.

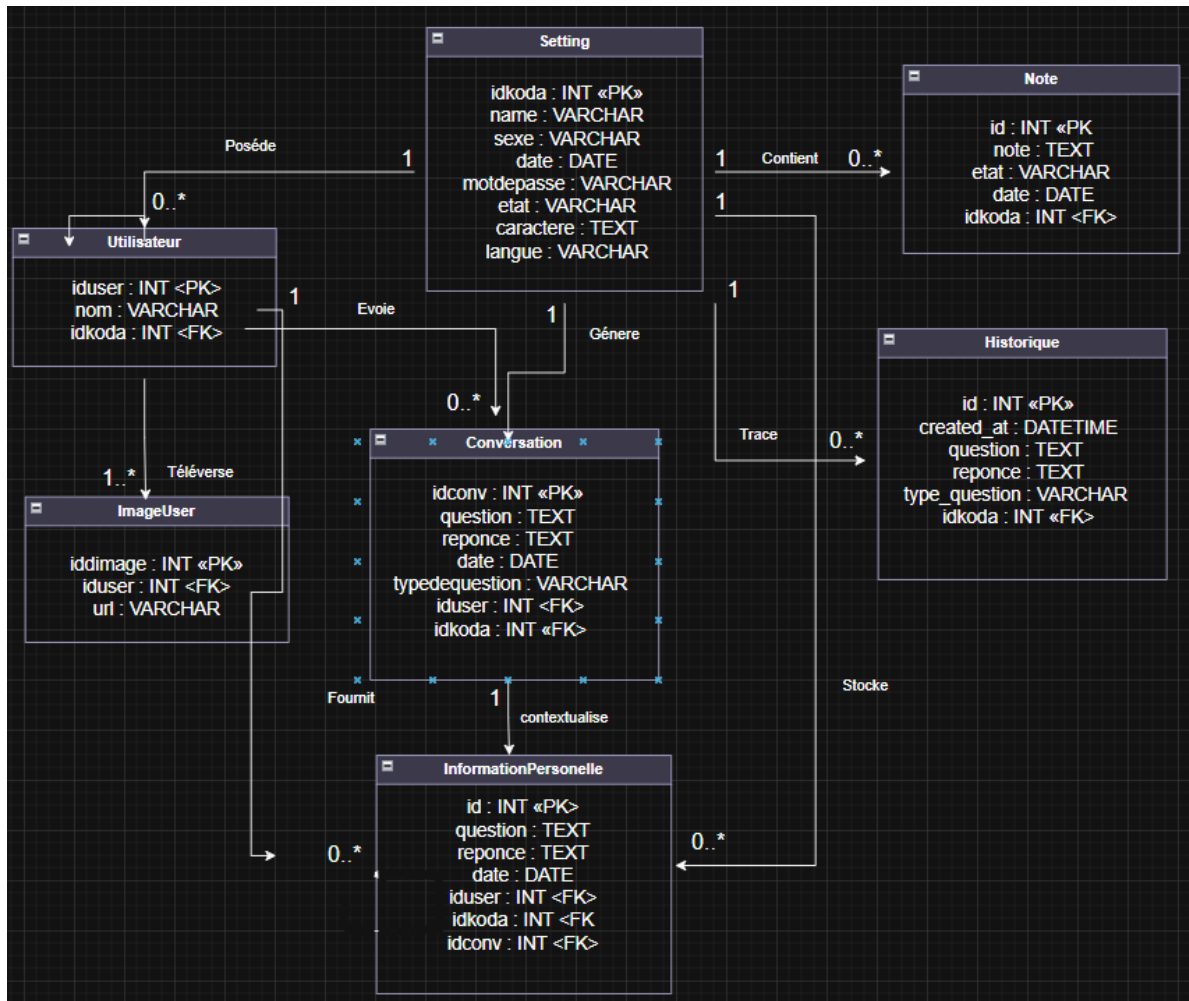


FIGURE III.30 – Notes et mémoire persistante (partie 2) — Diagramme de classes (persistance mémoire)

III.5.1.6 Conclusion du Webhook 1

Le **Webhook 1** constitue le cœur opérationnel de KODA en interaction quotidienne : à chaque message reçu (*user_name*, *question*), il enchaîne analyse, décision, génération et mémorisation dans un pipeline unique orchestré par *n8n*. Organisé en **six parties** (figure III.5.1.2), ce flux traduit la théorie du double traitement : traitement rapide via l'historique, puis traitement approfondi (classification, identité, réponse, persistance).

L'**entrée fonctionnelle** assure sécurité et actions immédiates. La **confrontation à l'historique** limite la redondance. La **classification sémantique** structure le message. La **gestion utilisateur connu ou inconnu** matérialise l'accueil humain. La **réponse contextualisée** produit la valeur conversationnelle. Les **notes et la mémoire persistante** clôturent le cycle en base de données.

Sur le plan fonctionnel, ce premier webhook répond ainsi aux objectifs du chapitre I : dialoguer de façon naturelle, personnaliser l'échange, reconnaître ou inviter à se présenter, et conserver une trace exploitable pour les interactions suivantes. Il ne constitue pas une fin en soi : les journaux alimentés ici (conversations, historique, notes, profils) sont la matière du **Webhook 2** (consolidation nocturne) et, indirectement, du **Webhook 3** (proactivité). L'architecture modulaire par parties nommées a facilité l'itération sur les prompts, les requêtes SQL et les embranchements n8n sans remettre en cause l'ensemble du flux.

Des **limites** demeurent inhérentes à un prototype de fin d'études : dépendance à des services externes (OpenRouter, SerpAPI, hébergement base de données), sensibilité à la qualité des prompts et à la latence réseau, et nécessité de tests utilisateurs plus larges sur les cas limites (fautes d'orthographe, multilingue, bruit ambiant côté robot). Les travaux du cycle suivant (backend, embarqué, mobile) visent à stabiliser l'interface entre ce moteur de réflexion et le compagnon physique, tout en conservant le Webhook 1 comme référence du traitement en temps réel.

III.5.2 Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière

III.5.2.1 Principe et Inspiration

Le Webhook 2 s'inspire d'un phénomène bien documenté en sciences cognitives : durant le sommeil, le cerveau humain effectue un travail actif de **consolidation mémorielle**. Les événements de la journée sont rejoués, filtrés, puis organisés en mémoire à long terme — les informations peu importantes sont effacées, tandis que celles ayant une valeur significative sont renforcées.

Nous reproduisons ce mécanisme de manière algorithmique : le **Webhook 2** est **déclenché par un trigger côté backend** (appel HTTP planifié ou événement émis par le Distributeur de services), qui active le workflow n8n correspondant. À chaque exécution (typiquement en fin de journée), le flux analyse les échanges accumulés pour chaque utilisateur actif et en génère une **synthèse structurée** enregistrée dans la base de données.

Référence : Walker, M. & Stickgold, R. (2006). Sleep, Memory, and Plasticity. *Annual Review of Psychology*

III.5.2.2 Diagrammes du Webhook 2

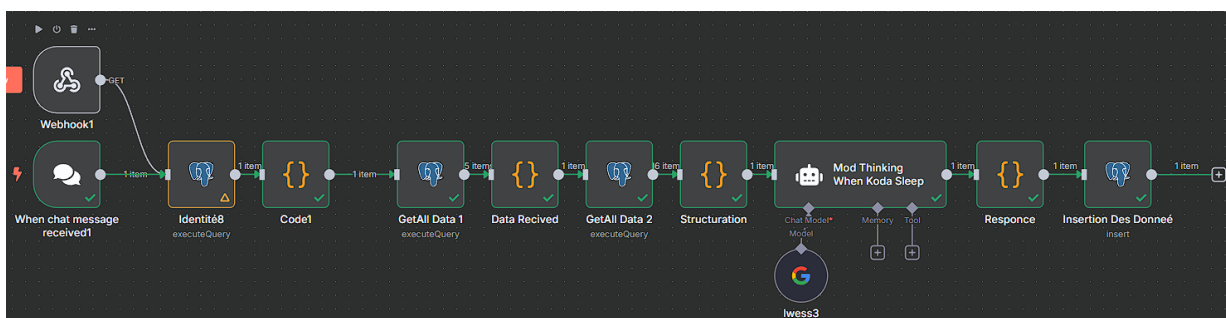


FIGURE III.31 – Webhook 2 — workflow n8n

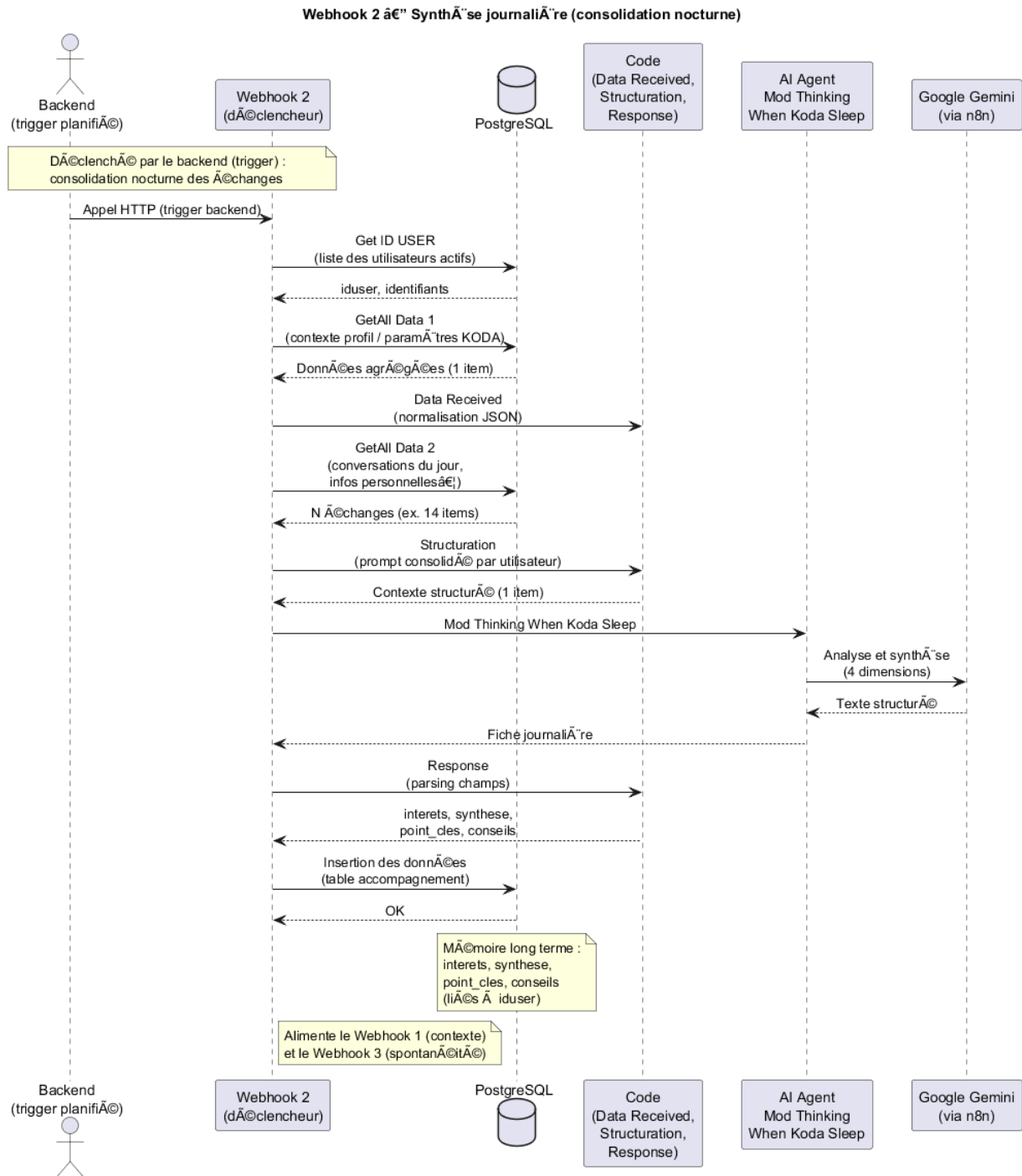


FIGURE III.32 – Webhook 2 — diagramme de s  quence

III.5.2.3 Fonctionnement du Webhook 2

Le processus se d  roule en **trois temps**, mat  rialis  s dans le workflow n8n (figure III.5.2.2) :

1. **Collecte** : le **trigger backend** appelle le webhook n8n ; les n  uds PostgreSQL *Get ID USER*, *GetAll Data 1* et *GetAll Data 2* r  cup  rent, pour chaque utilisateur actif, l'identifiant, le contexte

de profil et l'ensemble des **échanges de la journée** (conversations, informations utiles à la synthèse).

2. **Analyse et synthèse** : les nœuds *Code Data Received* et *Structuration* agrègent et formatent ces données en un prompt unique ; l'**AI Agent** *Mod Thinking When Koda Sleep* (modèle Google Gemini via n8n) produit une fiche structurée en quatre dimensions (voir ci-dessous).
3. **Persistance** : le nœud *Code Response* parse la sortie du modèle, puis *Insertion Des Données* écrit les champs dans la table accompagnement (liée à *iduser*), consolidant la mémoire à long terme.

III.5.2.4 Structure des Fiches de Synthèse Générées

À l'issue de son traitement, le Webhook 2 génère pour chaque utilisateur une fiche de synthèse couvrant **quatre dimensions complémentaires** :

- **Centres d'intérêt** : Identification des sujets qui engagent l'utilisateur, déduits des thématiques abordées dans ses échanges. Cette analyse permet de maintenir un profil d'intérêts à jour.
- **Synthèse de la journée** : Un résumé narratif des échanges significatifs de la journée. Ce résumé constitue la mémoire épisodique condensée de KODA pour cet utilisateur.
- **Points clés** : Extraction des informations factuelles importantes révélées durant la journée — décisions évoquées, projets mentionnés, problèmes soulevés. Ces éléments enrichissent la mémoire sémantique du profil.
- **Conseils personnalisés** : Génération de recommandations adaptées au profil et au contexte de l'utilisateur, permettant à KODA d'offrir une valeur proactive lors des interactions suivantes.

OUTPUT

Schema Table JSON

To make sure expressions after this node work, return the input items that produced each output item. [More info](#)

1 item

A interets La nature, l'identité et les limites du robot (personnalité, émotions, rêves, corps, capacité à faire des erreurs, évolution). • La mémoire du robot et sa capacité à se souvenir des interactions passées et des informations sur l'utilisateur (nom, âge, entourage). • La technologie, l'intelligence artificielle et ses concepts connexes (comparaison avec d'autres IA, termes techniques, langage de programmation, outils). • Les préférences et opinions du robot (musique, sport, sujets d'actualité ou sensibles). • Des informations personnelles et académiques (PFE, études, nationalité, spécialité, etc....

A synthese L'utilisateur est manifestement **curieux** et **testeur**, explorant les capacités et les limites de l'IA, en particulier sa mémoire et sa capacité à interagir de manière humaine. Il adopte un ton globalement **amical** et cherche à établir une connexion personnalisée avec le robot. Il manifeste un intérêt prononcé pour la **technologie** et l'IA, tout en projetant des traits et des questions humaines sur le robot. Il semble être **jeune ou en formation** (références aux études, PFE, ISET) et est probablement d'origine **tunisienne** ou proche de cette culture, vu l'usage de l'arabe et du dia...

A points_cles L'utilisateur valorise la **mémoire persistante** du robot, sa capacité à le reconnaître et à retenir des informations le concernant et sur son entourage. Il accorde de l'importance à la **personnalisation de l'interaction** et à la capacité du robot à avoir une 'personnalité' ou des 'opinions'. Il est intéressé par l'**intelligence et les capacités d'apprentissage** du robot. Enfin, la **pertinence culturelle et linguistique** (arabe, tunisien) est un point d'attention pour lui, ainsi que la capacité du robot à gérer des sujets complexes ou sensibles de manière adéquate.

A conseils L'IA doit adopter un ton **amical et engageant**, répondant aux salutations et aux marques d'intérêt personnel de manière chaleureuse. Il est crucial de faire preuve d'une **mémoire contextuelle** solide, en rappelant les informations précédemment données par l'utilisateur (son nom, âge si connu, personnes mentionnées) pour renforcer la personnalisation. Pour les questions sur l'identité ou la 'personnalité' du robot, l'IA doit expliquer son fonctionnement en tant que modèle d'IA de manière **claire et pédagogique**, sans nier l'intérêt de l'utilisateur, mais en évitant de se présenter comme u...

A acteur oussema

FIGURE III.33 – Webhook 2 — exemple de sortie structurée (quatre dimensions de la fiche journalière)

III.5.2.5 Persistance : relation utilisateur / accompagnement

Le nœud *Insertion Des Données* écrit une fiche par utilisateur dans la table accompagnement, reliée à utilisateur par la clé étrangère iduser. La figure III.5.2.5 précise cette liaison (un utilisateur, zéro ou plusieurs fiches de synthèse journalière).

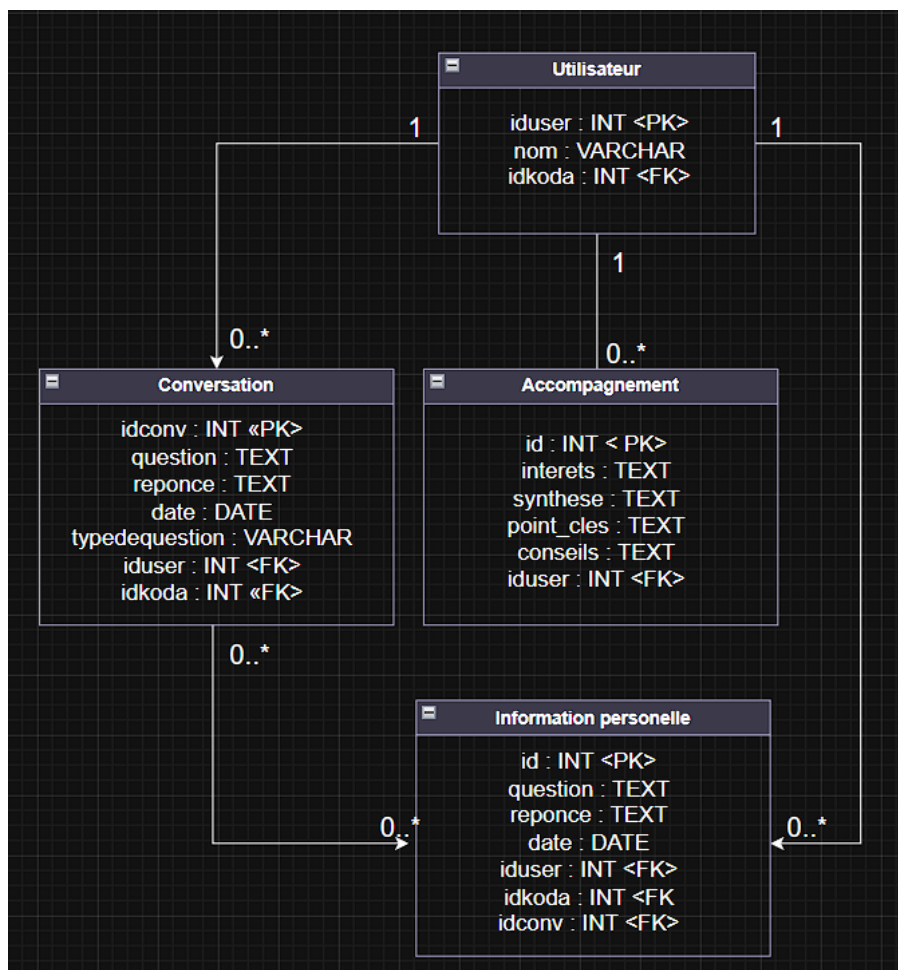


FIGURE III.34 – Webhook 2 — schéma de persistance (utilisateur → accompagnement : interets, synthese, point_cles, conseils)

Référence : Miller, G.A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2).

III.5.2.6 Impact sur le Système Global

Les synthèses du Webhook 2 constituent la **mémoire à long terme** de KODA, stockées dans accompagnement (figures III.5.2.5 et III.4.3). Elles sont consultées par le **Webhook 1** pour enrichir le contexte des réponses (branche mémorielle du la réponse contextualisée) et exploitées par le **Webhook 3** pour générer des interactions spontanées pertinentes (interests, daily_summary, key_points, personalized_advice).

III.5.2.7 Conclusion du Webhook 2

Le deuxième flux complète le premier en apportant une **consolidation différée** : là où le Webhook 1 réagit en temps réel et ne conserve qu'une fenêtre courte (historique, cinq messages), le Webhook 2 rejoue la journée entière pour en extraire une **mémoire durable** et structurée. Son architecture linéaire (collecte SQL → structuration → raisonnement IA → insertion) reste simple à maintenir. Le **trigger backend** assure la planification sans intervention manuelle et fait le lien entre

les journaux conversationnels et la personnalisation à long terme du compagnon.

Figures associées au Webhook 2 : workflow n8n (III.5.2.2), diagramme de séquence (III.5.2.2), exemple de sortie quatre dimensions (III.5.2.4), schéma utilisateur/accompagnement (III.5.2.5), vue globale des classes (III.4.3).

III.5.3 Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive

III.5.3.1 Principe : la spontanéité comme dimension humaine

Un compagnon humain ne se contente pas de répondre aux questions : il peut aussi **prendre la parole en premier**, avec un ton adapté à la personne devant lui. Les recherches en psychologie sociale montrent que les interactions les plus mémorables sont souvent celles initiées hors d'un simple échange question-réponse.

Le **Webhook 3** matérialise cette spontanéité. Son **rôle principal** est de recevoir un **nom de personne** (utilisateur connu, ex. Oussema) ou la valeur `unkonu / Inconnu`, puis de produire une **introduction orale** (`parole_du_robot`) accompagnée d'une **émotion visuelle** (`emotion_visuelle`) pour le robot. À chaque déclenchement, il **recharge les informations du Webhook 2** (centres d'intérêt, profils, points de vue, conseils stockés dans `accompagnement`) afin que la phrase d'accueil reste personnalisée et cohérente avec l'historique récent.

Référence : Dautenhahn, K. & Billard, A. (1999). Bringing up robots or the psychology of socially intelligent

III.5.3.2 Diagrammes du Webhook 3

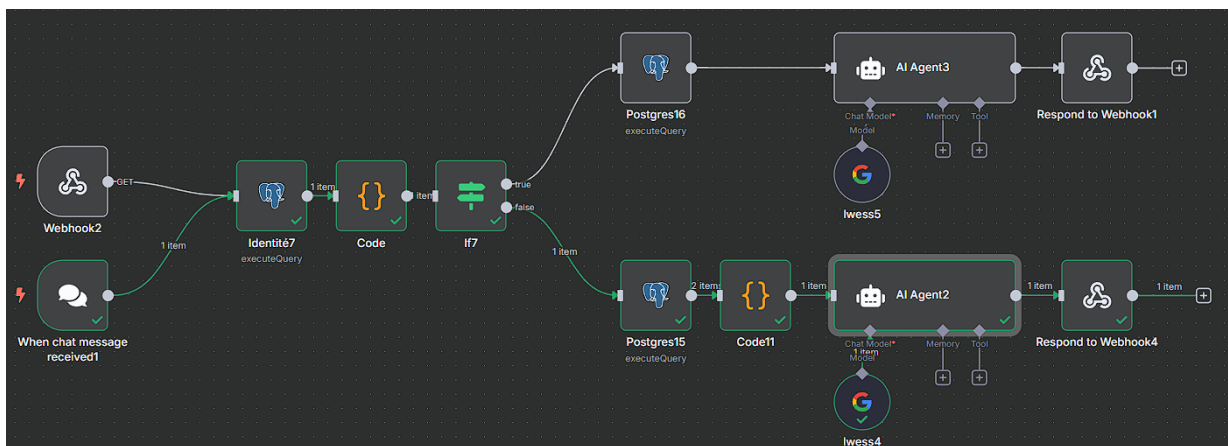


FIGURE III.35 – Webhook 3 — workflow n8n

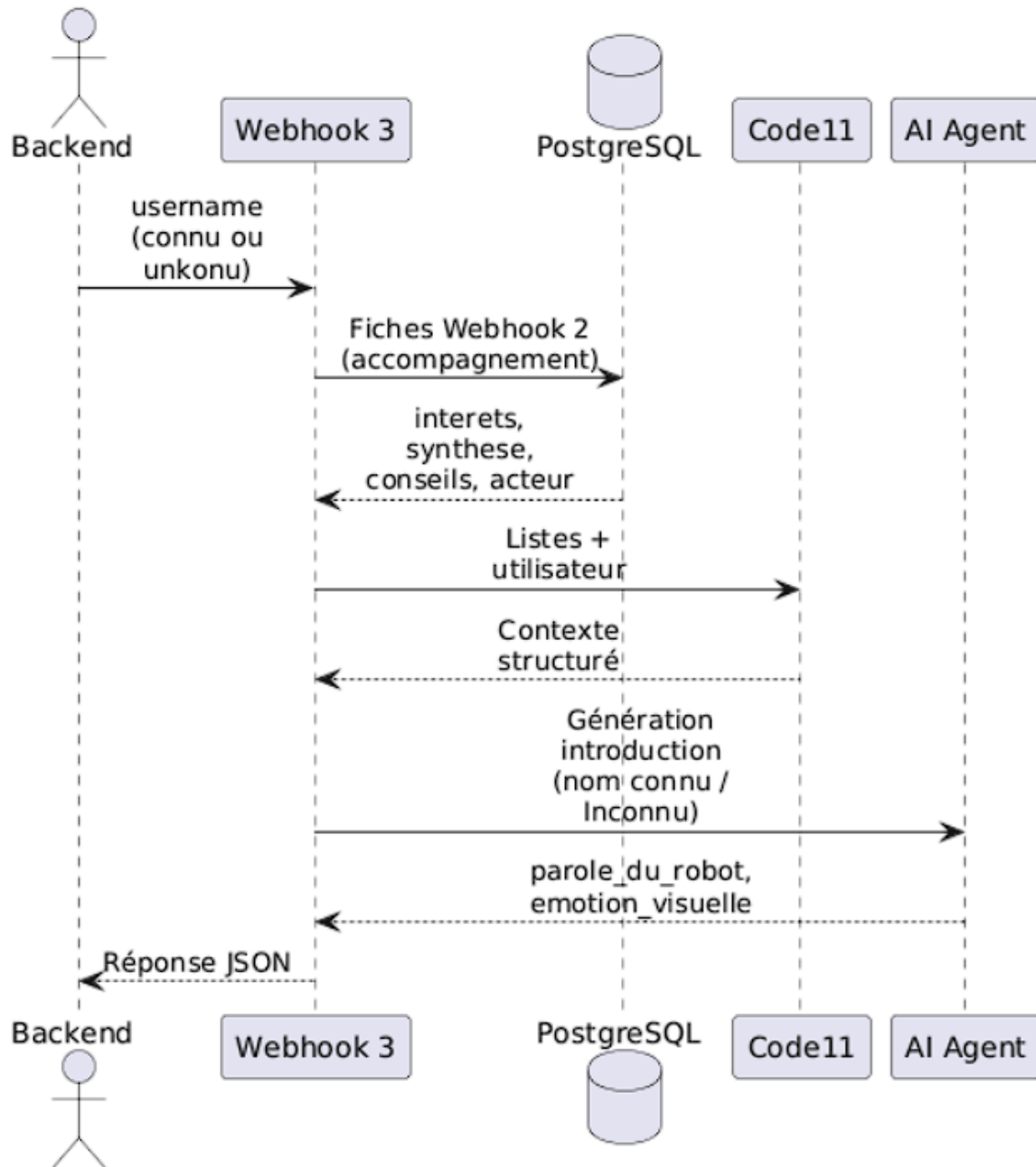
Webhook 3 — Introduction de parole

FIGURE III.36 – Webhook 3 — diagramme de séquence

III.5.3.3 Déclenchement et entrées

Le Webhook 3 est activé lorsque le **backend** ou le module de vision détecte une présence et transmet un identifiant (username, acteur). Le workflow (figure III.5.3.2) enchaîne *Identité7*, *Code*, puis *If7* vers **AI Agent3** ou **Code11 + AI Agent2**.

III.5.3.4 Préparation des données (nœud Code11)

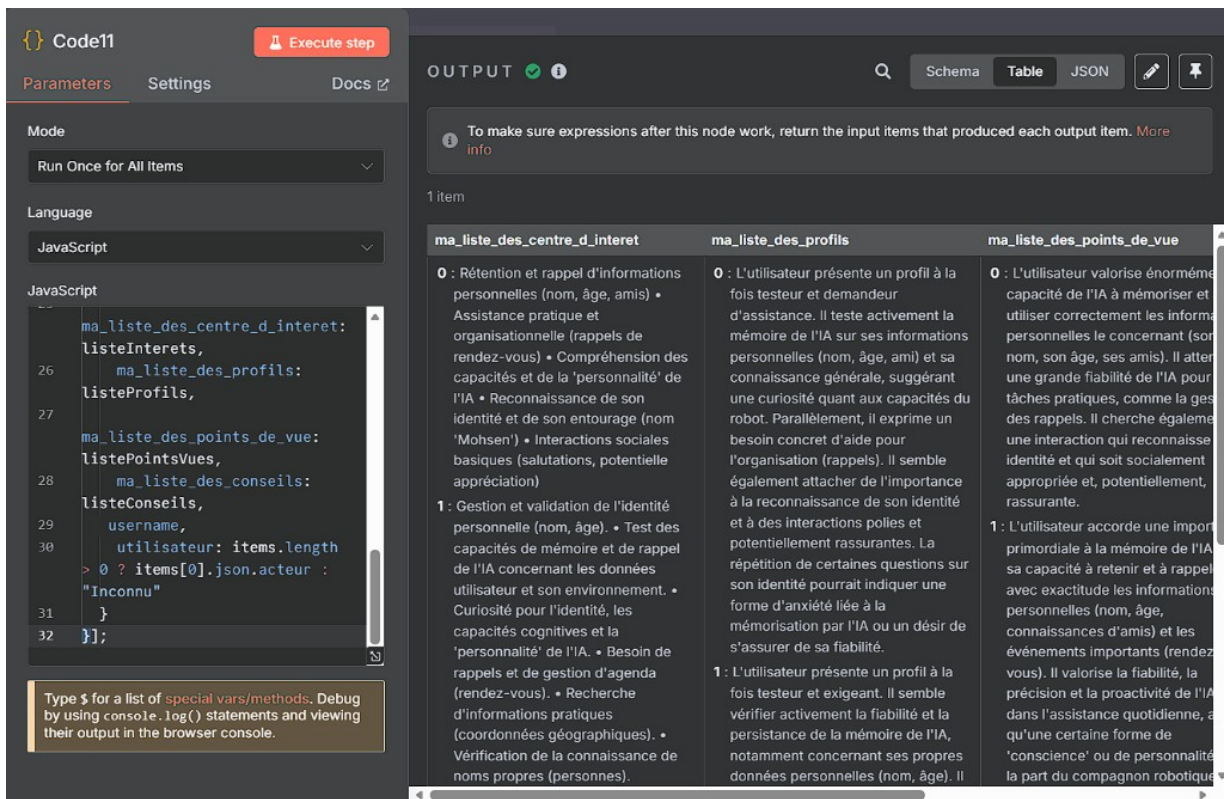


FIGURE III.37 – Webhook 3 — entrée du nœud Code11

Le nœud **Code11** consolide les listes issues de accompagnement (centres d'intérêt, profils, points de vue, conseils) et l'identifiant utilisateur.

III.5.3.5 Génération de l'introduction et sortie JSON

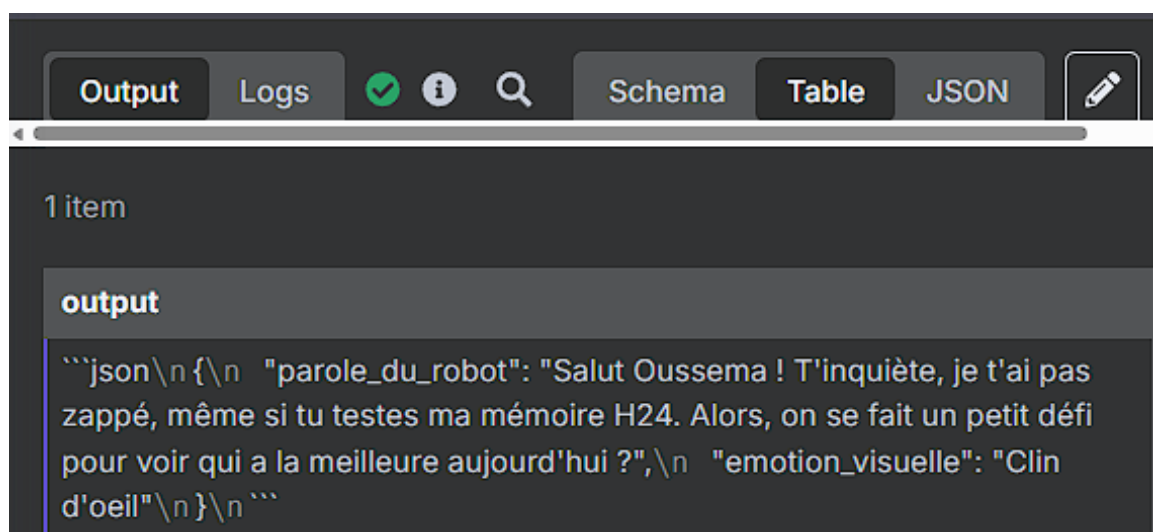


FIGURE III.38 – Webhook 3 — sortie JSON (parole_du_robot, emotion_visuelle)

L'**AI Agent** produit `parole_du_robot` et `emotion_visuelle` pour le robot (exemple utilisateur Oussema, figure III.5.3.5).

III.5.3.6 Conclusion du Webhook 3

Le troisième flux complète la chaîne cognitive : le Webhook 1 répond aux questions, le Webhook 2 consolide la mémoire, le Webhook 3 **initie la relation** en parlant d'abord, avec un contenu qui s'appuie systématiquement sur les synthèses journalières. La distinction **nom connu / inconnu** permet d'adapter l'accueil (personnalisation versus invitation à se présenter), tout en conservant une expérience cohérente avec la personnalité de KODA.

Référence : Reeves, B. & Nass, C. (1996). *The Media Equation*. Cambridge University Press.

III.6 Synthèse cognitive du Cycle 1

Les trois webhooks forment un **système cognitif cohérent** (voir modélisation globale en début de chapitre). Le tableau suivant résume les correspondances humaines / implémentations :

TABLE III.1 – Correspondance entre mécanismes cognitifs humains et implémentations de KODA

Mécanisme Humain	Référence Scientifique	Implémentation KODA
Double traitement (S1/S2)	Kahneman, 2011	Classification sémantique
Optimisation cognitive	Principe DRY	Confrontation à l'historique
Consolidation nocturne	Stickgold, 2005	Webhook 2 — Synthèse journalière
Regroupement mémoriel	Miller, 1956	4 dimensions de la synthèse Webhook 2
Spontanéité sociale	Clark, 1996 ; Dautenhahn & Billard, 1999	Webhook 3 — Interactions proactives

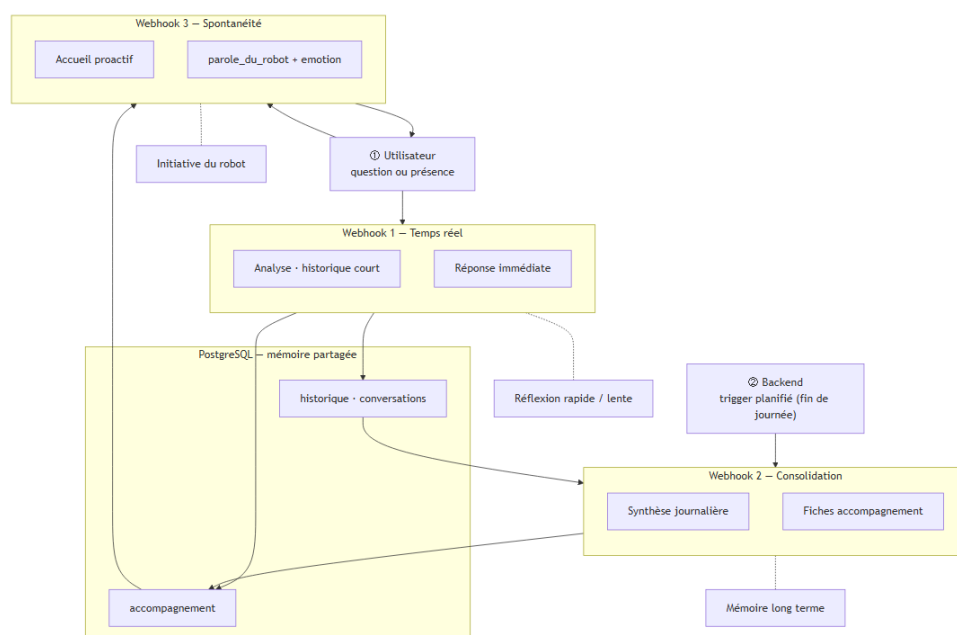


FIGURE III.39 – Architecture cognitive globale de KODA — interaction entre les trois webhooks

Référence : Russell, S. & Norvig, P. (2010). *Artificial Intelligence : A Modern Approach*, 3rd ed. Prentice Hall.

III.7 Conclusion

Ce chapitre a présenté l'architecture cognitive de KODA, structurée autour de trois webhooks complémentaires orchestrés par la plateforme n8n, en s'appuyant sur la théorie du double traitement, OpenRouter (la classification sémantique), SerpAPI (l'entrée fonctionnelle, branche musique) et PostgreSQL pour la persistance.

Le **Webhook 1** (six parties nommées) assure le traitement en temps réel : entrée fonctionnelle, confrontation à l'historique, classification sémantique, gestion unkonu, réponse contextualisée, notes et mémoire persistante. Le **Webhook 2**, déclenché par le **backend**, consolide la journée en fiches accompagnement. Le **Webhook 3** produit l'introduction de parole (`parole_du_robot`, `emotion_visuelle`) à partir de ces synthèses.

Ensemble, ces mécanismes — réactif, mémoriel et proactif — constituent le socle du Cycle 1. Le **chapitre suivant** (Cycle 2 — Distributeur de services) décrit comment le backend Python relie ce moteur n8n au robot, à l'application mobile et aux services cloud (Azure, Supabase).

_____ CHAPITRE IV _____

CYCLE 2 : DISTRIBUTEUR DE SERVICES

IV.1 Introduction

Le **chapitre IV** documente le **Cycle 2** du projet KODA : la conception, le développement et la validation du **Distributeur de Services**, composant central qui constitue la **couche de services** de l'écosystème. Ce cycle intervient après le **Cycle 1** (moteur de réflexion n8n) et précède les cycles embarqué (matériel et logiciel Raspberry Pi) et mobile (KodaMate).

IV.1.0.0.1 Rôle global du Distributeur.

Le Distributeur de Services n'est pas qu'un relais technique : il joue le rôle de **pivot fonctionnel** de KODA. Théoriquement, il assure trois fonctions indissociables :

Unification

Il traduit des interactions hétérogènes (parole du robot, saisie mobile, événements de présence) en un **langage commun** que le reste du système peut traiter : identité de l'interlocuteur, contexte mémoriel, demande formulée.

Personnalisation

Il garantit que chaque réponse s'appuie sur le **même profil utilisateur**, quelle que soit la voie d'accès : une question posée au robot et une saisie dans l'application mobile alimentent la même mémoire et le même moteur de réflexion.

Continuité

Il maintient la **expérience utilisateur** dans le temps : accueil proactif lors d'une détection de présence, dialogue vocal, suivi depuis le mobile — le robot et l'application ne sont que deux *faces* d'un même compagnon, et le Distributeur en est le garant.

En ce sens, le backend **matérialise la fonctionnalité globale** de KODA : il assemble des capacités techniques (vocal, visage, persistance, orchestration) pour produire une relation utilisateur-robot **cohérente, mémorisée et évolutive**. L'implémentation concrète (FastAPI, PostgreSQL, Azure, n8n) est détaillée dans les sections suivantes ; l'introduction se limite ici au **pourquoi** de ce composant.

IV.1.0.0.2 Objectifs et résultats attendus.

À l'issue de ce cycle, nous visons : **(i)** un backend déployable et testable indépendamment du robot et de n8n ; **(ii)** des interfaces stables (WebSocket robot, REST mobile, webhook n8n) permettant aux cycles suivants de progresser en parallèle ; **(iii)** une reconnaissance faciale opérationnelle (>94 % de précision) ; **(iv)** une chaîne vocale Azure STT/TTS fonctionnelle ; **(v)** une persistance relationnelle couvrant conversations, profils et agenda.

IV.1.0.0.3 Organisation du chapitre.

Le chapitre suit une progression **théorique puis opérationnelle** : d’abord le **cadre conceptuel** (pourquoi le Distributeur est le seul pivot du système), ensuite la **modélisation** (cas d’utilisation, classes, architecture globale), puis l’**architecture interne** du code, les **services et flux** détaillés, les **problèmes rencontrés** (réseau, CORS, concurrence), les **perspectives**, et enfin la conclusion.

Rôle principal du Distributeur de Services

Offrir au robot et à l’application mobile un **accès unifié** aux services techniques (vocal, visage, persistance, cloud) et au moteur n8n, afin de garantir **flexibilité** (évolution indépendante des composants) et **disponibilité** (validation, retry, journalisation) sans exposer les secrets ni multiplier les couplages.

IV.2 Fondements théoriques et positionnement du Distributeur

Avant d’aborder l’implémentation, il convient d’explicitier **pourquoi** le Distributeur est le **seul composant** capable de relier harmonieusement robot, application, base de données, n8n et cloud.

IV.2.1 Le principe du point de liaison unique

Dans une architecture distribuée, chaque lien direct entre deux composants crée une **dépendance** : changement d’URL n8n, rotation de clé Azure ou migration Supabase impose alors de modifier le Raspberry Pi, l’application mobile et les workflows. Le modèle retenu pour KODA inverse cette logique : seul le Distributeur connaît l’ensemble des partenaires externes ; robot et mobile ne connaissent **qu’une adresse backend** et un contrat d’interface stable.

Ce choix architectural, proche du **pattern API Gateway**, apporte deux propriétés essentielles :

Flexibilité

Remplacer Whisper par Azure STT, ou migrer un webhook n8n, se fait dans le backend sans recompiler le robot ni l’application mobile. Les cycles du modèle en spirale (chapitre II) peuvent avancer en parallèle tant que les contrats d’interface restent respectés.

Disponibilité

Le Distributeur centralise la gestion des erreurs (*timeout*, retry WebSocket, validation JSON, transactions atomiques en base). Un échec ponctuel d’un service cloud ne se propage pas brutalement à l’utilisateur : le backend peut renvoyer un message de repli ou mettre une commande en file d’attente.

IV.2.2 Services backend et fonctionnalité globale du système

La **fonctionnalité globale** de KODA — dialoguer, mémoriser, personnaliser, accompagner — ne réside dans aucun composant isolé. Elle émerge de la **composition de services** que le backend assemble :

1. **Service vocal** : capture → STT → traitement → TTS → restitution audio sur le robot ;
2. **Service d'identité** : reconnaissance faciale, profils utilisateurs, paramètres KODA ;
3. **Service de persistance** : conversations, historique contextuel, agenda, activités, notes ;
4. **Service d'orchestration cognitive** : délégation à n8n avec enrichissement du contexte depuis la base ;
5. **Service mobile** : authentification JWT, CRUD profils/agenda, consultation de l'historique.

Chaque service est implémenté comme un **cas d'usage** (Clean Architecture) et exposé via REST ou WebSocket. Le robot vocal et l'application mobile consomment ces services selon leurs besoins, mais partagent la **même logique métier** et la **même mémoire** : une question posée au robot et une saisie dans l'app aboutissent aux mêmes tables PostgreSQL et au même moteur n8n.

IV.2.3 Communication vocale et applicative : deux faces, un backend

- **Face robot (temps réel)** : le Raspberry Pi maintient une connexion **WebSocket** persistante pour les commandes, acquittements et flux audio. La latence et la fiabilité sont critiques ; le backend garantit un protocole JSON normalisé et une journalisation des commandes (statut « en attente », ACK, retry).
- **Face application (requête-réponse)** : KodaMate consomme des **API REST** authentifiées par JWT (Supabase Auth). Le rythme est moins contraignant, mais la cohérence des données avec le robot est impérative : modifier l'agenda dans l'app doit être visible lors de la prochaine interaction vocale.

Le Distributeur est le **garant de cette cohérence** : il unifie persistance, sécurité et orchestration pour produire, dans les deux cas, un **rôle et un output parfait** au sens fonctionnel — réponse adaptée, actions robot correctes, trace enregistrée en base.

IV.3 Modélisation et place dans l'architecture

La modélisation traduit le cadre théorique ci-dessus en artefacts UML : *que fait* le Distributeur (cas d'utilisation), *avec quelles entités* il travaille (schéma de classes), et *où* il se situe dans l'architecture multicouche KODA.

IV.3.1 Diagramme de cas d'utilisation général

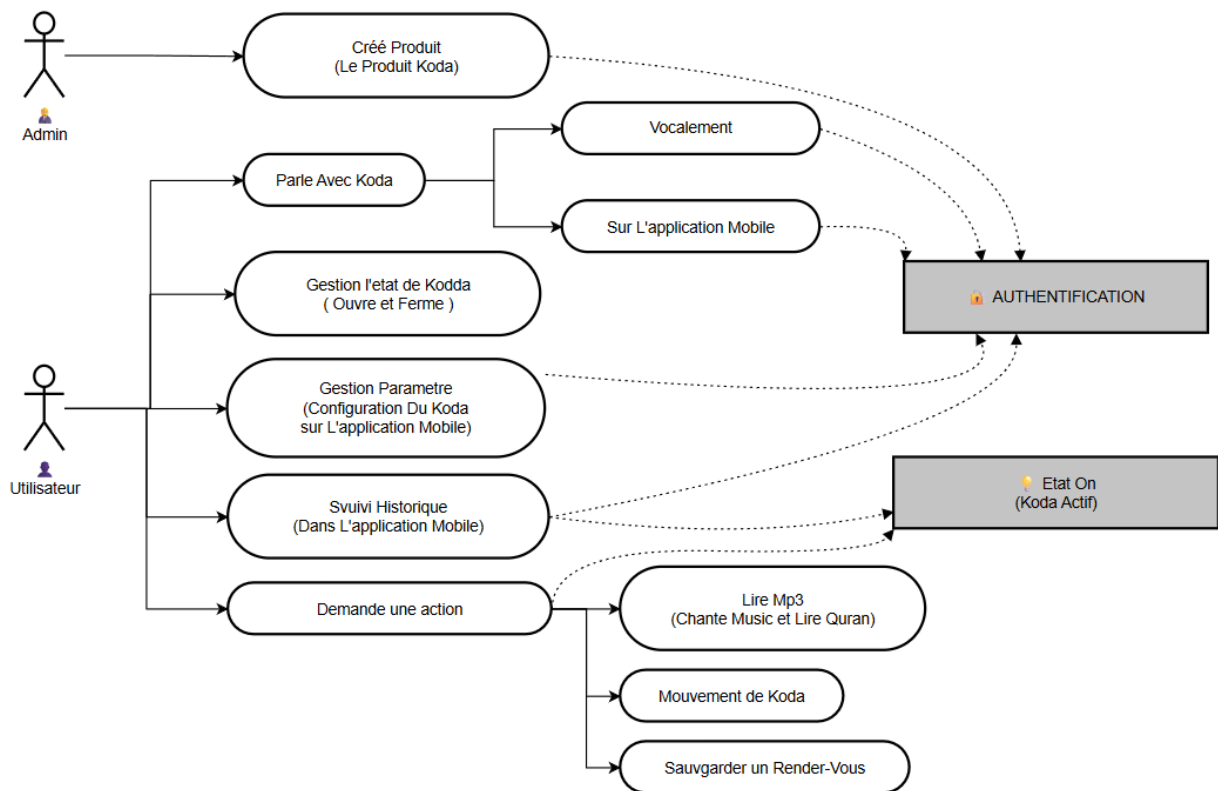


FIGURE IV.1 – Cycle 2 — diagramme de cas d'utilisation général du Distributeur de services

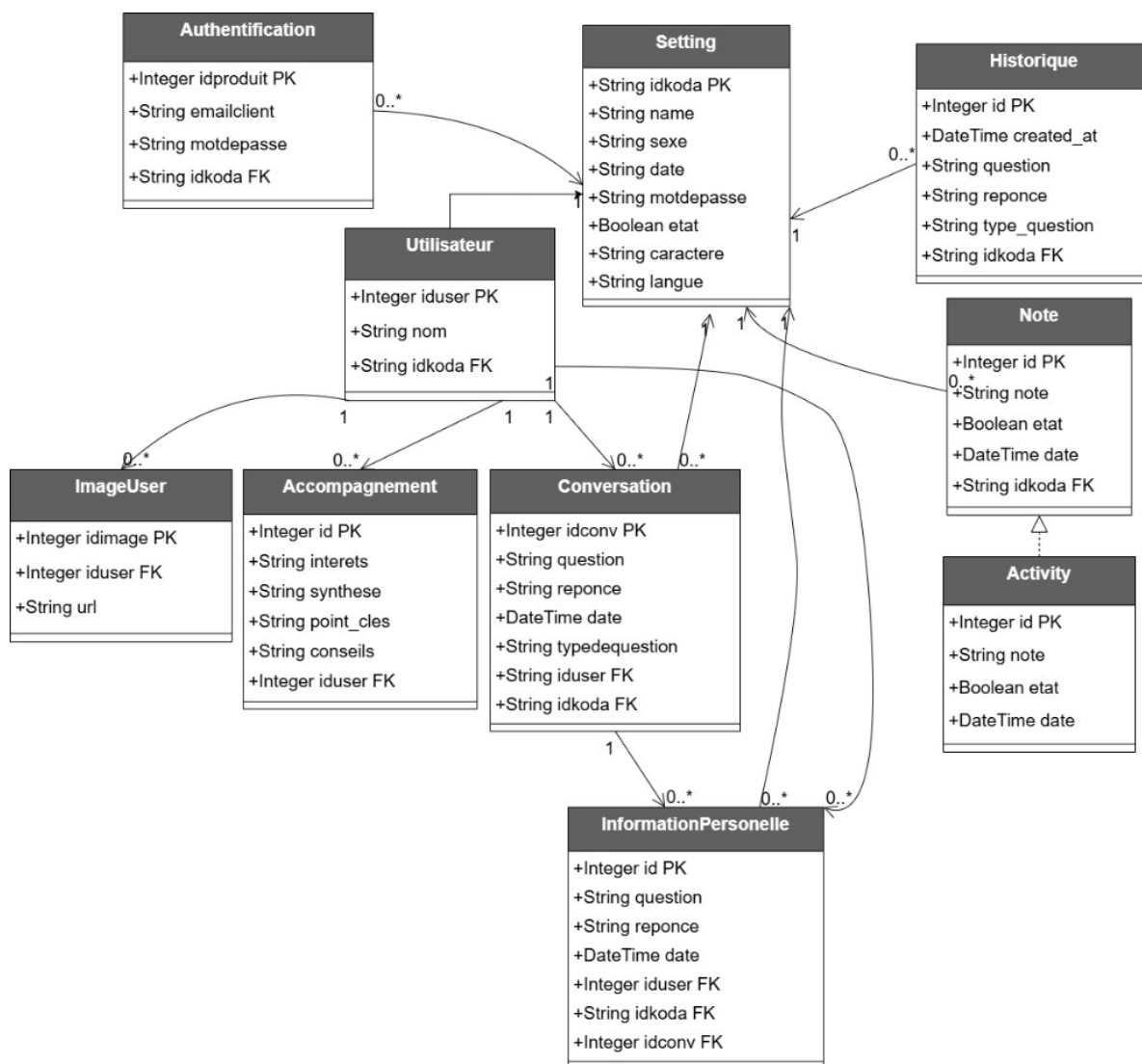


FIGURE IV.2 – Cycle 2 — schéma de classes général du backend (persistance PostgreSQL)

Le diagramme de cas d'utilisation regroupe les interactions typiques : traitement vocal, session WebSocket robot, reconnaissance faciale, authentification mobile, persistance et délégation n8n. Il sert de **carte de lecture** pour les sections IV.5 et IV.6.

Le schéma de classes (figure IV.3.1, placé sous le cas d'utilisation) matérialise les entités persistées par le Distributeur : Setting, Authentification, Utilisateur, Conversation, Historique, Note, Activity, InformationPersonnelle, Accompagnement et ImageUser, ainsi que leurs clés étrangères (idkoda, iduser, idconv). Il complète la vue fonctionnelle en montrant **quoi** est stocké en base, indépendamment des protocoles HTTP ou WebSocket.

IV.3.2 Architecture globale : pivot de flexibilité et de disponibilité

La figure II.2 (chapitre II) rappelle la décomposition multicouche de KODA : n8n pour la **réflexion**, le **backend** pour la **distribution des services**, le **Raspberry Pi** pour l'**action** physique, l'**application mobile** pour la **configuration**, PostgreSQL pour la **mémoire** durable.

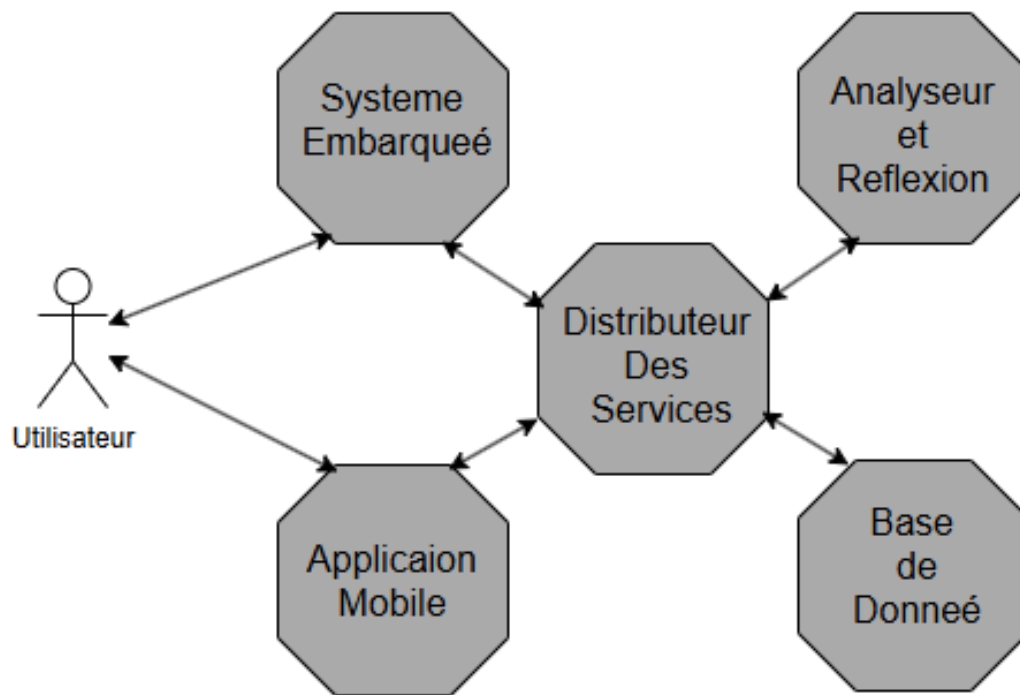


FIGURE IV.3 – Architecture multicouche de KODA — place du Distributeur de services (Cycle 2)

Le Distributeur est **indispensable** car il constitue le seul point d'entrée **homogène** vers les ressources partagées (cf. § IV.2) : une modification de la base, d'un webhook n8n ou d'une clé Azure se fait au même endroit, ce qui respecte le modèle en spirale (cycle 2 livrable et testable sans réécrire le robot ni les workflows). Les cycles 1, 3, 4 et 5 s'appuient sur des **interfaces stables** exposées par ce backend. La figure IV.3.2 détaille ce pivot central : seul le Distributeur connaît n8n, PostgreSQL et les services cloud ; robot et mobile ne maintiennent qu'une connexion vers lui.

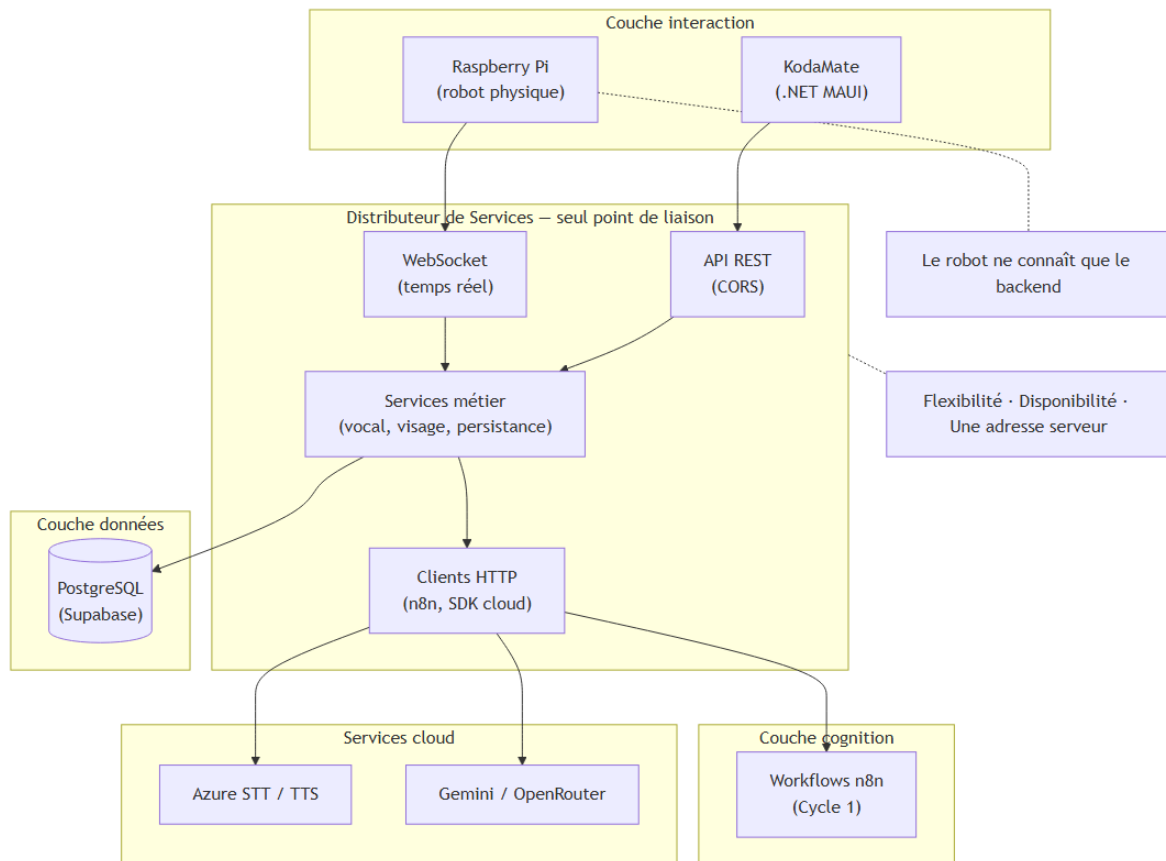


FIGURE IV.4 – Cycle 2 — Distributeur pivot central (composants et flux)

IV.4 Architecture interne du backend

Cette section décrit **comment** le Distributeur est structuré en code : choix technologiques, principes Clean Architecture et organisation des répertoires.

IV.4.1 Choix du langage : Python

Le choix de Python comme langage de développement du backend résulte d'une analyse rigoureuse des besoins fonctionnels et techniques du projet. Trois raisons majeures ont orienté cette décision.

Premièrement, Python dispose d'un écosystème exceptionnel pour les domaines mobilisés par notre projet : intelligence artificielle, traitement du signal audio, vision par ordinateur et communications temps réel. Des bibliothèques telles que `face_recognition`, `NumPy` ou les SDK officiels d'Azure et de Google (Gemini) sont nativement disponibles et parfaitement documentées.

Deuxièmement, le framework **FastAPI** — retenu pour la construction des endpoints REST — est entièrement basé sur Python. Il offre des performances comparables à Node.js grâce à son support natif de l'asynchronisme (`asyncio`), tout en permettant la génération automatique de la documentation Swagger et la validation des données via Pydantic.

Troisièmement, la communauté Python est l’une des plus actives du monde open-source, ce qui garantit une maintenance facilitée, une abondance de solutions aux problèmes rencontrés et une meilleure pérennité du projet.

TABLE IV.1 – Comparaison des technologies backend envisagées

Critère	Python + FastAPI	Node.js + Express	Java Spring Boot
Bibliothèques IA/ML	✓ Excellent	~ Limité	~ Limité
Performance async	✓ asyncio	✓ Natif	✓ Natif
Intégration Azure SDK	✓ Officiel	✓ Officiel	✓ Officiel
Facilité de développement	✓ Très rapide	✓ Rapide	~ Verbeux
Reconnaissance visage	✓ face_recog	× Inexistant	~ Complexe
Déploiement Docker	✓ Simple	✓ Simple	~ Lourd

IV.4.2 Clean Architecture : principes et application

La **Clean Architecture**, théorisée par Robert C. Martin (*Uncle Bob*), repose sur un principe fondamental : séparer les préoccupations en couches concentriques indépendantes, de sorte que les règles métier du domaine ne dépendent d’aucune technologie externe. Cette indépendance garantit la testabilité, la maintenabilité et l’évolutivité du système.

Dans la pratique, le backend est aussi pensé selon l’**architecture en oignon** (*Onion Architecture*, Palermo) : le **domaine** occupe le centre, enveloppé par les **services applicatifs**, puis par les **interfaces** vers l’extérieur, et enfin par l’**infrastructure** (web, persistance, SDK). L’oignon et la Clean Architecture partagent la même règle d’or : *aucune couche intérieure ne doit importer une couche extérieure*. Les noms de couches ci-dessous s’alignent sur cette lecture « anneaux » du code du Distributeur.

Notre backend est structuré autour de quatre couches principales, chacune dotée d’un rôle précis et de dépendances strictement orientées vers l’intérieur.

La couche **Entities** contient les objets métier purs : Person, Session, Command et Event. Ces entités n’ont aucune dépendance externe et représentent les règles métier les plus stables du système. La couche **Application Use Cases** orchestre les scénarios fonctionnels : reconnaissance de personnes, traitement des commandes robot et gestion des sessions de communication. La couche **Interface Adapters** assure la traduction entre le monde extérieur et les cas d’usage internes (routeurs, dépôts, sérialiseurs). Enfin, la couche **Frameworks & Drivers** regroupe tous les détails d’implémentation : FastAPI, les connecteurs Supabase, les SDK Azure et Gemini. Ces quatre anneaux concentriques orientent les dépendances vers l’intérieur.

IV.4.3 Structure des répertoires du projet

Le code du Distributeur est organisé selon une **arborescence** qui reflète la Clean Architecture : le point d'entrée `main.py` est configuré par `config/`, expose les routes via `controllers/`, délègue la logique à `application/`, s'appuie sur le `domain/` et externalise Supabase, Azure et n8n dans `infrastructure/`, avec `presentation/` pour les DTO. À la racine du projet figurent `.env`, `requirements.txt`, `migrations/` et `tests/unit/`.

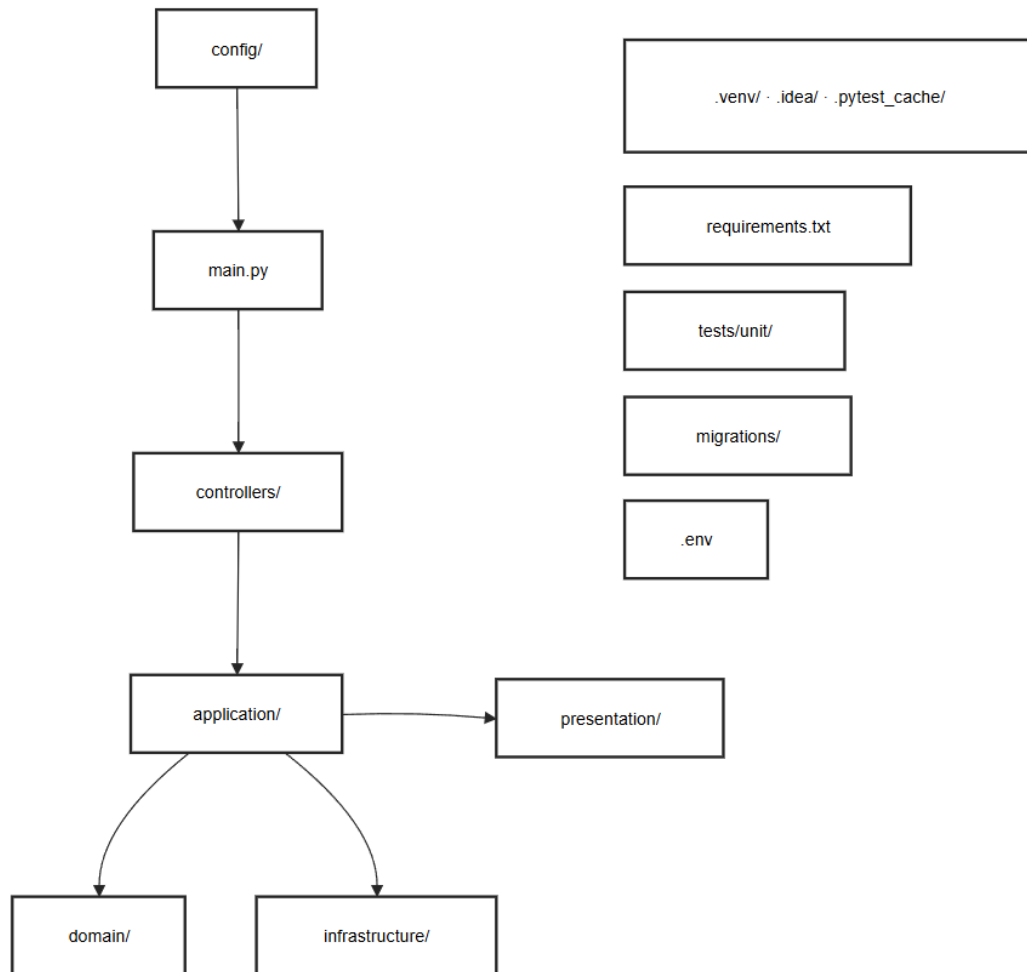


FIGURE IV.5 – Architecture des dossiers du projet backend Distributeur (`src/`)

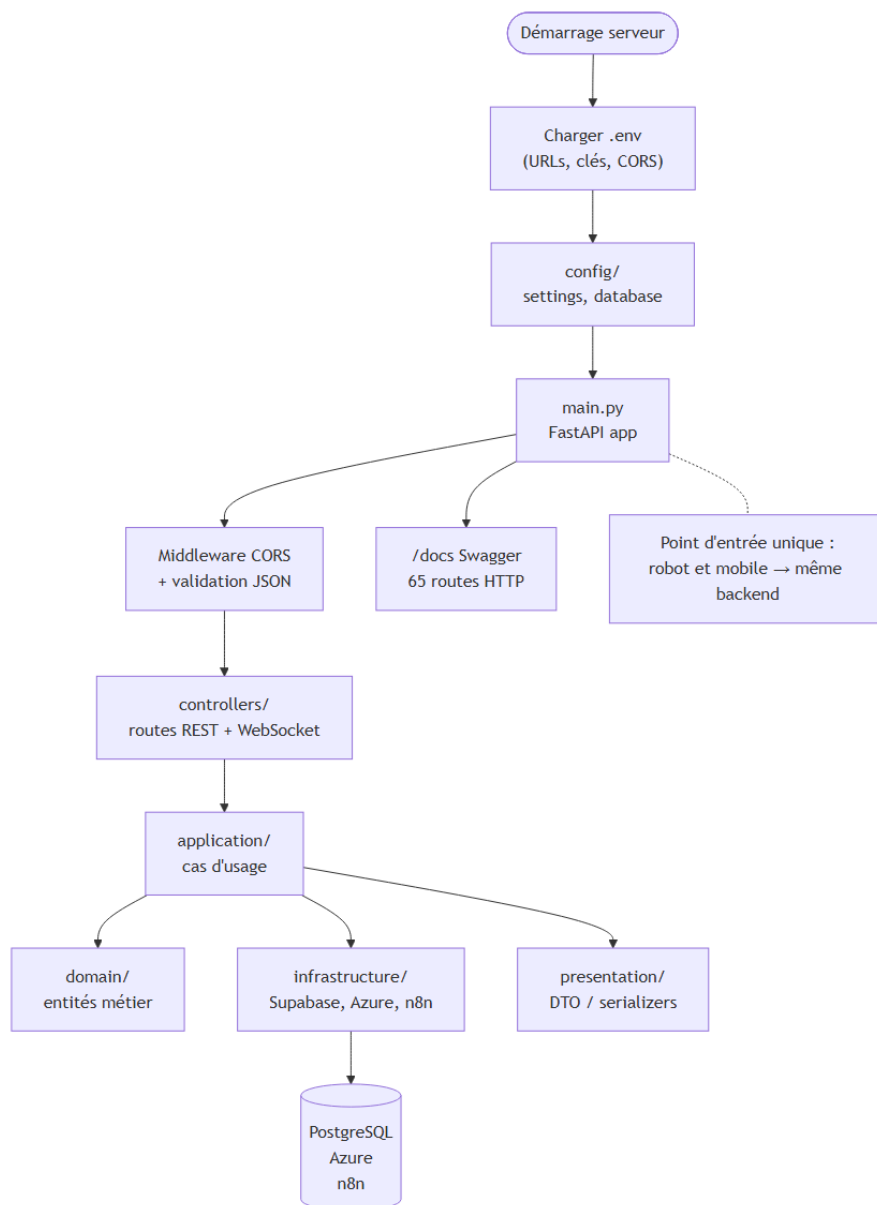


FIGURE IV.6 – Exécution du programme backend (Distributeur)

La figure IV.4.3 résume le flux de dépendances : `config` → `main.py` → `controllers` → `application`, puis branches vers `domain`, `infrastructure` et `presentation`.

IV.5 Services exposés et flux de communication

Le § IV.2 a posé le **cadre théorique** : le Distributeur assemble des services pour produire la fonctionnalité globale de KODA. Cette section décrit **concrètement** chaque flux : protocoles, séquences et rôle dans l'expérience utilisateur (vocale ou applicative).

IV.5.1 Service dédié vers le système embarqué (robot)

L'interface avec le **robot physique** (Raspberry Pi et microcontrôleurs) repose sur **deux actions vocales** initiées par le **système embarqué** auprès du Distributeur. Le robot ne contacte **jamais**

directement n8n, Supabase ou Azure : le backend centralise l’orchestration, la sécurité et la synthèse audio.

IV.5.1.1 Cas d’utilisation robot ↔ backend

La figure IV.5.1.1 présente les deux cas d’utilisation vus du côté embarqué :

- **Demander une réponse vocale** : le robot transmet une question ou un flux audio ; le Distributeur s’appuie sur **Azure Speech Services** (STT/TTS) et sur l’**Analyseur et Réflexion (Flux 1** — Webhook 1, chapitre III), puis persiste ou consulte la **base de données** ;
- **Demander une introduction de parole vocale** : lors d’une détection de présence, le robot sollicite un **accueil proactif** ; le Distributeur invoque l’**Analyseur et Réflexion (Flux 3** — Webhook 3) et retourne le texte synthétisé en **audio pré-enregistré** via Azure TTS.

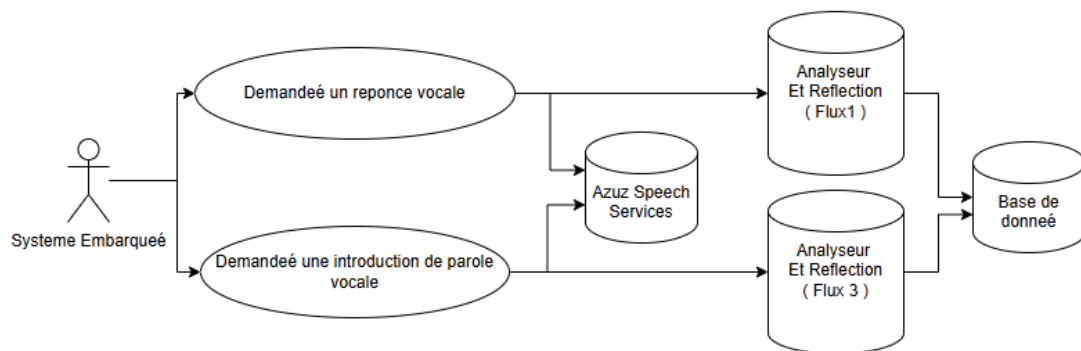


FIGURE IV.7 – Cycle 2 — cas d’utilisation du système embarqué vis-à-vis du Distributeur de services

Les sous-sections suivantes détaillent le **diagramme de séquence** de chaque scénario.

IV.5.1.2 Action 1 — Demander une réponse vocale (Flux 1)

La première action correspond au cas d’utilisation « **Demander une réponse vocale** » (figure IV.5.1.1). Lors d’un dialogue, le système embarqué transmet au Distributeur une **photo** (identification) et un **flux audio** (question). Le backend :

1. identifie l’interlocuteur via le **service de reconnaissance faciale** (`getUserName`) ;
2. transcrit l’audio en texte via **Azure STT** ;
3. normalise la langue (traduction vers le français si nécessaire) ;
4. invoque le **Webhook 1** (Flux 1 — Analyseur et Réflexion, chapitre III) pour produire la réponse ;
5. traduit éventuellement la réponse en arabe ;
6. synthétise la phrase via **Azure TTS** ;
7. retourne au robot le **JSON**, le **flux audio** et les commandes d’affichage ;

8. persiste l'échange dans PostgreSQL (étape optionnelle).

Le Raspberry Pi se contente de **capturer** la parole et la photo, puis de **restituer** la réponse (haut-parleur et écran Nextion). Pour une meilleure lisibilité, la figure IV.5.1.2 présente une **vue synthétique** en trois phases ; les figures IV.5.1.2 et IV.5.1.2 détaillent respectivement l'**entrée** (capture, identification, STT) et la **sortie** (Flux 1, TTS, restitution).

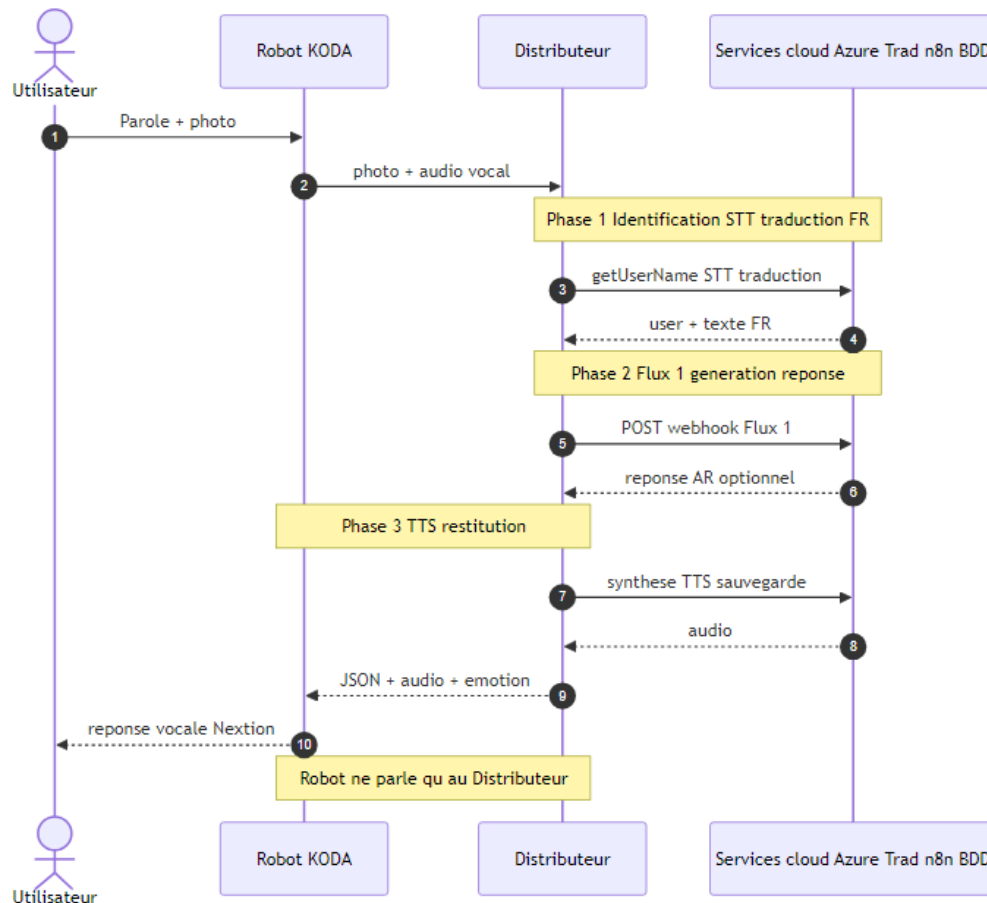


FIGURE IV.8 – Action 1 — vue compacte du pipeline vocal (Flux 1)

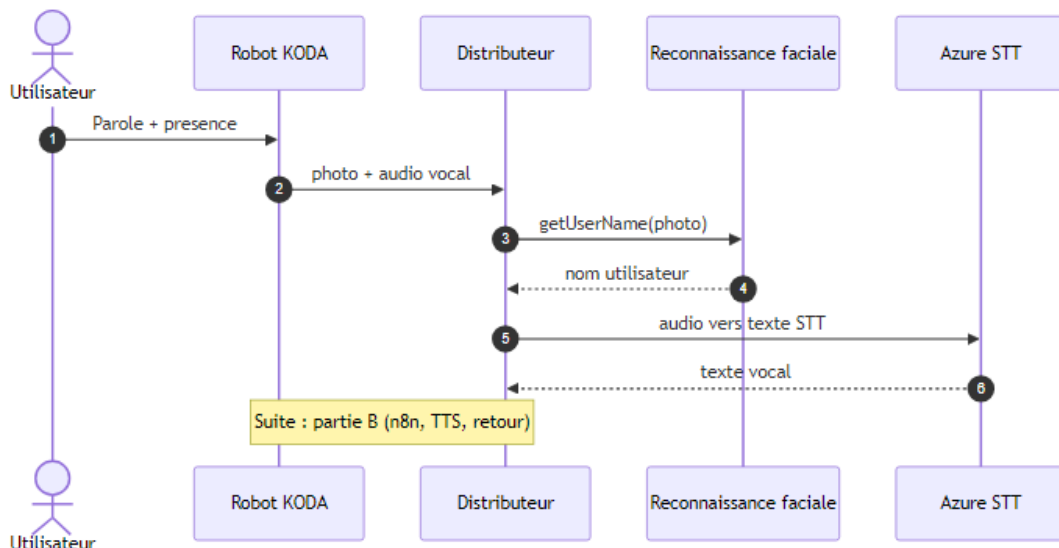


FIGURE IV.9 – Action 1 (a) — entrée : capture, identification faciale et transcription STT

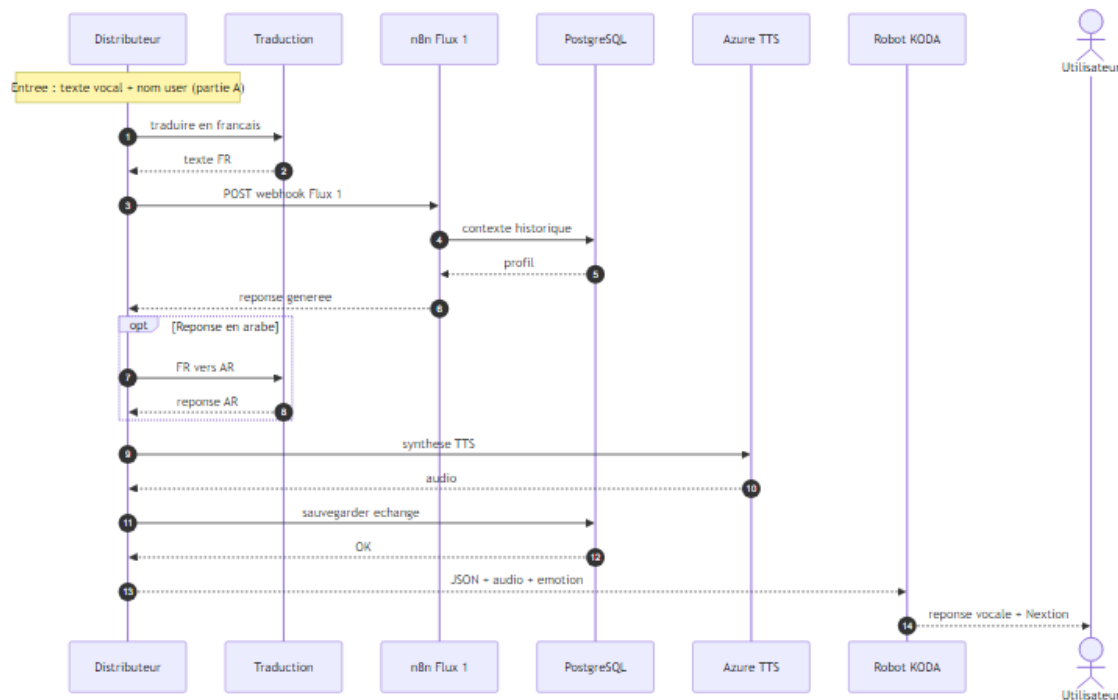


FIGURE IV.10 – Action 1 (b) — sortie : Flux 1, synthèse TTS et restitution au robot

IV.5.1.2.1 Lien avec le Cycle 1.

Le Flux 1 correspond au **Webhook 1** documenté au chapitre III (Analyseur et Réflexion). Le Distributeur centralise l'accès aux services Azure, à la traduction et à n8n : l'embarqué n'expose aucune clé API cloud.

IV.5.1.3 Action 2 — Demander une introduction de parole (Flux 3)

La seconde action correspond au cas d'utilisation « **Demander une introduction de parole vocale** » (figure IV.5.1.1). Elle intervient lorsque le robot **détecte une présence** (caméra, reconnaissance faciale) et souhaite **prendre la parole en premier** pour accueillir l'utilisateur. Le système embarqué envoie une requête HTTP au Distributeur (identifiant username ou acteur, personne connue ou inconnu). Le backend :

1. invoque le **Webhook 3** (Flux 3 — spontanéité, chapitre III) ;
2. récupère depuis n8n le texte `parole_du_robot` et l'émotion `emotion_visuelle`, enrichis par les fiches accompagnement (Webhook 2) ;
3. synthétise la phrase via **Azure TTS** ;
4. retourne au robot le **JSON** et le **flux audio pré-synthétisé**.

Le Raspberry Pi se contente de **restituer** l’audio (haut-parleur) et d’afficher l’émotion sur l’écran Nextion. Comme pour l’Action 1, la figure IV.5.1.3 résume le scénario en **deux phases** ; les figures IV.5.1.3 et IV.5.1.3 en détaillent l’**entrée** (détection, identification) et la **sortie** (Flux 3, TTS, restitution).

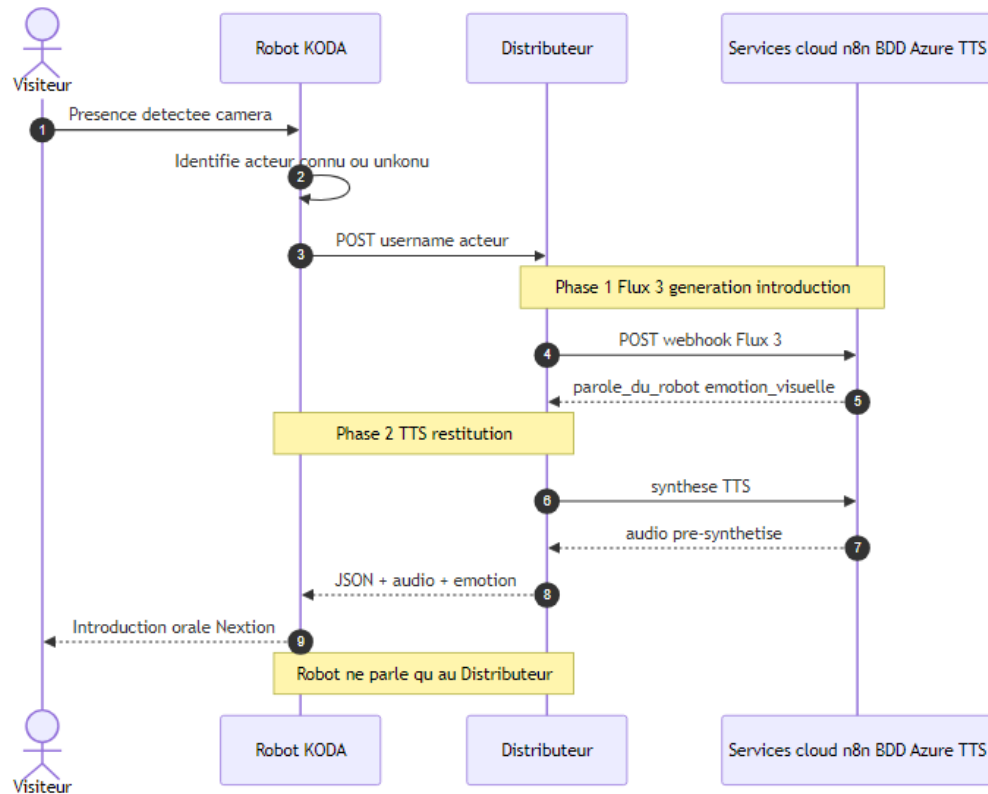


FIGURE IV.11 – Action 2 — vue compacte de l’introduction de parole (Flux 3)

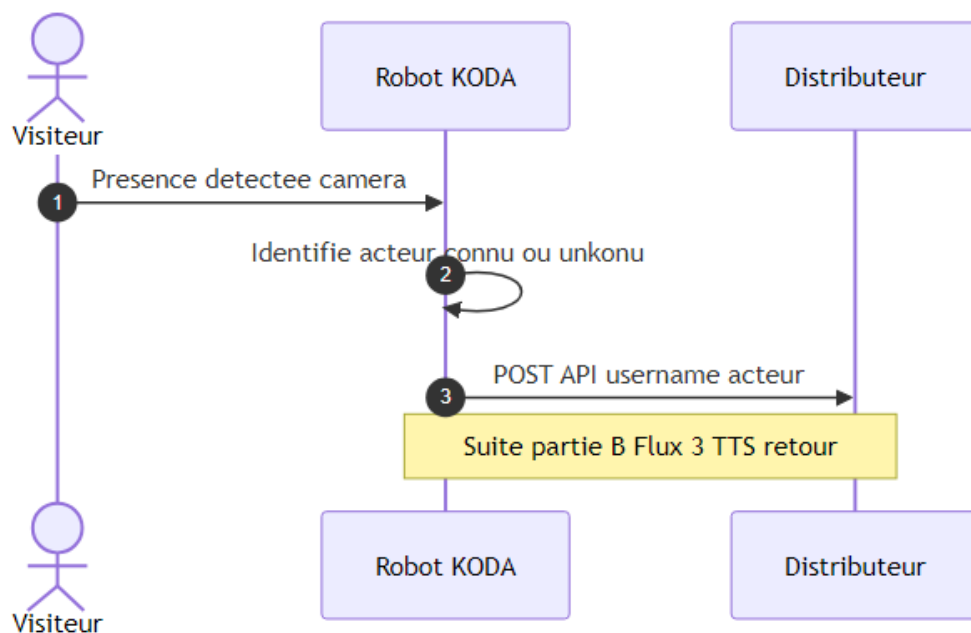


FIGURE IV.12 – Action 2 (a) — entrée : détection de présence et requête au Distributeur

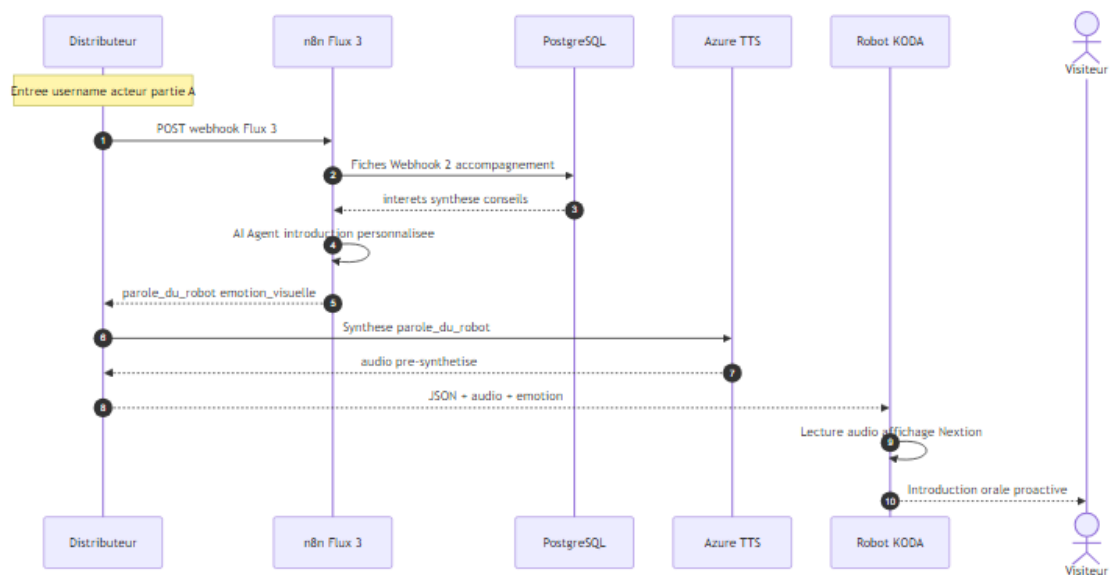


FIGURE IV.13 – Action 2 (b) — sortie : Flux 3, synthèse TTS et restitution au robot

IV.5.1.3.1 Lien avec le Cycle 1.

Le Flux 3 correspond au **Webhook 3** documenté au chapitre III (spontanéité et interaction proactive). Le Distributeur en est le **seul point d'accès** côté robot : il traduit l'appel embarqué en webhook n8n, puis en réponse audio exploitable par l'embarqué.

IV.5.1.4 Canal WebSocket (temps réel)

Outre les requêtes HTTP du pipeline vocal, le robot maintient un canal **WebSocket** persistant avec le Distributeur pour les acquittements, le suivi d'état et l'envoi de commandes moteur ou d'expressions en temps quasi réel. Le protocole repose sur des messages JSON typés (`status`, `action`, `command_result`); en cas de coupure réseau, le client embarqué applique une **reconnexion avec backoff exponentiel** (cf. tableau IV.11).

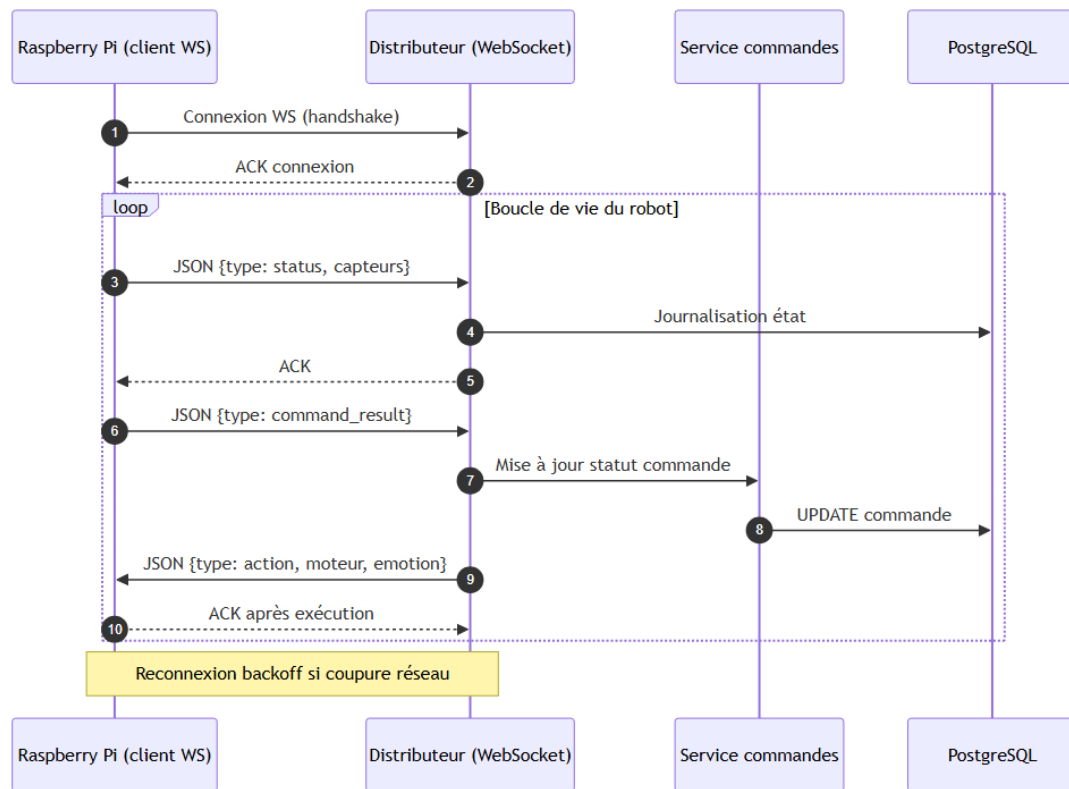


FIGURE IV.14 – Cycle 2 — échanges WebSocket robot ↔ Distributeur (temps réel)

IV.5.1.5 Chaîne vocale complète (Azure STT / TTS)

La dimension vocale est au cœur de l’expérience KODA. Le robot **capture** l’audio localement ; le backend prend en charge l’intégralité du traitement cloud et cognitif, puis renvoie un **output unifié** (audio synthétisé, commandes d’affichage et de mouvement). Ce découpage garantit que l’embarqué reste léger et que les clés Azure ne quittent jamais le serveur.

Le pipeline théorique s’articule ainsi :

1. capture audio sur le robot → envoi au backend (HTTP ou WebSocket) ;
2. transcription **Speech-to-Text** (Azure) ;
3. enrichissement du contexte depuis PostgreSQL (historique, profil) ;
4. traitement conversationnel via **webhook n8n** (Cycle 1) ;
5. synthèse **Text-to-Speech** (Azure) ;
6. retour audio et actions vers le robot.

Les routes REST associées sont recensées au tableau IV.6 (/api/audio/*).

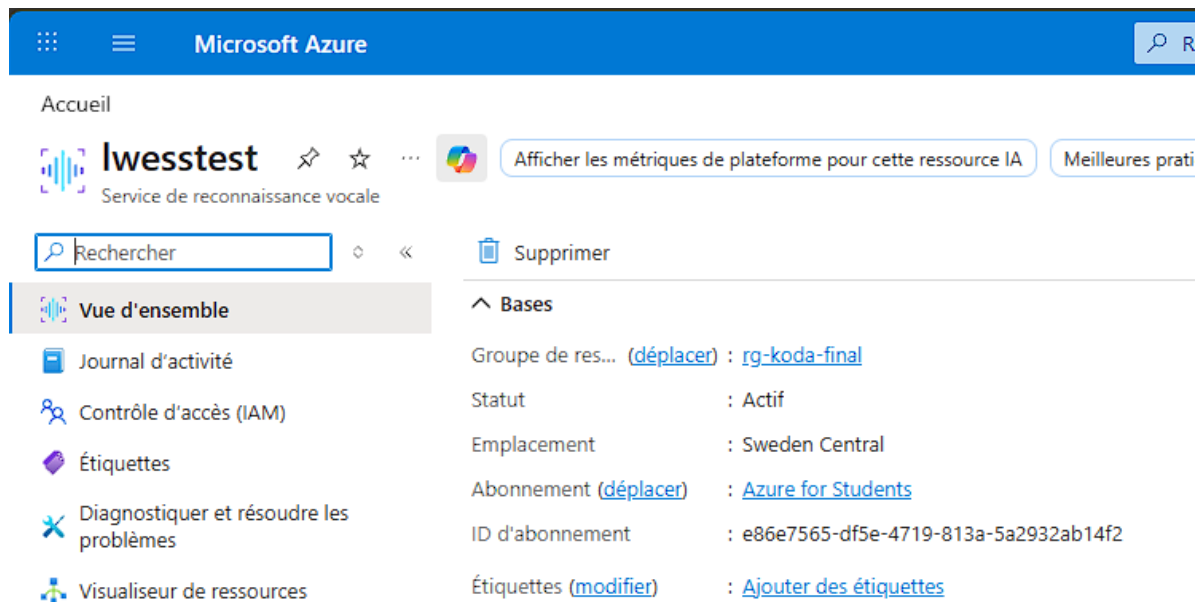


FIGURE IV.15 – Configuration Azure Speech (STT/TTS)

```

• PS C:\Users\l\Desktop\o\PFE\speatchtotext> python .\test_koda_speech.py
Koda est prêt à parler. Entre ton texte :
> bonjour je m'apelle oussema
Succès ! Koda a dit : [bonjour je m'apelle oussema ]
○ PS C:\Users\l\Desktop\o\PFE\speatchtotext>

```

FIGURE IV.16 – Test Text-to-Speech

```

• PS C:\Users\l\Desktop\o\PFE\speatchtotext> python .\koda_listen.py
=====
🔊 KODA LISTEN - Reconnaissance vocale
=====

Koda t'écoute... Parle en français !

✅ Koda a compris !

=====
📄 TEXTE RECONNU : Bonjour, je m'appelle Houssen.
=====

⌚ Affichage pendant 3 secondes...
✅ Terminé !

```

FIGURE IV.17 – Test Speech-to-Text

IV.5.1.6 Reconnaissance des personnes

La reconnaissance faciale constitue le **service d'identité** du Distributeur. Son rôle théorique est de transformer une **présence physique** devant le robot en une **identité numérique** exploitable par le reste du système : sans cette étape, KODA ne pourrait ni personnaliser la salutation, ni retrouver l'historique conversationnel, ni orienter le raisonnement n8n vers le bon profil (accompagnement, notes, préférences).

IV.5.1.6.1 Principe et modèle de données.

L'identification repose sur une **bibliothèque de visages connus** : à l'inscription (depuis l'application mobile ou un flux d'enregistrement), plusieurs photos de référence sont capturées et converties en **encodages vectoriels** (128 dimensions) stockés dans la table `imageuser`, liée à utilisateur. Lors d'une interaction, le robot transmet une capture caméra au Distributeur via `POST /api/identify-face`; le backend détecte le visage, calcule son encodage et le compare à la base de référence. L'identité retenue est celle dont la **distance euclidienne** est minimale, sous réserve d'un **seuil de confiance** (0,5 dans notre configuration).

IV.5.1.6.2 Pipeline d'identification.

Le traitement suit quatre étapes conceptuelles :

1. **Capture** : le Raspberry Pi envoie une image (multipart);
2. **Détection et encodage** : extraction du visage et calcul du vecteur 128D (`face_recognition / dlib`);
3. **Comparaison** : recherche du profil le plus proche dans PostgreSQL;
4. **Décision** : retour du username identifié et d'un score, ou de l'état `unknown` (personne inconnue) déclenchant un scénario d'accueil générique ou d'invitation à se présenter (Flux 3, chapitre III).

Ce pipeline intervient en **amont** de chaque dialogue vocal (Action 1, figure IV.5.1.2) et lors de la détection de présence (Action 2, figure IV.5.1.3) : l'identité conditionne le ton, le contenu et le routage vers n8n.

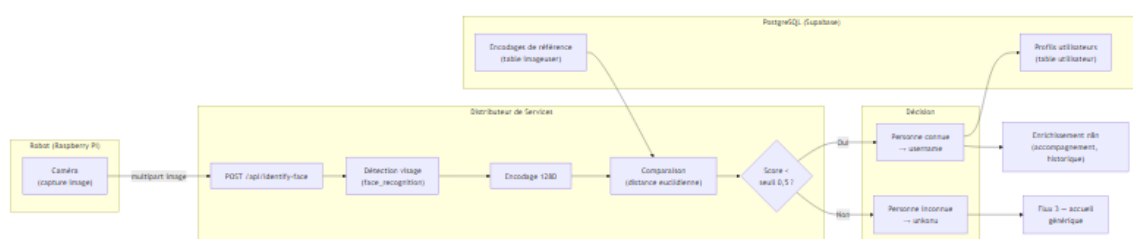


FIGURE IV.18 – Pipeline de reconnaissance faciale — de la capture caméra à l’identité utilisateur

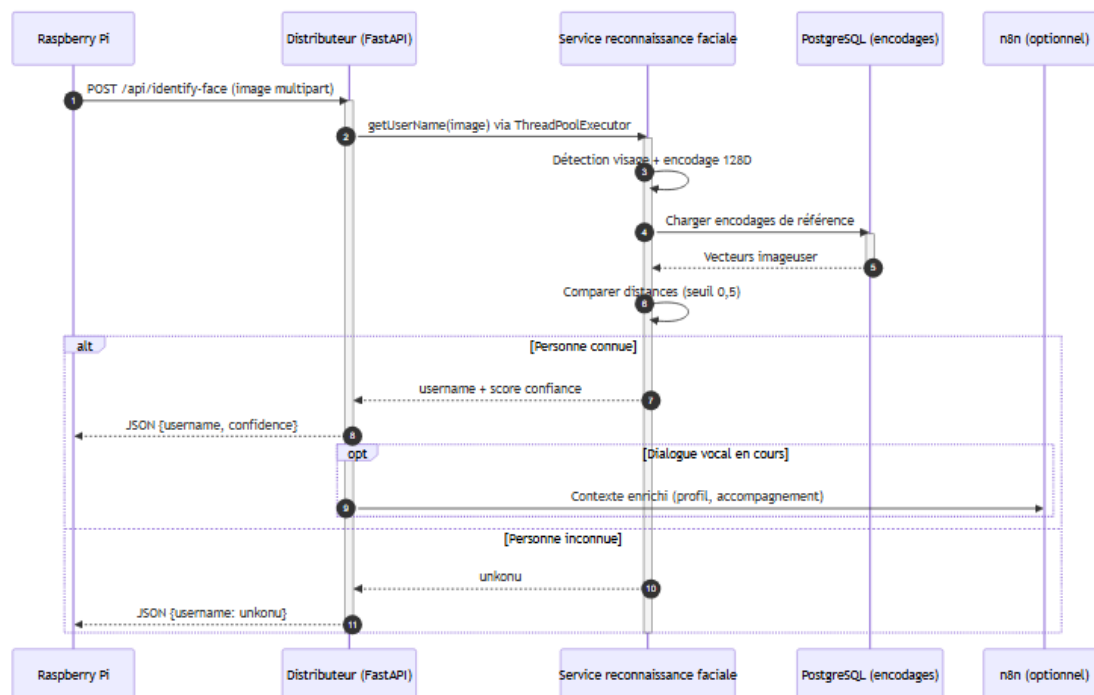


FIGURE IV.19 – Diagramme de séquence — identification faciale (robot, Distributeur, base de référence)



FIGURE IV.20 – Exemple d’images de référence en mémoire (profils utilisateurs)

fig :ch4-face-dataset

IV.5.1.6.3 Cas particuliers.

Trois situations méritent d’être distinguées :

- **Personne connue** : match sous le seuil → personnalisation immédiate (salutation par le prénom, contexte accompagnement) ;
- **Personne inconnue** : aucun match satisfaisant → état unkonu, accueil générique et possibilité d’enregistrement ultérieur ;
- **Enregistrement** : création d’un profil utilisateur et de ses encodages via les endpoints CRUD (/api/users, /api/user-images), sans intervention directe du robot sur la base.

IV.5.1.6.4 Contraintes et intégration.

Le calcul d'encodage étant **CPU-intensif**, le traitement est délégué à un `ThreadPoolExecutor` (`run_in_executor`) afin de ne pas bloquer la boucle asynchrone FastAPI (cf. tableau IV.11). Les encodages fréquents peuvent être mis en cache (Redis) pour réduire la latence sous charge. Le robot ne charge **jamais** la bibliothèque de vision : seul le Distributeur détient les modèles et les références, ce qui respecte le principe du point de liaison unique (§ IV.2).

Résultats de validation

Tests en conditions réelles : taux de reconnaissance supérieur à **94 %** (seuil de distance 0,5), temps moyen **180 ms** par image sur le serveur de déploiement.

IV.5.2 Service dédié vers l'application mobile (KodaMate)

L'application **KodaMate** (.NET MAUI 8, Cycle 5) constitue l'interface utilisateur mobile du système. Elle consomme un **sous-ensemble** des API REST du Distributeur ; elle ne contacte **jamais** directement Supabase ni n8n. Toute opération métier transite par le backend, seul habilité à accéder à la base, aux webhooks et aux clés de services cloud. Le catalogue complet des routes est donné au § IV.6 ; cette sous-section en décrit le **fonctionnement** et les **flux** principaux.

IV.5.2.1 Contexte architectural

KodaMate s'inscrit dans une architecture **client–serveur en trois couches** :

1. **Présentation** : application mobile KodaMate (.NET MAUI) sur smartphone Android (ou autre cible MAUI) ;
2. **Services** : backend Distributeur (FastAPI / Python), API REST centralisée hébergée sur le PC ou le VPS du projet ;
3. **Données** : PostgreSQL distant (Supabase), accessible **exclusivement** via le backend (ORM SQLAlchemy / asyncpg).

Cette séparation garantit la cohérence avec le pipeline vocal du robot : robot, mobile et n8n partagent la **même mémoire** PostgreSQL, mais seul le Distributeur en assure l'accès contrôlé.

IV.5.2.2 Principe de communication

L'échange s'effectue en **HTTP/JSON** sur le réseau local Wi-Fi (ou Internet en déploiement distant) :

- **URL de base** : `http://<IP-du-PC>:8000/api` (configurable) ;

- **Protocole** : REST (GET, POST, PUT) ;
- **Format** : corps et réponses en JSON ;
- **Timeout côté mobile** : 60 secondes (latences Supabase et n8n) ;
- **Validation serveur** : middleware FastAPI transversal (CORS, parsing).

Au **démarrage**, l'application vérifie la disponibilité du serveur via l'écran de connexion. Après authentification réussie, une **session locale** (email, idproduit, idkoda) est conservée dans les préférences MAUI (Preferences) pour les écrans suivants.

IV.5.2.3 Flux fonctionnels principaux

IV.5.2.3.1 Authentification.

L'utilisateur saisit email et mot de passe. L'application envoie : POST /api/auth/login avec {emailclient, motdepasse}. Le backend interroge la table d'authentification sur Supabase, valide les identifiants et renvoie {idproduit, emailclient, idkoda}. Ces données alimentent la configuration robot, le chat et l'historique.

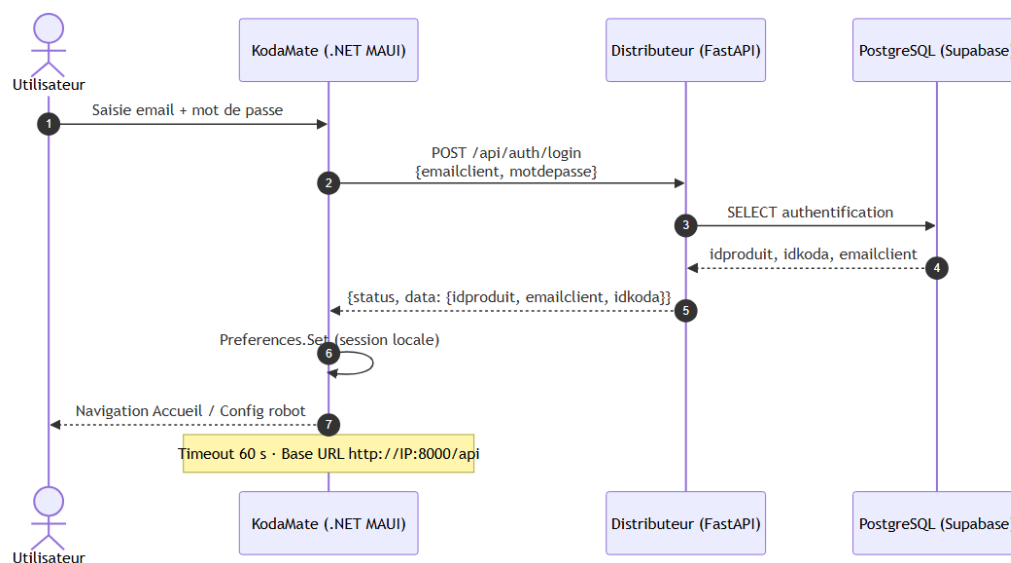


FIGURE IV.21 – Authentification mobile — séquence POST /api/auth/login

IV.5.2.3.2 Configuration et contrôle du robot.

- lecture des paramètres : GET /api/settings (nom, langue, état on/off, caractère, etc.) ;
- enregistrement : PUT /api/settings ;
- allumage / extinction : POST /api/power-on et POST /api/power-off.

L'état du robot (etat) provient de la table setting en base.

IV.5.2.3.3 Chat intelligent (délégation n8n).

Depuis l'onglet Chat, l'utilisateur envoie un message. L'application :

1. ajoute un suffixe identifiant au texte (ex. « oussama »);
2. appelle POST /api/trigger-n8n avec {question: "..."};
3. le backend transmet la requête au workflow n8n (variable N8N_WEBHOOK_URL);
4. n8n traite la question (IA, notes, contexte) et renvoie un JSON contenant le champ output;
5. KodaMate extrait et affiche uniquement le texte de output.

Le mobile **ne connaît pas** l'URL n8n : seul le Distributeur y accède, ce qui respecte le principe du point de liaison unique (§ IV.2).

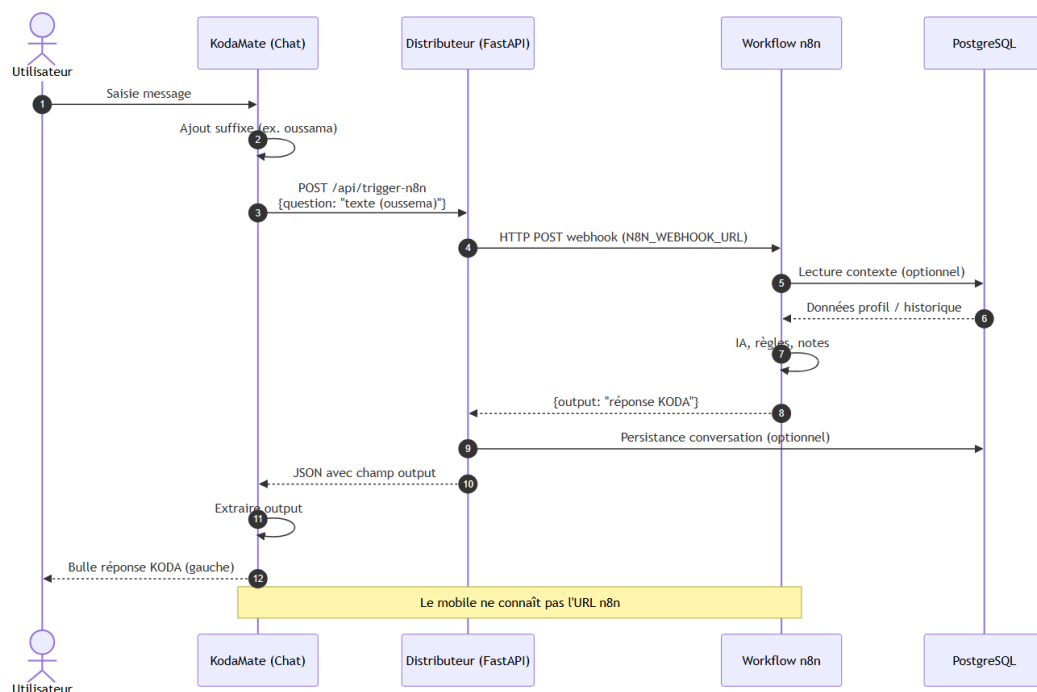


FIGURE IV.22 – Chat mobile — séquence POST /api/trigger-n8n

IV.5.2.3.4 Historique des conversations.

L'onglet Historique charge les échanges via GET /api/conversations (filtres optionnels date_from, date_to), ou en secours GET /api/conversations/last100. Les messages s'affichent en bulles (question utilisateur à droite, réponse KODA à gauche), groupés par date.

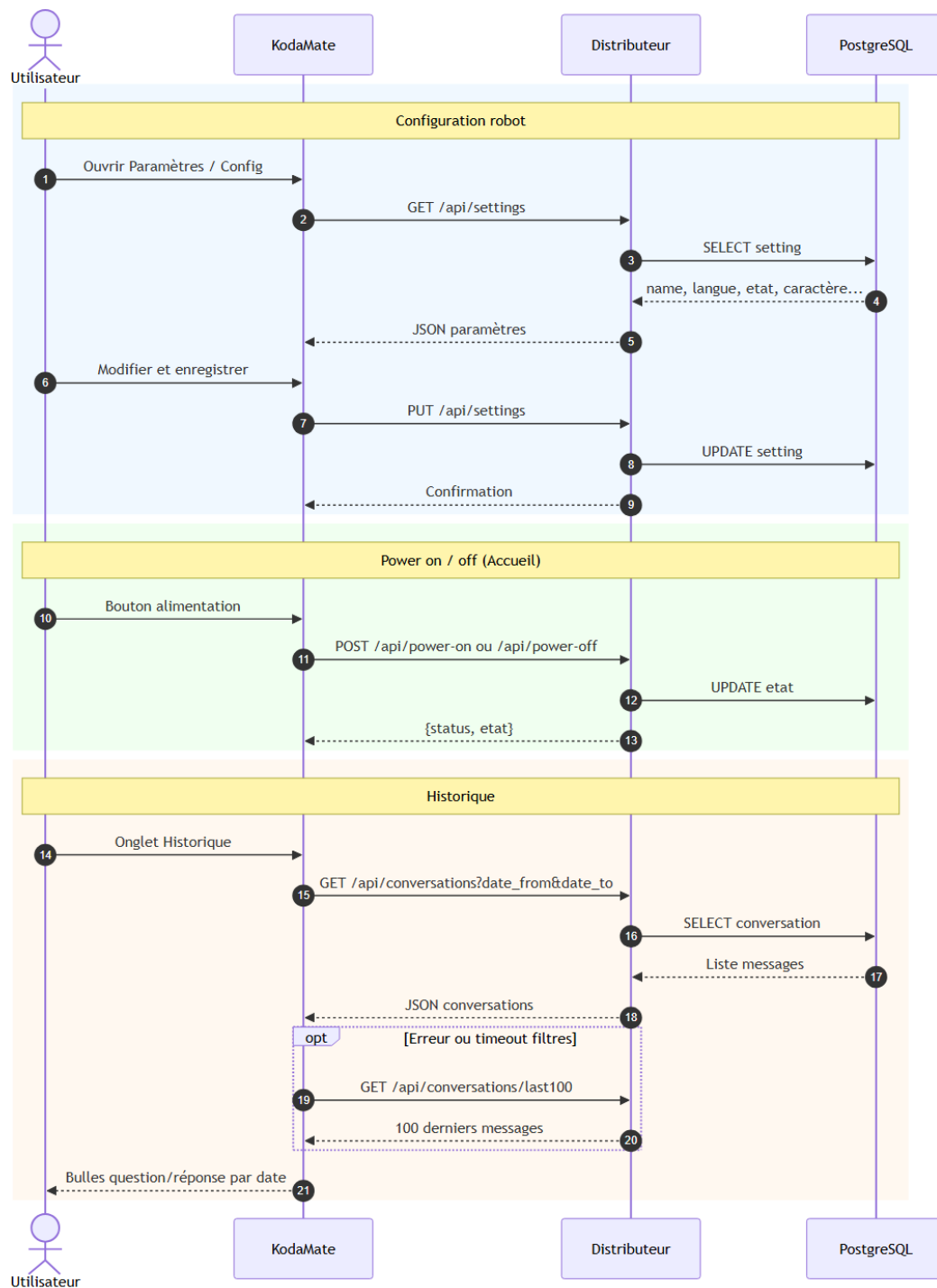


FIGURE IV.23 – Paramètres, power et historique — séquences REST KodaMate

IV.5.2.3.5 Données complémentaires (hors backend).

- **Météo** (écran Accueil) : appel direct à l'API externe Open-Meteo, sans passer par le Distributeur ;
- **Notifications** : service simulé localement (MockFirebaseService), non branché au backend à ce stade ;
- **Wi-Fi** : scan Android natif, sans endpoint backend.

```

PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/api/users" -H
"accept: application/json"
[{"id":"98a327fd-0ece-4afc-8b06-5a6499a9ce04","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/
storage/v1/object/public/photos_utilisateurs/s1.jpg"}, {"id":"d58ff93a-26d1-4785-867d-4934351cb4e2","nom":"sala
h","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s2.jpg"}, {"i
d":"798bd831-0837-4c6c-93a1-f9654f22d074","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/stor
age/v1/object/public/photos_utilisateurs/s3.jpg"}, {"id":"ec7abfae-1460-473e-a5ab-c7dd65f091ea","nom":"salah","
image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s4.jpg"}, {"id":"
2a6df721-a61d-42e7-9ee3-f0a9e9acf756","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/storage/
v1/object/public/photos_utilisateurs/s5.jpg"}, {"id":"77af3963-e661-423f-8b48-5a70f79cf668","nom":"salah","imag
e":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s6.jpg"}, {"id":"b788
4803-0039-474f-9c99-eb44d7a07063","nom":"salah","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/s7.jpg"}, {"id":"0c8f6a3c-8a3b-45e8-ac7e-191c1ee861bd","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o1.jpg"}, {"id":"ac2884
e6-b761-405a-a2a9-93f4822c52e5","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o2.jpg"}, {"id":"6360315d-0c7d-4194-a518-393018d183e7","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o3.jpg"}, {"id":"e2e5ba
d9-2b00-42c6-b59d-392cf9c10673","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o4.jpg"}, {"id":"de605ab3-4d9c-4aac-b26e-de47c141c22a","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o5.jpg"}, {"id":"51b91c
f9-8512-42d7-84e2-efe9d043bef5","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o6.jpg"}, {"id":"38e05d5f-149f-444f-874e-9ede96e85696","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o7.jpg"}, {"id":"912fb4
8b-dc88-4314-911d-e5896de7f939","nom":"oussema","image":"https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o8.jpg"}, {"id":"8a0deca6-72fe-48ce-a2af-2802c6cc90db","nom":"oussema","image"
:"https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o9.jpg"}]
PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/api/agendas"
-H "accept: application/json"
[{"id":1,"titre":"aaaa","note":"aaaa","etat":true,"contenu":"aaaa","date_modification":"2026-04-12T16:37:12.74
2374+00:00"}]
PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X POST "http://127.0.0.1:8000/api/power-on"
-H "accept: application/json"
{"status":"power_on","etat":1}
PS C:\Users\l\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/test"
{"message":"Test OK"}

```

FIGURE IV.24 – Exécution de quelques endpoints (tests manuels)

IV.5.3 Service dédié vers la base de données (PostgreSQL / Supabase)

La couche de persistance repose sur **PostgreSQL** hébergé sur **Supabase** (interface d'administration, SQL, gestion des accès). Le backend y accède via **SQLAlchemy** (ORM) et **asyncpg** (asynchrone), selon la chaîne : *API REST* → *cas d'usage* → *repository* → *PostgreSQL*. C'est la **mémoire partagée** entre interactions vocales et applicatives.



FIGURE IV.25 – Interface Supabase — administration PostgreSQL

IV.5.3.1 Modèle relationnel et rôle des tables

Le modèle couvre les dimensions conversationnelles, contextuelles et organisationnelles. Principales tables :

- **conversation** (**id, question, reponce, date, type_de_question, personne**) : archive l'ensemble des conversations entre le robot et l'humain.
- **historique** (**id, date, question, reponce, type_de_question**) : conserve les 5 derniers messages pour la mémoire contextuelle immédiate.
- **information_personnel** (**id, question, reponce, date, acteur, type_de_question**) : stocke les informations personnelles liées à l'humain ou au robot.
- **note** (**id, note, etat**) : enregistre les rendez-vous et dates importantes de l'utilisateur.
- **setting** (**id, name, sexe, date, motdepasse, etat, caractere, langue**) : centralise l'identité et les paramètres généraux de KODA.
- **utilisateur** (**id, nom, image_url**) : référence les personnes connues par KODA avec leur nom et leur image.
- **agenda** (**id, titre, note, etat, contenu, date_modification**) : enregistre les éléments d'agenda saisis depuis l'application mobile.
- **activity** (**id, titre, description, etat, date, createdat**) : stocke les activités de l'utilisateur provenant de l'application mobile.
- **accompagnement** (**id, interets, synthese, point_cles, conseils, acteur**) : conserve l'analyse quotidienne consolidée pour chaque personne connue.

Le détail des relations entre tables est donné par la figure IV.3.1 (§ IV.3).

IV.5.4 Service dédié vers le workflow n8n (Cycle 1)

Le moteur de workflow **n8n** (Cycle 1) joue le rôle d'**orchestrateur cognitif** : il reçoit une question enrichie, applique la logique de traitement et renvoie une réponse structurée. Le backend ne duplique pas cette intelligence : il **prépare le contexte** (identité, historique), **appelle le web-hook**, **valide la réponse** et **persiste** l'échange. Cette séparation permet de modifier rapidement le comportement métier dans n8n sans impacter la couche d'exposition API ni le robot.

Le scénario standard suit les étapes suivantes :

1. récupération du **nom de l'utilisateur** (ou identifiant) et de la **question** formulée ;
2. construction d'un payload JSON normalisé (ex. : `{"name": "...", "question": "..."};`);
3. envoi au webhook n8n via HTTP POST (timeout configurable) ;
4. exécution du workflow : enrichissement, appels LLM, règles de décision ;
5. retour de la **réponse finale** et métadonnées éventuelles ;

6. validation, journalisation et renvoi au robot ou à l'application cliente.

Sur le plan qualité, cette interface inclut la gestion des *timeouts*, le contrôle des erreurs de webhook et la validation du schéma de réponse.

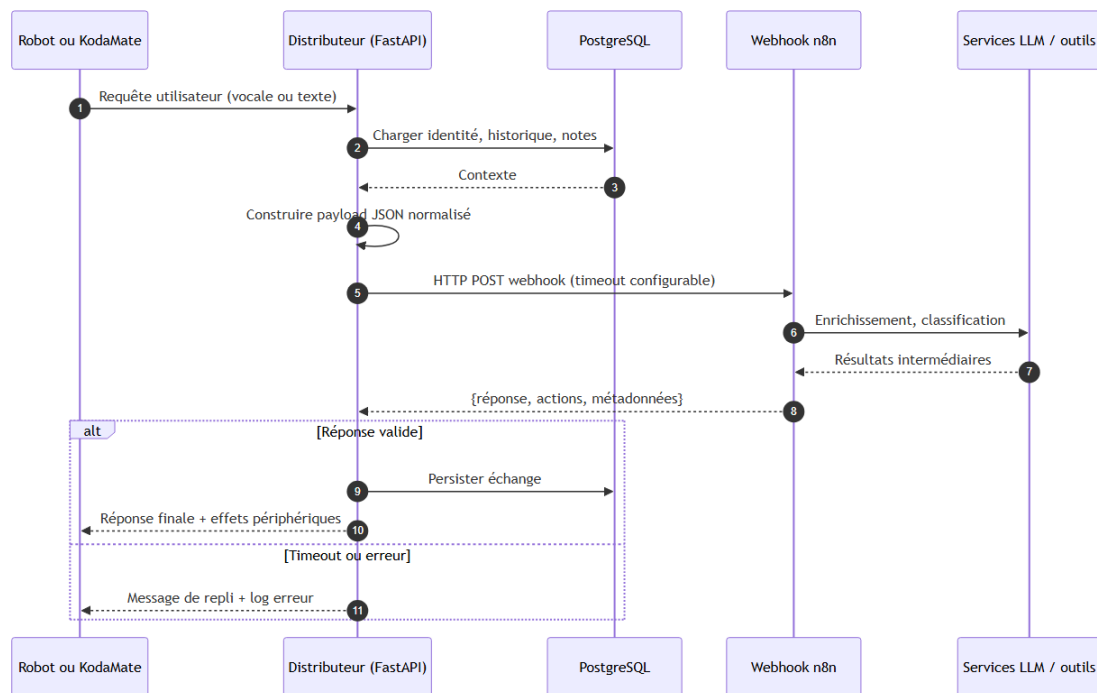


FIGURE IV.26 – Délégation au workflow n8n — séquence robot ou mobile → Distributeur → n8n

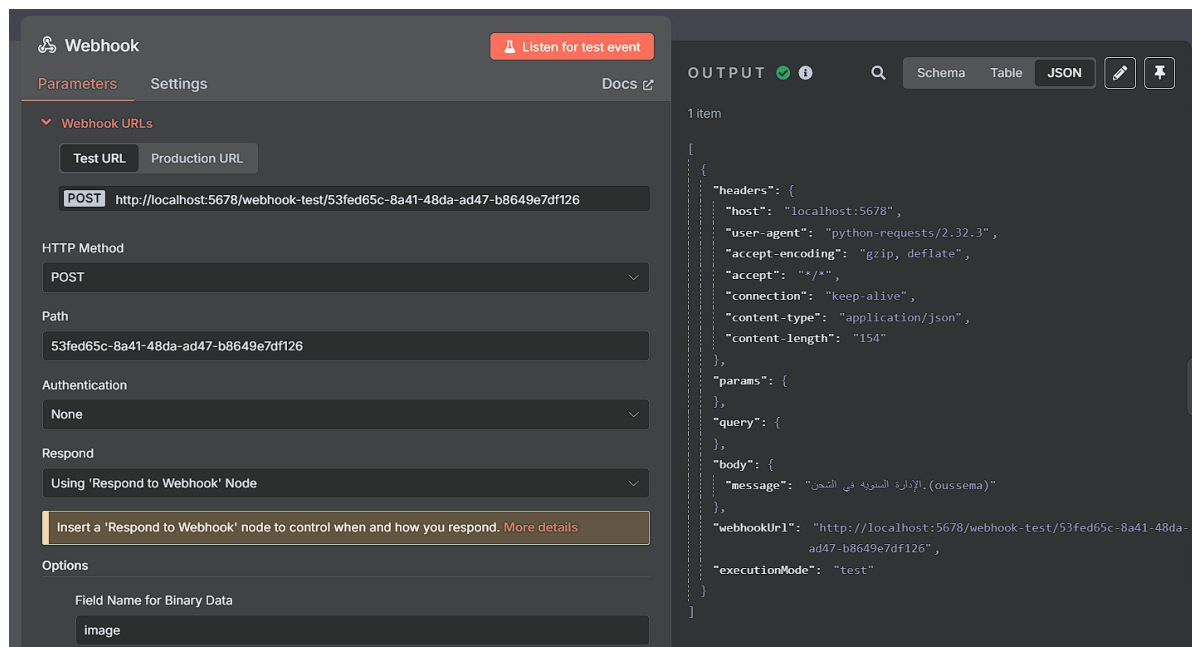


FIGURE IV.27 – Communication avec n8n — exemple de payload en entrée de webhook

IV.6 Catalogue des endpoints REST du Distributeur

Le Distributeur expose **65 routes HTTP** au total : **60 endpoints métier** sous le préfixe `/api` et **5 routes** de documentation ou de santé à la racine. Plutôt qu’une liste plate, ce catalogue les regroupe selon **l’axe de communication** qu’ils matérialisent : avec le **système embarqué**, l’**application mobile**, la **base PostgreSQL**, le **workflow n8n**, ou les outils d’**exploitation**. Certains endpoints sont **partagés** entre plusieurs clients (ex. : `/api/conversations` alimenté par le robot et consulté par KodaMate).

TABLE IV.2 – Répartition des 65 endpoints par axe de communication

Axe de communication	Nb	Rôle principal
Exploitation / documentation	5	Santé serveur, Swagger, OpenAPI (<code>/</code> , <code>/docs...</code>)
Persistance PostgreSQL (Supabase)	46	CRUD sur utilisateurs, conversations, notes, paramètres, etc.
Système embarqué (Raspberry Pi)	5	Audio Azure, reconnaissance faciale, mot-clé de réveil
Délégation n8n / IA	2	Déclenchement workflow et pipeline vocal complet
Application mobile (KodaMate)	9 actifs	Consommés à l’écran ; 5 endpoints « notes » codés mais non branchés
Total backend	65	60 métier + 5 docs/racine

Remarque : le décompte par axe peut se chevaucher : un endpoint de persistance est invoqué indifféremment par le robot, n8n (via le backend) ou l’application mobile.

IV.6.1 Infrastructure et documentation (hors `/api`)

Ces routes servent au **déploiement**, aux tests de connectivité et à la documentation interactive FastAPI.

TABLE IV.3 – Endpoints racine — exploitation et documentation

#	Méth.	Endpoint	Description
1	GET	<code>/</code>	Santé serveur (message, préfixes de routes)
2	GET	<code>/test</code>	Test simple de disponibilité
3	GET	<code>/docs</code>	Interface Swagger UI (FastAPI)
4	GET	<code>/redoc</code>	Documentation ReDoc
5	GET	<code>/openapi.json</code>	Schéma OpenAPI exportable

IV.6.2 Communication avec PostgreSQL (persistance Supabase)

Ces endpoints traduisent la couche **mémoire durable** du système. Ils sont consommés par le backend lui-même au service du robot, de n8n ou de KodaMate. La chaîne interne reste : *route REST* → *cas d'usage* → *repository* → *PostgreSQL*.

IV.6.2.1 Santé de la base

#	Méth.	Endpoint	Description
6	GET	/api/db-test	Test de connexion Supabase / PostgreSQL

TABLE IV.4 – Endpoint de diagnostic base de données

IV.6.2.2 Entités persistées (CRUD)

L'endpoint `POST /api/auth/login` (#47) appartient à la persistance des comptes tout en constituant le **point d'entrée** de l'application mobile (cf. § IV.6.5).

IV.6.3 Communication avec le système embarqué (Raspberry Pi)

Ces routes sont **primordialement** consommées par le Raspberry Pi (Cycle 4) pour la voix, la vision et le mot de réveil. Elles s'ajoutent au canal **WebSocket** (temps réel) décrit au § IV.5.

Le robot s'appuie également, de façon indirecte, sur les endpoints de **persistance** (utilisateurs, historique, conversations) lorsque n8n ou le backend enregistrent une interaction vocale.

IV.6.4 Communication avec le workflow n8n (Cycle 1)

`POST /api/trigger-n8n` est aussi invoqué par **KodaMate** depuis l'écran Chat ; c'est l'exemple le plus visible d'un endpoint **croisé** entre robot (via pipeline vocal) et application mobile (saisie texte).

IV.6.5 Communication avec l'application mobile (KodaMate)

L'application **KodaMate** (.NET MAUI) n'utilise qu'un **sous-ensemble** du catalogue : neuf endpoints backend sont effectivement appelés à l'écran, auxquels s'ajoute une API météo externe. Les endpoints « alimentation robot » (`power-on/off`) sont exclusifs au mobile ; le Wi-Fi et les notifications restent locaux (pas de route backend).

TABLE IV.5 – Endpoints CRUD — communication avec PostgreSQL par entité

#	Méth.	Endpoint	Entité / table
<i>Utilisateurs (utilisateur)</i>			
7–11	GET/POST/PUT/DELETE	/api/users, /api/users/{iduser}	Profil des personnes connues de KODA
<i>Images utilisateur (imageuser)</i>			
12–17	GET/POST/PUT/DELETE	/api/user-images..., /by-user/{iduser}	Photos et encodages faciaux
<i>Historique contextuel (historique)</i>			
18–21	GET/POST/DELETE	/api/history, /api/history/{historique_id}	Fenêtre des derniers messages (mémoire immédiate)
<i>Conversations (conversation)</i>			
22–27	GET/POST/PUT/DELETE	/api/conversations..., /last100	Archive des échanges robot–humain
<i>Notes et rendez-vous (note)</i>			
28–33	GET/POST/PUT/PATCH/DELETE	/api/notes..., /etat?etat=	Agenda, rappels, bascule d'état
<i>Paramètres robot (setting)</i>			
34–35	GET/PUT	/api/settings	Identité KODA (nom, langue, caractère, état)
<i>Informations personnelles</i>			
36–40	GET/POST/PUT/DELETE	/api/personal-info..., /user/{iduser}	Faits mémorisés sur l'utilisateur
<i>Accompagnements</i>			
41–46	GET/POST/PUT/DELETE	/api/accompagnements..., /user/{iduser}	Synthèses et conseils quotidiens
<i>Inscriptions produit (auth/registrations)</i>			
47–52	POST/GET/PUT/DELETE	/api/auth/login, /api/auth/registrations...	Authentification et comptes KodaMate
<i>Activités (alias notes)</i>			
54–58	GET/POST/PUT/DELETE	/api/activities...	Activités utilisateur saisies depuis le mobile

TABLE IV.6 – Endpoints orientés robot — audio, vision et mot-clé

#	Méth.	Endpoint	Description
59	POST	/api/identify-face	Reconnaissance faciale (multipart, fichier image)
62	POST	/api/audio/speech-to-text	Transcription Azure STT
63	POST	/api/audio/text-to-speech	Synthèse Azure TTS
64	POST	/api/audio/speech-to-n8n-to-speech	Pipeline vocal complet (STT → n8n → TTS)
65	GET	/api/keywords/robot-name	Mot-clé / nom de réveil du robot

TABLE IV.7 – Endpoints de délégation au moteur n8n

#	Méth.	Endpoint	Description
53	POST	/api/trigger-n8n	Envoi {question} au webhook n8n, retour {output}
64	POST	/api/audio/speech-to-n8n-to-speech	Variante vocale intégrée (STT + n8n + TTS)

TABLE IV.8 – Endpoints backend effectivement utilisés par KodaMate (UI)

#	Méth.	Endpoint	Écran	Entrée	Sortie
1	GET	/	Démarrage, Login, session	—	{message, routes_prefix}
2	POST	/api/auth/login	Connexion	emailclient, motdepasse	{status, data:{idproduit, idkoda}}
3	GET	/api/settings	Accueil, Paramètres, Config	—	nom, langue, caractère, etat, idkoda...
4	PUT	/api/settings	Paramètres, Config robot	corps setting	lignes mises à jour
5	POST	/api/power-on	Accueil (power)	—	{status: power_on, etat: 1}
6	POST	/api/power-off	Accueil (power)	—	{status: power_off, etat: 0}
7	POST	/api/trigger-n8n	Chat (+ photo)	{question}	{output:...}
8	GET	/api/conversations?date_from={date}&date_to={date}	Historique (date)	dates YYYY-MM-DD	liste conversations
9	GET	/api/conversations/latest	Historique (repli)	—	100 derniers messages max

IV.6.5.1 Endpoints consommés à l'écran

IV.6.5.2 Endpoints codés mais non branchés à l'interface

Cinq routes `/api/notes` sont implémentées dans `DistributeurApiService` mais aucun écran ne les invoque encore : CRUD complet + PATCH `.../etat`.

IV.6.5.3 API externe et services locaux

TABLE IV.9 – Appels réseau complémentaires de KodaMate (hors Distributeur)

#	Méth.	URL / service	Écran
15	GET	<code>api.open-meteo.com/v1/forecast...</code>	Accueil (météo)
—	—	<code>MockFirebaseService</code> (local)	Notifications
—	—	<code>Scan Wi-Fi Android</code> (local)	Paramètres réseau

IV.6.5.4 Cartographie écran → endpoints

TABLE IV.10 – Mapping des écrans KodaMate vers les endpoints backend

Écran	Endpoints invoqués
Démarrage / Session	GET <code>/</code> , GET <code>/api/settings</code>
Login	GET <code>/</code> , POST <code>/api/auth/login</code>
Config robot	GET <code>/api/settings</code> , PUT <code>/api/settings</code>
Accueil	GET <code>/api/settings</code> , POST <code>/api/power-on</code> , POST <code>/api/power-off</code> , Open-Meteo
Chat	POST <code>/api/trigger-n8n</code>
Historique	GET <code>/api/conversations</code> , GET <code>/api/conversations/last100</code>
Paramètres	GET <code>/api/settings</code> , PUT <code>/api/settings</code>

Synthèse de consommation — KodaMate

- **65** endpoints backend au total (60 métier + 5 documentation) ;
- **9** endpoints backend actifs à l'écran + **1** API externe (météo) ;
- **5** endpoints `/api/notes` prêts dans le code client, non branchés ;
- **56** routes backend réservées au robot, à l'audio, aux CRUD avancés ou à une évolution future de l'UI mobile.

IV.7 Problèmes rencontrés et solutions apportées

Le développement et l'intégration du backend ont révélé des difficultés **techniques** (performance, concurrence) et **d'intégration** (réseau, CORS). Le tableau IV.11 synthétise les principaux cas ; les

deux sous-sections suivantes détaillent les problèmes de connectivité les plus impactants pour la chaîne robot–backend–mobile.

TABLE IV.11 – Problèmes techniques rencontrés et solutions apportées

Problème rencontré	Impact	Solution adoptée
Latence élevée de la reconnaissance faciale sous charge	Blocage du thread principal FastAPI	Utilisation de <code>run_in_executor</code> pour déléguer le traitement au <code>ThreadPoolExecutor</code>
Déconnexions intempestives du WebSocket robot	Perte de commandes, état incohérent	Reconnexion automatique avec <i>backoff</i> exponentiel
Conflits de concurrence sur la base de données	Doublons de sessions et d'événements	Transactions atomiques et <code>SELECT FOR UPDATE</code>
Taille des encodages faciaux (128 flottants / profil)	Comparaisons massives lentes	Indexation vectorielle et cache Redis pour les encodages fréquents
Quota Azure STT dépassé lors des tests intensifs	Interruption du service vocal	Système de quota local et <i>fallback</i> Whisper (OpenAI)
Conflit d'adresses serveur (Raspberry Pi vs backend)	Robot injoignable ou connecté au mauvais hôte (<code>localhost</code> , IP obsolète)	Fichier de configuration centralisé (<code>.env</code>) avec URL publique ou IP LAN du VPS ; distinction explicite URL backend / URL n8n
Erreurs CORS (backend vs application mobile)	Requêtes MAUI / client HTTP bloquées (politique CORS)	<code>CORSMiddleware</code> FastAPI avec liste d'origines autorisées (émulateur, device, production)

IV.7.1 Conflit d'adressage Raspberry Pi / backend

Lors des premiers tests d'intégration, le Raspberry Pi était configuré avec `http://localhost:8000` ou une adresse IP de poste de développement devenue inaccessible après déploiement du backend sur un VPS. Symptômes : WebSocket robot en échec permanent, pipeline vocal silencieux, commandes non acquittées.

Analyse : le robot, le backend et n8n possèdent chacun une URL distincte ; confondre l'adresse du poste local avec celle du serveur de production crée un **conflit de routage** invisible côté logs n8n (le workflow n'est jamais appelé car la requête n'atteint pas le backend).

Solution : externalisation de l'URL backend dans un fichier `.env` partagé entre environnements (dev LAN, staging, production), tests de connectivité (`curl` / ping WebSocket) avant chaque session robot, et documentation d'une **base URL unique** consommée par l'embarqué (Cycle 4).

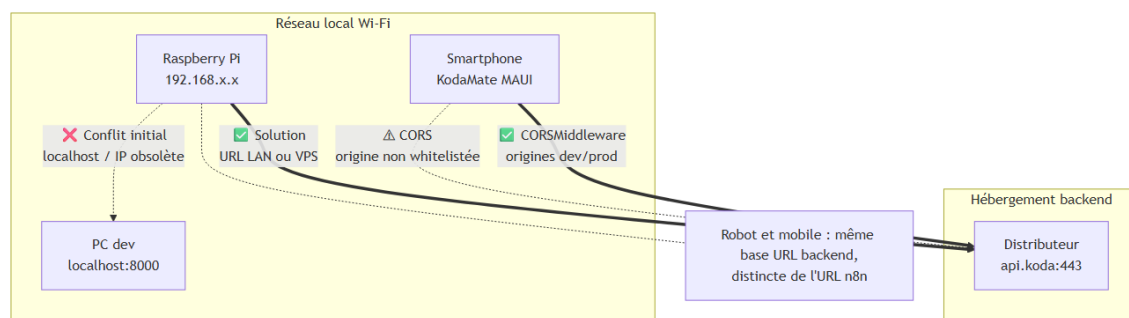


FIGURE IV.28 – Conflit d’adressage et CORS — topologie réseau et solutions

IV.7.2 Problème CORS entre backend et application mobile

L’application KodaMate (.NET MAUI), exécutée sur émulateur Android ou appareil physique, envoie des requêtes HTTP cross-origin vers le backend FastAPI. Sans en-têtes `Access-Control-Allow-Origin` adaptés, le client rejette les réponses (« blocked by CORS policy ») même si le serveur traite correctement la requête.

Solution : configuration explicite du `CORSMiddleware` : origines autorisées (`http://localhost:*`, IP LAN du poste de dev, domaine de production), méthodes (`GET`, `POST`, `PUT`, `DELETE`, `OPTIONS`) et en-têtes (`Authorization`, `Content-Type`). En environnement de production, la liste est restreinte aux domaines KodaMate déployés.

Un défi transversal a également concerné la gestion des dépendances circulaires entre modules dans une architecture Clean. Python, contrairement à Java, n’impose pas nativement l’inversion de dépendances, ce qui peut conduire à des couplages non voulus. Ce problème a été résolu par l’utilisation systématique d’interfaces (classes abstraites Python) et d’un conteneur d’injection de dépendances (`dependency_injector`).

IV.8 Perspectives et évolutions futures

Le Cycle 2 pose une **fondation** ; plusieurs axes d’évolution sont identifiés pour les cycles suivants et la mise en production :

- **Observabilité** : métriques Prometheus, traçage des appels n8n/Azure et tableaux de bord de disponibilité du backend ;
- **Résilience vocale** : bascule automatique STT/TTS multi-fournisseurs selon quota et latence ;
- **Sécurité renforcée** : rotation des JWT, rate limiting sur les endpoints publics, chiffrement des flux WebSocket en production ;
- **Scalabilité** : déploiement multi-instances derrière un reverse proxy, avec sessions WebSocket « sticky » ;

- **Alignement mobile–robot** : notifications push depuis le backend lorsqu'un événement agenda modifié dans l'app doit être annoncé vocalement par KODA (Cycle 5).

Ces perspectives s'appuient sur la **flexibilité** intrinsèque du Distributeur : tant que les contrats d'interface (REST, WebSocket, webhook) restent stables, les composants amont et aval peuvent évoluer sans remise en cause du cœur du système.

IV.9 Conclusion

Le Cycle 2 a permis de concevoir, de développer et de valider le composant le plus structurant de l'écosystème KODA : le **Distributeur de Services**. Ce backend Python / FastAPI, organisé selon la Clean Architecture, **matérialise la fonctionnalité globale** du système en offrant au robot et à l'application mobile un accès unifié aux services vocaux, à la persistance, au moteur n8n et aux APIs cloud.

L'approche retenue — **un seul pivot** plutôt que des liaisons directes entre composants — garantit la **flexibilité** des évolutions (spirale de développement) et la **disponibilité** des échanges (validation, retry, journalisation). Les résultats obtenus (reconnaissance faciale >94 %, chaîne Azure STT/TTS, persistance relationnelle, API mobile) constituent une base testable sur laquelle s'appuient les cycles embarqué (III–IV) et mobile (V).

Les difficultés d'intégration (adressage réseau Raspberry Pi, CORS KodaMate) ont mis en évidence l'importance d'une configuration d'environnement rigoureuse ; elles sont documentées pour faciliter le déploiement futur. Le chapitre suivant (Cycle 3) traite du **matériel embarqué** sur lequel le backend vient s'interfacer physiquement.

Points clés du Cycle 2

- Distributeur = pivot unique pour flexibilité et disponibilité
- Services backend = fonctionnalité globale (vocal + application + mémoire)
- Catalogue de **65 endpoints REST** répartis par axe de communication
- KodaMate : 9 endpoints actifs + 5 notes préparés + 1 API météo externe
- Flux détaillés : WebSocket robot, pipeline vocal, REST mobile, webhook n8n
- Problèmes intégration : conflit adresses serveur, CORS mobile
- Reconnaissance faciale >94 % ; Azure STT/TTS ; persistance Supabase

———— CHAPITRE V ————

CYCLE 3 : SYSTÈME EMBARQUÉ — PARTIE MATÉRIELLE

V.1 Introduction

Après le **Cycle 1** (moteur de réflexion n8n) et le **Cycle 2** (Distributeur de Services), le **Cycle 3** porte sur le **corps matériel** de KODA : choix des composants, câblage, alimentation et validation physique du prototype.

Ce chapitre reste volontairement centré sur le **hardware** et les **concepts d'intégration**. Le logiciel embarqué (scripts Raspberry Pi, protocoles série, audio, caméra) fera l'objet du **Cycle 4**.

Nous présentons l'architecture globale, la **conception 3D du châssis** (Blender), chaque sous-système (contrôle, actionneurs, audio, vision, affichage, toucher et orientation inertielle), le schéma de câblage, puis une campagne de **tests unitaires par composant** suivie d'un **test d'intégration** complète de la plateforme.

Objectif du Cycle 3

Concevoir, assembler et valider la plateforme matérielle du robot KODA, intégrant mobilité, interaction vocale, vision et expression faciale, prête à accueillir la couche logicielle embarquée du Cycle 4.

Après la première version matérielle, deux améliorations ont été intégrées au prototype : deux capteurs de touche pour introduire un **sens tactile**, et un gyroscope MPU5060 branché sur l'Arduino pour renforcer le contrôle de la rotation. Ces éléments sont documentés dans ce cycle car ils modifient directement le corps physique et le câblage du robot.

V.2 Architecture Matérielle Globale

L'architecture matérielle de KODA repose sur une conception **biprocasseur** articulée autour de deux unités de traitement complémentaires :

- **Raspberry Pi 4 Model B** : unité centrale intelligente, responsable du traitement des flux audio et vidéo, de la communication réseau avec le backend, et du pilotage de l'écran d'expression faciale.
- **Arduino Uno + Shield L293D** : unité de commande temps réel, dédiée au contrôle des actionneurs (servomoteurs et moteurs DC), recevant ses instructions du Raspberry Pi via une liaison série USB.

Cette séparation est motivée par un principe d'ingénierie fondamental : le Raspberry Pi, fonctionnant sous un système d'exploitation Linux, n'offre pas de garanties temps réel strictes pour le pilotage de signaux PWM. L'Arduino, en revanche, exécute un programme monotâche cadencé à

16 MHz, garantissant une précision temporelle adaptée au contrôle moteur.

Le tableau V.1 synthétise l'ensemble des composants matériels du robot.

TABLE V.1 – Inventaire des composants matériels de KODA

Catégorie	Composant	Rôle
Unité centrale	Raspberry Pi 4 Model B	Traitement IA, réseau, audio/vidéo
Commande moteur	Arduino Uno + Shield L293D	Pilotage PWM des actionneurs
Mobilité	2 moteurs DC (M1, M3)	Déplacement du robot (traction différentielle)
Articulations	3 servomoteurs (S1, S2, S3)	Bras droit, bras gauche ; S3 = écran Nextion
Entrée audio	ReSpeaker 4-Mic Array USB	Capture vocale + direction de la voix
Sortie audio	Jack + PAM8403 + HP ; Bluetooth	Restitution filaire ou sans fil
Vision	Caméra Raspberry Pi (CSI)	Acquisition d'images pour identification
Affichage	Nextion NX4832T035_011 (3,5")	Visage animé du robot
Sens tactile	2 capteurs de touche capacitifs	Détection du toucher humain et déclenchement d'une réaction de chatouillement
Orientation inertielle	Gyroscope / IMU MPU5060	Mesure de rotation pour améliorer la précision d'orientation du robot
Alimentation Pi	Power Bank 5V / 3A	Alimentation du Raspberry Pi
Alimentation moteurs	3 × piles 3,7V	Alimentation du Shield L293D

V.3 Conception mécanique et châssis 3D (Blender)

Au-delà du câblage électronique, KODA repose sur un **habillage mécanique** imprimé en 3D, conçu sous **Blender**. Le point de départ n'était pas une création entièrement *from scratch*, mais un **modèle 3D existant** (humanoid simplifié) que nous avons dû **adapter** au prototype réel : découpe des volumes, redimensionnement des pièces, ajout d'ouvertures manquantes (logement écran Nextion, passage caméra sur le torse, fixation des bras) et harmonisation avec la base à chenilles.

Cette étape a constitué un **vrai défi** du Cycle 3 : chaque modification impacte l'encombrement interne (câbles, Raspberry Pi, batterie) et la cohérence visuelle du robot. La figure V.1 illustre la vue éclatée des pièces modélisées ; la figure V.20 montre le résultat physique après impression et assemblage.

Choix mécaniques retenus sur le prototype :

- **Caméra** : fixée sur le **torse** (corps), face avant, indépendante du servo S3.
- **Écran Nextion** : intégré dans la **tête**, entraîné par le servo S3 (rotation du visage animé seulement).
- **Base** : châssis à chenilles pour la mobilité (moteurs M1/M3).

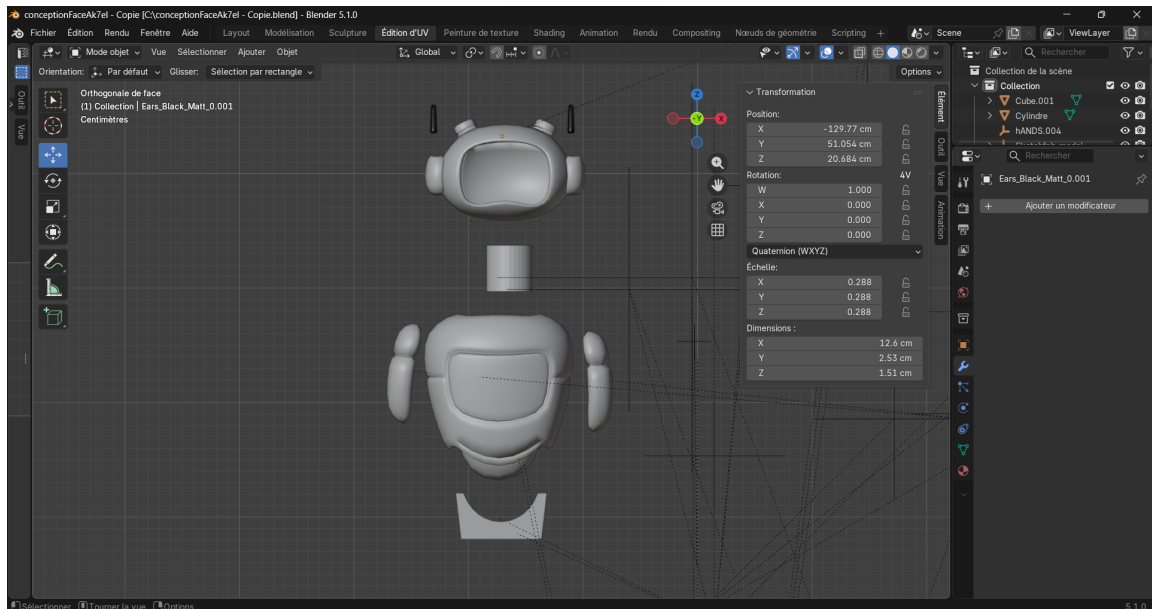


FIGURE V.1 – Conception 3D du châssis KODA sous Blender (vue éclatée des pièces)

V.4 Description Détaillée des Composants

V.4.1 Unité Centrale : Raspberry Pi 4 Model B

Le **Raspberry Pi 4 Model B** constitue le cœur du robot KODA. Il s'agit d'un ordinateur monocarte (SBC — *Single Board Computer*) basé sur un processeur ARM Cortex-A72 quadricœur cadencé à 1,5 GHz, équipé de 4 Go de mémoire RAM LPDDR4.

Dans l'architecture de KODA, le Raspberry Pi assure les fonctions suivantes :

- **Communication réseau** : connexion Wi-Fi au backend (Distributeur de Services) pour l'échange de données avec les workflows n8n et la base de données.
- **Traitement audio** : ReSpeaker en entrée (USB) ; sortie soit via le **jack 3,5 mm** et l'amplificateur PAM8403, soit via **Bluetooth** vers un haut-parleur externe.
- **Traitement vidéo** : acquisition d'images via la caméra pour la reconnaissance faciale.
- **Lecture des capteurs** : acquisition des états des capteurs de touche via GPIO ; le gyroscope MPU5060 est lu par l'Arduino, puis exploité à travers la liaison USB série.
- **Pilotage de l'écran Nextion** : communication série UART pour l'affichage des expressions faciales.

- **Commande de l'Arduino** : envoi d'instructions de mouvement via la liaison série USB.



FIGURE V.2 – Raspberry Pi 4 Model B utilisé comme unité centrale de KODA

TABLE V.2 – Spécifications techniques du Raspberry Pi 4 Model B

Caractéristique	Valeur
Processeur	Broadcom BCM2711, Cortex-A72 quad-core @ 1,5 GHz
Mémoire	4 Go LPDDR4
Connectivité	Wi-Fi 802.11ac, Bluetooth 5.0
Ports USB	2× USB 3.0 + 2× USB 2.0
Sortie audio	Jack 3,5 mm stéréo
Interface caméra	Port CSI (Camera Serial Interface)
GPIO	40 broches (dont UART, I2C, SPI)
Alimentation	5V / 3A via USB-C

V.4.2 Unité de Commande Moteur : Arduino Uno + Shield L293D

L'**Arduino Uno** est une carte de développement à microcontrôleur ATmega328P, cadencé à 16 MHz. Il est couplé au **Shield L293D**, un circuit intégré de pilotage moteur (*motor driver*) qui permet de contrôler simultanément jusqu'à 4 moteurs DC et 2 servomoteurs.

Dans KODA, l'Arduino reçoit ses instructions du Raspberry Pi via une **liaison série USB** et pilote :

- **2 moteurs DC** (bornes **M1** et **M3** du shield) : déplacement par traction différentielle.
- **3 servomoteurs** :
 - **S1** : bras droit
 - **S2** : bras gauche
 - **S3** : tête / cou (rotation de l'écran Nextion uniquement)

Le Shield L293D est alimenté indépendamment par **3 piles lithium de 3,7 V** connectées en série, fournissant une tension nominale d'environ 11,1 V nécessaire au fonctionnement des moteurs DC sous charge.



FIGURE V.3 – Carte Arduino Uno (microcontrôleur ATmega328P à 16 MHz)



FIGURE V.4 – Shield L293D empilé sur l'Arduino Uno (commande des moteurs DC et servomoteurs)

V.4.3 Système d'Actionneurs

V.4.3.1 Moteurs DC — Mobilité

Les deux moteurs à courant continu assurent la **locomotion** du robot par un système de **traction différentielle** : en faisant varier indépendamment la vitesse et le sens de rotation de chaque moteur, le robot peut avancer, reculer, tourner à gauche ou à droite. Ce principe simple et éprouvé est largement utilisé en robotique mobile (figure V.6).



FIGURE V.5 – Moteur DC utilisé pour la mobilité du robot (bornes M1 et M3 du shield)

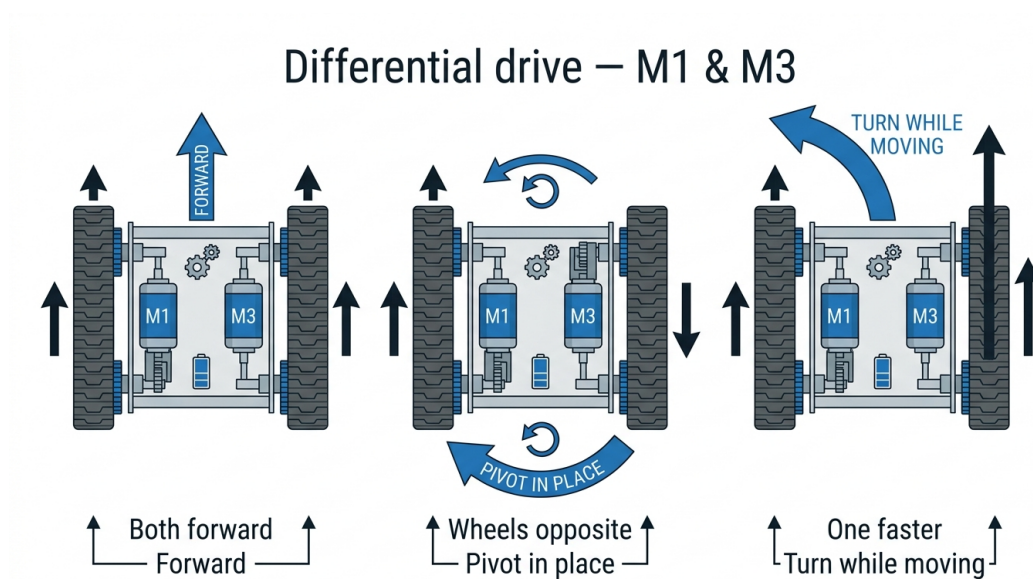


FIGURE V.6 – Principe de traction différentielle — moteurs M1 et M3

V.4.3.2 Servomoteurs — Articulations

Les trois servomoteurs offrent à KODA des capacités d'**expression corporelle** :

- **Servomoteur S1 (bras droit) et S2 (bras gauche)** : permettent de simuler des gestes d'accueil, de salutation ou d'indication directionnelle.

- **Servomoteur S3 (tête / cou)** : oriente uniquement l'écran **Nextion** (visage animé). La **caméra** est fixée sur le **châssis** (corps) du robot et ne suit pas ce mouvement.



FIGURE V.7 – Servomoteur utilisé pour les articulations (bras S1/S2 et rotation Nextion S3)

V.4.4 Système Audio

V.4.4.1 Entrée Audio : ReSpeaker 4-Mic Array USB

Le **ReSpeaker 4-Mic Array** de Seeed Studio est un périphérique USB intégrant quatre microphones MEMS disposés en configuration circulaire. Cette disposition permet deux fonctionnalités essentielles :

- **Capture vocale omnidirectionnelle** : enregistrement de la voix de l'utilisateur quelle que soit sa position autour du robot.
- **Détection de la Direction d'Arrivée (DOA — *Direction of Arrival*)** : identification de l'angle d'où provient la voix. Le robot **pivote sur place** grâce aux **deux moteurs DC** (M1, M3) pour s'orienter vers l'interlocuteur, puis la **caméra** (fixée sur le corps) permet la détection du visage (figure V.10).

Le ReSpeaker est connecté au Raspberry Pi via un **port USB** et est reconnu comme un périphérique audio standard sous Linux.



FIGURE V.8 – ReSpeaker 4-Mic Array USB : quatre microphones MEMS pour capture omnidirectionnelle et détection de direction (DOA)

V.4.5 Capteurs Sensoriels Ajoutés : toucher et orientation

V.4.5.1 Gyroscope MPU5060 pour la rotation

Le **MPU5060** est ajouté pour améliorer la précision de l'orientation du robot pendant les rotations. Sans retour inertiel, une rotation commandée par durée reste approximative : l'angle réellement parcouru dépend de la charge des piles, du frottement des chenilles, de l'inertie du châssis et de l'état du sol. Le rôle du gyroscope est donc de fournir une mesure physique de la rotation effectuée, afin de comparer le mouvement réel avec l'angle demandé.

Le scénario matériel retenu est le suivant. Le ReSpeaker estime l'angle de provenance de la voix grâce à la DOA. Cet angle devient une consigne de rotation pour la base mobile : le Raspberry Pi transmet la consigne à l'Arduino par **USB série**. L'Arduino pilote alors les moteurs **M1** et **M3**, lit le MPU5060 pendant la rotation, puis arrête ou corrige le mouvement lorsque l'angle réellement parcouru se rapproche de la consigne. La caméra, fixée sur le torse, se retrouve ainsi mieux orientée vers la personne détectée.

Le module MPU5060 n'est donc pas câblé directement au Raspberry Pi. Il est branché sur l'Arduino via le bus **I2C** de la carte (**SDA** sur A4, **SCL** sur A5 sur Arduino Uno), avec une alimentation fournie par l'Arduino et une masse commune. Ce choix place la mesure inertielle au plus près de la commande moteur temps réel : le Raspberry envoie un angle cible, tandis que l'Arduino ferme localement la rotation avec le retour du gyroscope.

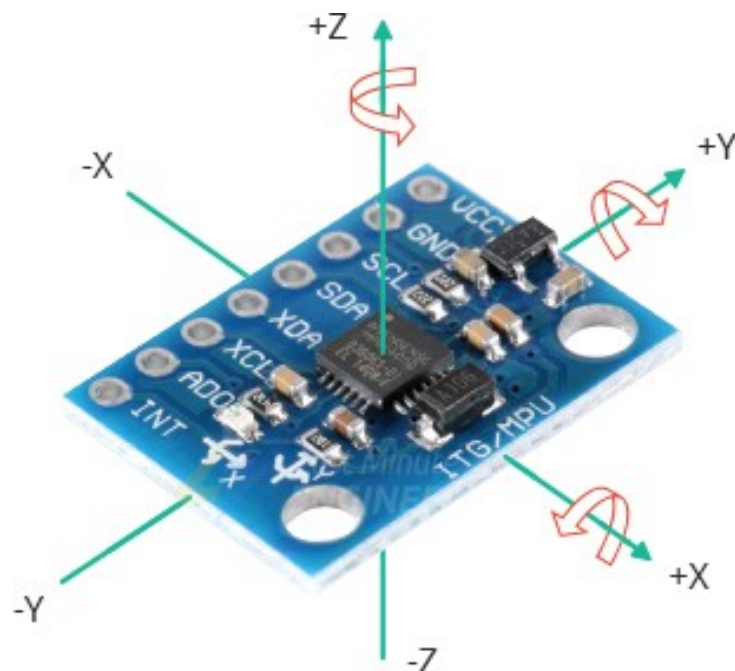


FIGURE V.9 – Module gyroscopique MPU5060 utilisé pour améliorer la précision de rotation

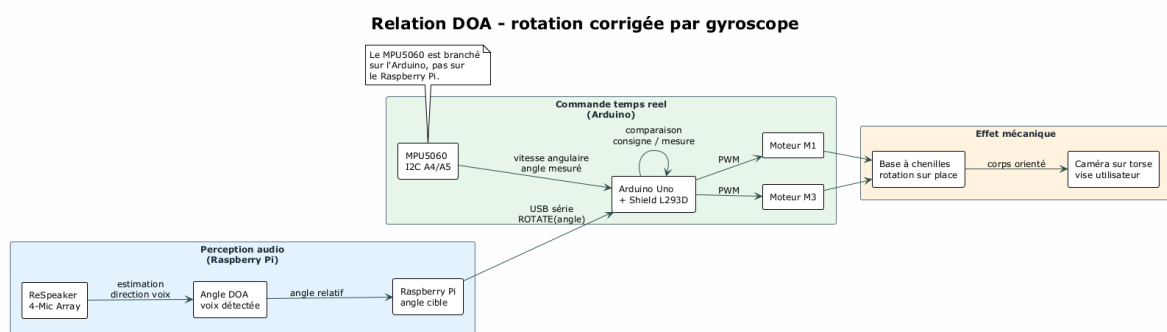


FIGURE V.10 – Relation conceptuelle : DOA du ReSpeaker → consigne Raspberry → Arduino (M1, M3 + MPU5060) → caméra du torse orientée

V.4.5.2 Capteurs de touche capacitifs

Deux **capteurs de touche capacitifs** sont ajoutés afin de donner au robot un sens tactile simple. Chaque capteur fournit une sortie numérique (*touché / non touché*) reliée directement à une entrée GPIO du Raspberry Pi. Le premier capteur est câblé sur **GPIO 17** et le second sur **GPIO 27**, avec une alimentation logique en **3,3 V** et une masse commune.

Ces capteurs ne participent pas à l'orientation du robot. Leur rôle est de détecter un contact physique volontaire de l'utilisateur. Dans la couche logicielle du Cycle 4, ce signal peut devenir une interruption prioritaire : KODA arrête l'action en cours, change d'expression et déclenche une réaction de rire pour simuler une réponse de chatouillement.

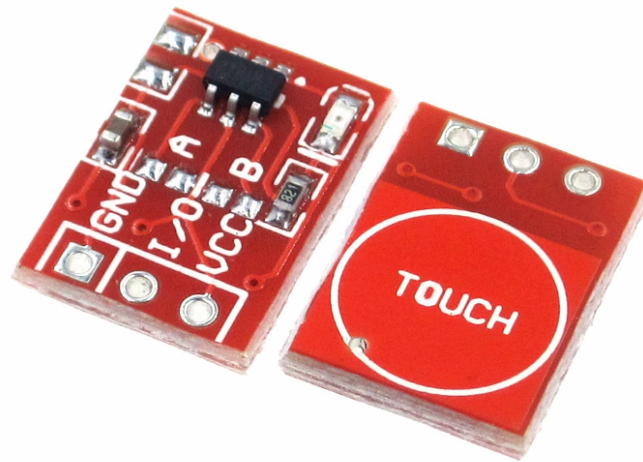


FIGURE V.11 – Capteurs de touche capacitifs ajoutés pour le sens tactile du robot

V.4.6 Sortie Audio : double chaîne (jack filaire et Bluetooth)

KODA dispose de **deux modes de restitution** de la parole, tous deux pilotés depuis le Raspberry Pi :

- **Chaîne filaire** : sortie **jack 3,5 mm** du Pi → amplificateur **PAM8403** → deux haut-parleurs (gauche / droite). Solution compacte, intégrée au châssis.
- **Chaîne sans fil** : le Bluetooth intégré du Raspberry Pi 4 peut envoyer le flux audio vers un **haut-parleur Bluetooth** (appairage classique). Utile pour les essais ou une sortie plus puissante sans modifier le câblage jack.

Le choix entre les deux dépend du contexte de démonstration ; la couche logicielle (Cycle 4) sélectionnera la carte audio ALSA appropriée.



FIGURE V.12 – Amplificateur audio PAM8403 (chaîne filaire entre la sortie jack du Pi et les haut-parleurs)



FIGURE V.13 – Paire de haut-parleurs gauche/droite alimentés par le PAM8403

V.4.7 Système de Vision : Caméra Raspberry Pi

La **caméra officielle Raspberry Pi** est connectée via le port **CSI** (nappe dédiée). Elle fournit les images nécessaires à l'**identification visuelle** des utilisateurs.

Au niveau matériel, seule la **qualité de capture** (netteté, le serveur distant et le traitement logiciel seront détaillés au **Cycle 4**.



FIGURE V.14 – Caméra Raspberry Pi connectée au port CSI (acquisition d’images pour l’identification visuelle)

V.4.8 Interface d’Expression Faciale : Nextion NX4832T035_011

L’écran **Nextion NX4832T035_011** (3,5 pouces) intègre son propre microcontrôleur : le Raspberry Pi n’affiche pas des pixels en temps réel, mais envoie des **commandes série** pour changer de page ou d’expression (sourire, surprise, écoute, etc.) déjà stockées dans la mémoire de l’écran.

Alimentation et liaison (câblage retenu sur le prototype) :

- **5 V** et **GND** du Nextion reliés au **5 V** et **GND** du GPIO du Raspberry Pi (masse commune).
- **RX** du Nextion (fil jaune) → broche **8** du Pi (**GPIO 14**, ligne **TX** du Pi).
- **TX** du Nextion (fil bleu) → broche **10** du Pi (**GPIO 15**, ligne **RX** du Pi).

Ce câblage UART permet au Pi d’animer le **visage** de KODA sans surcharger le bus USB, tout en gardant une interface visuelle dédiée à l’interaction humaine.



FIGURE V.15 – Écran Nextion NX4832T035_011 (3,5") affichant le visage animé de KODA

V.4.9 Système d’Alimentation

Le robot KODA dispose de **deux sources d’alimentation indépendantes**, séparant volontairement la partie logique de la partie motrice :

TABLE V.3 – Architecture d’alimentation de KODA

Source	Caractéristiques	Composants alimentés
Power Bank	5V / 3A, USB-C	Raspberry Pi 4 (et par extension : ReSpeaker, caméra, écran Nextion, capteurs tactiles et liaison USB avec l’Arduino)
3 × piles Li-ion	3,7V chacune (11,1V total)	Shield L293D, moteurs DC, servomoteurs

Cette séparation protège le Raspberry Pi contre les perturbations électromagnétiques et les chutes de tension provoquées par les appels de courant des moteurs.



FIGURE V.16 – Power Bank 5 V / 3 A alimentant le Raspberry Pi (chaîne logique)



FIGURE V.17 – 3 piles lithium 3,7 V en série (11,1 V total) alimentant le Shield L293D et les moteurs

V.5 Schéma de Câblage Global

La figure V.18 présente le **schéma de montage** du prototype : vue type *Fritzing*, avec l'ensemble des connexions entre Raspberry Pi, Arduino, capteurs, actionneurs et alimentations. Les ajouts récents enrichissent ce schéma avec deux capteurs de touche reliés aux GPIO du Raspberry Pi et un gyroscope MPU5060 relié à l'Arduino pour le suivi inertiel de la rotation.

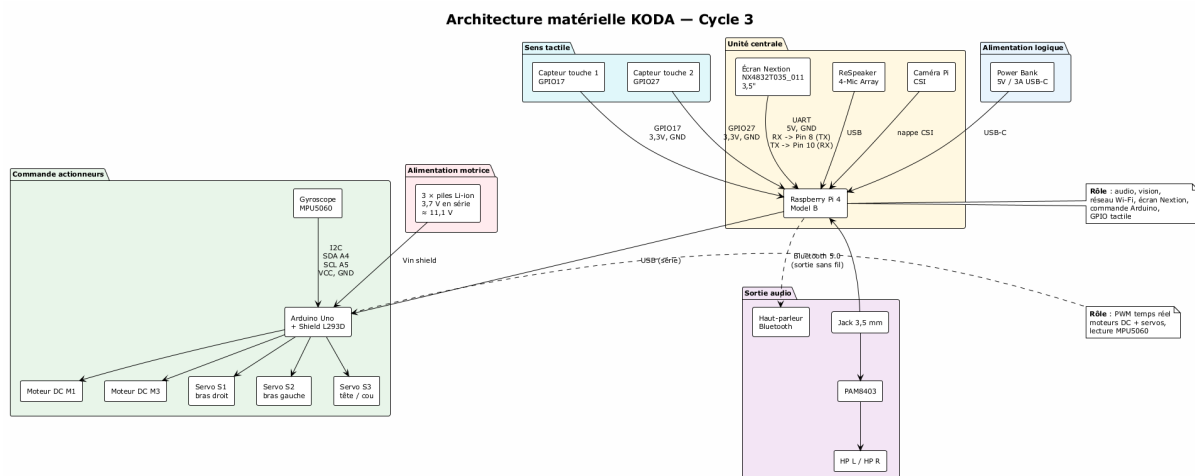
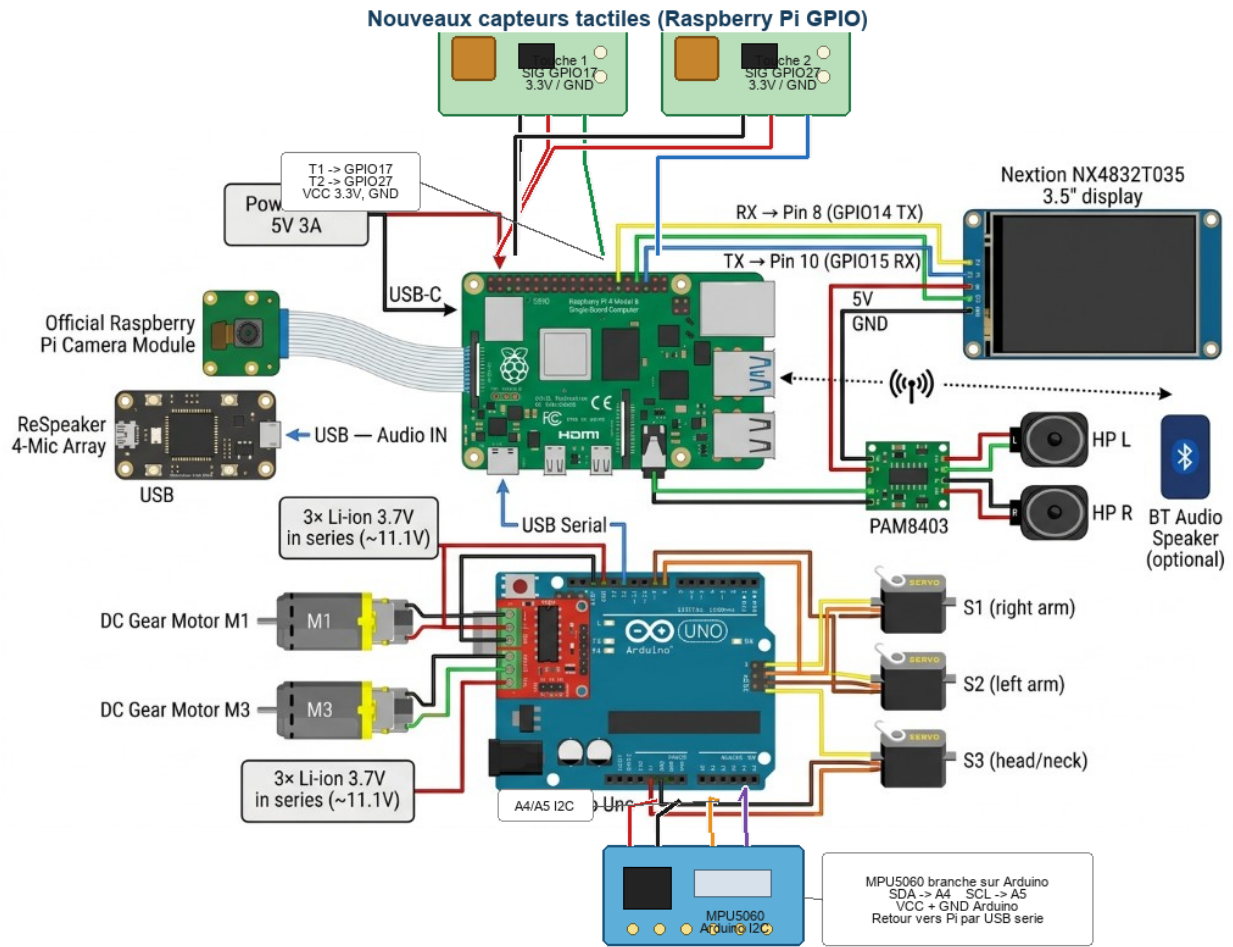


FIGURE V.19 – Architecture matérielle logique mise à jour avec toucher et gyroscope

Le tableau V.4 récapitule les connexions pour la rédaction et la maintenance du prototype.

V.6 Assemblage et points d'attention

L'intégration physique a mis en évidence quelques **contraintes** récurrentes, sans entrer dans le détail logiciel (Cycle 4) :



Ajouts Cycle 3 : deux capteurs de touche capacitifs connectés au Raspberry Pi (SIG GPIO17, SIG GPIO27, 3.3V, GND).
MPU5060 connecté à l'Arduino Uno par I2C (SDA -> A4, SCL -> A5, VCC, GND) ; retour de rotation vers le Raspberry par USB serie.

FIGURE V.18 – Schéma de montage matériel de KODA — style Fritzing (Cycle 3)

TABLE V.4 – Récapitulatif des connexions entre composants

Composant	Connecté à	Interface	Détail
Arduino Uno	Raspberry Pi	USB	Commandes moteur (série)
ReSpeaker 4-Mic	Raspberry Pi	USB	Entrée audio + DOA
PAM8403 + HP L/R	Raspberry Pi	Jack 3,5 mm	Sortie filaire
HP Bluetooth	Raspberry Pi	Bluetooth	Sortie sans fil (option)
Caméra Pi	Raspberry Pi	CSI	Nappe caméra
Nextion NX4832T035_011	Raspberry Pi	UART + alim	5V/GND GPIO ; RX→ pin 8 ; TX→ pin 10
Capteurs de touche	Raspberry Pi	GPIO numérique + alim	3,3V/GND ; capteur principal sur GPIO 17 ; second capteur sur GPIO 27
Gyroscope MPU5060	Arduino Uno	I2C + alim	VCC/GND Arduino ; SDA→ A4 ; SCL→ A5
Moteurs DC M1, M3	Shield L293D	Bornes M1, M3	Traction différentielle
Servos S1, S2, S3	Shield L293D	Servo 1, 2, 3	Bras D/G ; S3 = Nextion
Power Bank 5V/3A	Raspberry Pi	USB-C	Alimentation logique
3 × piles 3,7V (série)	Shield L293D	Vin	Alimentation motrice

- **Séparation des alimentations** : éviter de nourrir les moteurs depuis le 5 V du Pi ; les perturbations PWM peuvent provoquer des réinitialisations ou du bruit sur l’audio.
- **UART Nextion** : respecter le croisement TX/RX (fil RX de l’écran vers TX du Pi) et une masse commune stable.
- **USB multiples** : ReSpeaker, Arduino et alimentation compétent pour les ports USB du Pi ; une alimentation 5 V/3 A fiable (power bank) est indispensable.
- **GPIO et I2C** : les capteurs de touche doivent partager la masse du Raspberry Pi et rester en logique 3,3 V ; le gyroscope MPU5060 est raccordé au bus I2C de l’Arduino, ce qui impose des fils courts entre le module et la carte de commande moteur.
- **Encombrement mécanique** : la nappe CSI (caméra sur le corps) et les câbles des servos (dont S3 pour le Nextion) doivent rester libres de frottement lors des mouvements articulés.

V.7 Validation matérielle — tests par composant

Chaque sous-système a été validé **individuellement** avant le test global (contrôle visuel ou fonctionnel, sans détailler les scripts). Le tableau V.5 ci-dessous synthétise cette démarche.

TABLE V.5 – Tests unitaires par composant (Cycle 3)

Composant	Procédure (concept)	Critère OK
Power Bank + Pi	Mise sous tension, boot Raspberry Pi OS	LED activité, pas de brown-out
ReSpeaker USB	Enregistrement court, niveau sonore	Signal audible, périphérique reconnu
Sortie jack + PAM8403	Lecture d'un son test sur les HP	Son clair L/R
Sortie Bluetooth	Appairage HP BT, lecture test	Son sur HP externe
Caméra CSI	Capture d'une image fixe	Image nette, pas de bandes
Nextion UART	Changement de page ou d'expression	Affichage réactif
Capteurs de touche	Lecture des entrées GPIO puis contact physique	Passage stable de repos à touché
Gyroscope MPU5060	Lecture I2C par l'Arduino puis retour sur le port série	Valeurs cohérentes en rotation, visibles côté Arduino/USB
Arduino + USB	Envoi d'une commande simple (moniteur série)	Réponse attendue
Moteurs M1, M3	Avancer, reculer, pivoter	Roues cohérentes
Servos S1, S2, S3	Balayage angle min-max	Mouvement fluide, sans à-coups
Alim. 3 piles (série)	Charge moteurs à vide	Tension stable sur Vin

V.7.1 Commandes de test par composant

Chaque composant a été vérifié avec une commande courte exécutée directement depuis le Raspberry Pi (*shell* ou *Python*), ou via un mini-script côté Arduino. Les extraits ci-dessous donnent la procédure exacte employée pour confirmer le bon fonctionnement.

V.7.1.0.1 Power Bank + Raspberry Pi.

On vérifie qu’aucun *brown-out* (chute de tension) n’a été détecté par le firmware du Pi après mise sous tension.

```
# Doit retourner 0x0 ; tout bit non nul indique under-voltage ou  
# throttling.  
vcgencmd get_throttled  
# Tension sur le rail 5V (volts) :  
vcgencmd measure_volts core
```

Listing V.1 – Diagnostic alimentation vcgencmd

V.7.1.0.2 ReSpeaker 4-Mic Array USB.

Test en deux temps : enregistrement audio brut via ALSA/PipeWire, puis lecture de l’angle de provenance du son (*DOA*) via le registre interne du microcontrôleur XMOS du ReSpeaker.

```
# 1. Enregistrer 5 s en mono 16 kHz S16_LE (format attendu par Vosk/  
# Azure STT)  
arecord -D pipewire -f S16_LE -r 16000 -c 1 -d 5 /tmp/test_mic.wav  
aplay /tmp/test_mic.wav # écoute de controle  
  
# 2. Detection de la Direction of Arrival (DOA) via tuning.py de Seeed  
python3 -c "from usb_4_mic_array.tuning import Tuning; import usb.core;  
    \  
dev=usb.core.find(idVendor=0x2886, idProduct=0x0018); \  
t=Tuning(dev); print('DOA =', t.direction, 'deg')"
```

Listing V.2 – Enregistrement 5 s et lecture du DOA

V.7.1.0.3 Sortie jack + amplificateur PAM8403.

On lit un signal sinusoïdal stéréo connu pour vérifier les deux canaux (gauche/droite) de l’amplificateur et des haut-parleurs.

```
# Son systeme livre avec alsa-utils : "Front Center" annonce L puis R
```

```

aplay /usr/share/sounds/alsa/Front_Center.wav
aplay /usr/share/sounds/alsa/Front_Left.wav
aplay /usr/share/sounds/alsa/Front_Right.wav

```

Listing V.3 – Lecture d’un son test stereo

V.7.1.0.4 Sortie Bluetooth.

Appairage du haut-parleur Bluetooth puis lecture d’un WAV via le serveur audio PipeWire (qui route vers le sink `bluez_output.<MAC>.a2dp-sink`).

```

# Appairage initial (a faire une seule fois)
bluetoothctl power on
bluetoothctl pair      AD:1C:99:E7:9B:78
bluetoothctl trust     AD:1C:99:E7:9B:78
bluetoothctl connect  AD:1C:99:E7:9B:78

# Lecture vers le sink Bluetooth
paplay --device=bluez_output.AD_1C_99_E7_9B_78.a2dp-sink /tmp/test_mic.
wav

```

Listing V.4 – Appairage et lecture sur HP Bluetooth

V.7.1.0.5 Caméra Raspberry Pi (CSI).

Capture d’une image fixe avec l’utilitaire `rpicas-still` et contrôle visuel du fichier produit.

```

# Capture une photo en 1920x1080 apres 2 s d'autofocus
rpicas-still --width 1920 --height 1080 -t 2000 -o /tmp/test_cam.jpg

# Verifier le fichier (taille typique : > 200 Ko si l'image est nette)
ls -lh /tmp/test_cam.jpg

```

Listing V.5 – Capture d’une image avec `rpicas-still`

V.7.1.0.6 Écran Nextion (UART).

Envoi d’une commande page via la liaison série `/dev/serial0` (GPIO 14/15 du Pi). Chaque commande Nextion se termine par trois octets `0xFF`.

```

import serial

ser = serial.Serial("/dev/serial0", baudrate=9600, timeout=1)

# Trois 0xFF marquent la fin de chaque instruction Nextion

```



```
END = b"\xff\xff\xff"
ser.write(b"page 2" + END)           # bascule sur l'expression "joyeux"
ser.write(b"t0.txt=\"Hello\"" + END)
ser.close()
```

Listing V.6 – Changer la page affichée par le Nextion

V.7.1.0.7 Capteurs de touche (GPIO).

Les capteurs tactiles sont vérifiés en lisant directement leurs sorties numériques. Le premier capteur est raccordé au **GPIO 17**; le second est raccordé au **GPIO 27**.

```
from gpiozero import DigitalInputDevice
from time import sleep

touch_left = DigitalInputDevice(17, pull_up=False)
touch_right = DigitalInputDevice(27, pull_up=False)

while True:
    print("gauche=", touch_left.value, "droite=", touch_right.value)
    sleep(0.2)
```

Listing V.7 – Lecture des capteurs de touche sur GPIO

V.7.1.0.8 Gyroscope MPU5060 côté Arduino (I2C).

Le gyroscope est validé depuis l'Arduino, car c'est cette carte qui le lit pendant la rotation. Le test consiste à réveiller le module MPU5060 sur le bus I2C de l'Arduino, lire l'axe Z du gyroscope, puis afficher les valeurs sur le port série USB.

```
#include <Wire.h>

const byte MPU_ADDR = 0x68;
const byte PWR_MGMT_1 = 0x6B;
const byte GYRO_ZOUT_H = 0x47;

int16_t readWord(byte reg) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(reg);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, (byte)2);
    int16_t high = Wire.read();
```

```

    int16_t low = Wire.read();
    return (high << 8) | low;
}

void setup() {
    Serial.begin(9600);
    Wire.begin(); // SDA=A4, SCL=A5 sur Arduino Uno
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(PWR_MGMT_1);
    Wire.write(0); // reveil du capteur
    Wire.endTransmission(true);
}

void loop() {
    Serial.print("gyro_z_raw=");
    Serial.println(readWord(GYRO_ZOUT_H));
    delay(200);
}

```

Listing V.8 – Lecture du MPU5060 par l’Arduino via I2C

```

import serial

ard = serial.Serial("/dev/ttyUSB0", baudrate=9600, timeout=2)
while True:
    print(ard.readline().decode(errors="replace").strip())

```

Listing V.9 – Lecture du retour gyroscope depuis le Raspberry via USB série

V.7.1.0.9 Arduino + Shield L293D (commande série USB).

Le sketch Arduino lit un octet sur le port série; le Pi envoie un caractère pour déclencher un mouvement (F = forward, B = backward, S = stop). On vérifie ici le canal de communication uniquement.

```

import serial, time
ard = serial.Serial("/dev/ttyUSB0", baudrate=9600, timeout=2)
time.sleep(2) # laisser l'Arduino rebooter apres
              ouverture du port
ard.write(b"F") # forward 1 s puis stop
time.sleep(1)
ard.write(b"S")

```

```
print(ard.readline().decode(errors="replace").strip()) # ACK eventuel
ard.close()
```

Listing V.10 – Envoi d’une commande au sketch Arduino

V.7.1.0.10 Moteurs DC (M1, M3).

Extrait du sketch Arduino utilisant la bibliothèque AFMotor (Adafruit Motor Shield v1, équivalent L293D). Ce *loop* fait avancer puis reculer le robot pendant une seconde dans chaque sens.

```
#include <AFMotor.h>
AF_DCMotor m1(1); // borne M1
AF_DCMotor m3(3); // borne M3
void setup() {
    m1.setSpeed(200); m3.setSpeed(200); // 0-255
}
void loop() {
    m1.run(FORWARD); m3.run(FORWARD); delay(1000);
    m1.run(BACKWARD); m3.run(BACKWARD); delay(1000);
    m1.run(RELEASE); m3.run(RELEASE); delay(2000);
}
```

Listing V.11 – Test des moteurs M1 et M3 (Arduino + L293D)

V.7.1.0.11 Servomoteurs (S1, S2, S3).

Balayage de l’angle minimum au maximum pour les trois servomoteurs branchés sur les broches Servo_1, Servo_2 et Servo_3 du shield L293D.

```
#include <Servo.h>
Servo s1, s2, s3; // bras droit, bras gauche, tete (Nextion)
void setup() {
    s1.attach(9); s2.attach(10); s3.attach(11);
}
void loop() {
    for (int a = 0; a <= 180; a += 5) {
        s1.write(a); s2.write(180 - a); s3.write(a / 2);
        delay(30);
    }
}
```

Listing V.12 – Balayage des trois servomoteurs

V.7.1.0.12 Alimentation 3 piles 3,7 V (Vin shield).

Mesure au multimètre sur les bornes Vin du Shield L293D, sous charge (moteurs en rotation).

La tension doit rester stable autour de 11 V.

```
# Aucune commande logicielle : mesure physique au multimetre.  
# - A vide : ~12.4 V sur Vin du shield  
# - Sous charge : ~11.0 V (chute typique d'environ 1 V due au courant  
# moteurs)  
# Si la tension chute sous 9 V -> recharger ou remplacer les piles.
```

Listing V.13 – Procedure de mesure (notes oscilloscope/multimetre)

V.7.2 Test d'intégration matérielle

Le **test complet** consiste à alimenter l'ensemble, vérifier que le Pi détecte simultanément micro, caméra, écran et capteurs de touche, que l'Arduino répond toujours sur USB et renvoie les informations du gyroscope, et qu'un scénario court combine : capture vocale, mouvement des servos, rotation des roues, changement d'expression Nextion, détection d'un toucher et lecture inertielle, sans coupure d'alimentation ni conflit USB. Ce test valide la **plateforme prête pour le Cycle 4** (logiciel embarqué et liaison réseau avec le backend).



FIGURE V.20 – Prototype physique KODA après impression 3D et montage

V.8 Conclusion

Ce chapitre a présenté l'intégralité de la plateforme matérielle du robot KODA. L'architecture biprocesseur retenue — Raspberry Pi pour l'intelligence et l'Arduino pour le contrôle moteur — garantit à la fois la puissance de calcul nécessaire aux traitements IA et la précision temporelle requise pour le pilotage des actionneurs.

Chaque composant a été sélectionné et intégré pour répondre aux exigences fonctionnelles définies au chapitre II : mobilité, interaction vocale bidirectionnelle, reconnaissance visuelle,

expression émotionnelle, perception tactile et orientation plus précise. La plateforme matérielle est désormais prête à accueillir la couche logicielle embarquée, objet du **Cycle 4** présenté dans le chapitre suivant.

Points clés du Cycle 3

- Architecture biprocesseur : Raspberry Pi 4 + Arduino Uno / L293D
- 5 actionneurs : M1/M3 (DC) + S1/S2 (bras) + S3 (Nextion)
- Audio : ReSpeaker (entrée) ; sortie jack/PAM8403 ou Bluetooth
- Capteurs ajoutés : deux touches capacitives pour le toucher et MPU5060 côté Arduino pour la rotation
- Nextion NX4832T035_011 (UART GPIO 8/10, alim. 5 V Pi)
- Tests unitaires puis intégration ; logiciel au Cycle 4

———— CHAPITRE VI ————

CYCLE 4 : SYSTÈME EMBARQUÉ — PARTIE LOGICIELLE RASPBERRY PI

VI.1 Introduction

Le **Cycle 3** a validé la plateforme matérielle de KODA : Raspberry Pi, Arduino, ReSpeaker, écran Nextion, caméra, sortie audio et alimentation. Le **Cycle 4** transforme cette plateforme physique en un **robot compagnon autonome** capable de réagir en continu à la voix et au toucher, de dialoguer avec l'utilisateur et d'exécuter les décisions produites par le Distributeur de Services et le moteur n8n.

Ce chapitre présente la **partie logicielle embarquée Raspberry Pi** telle qu'elle est implémentée dans le dossier `codeRaspberry/`. Le programme principal est lancé par la commande `python -m app.main` et repose sur une architecture Python **asynchrone** en couches, organisée autour de **services métier** de haut niveau et d'**adaptateurs matériels** interchangeableables. Le Raspberry Pi n'est pas le cerveau conversationnel complet de KODA ; il joue plutôt le rôle de **corps opérationnel** : il écoute en permanence un mot de réveil, s'oriente vers le locuteur grâce à la DOA du ReSpeaker, capture la question jusqu'au silence, interroge le backend FastAPI, affiche une expression sur l'écran Nextion, prononce la réponse synthétisée, joue la musique téléchargée et transmet les mouvements à l'Arduino qui ferme la boucle de rotation avec un gyroscope MPU6050.

Par rapport au prototype initial, la version actuelle du logiciel ajoute :

- un moteur de mot de réveil **hybride** (Vosk local + Azure WebSocket) pour combiner faible latence et robustesse multilingue ;
- une **rotation en boucle fermée MPU6050** qui remplace la temporisation open-loop par une intégration gyroscopique côté Arduino ;
- un **capteur tactile** avec interruption GPIO qui suspend toute action en cours pour jouer une réaction « chatouilles » ;
- une **reconnaissance faciale** déléguée au backend, mise en cache et rafraîchie en arrière-plan, qui personnalise les réponses ;
- un **lecteur de musique** qui télécharge le morceau depuis le backend (la Raspberry n'ayant pas accès Internet sur le réseau ISET) ;
- un **registre de sous-processus** qui garantit que `sox`, `paplay` ou `rpircam` ne laissent pas de zombies après un arrêt brutal.

Objectif du Cycle 4

Implémenter et valider le logiciel embarqué Raspberry Pi qui relie les composants matériels du robot au backend FastAPI : mot de réveil hybride, orientation closed-loop, capture vocale VAD, dispatch des cinq types d'actions n8n (texte, musique, mouvement, sommeil, erreur), affichage Nextion, reconnaissance faciale, capteur tactile et arrêt propre. Les cibles de latence sont **< 300 ms** pour le mot de réveil et **< 3 s** pour le premier token vocal de la réponse.

VI.2 Architecture logicielle globale

Le logiciel Raspberry Pi est conçu comme une **couche d'orchestration locale**. Il démarre tous les adaptateurs en parallèle, vérifie les dépendances matérielles au boot, puis exécute une boucle asynchrone unique qui alterne entre un mode **passif** (attente du mot de réveil) et un mode **actif** (dialogue avec l'utilisateur). Les traitements lourds de compréhension, de décision et de synthèse vocale restent délégués au **Distributeur FastAPI**, qui relaie l'information vers n8n, Supabase et les services cloud (Azure Speech).

L'architecture suit quatre niveaux complémentaires :

- **Point d'entrée** (app/) : `main.py`, qui exécute la séquence de démarrage parallèle, installe les handlers SIGINT/SIGTERM, lance la boucle *wake-word* + *touch* puis effectue l'arrêt propre.
- **Configuration** (app/config.py) : centralise les dataclasses chargées depuis `.env` (réseau backend, ReSpeaker, Vosk, hybride wake-word, rotation, touch, face recognition, caméra, Bluetooth).
- **Services métier** (services/) : modules indépendants du matériel — conversation, wake word, écoute VAD, affichage, mouvement, musique, voix, vision, capteur tactile.
- **Adaptateurs** (adapters/) : interfaces concrètes vers le monde externe — HTTP/WebSocket vers le backend, ReSpeaker (sox), Nextion (UART), Arduino (USB série), audio (Bluetooth/PulseAudio), caméra (MJPEG ou `rpivid`).

VI.3 Architecture du code et diagramme de classes

Le projet `codeRaspberry` est volontairement séparé en dossiers courts et lisibles. Chaque adaptateur peut être testé indépendamment du robot complet, et le programme principal ne manipule jamais directement les détails série, sox ou HTTP : il passe par les services, qui passent par les adaptateurs.

Le diagramme de classes UML ci-dessous présente les objets logiciels qui coopèrent pendant

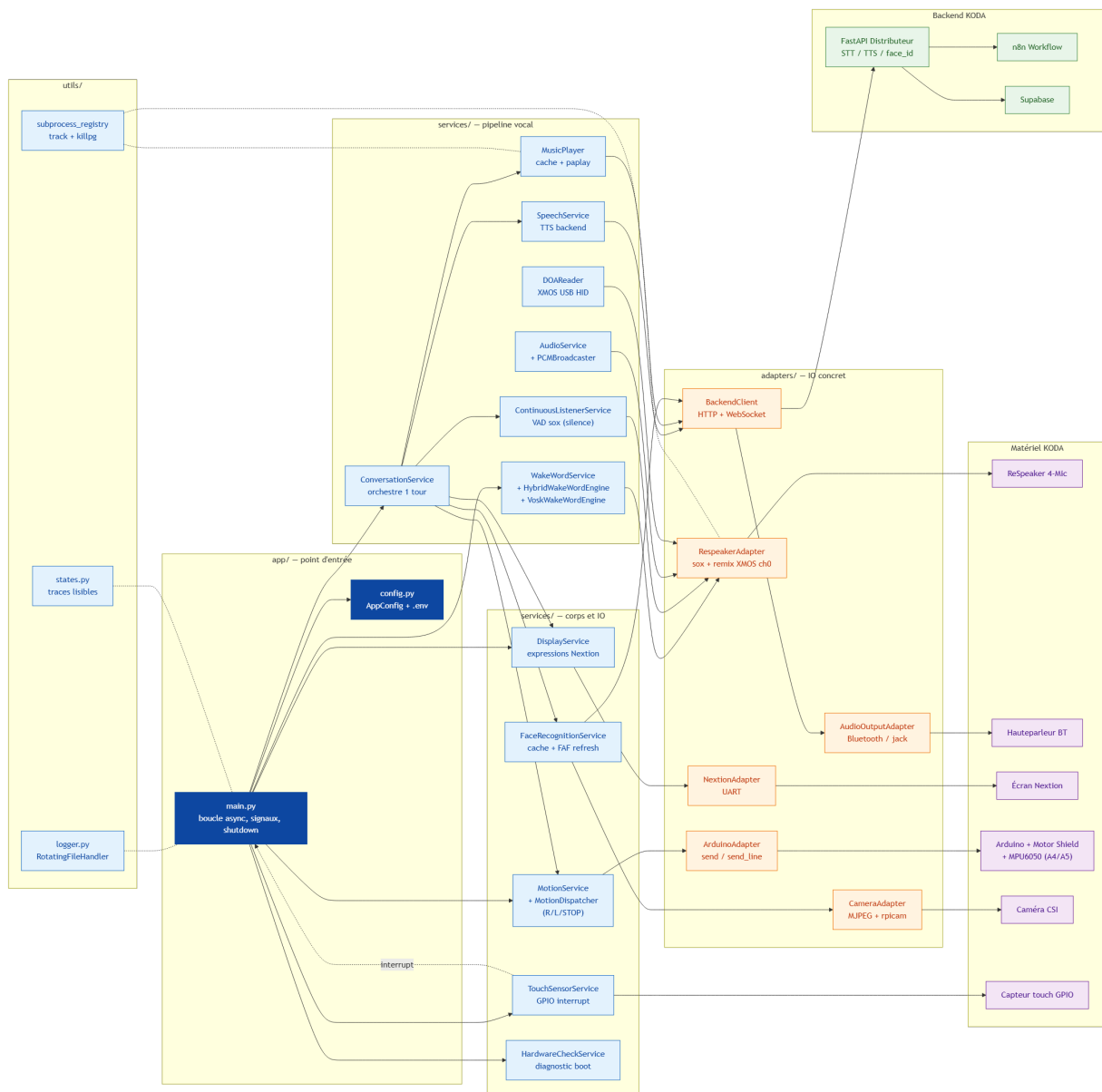
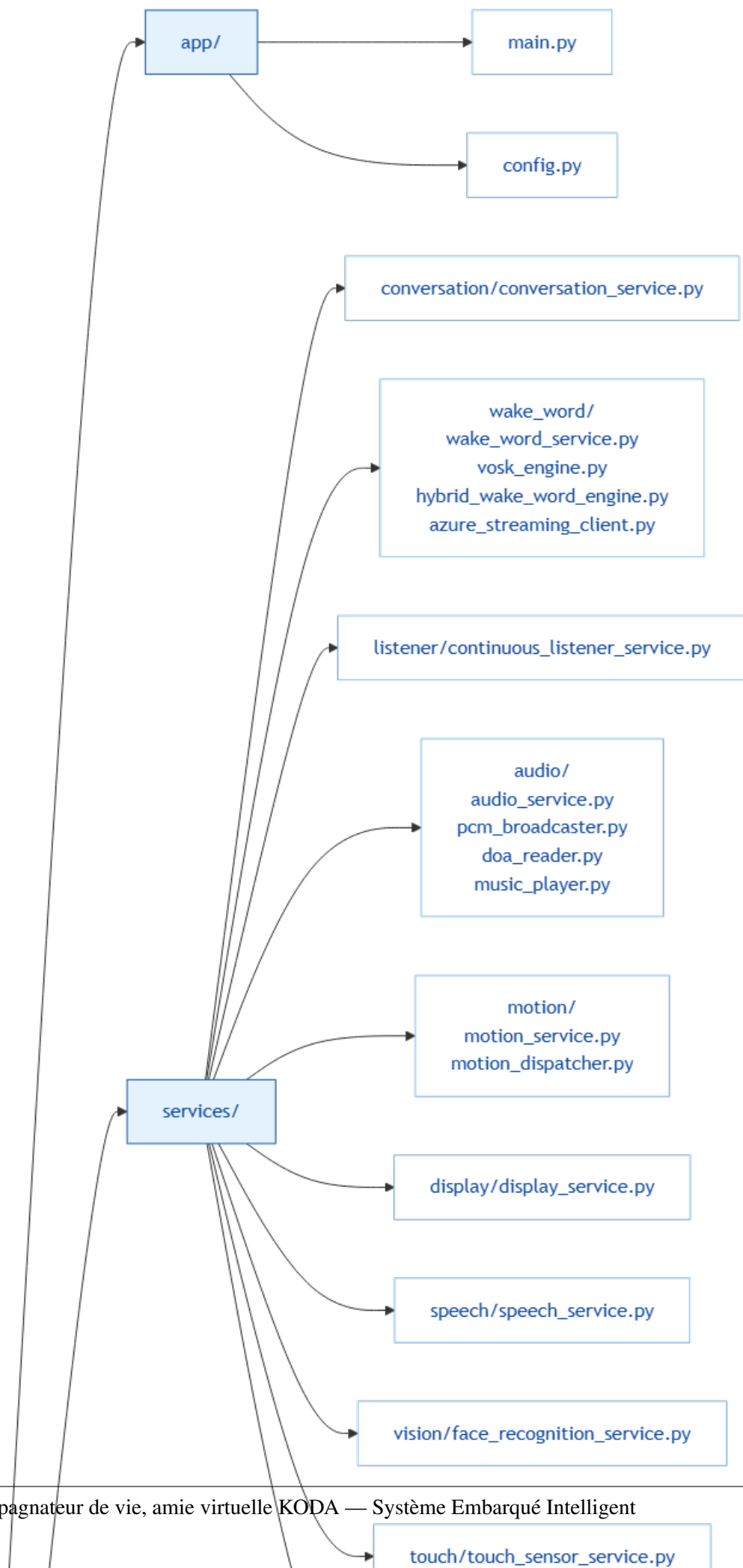


FIGURE VI.1 – Cycle 4 — architecture logicielle globale du logiciel Raspberry Pi (services, adaptateurs et environnement externe)



une interaction. `ConversationService` est l'orchestrateur fonctionnel d'un tour de parole : il récupère l'audio via `ContinuousListenerService`, appelle `BackendClient`, puis déclenche l'action retournée via `SpeechService`, `MotionDispatcher`, `MusicPlayer` ou `DisplayService`. La détection du mot de réveil est portée par `WakeWordService` et son moteur `HybridWakeWordEngine`, qui multiplexe Vosk et un Websocket Azure.

VI.4 Configuration et lancement

L'exécution se fait depuis le dossier `codeRaspberry/`. Les variables d'environnement permettent d'adapter le même code à un robot donné sans modifier les sources Python. Toutes les valeurs ont un défaut raisonnable dans `app/config.py` ; le fichier `.env` ne contient que les *overrides* propres au robot (adresse backend, identifiant, MAC du hauteparlleur Bluetooth, etc.).

```
source .venv/bin/activate
export BACKEND_URL=http://192.168.69.185:8000
export ROBOT_ID=koda-01
export RESPEAKER_DEVICE=pipewire           # ALSA -> PipeWire,
      pas plughw:3,0
export VOSK_LANGUAGE=ar                     # mgb2 arabe
export HYBRID_WAKE_WORD_ENABLED=1
export HYBRID_WAKE_WORD_AWAITING_TIMEOUT=4.0
export ROTATION_ENABLED=1
export TOUCH_ENABLED=1                     # GPIO BCM 17 par
      default
export FACE_RECOGNITION_ENABLED=1
python -m app.main
```

Listing VI.1 – Lancement du logiciel embarqué Raspberry Pi

Les paramètres essentiels sont :

- `BACKEND_URL` : adresse HTTP du Distributeur FastAPI sur le même réseau que la Raspberry (utilisée aussi pour la Websocket `/api/ws/wake-word/{robot_id}`).
- `RESPEAKER_DEVICE` : périphérique de capture sox. `pipewire` est préféré car la firmware XMOS refuse parfois le `set_freq 16 kHz` issu de `plughw:3,0`.
- `VOSK_LANGUAGE` : code de langue du modèle Vosk (ar = arabe mgb2, fr, en, etc.).
- `HYBRID_WAKE_WORD_*` : active la double détection Vosk + Azure WS et le délai d'attente Azure après un déclenchement Vosk.
- `ROTATION_ENABLED`, `TOUCH_ENABLED`, `FACE_RECOGNITION_ENABLED` : commutateurs princi-

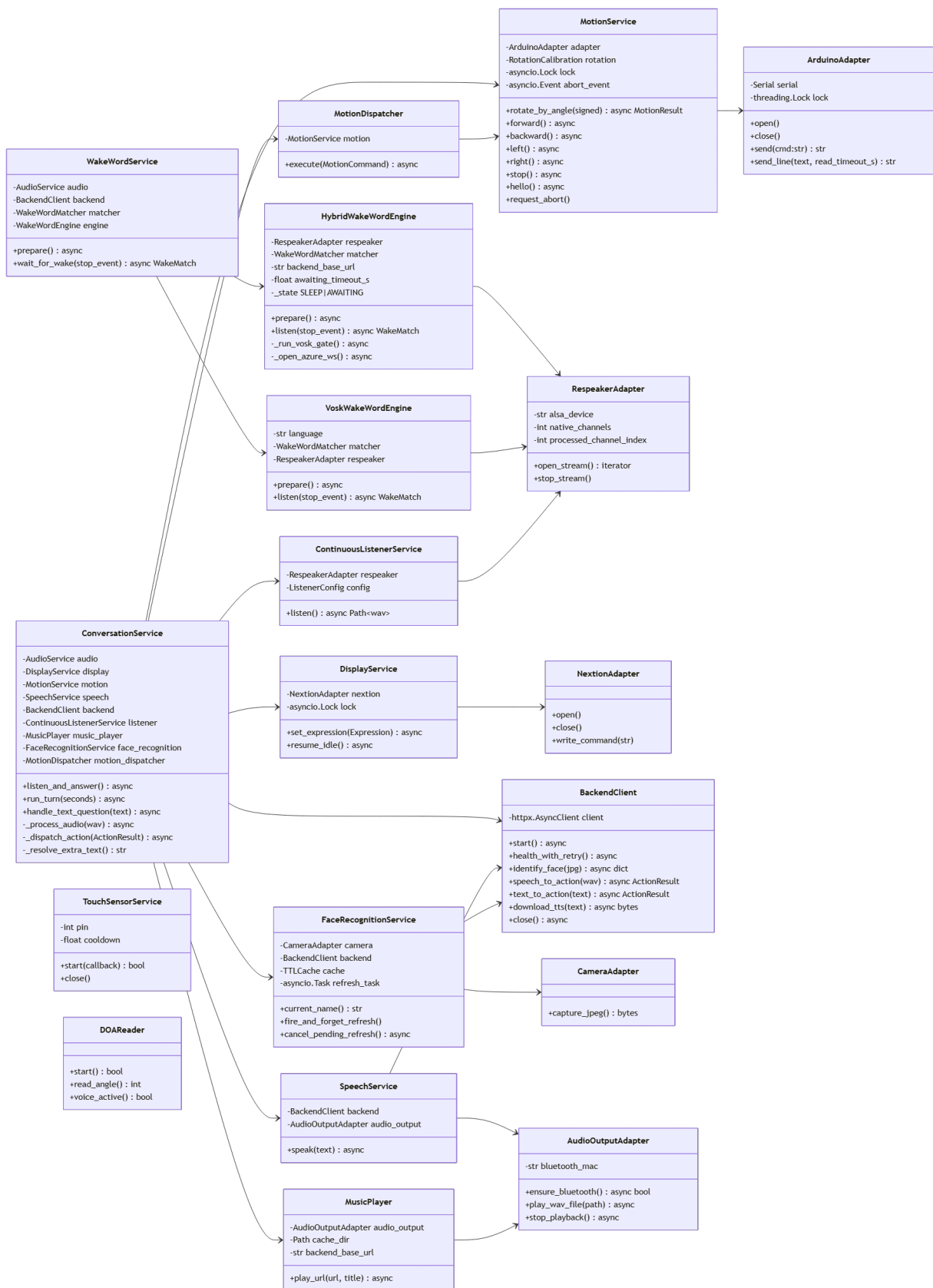


FIGURE VI.3 – Cycle 4 — diagramme de classes UML : services métier et adaptateurs matériels

paux qui permettent de démarrer en mode dégradé contrôlé si un composant manque.

- **Audio sortie** : MAC Bluetooth ou `pulse_sink` dans `AUDIO_OUTPUT_*`. Bascule automatique vers le sink par défaut si le speaker BT n'est pas joignable.

VI.5 Démarrage parallèle et diagnostic matériel

Au démarrage, `app.main` installe un `ThreadPoolExecutor` dédié aux appels bloquants (`KODA_THREAD_WORKER`), exécute la fonction `run_full_check()` qui dresse un rapport composant par composant, puis ouvre les adaptateurs série Nextion et Arduino en **parallèle**. Tous les services qui n'ont aucune dépendance les uns sur les autres sont ensuite lancés dans un unique `asyncio.gather` : `backend.start()`, sonde `/health`, connexion Bluetooth, chargement du modèle Vosk (8 s sur Pi 4), initialisation USB de la `DOAReader`, retour de l'écran à son frame idle et geste de salutation. Le *coût total* du boot est ramené à celui de la tâche la plus lente (généralement Bluetooth ou Vosk) au lieu de la somme des temps unitaires.

L'option `return_exceptions=True` du `gather` garantit qu'**un échec de bootstrap (Bluetooth absent, backend indisponible, hautepareur déconnecté) ne tue pas les autres** : chaque tâche est isolée, loguée et marquée comme dégradée. Le robot peut ainsi démarrer même si la caméra ou le speaker manquent.

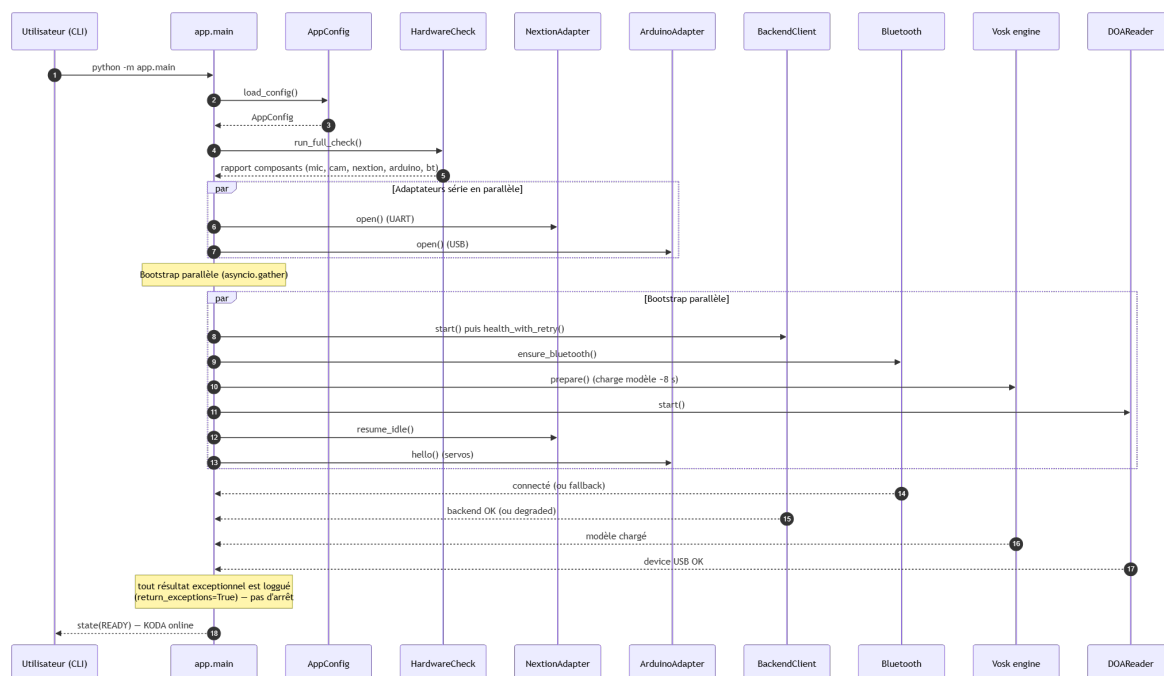


FIGURE VI.4 – Cycle 4 — séquence de démarrage parallèle (`app.main.run`)

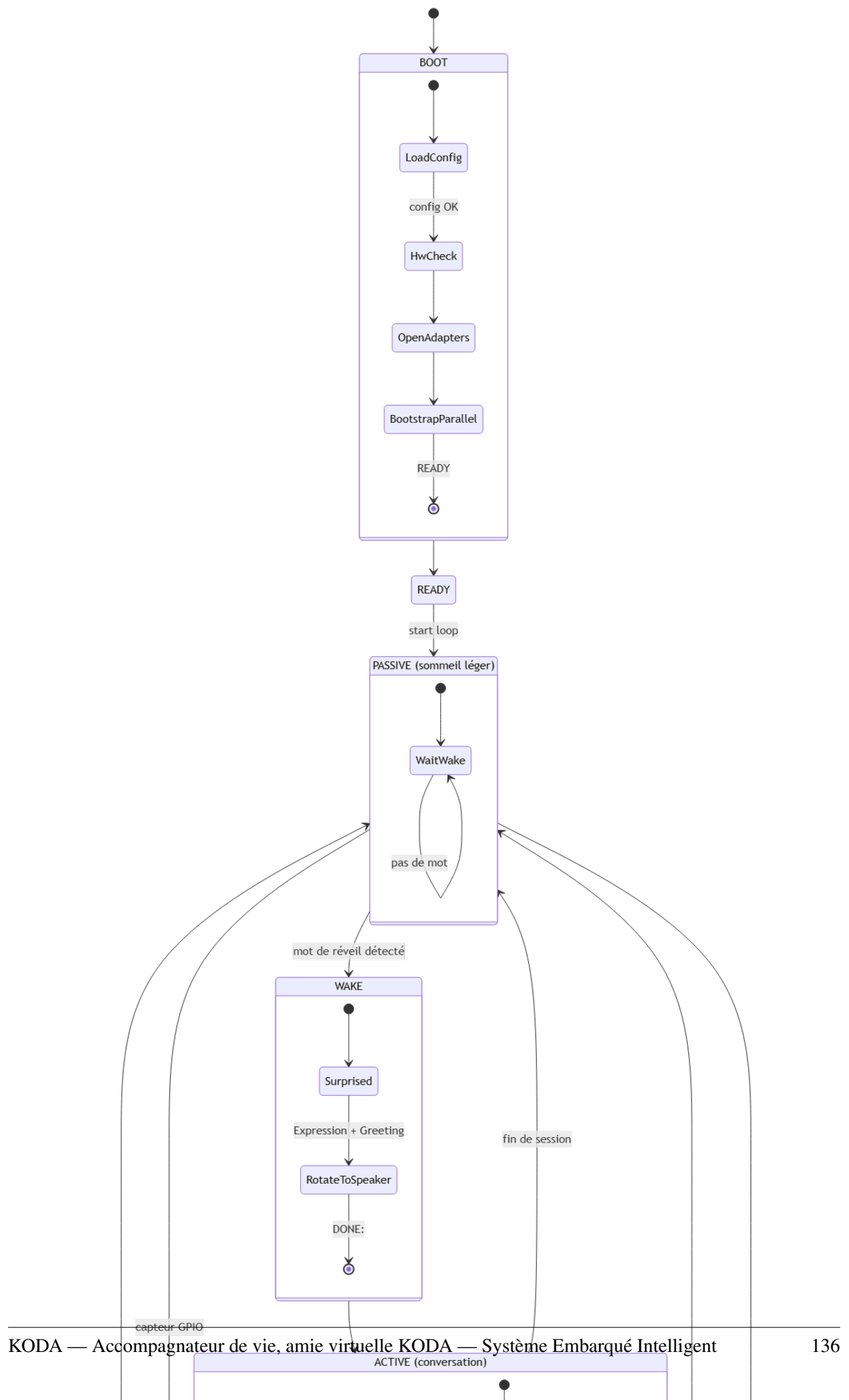
VI.6 Boucle principale et machine d'états

Après l'initialisation, le robot entre dans la fonction `_run_robot_loop_with_touch`, qui combine la boucle comportementale principale et la surveillance du capteur tactile via un `asyncio.wait(..., return_when=FIRST_COMPLETED)`. En mode passif, il écoute le mot de réveil ; détecté, il affiche `SURPRISED`, s'oriente vers l'utilisateur par DOA puis MPU6050, prononce une salutation en parallèle, et lance la capture VAD de la question. Le retour du backend décide ensuite du comportement : réponse vocale, musique, mouvement, sommeil ou erreur. Une interruption tactile peut, à tout moment, annuler la tâche en cours et déclencher une réaction « chatouilles » avant le retour à l'état passif.

VI.7 Pipeline vocal : mot de réveil hybride Vosk + Azure

Le pipeline vocal est le cœur de l'expérience utilisateur. Il combine trois traitements complémentaires :

- **Détection locale du mot de réveil par Vosk** : le moteur `VoskWakeWordEngine` consomme un flux PCM 16 kHz mono issu du `ReSpeaker` via `sox` (`remix 1` sur le canal `XMOS` traité `ch0`, suivi d'un `-c 1` obligatoire en sortie `raw`). Aucun appel cloud n'est fait tant qu'un candidat n'est pas détecté.
- **Confirmation par Azure WebSocket** : lorsque `HYBRID_WAKE_WORD_ENABLED=1`, le moteur `HybridWakeWordEngine` bascule de l'état `SLEEP` à `AWAITING` dès le premier déclenchement Vosk. Il ouvre alors un `WebSocket` vers `/api/ws/wake-word/{robot_id}` sur le backend (qui relaie le PCM à Azure Speech) et fait une *course* : le premier moteur qui confirme le mot-clé l'emporte ; sinon, le moteur revient en `SLEEP` après `awaiting_timeout_s`.
- **Écoute active VAD** : après le réveil, `ContinuousListenerService` capture la question complète jusqu'à une période de silence configurable, au lieu d'imposer une durée fixe ; un *seuil minimum* (`min_speech_seconds`) évite d'envoyer un WAV de 44 octets quand `sox` est encore en train de démarrer.
- **Traitement distant** : le WAV est envoyé en HTTP au backend sur `POST /api/audio/speech-to-action`. La réponse est un objet `ActionResult` contenant le type d'action, le texte reconnu, le texte à prononcer, l'audio TTS encodé en base64 et éventuellement une URL de musique ou une commande de mouvement.



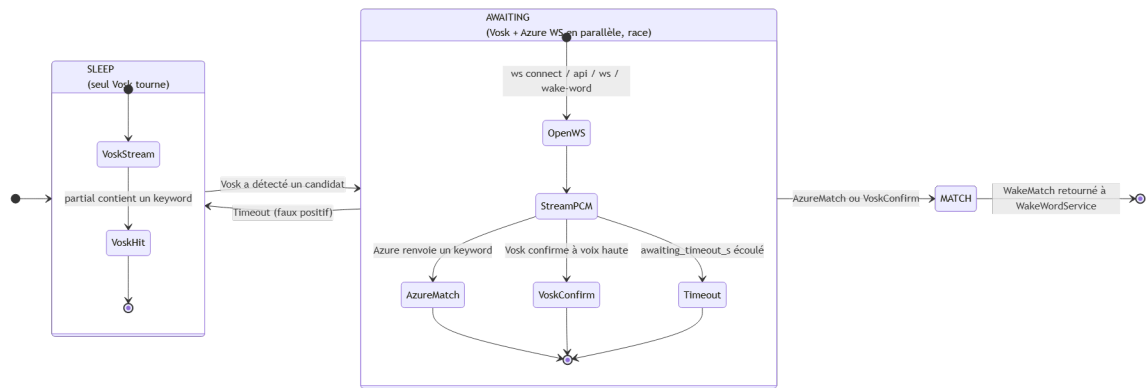


FIGURE VI.6 — Cycle 4 — machine d'états interne du moteur HybridWakeWordEngine (Vosk gate, Azure race, timeout)

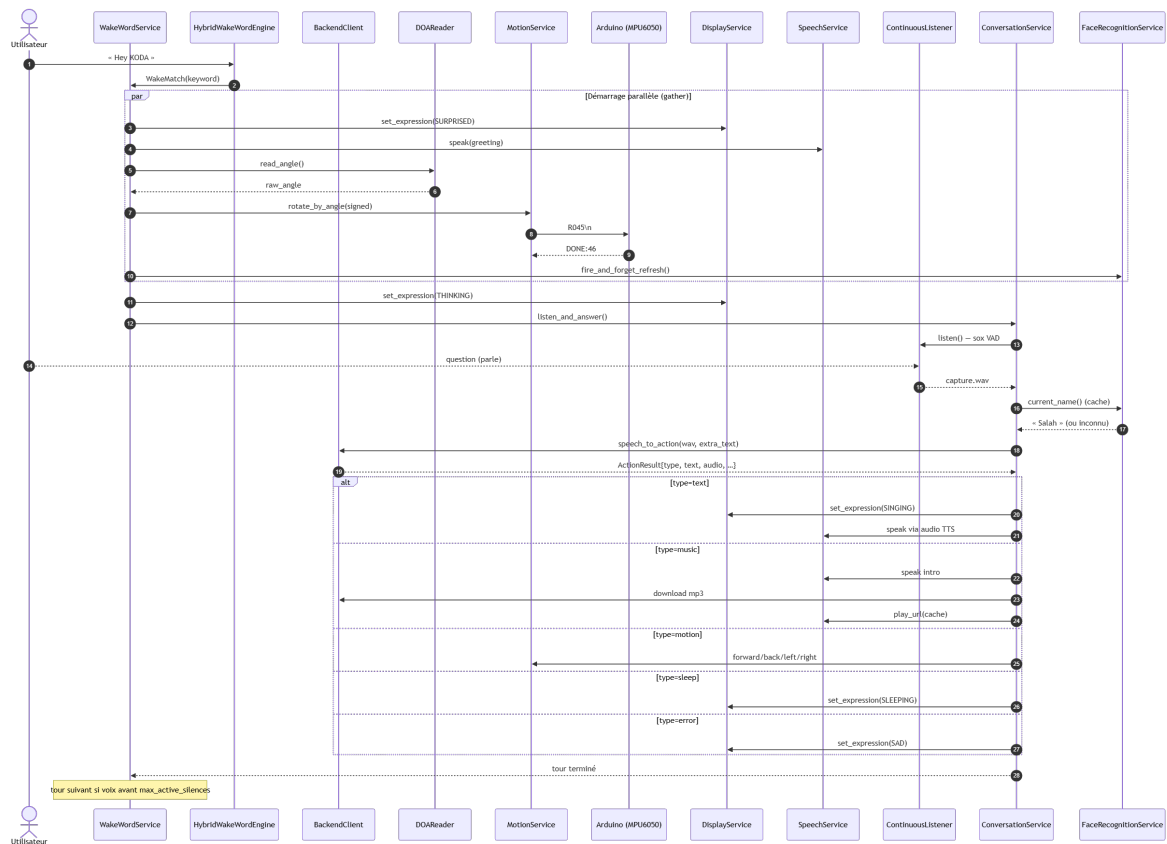


FIGURE VI.7 — Cycle 4 — séquence complète d'un tour de parole : wake → rotation → greeting → écoute → backend → dispatch

VI.8 Capture audio multi-consommateurs

Plusieurs services veulent lire le même flux microphone en même temps : Vosk pour le mot de réveil, le client WebSocket Azure pendant AWAITING, et la VAD lors de la capture de la question. Plutôt que de relancer sox à chaque besoin (ce qui provoquait des conflits de périphérique USB), le service PCMBroadcaster adopte un schéma **un producteur, N consommateurs** : un unique processus sox écrit dans une file de trames, chaque consommateur dispose d'une queue `asyncio` **bornée**. Un consommateur lent voit ses anciennes trames droppées sans pénaliser les autres.

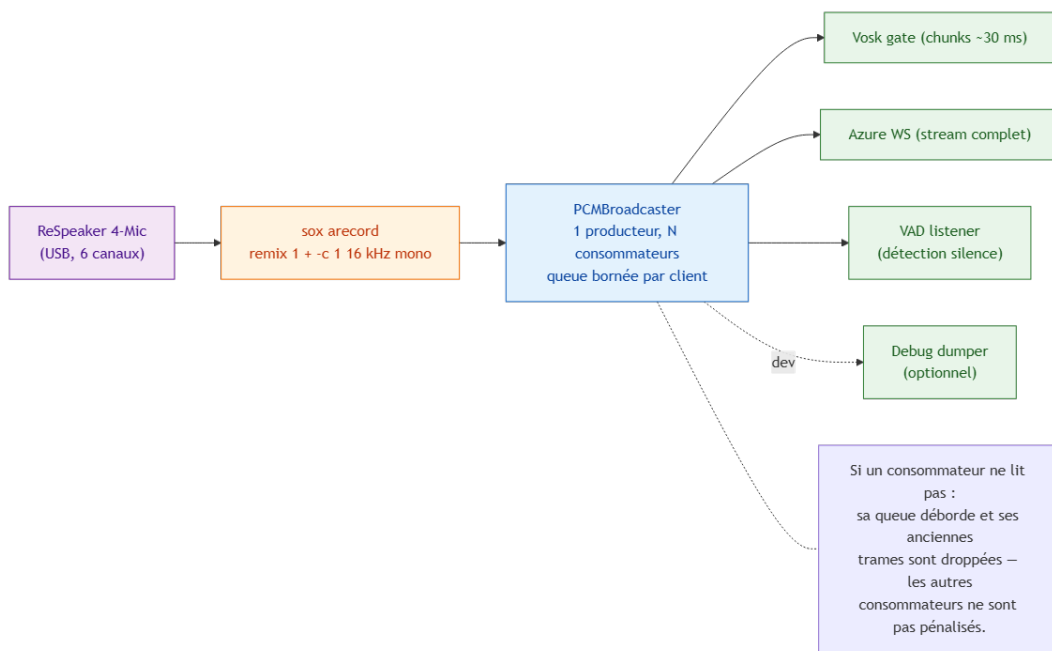


FIGURE VI.8 – Cycle 4 — broadcaster PCM : un producteur sox, plusieurs consommateurs (Vosk, Azure WS, VAD)

VI.9 Rotation closed-loop avec MPU6050

Pour orienter le robot vers le locuteur, le Raspberry Pi lit la direction d'arrivée du son (DOA) fournie par le ReSpeaker via le registre USB HID du XMOS, puis convertit cet angle brut en rotation signée $[-180^\circ, +180^\circ]$ selon la convention *positif = horaire, négatif = anti-horaire*, en tenant compte du `front_offset_deg` et du flag `invert_direction` pour absorber l'orientation mécanique du micro.

L'innovation majeure du Cycle 4 est de remplacer la temporisation **open-loop** (durée = angle / vitesse) par une rotation **en boucle fermée** basée sur le gyroscope MPU6050 câblé sur les broches A4 (SDA) et A5 (SCL) de l'Arduino. Le firmware `koda_arduino.ino` expose un protocole étendu en plus du jeu historique de commandes mono-caractère :

```
# Commandes legacy (immediates, sans reponse)
```

```

F B S L R          # forward/backward/stop/left/right (
    continu)
H T G D A          # hello, head, left arm, right arm, all
    servos
+ -                # speed up / down
?                  # status -> STATUS:speed=N, gyro=ok/nok\n

# Protocole etendu rotation (bloquant, avec reponse)
R045\n             # tourner a droite de 45 degres
L180\n             # tourner a gauche de 180 degres
DONE:46\n          # succes (degres reellement parcourus)
ERR:timeout\n      # echec (safety 6 s firmware)
ERR:nogyro\n       # MPU6050 absent / I2C muet au boot

```

Listing VI.2 – Protocole de rotation MPU6050 (firmware Arduino koda_arduino.ino)

Côté Pi, `MotionService.rotate_by_angle` envoie `send_line("R045", read_timeout_s=...)` et attend la ligne `DONE:<actual>`. Le firmware, lui, intègre la vitesse angulaire ω_z du gyroscope à 200 Hz, freine sur les 6 derniers degrés en abaissant le PWM à 110 (constante `ROTATION_BRAKE_SPEED`), stoppe les moteurs et attend 150 ms supplémentaires que le châssis se stabilise. La précision typique passe de $\pm 5\text{--}10^\circ$ (open-loop, sensible à la tension batterie et au frottement) à $\pm 2^\circ$ quelle que soit la charge.

Sécurités : un timeout firmware de 6 s coupe la rotation si le gyroscope renvoie des valeurs aberrantes ou si le châssis est bloqué ; le Pi reçoit alors `ERR:timeout` et envoie un `STOP` défensif. Si la MPU6050 n’a pas répondu à l’I2C au boot, toute commande `Rxxx/Lxxx` retourne immédiatement `ERR:nogyro` sans alimenter les moteurs. Une consigne `|angle|` inférieure à `deadband_deg` (par défaut 5°) est ignorée pour préserver la batterie. Côté Pi, le `asyncio.Lock` du `MotionService` sérialise les commandes Arduino ; les appels concurrents (par exemple `hello()` pendant une rotation) sont mis en file sans blocage du reste de la boucle.

VI.10 Dispatch des actions retournées par le backend

Le backend interprète la réponse de `n8n` (qui peut prendre six formes différentes : JSON texte, JSON musique, texte direction brut, « I am Closed », « Yo Yo », message de sommeil) et la normalise en un `ActionResult` structuré. Côté Raspberry, `ConversationService` applique alors cinq branches métier :

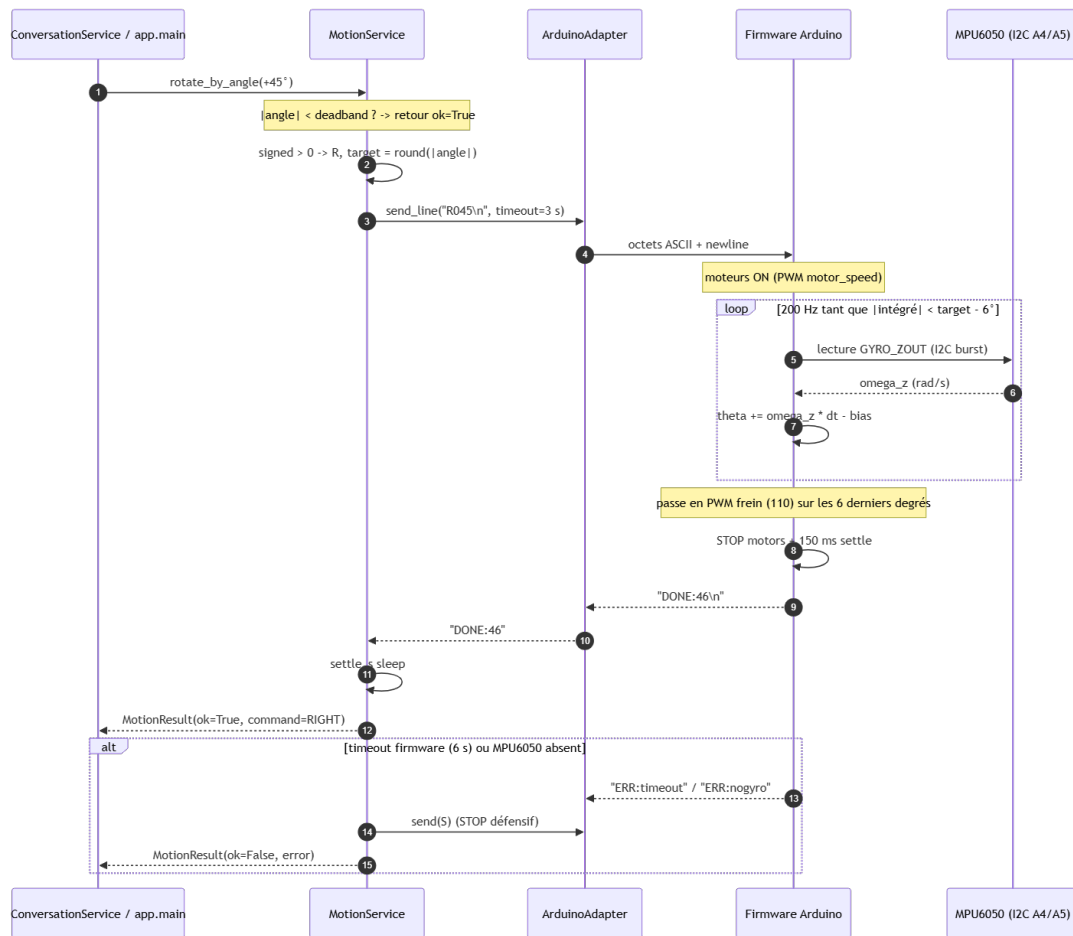


FIGURE VI.9 – Cycle 4 — rotation en boucle fermée : Pi → Arduino → MPU6050 → DONE/ERR

TABLE VI.1 – Cycle 4 — les cinq types d’actions dispatchés côté Pi

type	Comportement	Services impliqués
text	Expression SINGING, lecture de l’audio TTS reçu en base64, retour idle	Display, Speech, AudioOutput
music	Annonce vocale, téléchargement depuis le backend (cache local cache/music/), lecture avec paplay	Speech, MediaPlayer, BackendClient
motion	Commande forward, backward, left, right ou stop via MotionDispatcher	Motion, Arduino
sleep	Message d’au revoir, expression SLEEPING, retour en mode passif	Speech, Display
error	Expression SAD brève, log structuré, reprise de la boucle	Display

VI.11 Affichage Nextion et expressions

L’écran Nextion n’est pas piloté pixel par pixel : les images et animations sont préparées dans l’éditeur Nextion, et le Pi se contente d’envoyer des commandes série courtes (tm0.en=0, page facegif, ref). DisplayService sérialise ces commandes avec un verrou asyncio afin d’éviter qu’une expression en cours de transition soit écrasée par une autre. Pendant une réponse vocale, le service peut désactiver temporairement le timer de clignement, afficher une expression fixe, puis restaurer l’état idle.

TABLE VI.2 – Expressions logicielles envoyées à l’écran Nextion

Expression	Usage principal
NEUTRAL	Attente et retour idle
SURPRISED	Mot de réveil détecté
THINKING	Capture ou traitement de la question
SINGING	Réponse vocale ou musique
HAPPY	Interruption tactile (chatouilles)
SAD	Erreur ou échec de traitement
SLEEPING	Passage en sommeil ou arrêt

VI.12 Reconnaissance faciale

Lorsque `FACE_RECOGNITION_ENABLED=1`, `FaceRecognitionService` déclenche en arrière-plan une capture `CameraAdapter` (MJPEG en priorité, fallback `rpicas-still`) dès la détection du mot de réveil. La photo est envoyée à `/api/identify-face` sur le backend ; le nom retourné est mis en cache (TTL configurable, par défaut 60 s) puis injecté comme `extra_text` dans la requête `speech-to-action`. Le prompt côté n8n peut ainsi personnaliser la réponse (« Bonjour Salah ») sans appel additionnel.

Le rafraîchissement est *fire-and-forget* : la tâche `asyncio` est tracée par `self._refresh_task` pour qu’au plus une requête soit en vol à la fois (les tirs suivants sont court-circuités tant que la tâche précédente n’est pas terminée). Un `done_callback` libère la référence pour éviter toute fuite. Lors de l’arrêt, `cancel_pending_refresh()` attend la fin de la requête en cours pendant un court délai avant de la couper.

VI.13 Capteur tactile et interruption

Un capteur tactile capacitif est câblé sur le GPIO BCM 17 par défaut. `TouchSensorService` utilise `gpiozero` (fallback `RPi.GPIO`) et publie l’événement sur un `asyncio.Event` via `loop.call_soon_threading`. La boucle principale attend simultanément trois événements : fin du comportement courant, signal d’arrêt et toucher.

Lorsque le touch déclenche en premier, `_handle_touch_interrupt` est exécuté :

1. `motion.request_abort()` pour couper toute `sleep` d’attente moteur ;
2. `behavior_task.cancel()` pour arrêter la conversation ;
3. `audio_output.stop_playback()` et `motion.stop()` en parallèle ;
4. `kill_tracked_subprocesses(grace_s=0.25)` pour libérer `sox/paplay/rpicam` restés actifs ;
5. `display.set_expression(HAPPY)` puis lecture du WAV de rire (ou `speech.speak` en fallback).

Une fois la réaction terminée, une nouvelle `behavior_task` est créée et la boucle reprend. Cela permet d’**interrompre** une réponse en cours sans laisser de processus enfants orphelins.

VI.14 Robustesse, registre de sous-processus et arrêt propre

La robustesse du logiciel embarqué repose sur plusieurs choix :

- **Asynchronisme strict** : les lectures audio, appels HTTP, commandes série et lectures vidéo sont orchestrés par `asyncio`, sans thread bloquant. Les appels bloquants inévitables (`sox`,

`serial.write`, ouverture USB) sont déportés via `asyncio.to_thread` sur un `ThreadPoolExecutor` dédié.

- **Verrous locaux** : `DisplayService` et `MotionService` protègent leurs ressources série respectives contre les accès concurrents. Le `MotionService` ne tient jamais son verrou pendant un `sleep` d'attente moteur — règle cardinale pour ne pas geler le reste de la boucle.
- **Reprise sur crash** : la boucle principale extrait chaque itération dans `_wake_word_iteration` ; toute exception (Vosk qui crashe, WebSocket Azure qui se ferme, USB qui se déconnecte) est loguée puis suivie d'une attente de 2 s avant la prochaine tentative. Le robot ne meurt jamais sur un `STACK TRACE`.
- **Registre de sous-processus** (`utils/subprocess_registry.py`) : tout `subprocess.Popen` suspect (`sox`, `arecord`, `paplay`, `rpicam`, `yt-dlp`) est créé avec `start_new_session=True` et enregistré. À l'arrêt, `kill_tracked_subprocesses()` envoie un `SIGTERM` puis un `SIGKILL` sur le *groupe de processus* (via `os.killpg`) ; un `pkill_orphans()` balaie ensuite les binaires connus pour rattraper les fuites éventuelles.
- **Logs structurés** : `utils/logger.py` configure un `RotatingFileHandler` avec horodatage millisecondes, ce qui permet de corréler une trace Pi avec une trace backend lors d'un bug transitoire.
- **Diagnostics explicites** : les composants absents sont loggués avec leur nom et leur message d'erreur, et listés au début de chaque démarrage (« 5/7 components detected »).
- **Arrêt propre** : sur `SIGINT` ou `SIGTERM`, le robot affiche `SLEEPING`, arrête les moteurs, annule la tâche de rafraîchissement faciale en cours, tue les sous-processus tracés, ferme le client HTTP et libère les ports série.

VI.15 Tests et validation logicielle

La validation du Cycle 4 combine tests automatisés Pytest et essais réels sur le Raspberry Pi.

TABLE VI.3 – Tests de validation du logiciel Raspberry Pi

Type de test	Procédure	Critère OK
Configuration	<code>pytest tests/test_config.py</code> avec defaults et overrides	Toutes valeurs chargées
Imports	<code>pytest tests/test_imports.py</code>	Tous les modules importables
Hardware checks	<code>python -m services.hardware_check</code> sur le Pi	Rapport composant par composant
Mot de réveil	Prononcer le mot devant le ReSpeaker en mode hybride	Vosk déclenche, Azure confirme, WAKE
VAD question	Poser une question puis se taire	WAV complet jusqu'au silence (pas 44 octets)
Backend action	Envoyer une question vocale	ActionResult reçu avec type valide
Rotation MPU6050	<code>pytest tests/test_motion_concurrent.py</code> (7 cas)	Protocole Rxxx, DONE/ERR, deadband
WebSocket wake-word	<code>python tests/test_ws_wake_word.py respeaker</code>	Trames PCM streamées, match reçu
Face refresh	<code>pytest tests/test_face_recognition_lifecycle.py</code>	Au plus 1 task en vol, cancel propre
Touch interrupt	Toucher le capteur pendant une réponse	Audio coupé, sox tué, rire joué
Shutdown	Ctrl+C ou SIGTERM	Moteurs arrêtés, ports fermés, 0 zombie

Les commandes minimales de validation logicielle sont :

```

pytest tests/                                     # suite unitaire
complete
python tests/test_ws_wake_word.py respeaker      # smoke test

```



```
Websocket

python tests/test_mic_configs.py                # 6 configs
micro + RMS

python -m app.main                               # essai bout-en-
bout
```

Listing VI.3 – Commandes de validation du Cycle 4

VI.16 Limites et perspectives

Le logiciel embarqué actuel assure le lien principal entre l'utilisateur, le mouvement, l'affichage et le backend. Certaines améliorations restent toutefois possibles :

- streamer la STT côté Pi pendant la question pour économiser 3–5 s par tour (le backend reçoit aujourd'hui le WAV complet);
- ajouter un service systemd pour démarrer KODA automatiquement au boot avec un *watchdog* sur le port Arduino;
- enrichir la télémétrie (erreurs audio, latences backend, rotations en échec) et la pousser vers Supabase pour le suivi des démonstrations;
- stocker localement quelques réponses de secours si le backend devient temporairement indisponible;
- permettre la découverte automatique du backend par mDNS comme fallback à BACKEND_URL.

VI.17 Conclusion

Ce chapitre a présenté le logiciel embarqué Raspberry Pi qui donne vie au prototype KODA. Le programme `app.main` coordonne les services et adaptateurs nécessaires pour passer d'une plateforme matérielle assemblée à un compagnon capable d'écouter, de s'orienter au degré près grâce au gyroscope MPU6050, de parler, d'afficher des émotions, de réagir au toucher et d'exécuter cinq familles d'actions issues du backend.

Le choix d'une architecture asynchrone, séparée en services et adaptateurs, rend le système maintenable et robuste face aux contraintes du réel : matériel parfois absent, latence réseau, capture audio continue, sous-processus à surveiller, ports série partagés et arrêt propre. Le Cycle 4 complète ainsi le Cycle 3 en transformant le corps matériel en plateforme interactive connectée au cerveau n8n et au Distributeur de Services.

Points clés du Cycle 4

- Démarrage *parallèle* (`asyncio.gather`) avec mode dégradé contrôlé par `return_exceptions=True`
- Architecture Python asynchrone en couches : services métier + adaptateurs matériels
- Mot de réveil hybride Vosk (gate local) + Azure WebSocket (confirmation)
- Rotation *closed-loop* MPU6050 sur Arduino — précision $\pm 2^\circ$
- Capteur tactile avec interruption asynchrone et réaction « chatouilles »
- Reconnaissance faciale en cache, rafraîchie en arrière-plan (fire-and-forget tracé)
- Dispatch des cinq types d'actions n8n (texte, musique, mouvement, sommeil, erreur)
- Registre de sous-processus + `killpg` + logs rotatifs pour un arrêt propre

Le chapitre suivant, **Cycle 5**, sera consacré à l'application mobile cross-plateforme KodaMate, qui permet à l'utilisateur de configurer, contrôler et superviser son compagnon.

CHAPITRE VII

CYCLE 5 : APPLICATION MOBILE CROSS-PLATEFORME KODAMATE

VII.1 Introduction : pourquoi une application compagnon ?

KODA a été conçu comme un **compagnon de vie** : présence, dialogue, mémoire et personnalisation (chapitre I). Sur le plan matériel et cognitif, les cycles précédents ont donné au robot une voix, un corps et une capacité de réflexion. Il reste pourtant une attente essentielle côté humain : pouvoir **habiter la relation** avec son compagnon au-delà du salon — le configurer, le suivre, lui écrire, retrouver ce qui a été dit, même lorsque l'on n'est pas face au robot.

C'est le **besoin** que répond **KodaMate**. L'application n'est pas le « cerveau » de KODA : elle est le **prolongement relationnel** de l'écosystème, accessible depuis un téléphone ou un ordinateur. Elle permet à l'utilisateur de rester acteur de la vie de son compagnon : lui donner une identité, vérifier qu'il est disponible, échanger par écrit lorsque la voix n'est pas adaptée, et consulter la mémoire des conversations comme on relirait un carnet de bord partagé.

Rôle de KodaMate dans le projet :

- **Continuité** : maintenir le lien utilisateur-compagnon entre deux présences physiques du robot ;
- **Personnalisation** : traduire l'intention « mon KODA à moi » en paramètres concrets (nom, langue, caractère) ;
- **Supervision bienveillante** : savoir si le compagnon est actif ou au repos, sans confondre affichage local et réalité du système ;
- **Dialogue écrit** : offrir un canal complémentaire à la parole, aligné sur la même mémoire que le dialogue vocal ;
- **Confiance d'accès** : n'ouvrir les fonctions sensibles qu'après identification de l'utilisateur.

Ces besoins reprennent explicitement ceux formulés au chapitre II (*pilotage à distance, dialogue par message, personnalisation*). KodaMate en est la réponse concrète du Cycle 5.

Objectif du Cycle 5

Offrir une expérience mobile et desktop simple, continue et sécurisée, pour configurer, superviser et converser avec KODA — sans remplacer le robot ni dupliquer son intelligence, mais en prolongeant la relation déjà construite dans les cycles précédents.

La figure VII.1 résume les capacités attendues du point de vue de l'utilisateur ; la plupart supposent une authentification préalable, ce qui traduit l'exigence de confiance énoncée ci-dessus.

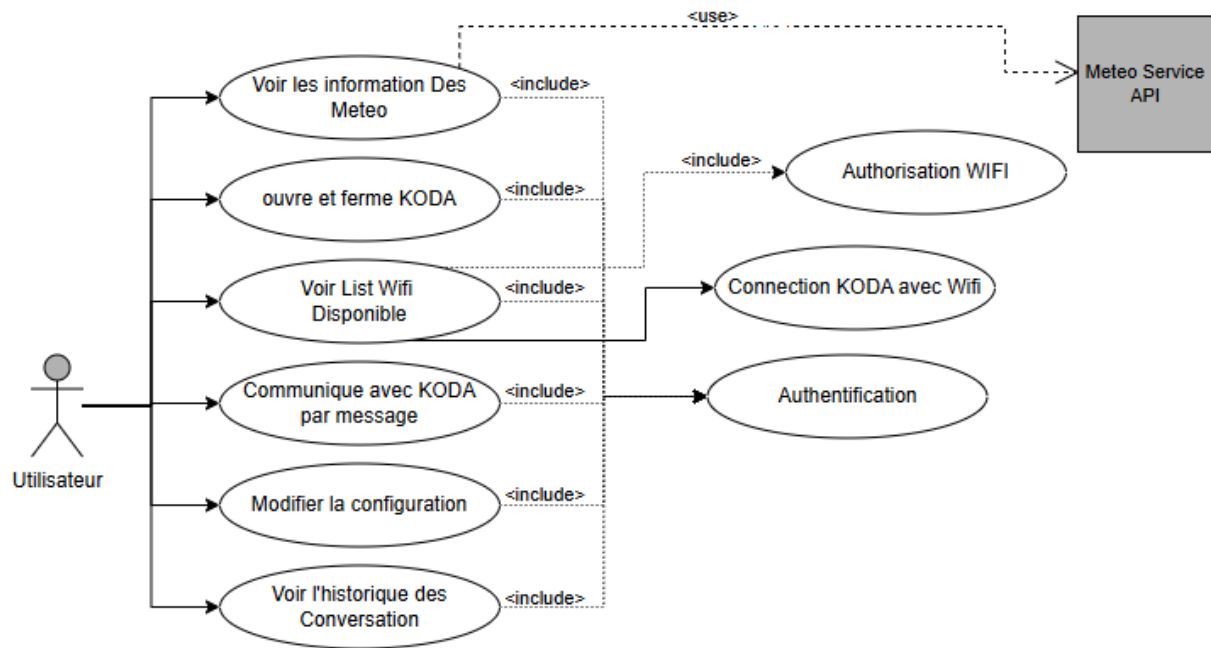


FIGURE VII.1 – Cycle 5 — cas d'utilisation de KodaMate (vue utilisateur)

VII.2 Place de KodaMate dans l'écosystème

Dans la vision globale du projet, KodaMate occupe la place du **interlocuteur à distance** : l'utilisateur exprime une intention (lire l'état, modifier un réglage, poser une question), et le système central — déjà présenté au chapitre IV — se charge de la réaliser en s'appuyant sur la mémoire et le raisonnement du chapitre III. Le robot physique reste le lieu de la présence ; l'application reste le lieu du **pilotage et du suivi**.

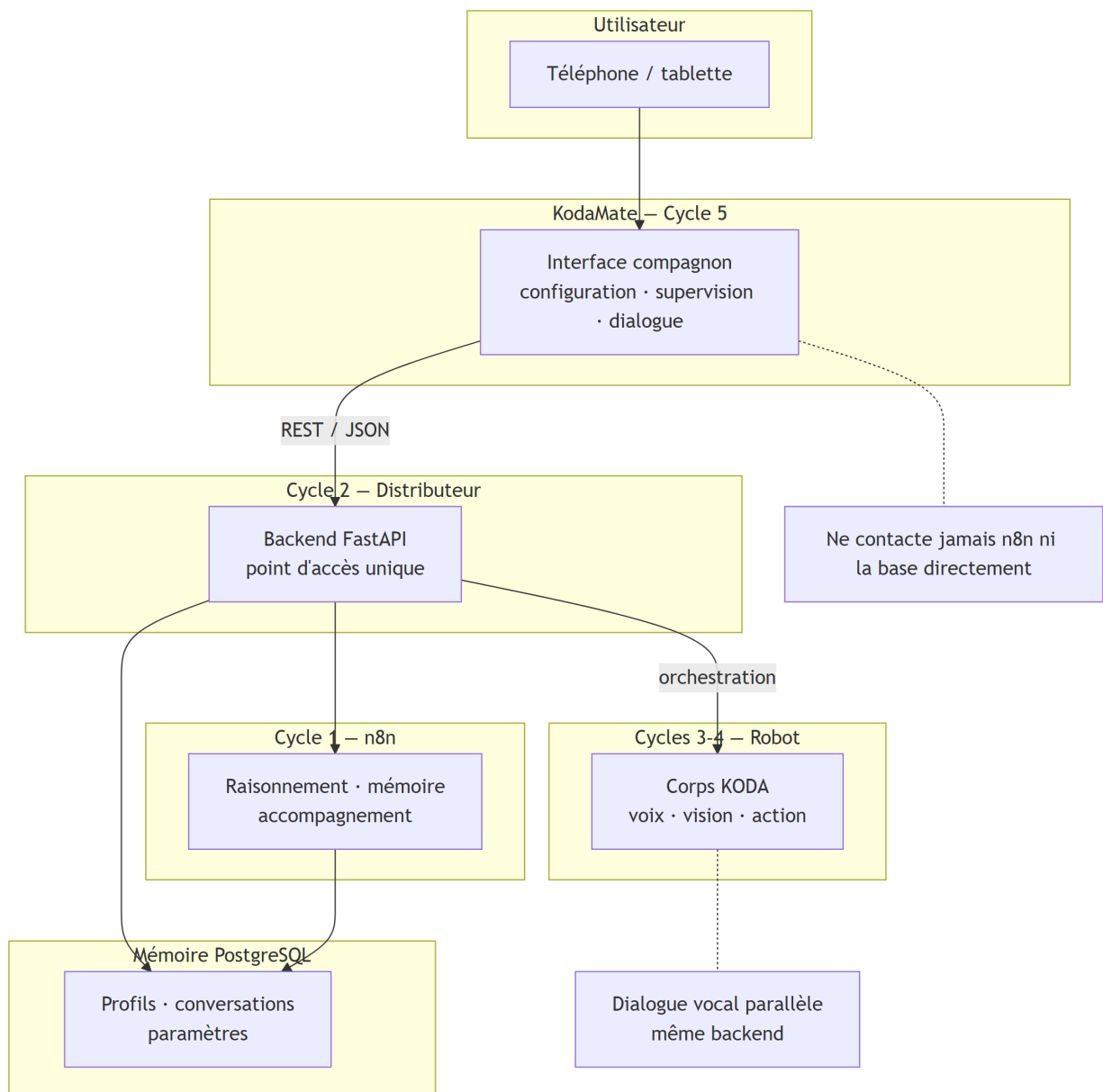


FIGURE VII.2 — Cycle 5 — place de KodaMate entre l'utilisateur, le Distributeur, la cognition n8n et le corps du robot

Cette répartition évite de fragmenter la relation : un même profil, une même histoire de conversations, que l'échange soit oral sur le robot ou écrit depuis le téléphone.

VII.3 Parcours et principes d'expérience

Avant d'aborder chaque écran, il est utile de décrire le **parcours vécu** par l'utilisateur. Celui-ci commence par une phase d'entrée (connexion, éventuelle première configuration), puis bascule vers un usage quotidien en quatre espaces : accueil, dialogue, historique et paramètres (figure VII.3).

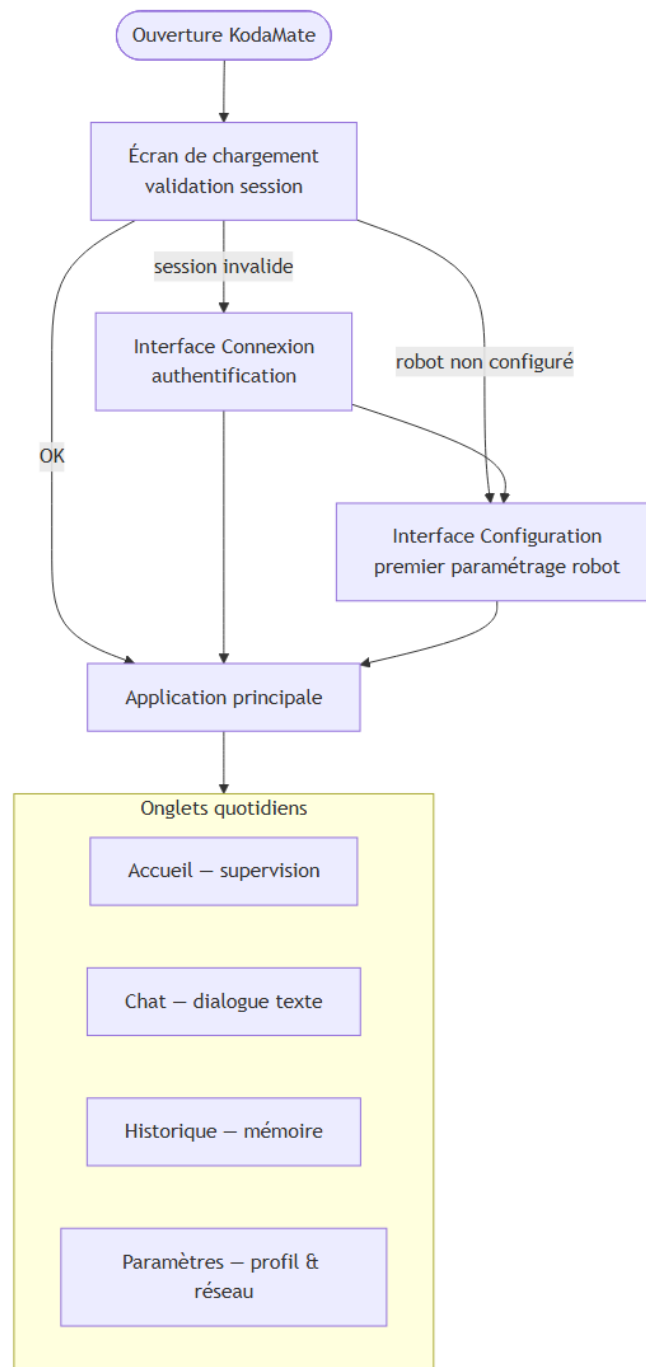
**FIGURE VII.3** – Cycle 5 — parcours utilisateur de KodaMate

TABLE VII.1 – Interfaces KodaMate et rôles utilisateur

Interface	Rôle pour l'utilisateur
Chargement	Accueillir l'utilisateur et orienter vers la bonne étape (connexion, configuration ou usage).
Connexion	Lier le compte utilisateur au Distributeur et mémoriser la session.
Configuration initiale	Personnaliser KODA (identité, langue, caractère) avant le premier usage.
Accueil	Superviser l'état du robot, la météo locale et la commande marche/arrêt.
Chat	Dialoguer par texte avec KODA, sur le même fond mémoriel que la voix.
Historique	Retrouver et filtrer les conversations passées.
Paramètres	Ajuster le profil, le réseau (simulation Wi-Fi) et quitter la session.

Chaque onglet de la phase quotidienne recharge ses informations lorsqu'il redevient visible, afin de refléter l'état réel du compagnon sans effort supplémentaire de l'utilisateur.

VII.4 Les interfaces utilisateur

Cette section décrit chaque écran selon le même fil : *rôle*, *fonctionnement*, puis *illustration* (capture d'écran lorsque le fichier est disponible dans public/).

VII.4.1 Interface de chargement

Rôle. C'est la première impression de KodaMate : un écran sobre avec le logo KODA, qui laisse le temps de vérifier si l'utilisateur peut entrer directement dans l'application ou doit se reconnecter.

Fonctionnement. Au lancement, l'application contrôle la session enregistrée localement, teste la disponibilité du serveur, puis interroge les paramètres du robot. Si la chaîne de vérification échoue, la session est effacée et l'utilisateur est renvoyé vers la connexion ; si le robot n'a jamais été configuré, vers l'écran de configuration ; sinon, vers les onglets principaux. L'adresse du serveur peut être saisie ou mémorisée ; en développement sur réseau local, elle est normalisée automatiquement (port, émulateur Android).

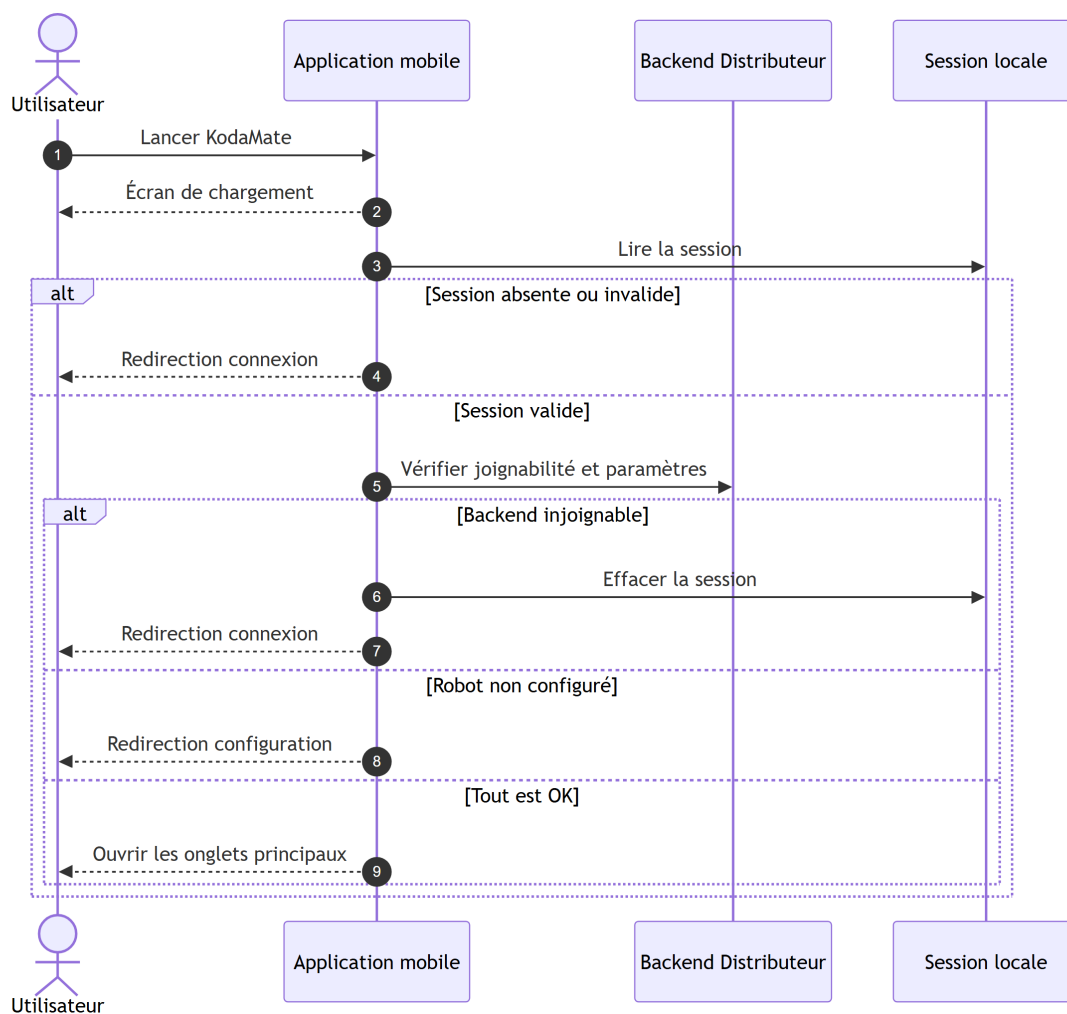


FIGURE VII.4 – Cycle 5 — séquence de démarrage et orientation de l'utilisateur

VII.4.2 Interface de connexion

Rôle. Établir le lien de confiance entre l'utilisateur et le Distributeur : sans connexion valide, aucune autre fonction n'est accessible.

Fonctionnement. L'utilisateur saisit l'adresse du Distributeur sur le réseau local (IP du poste de développement), peut *tester le serveur* avant de se connecter, puis s'authentifie par email et mot de passe. En cas d'échec réseau, un message explicite guide la vérification (backend actif, même Wi-Fi, adresse IP). Après connexion réussie, l'application mémorise les identifiants nécessaires aux appels suivants.



FIGURE VII.5 – Cycle 5 — connexion et test du serveur Distributeur

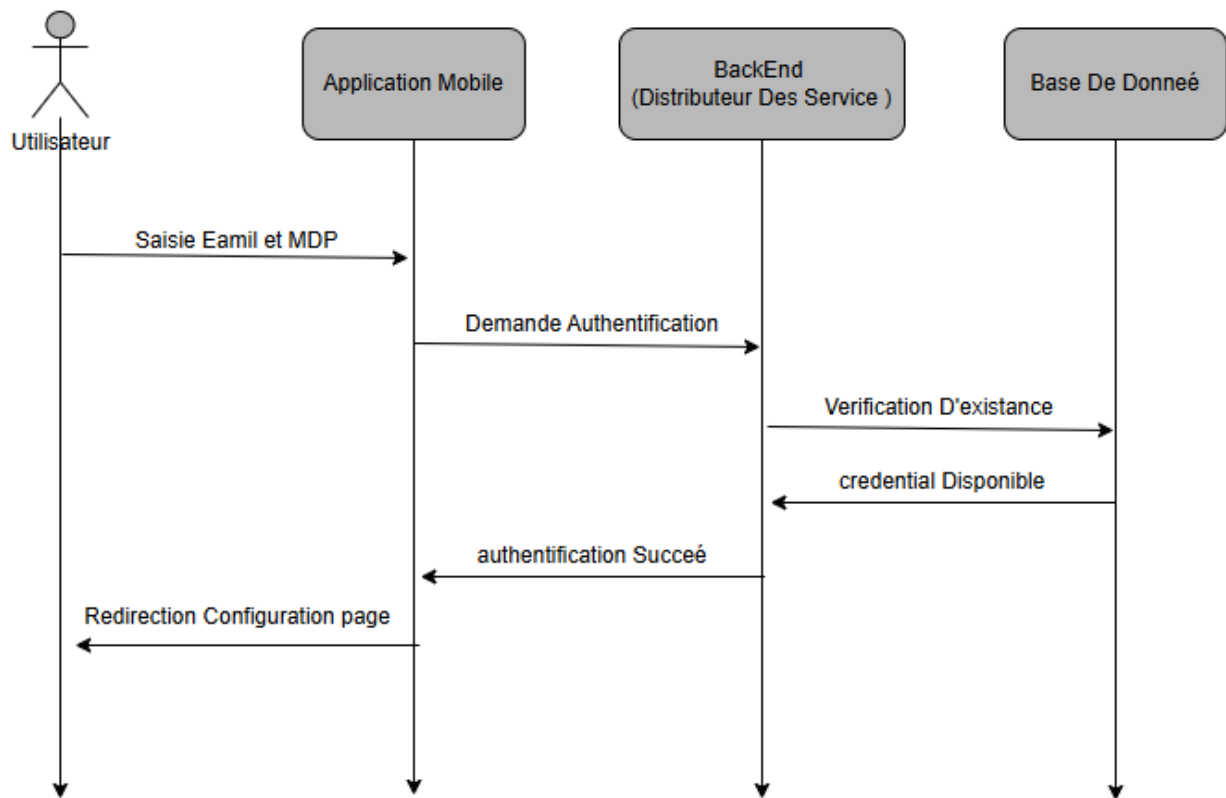


FIGURE VII.6 – Cycle 5 — séquence d’authentification utilisateur

VII.4.3 Interface de configuration du robot

Rôle. Transformer un compte technique en *compagnon nommé* : nom, langue, caractère, date de naissance symbolique, etc. Cette étape matérialise la personnalisation évoquée au Cycle 2 et prépare le dialogue n8n.

Fonctionnement. L’écran charge les paramètres existants depuis le backend, permet de les modifier, puis enregistre l’ensemble en une seule sauvegarde. Une fois validée, l’application marque le robot comme configuré et autorise l’accès aux onglets quotidiens. Les mêmes champs restent modifiables ultérieurement depuis l’écran Paramètres.

Configuration du robot

Renseignez les paramètres (table setting du backend).

Nom du robot *

mohsen

Sexe (personnage)

male

Date

2026-04-12

Mot de passe (setting)

.....

Caractère / personnalité

que, qui parle avec en ar-SA-HamedNeural

Configuration du robot

2026-04-12

Mot de passe (setting)

.....

Caractère / personnalité

que, qui parle avec en ar-SA-HamedNeural

Langue (locale BCP-47)

ar-SA-HamedNeural

Robot allumé (etat) ☒

Enregistrer et continuer

PUT /api/settings

FIGURE VII.7 – Cycle 5 — configuration initiale : identité du robot (gauche) et voix, langue et état d'allumage (droite)

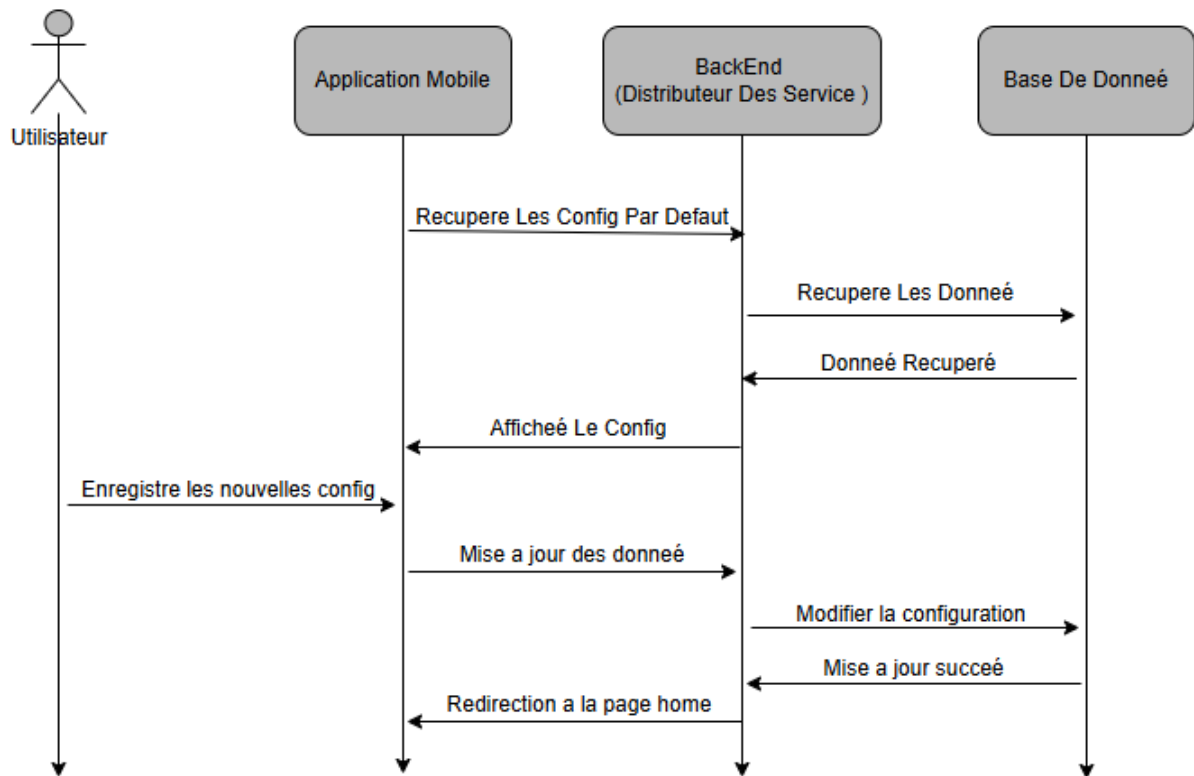


FIGURE VII.8 – Cycle 5 — séquence de lecture et mise à jour de la configuration

VII.4.4 Interface d'accueil (Home)

Rôle. Constituer le **tableau de bord** du compagnon : en un coup d'œil, l'utilisateur voit si KODA est joignable, dans quel état il se trouve, et peut agir sur l'alimentation logique (marche/arrêt). La météo d'Alger apporte un contexte ambiant léger, distinct du cœur métier mais utile pour l'expérience d'accueil.

Fonctionnement. L'écran combine deux sources : le Distributeur pour l'état réel du robot (lecture des paramètres, commandes d'allumage/extinction confirmées par relecture serveur) et l'API Open-Meteo pour la météo. En cas d'indisponibilité réseau, l'interface passe en mode hors ligne plutôt que d'afficher un état fictif. Le rafraîchissement intervient à chaque retour sur l'onglet.

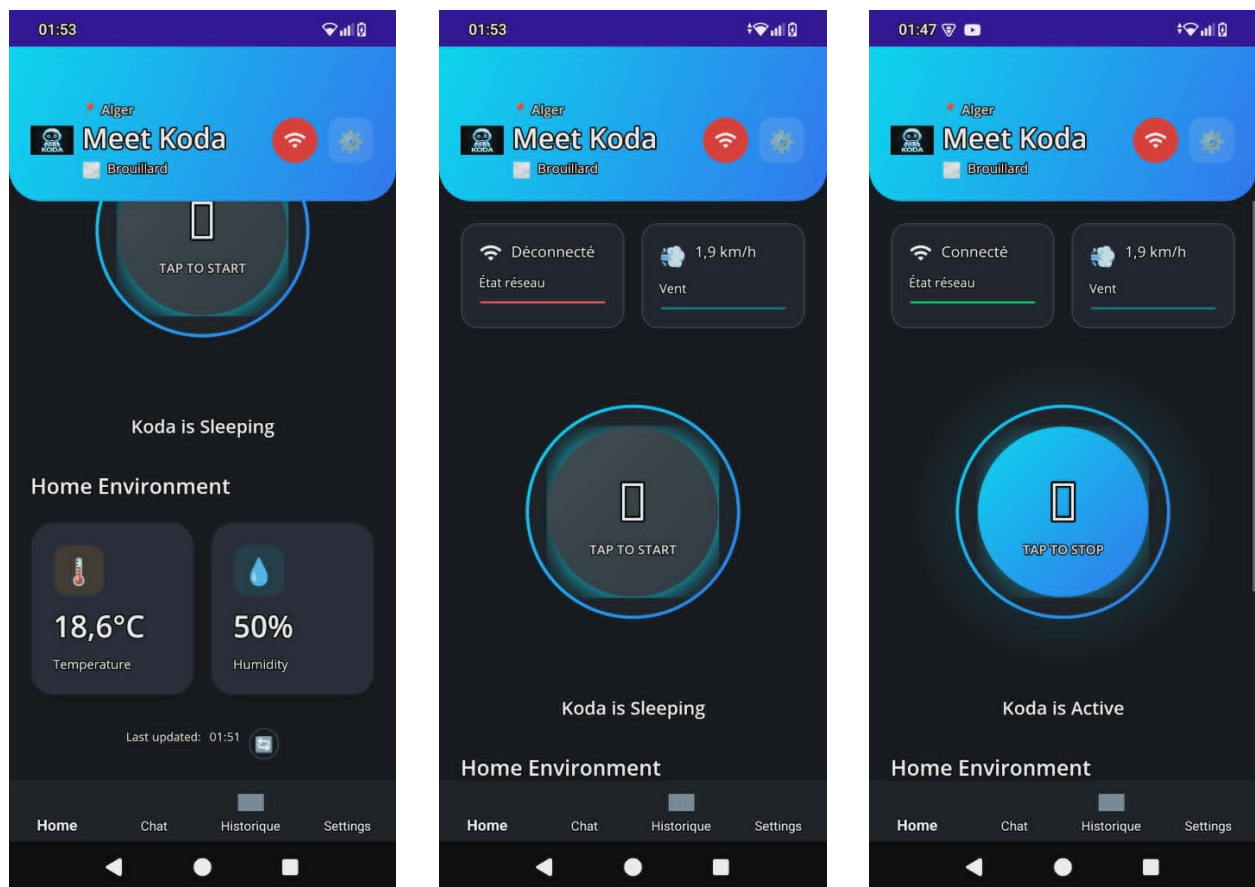


FIGURE VII.9 – Cycle 5 — accueil : veille et environnement (gauche), réseau déconnecté (centre), robot actif (droite)

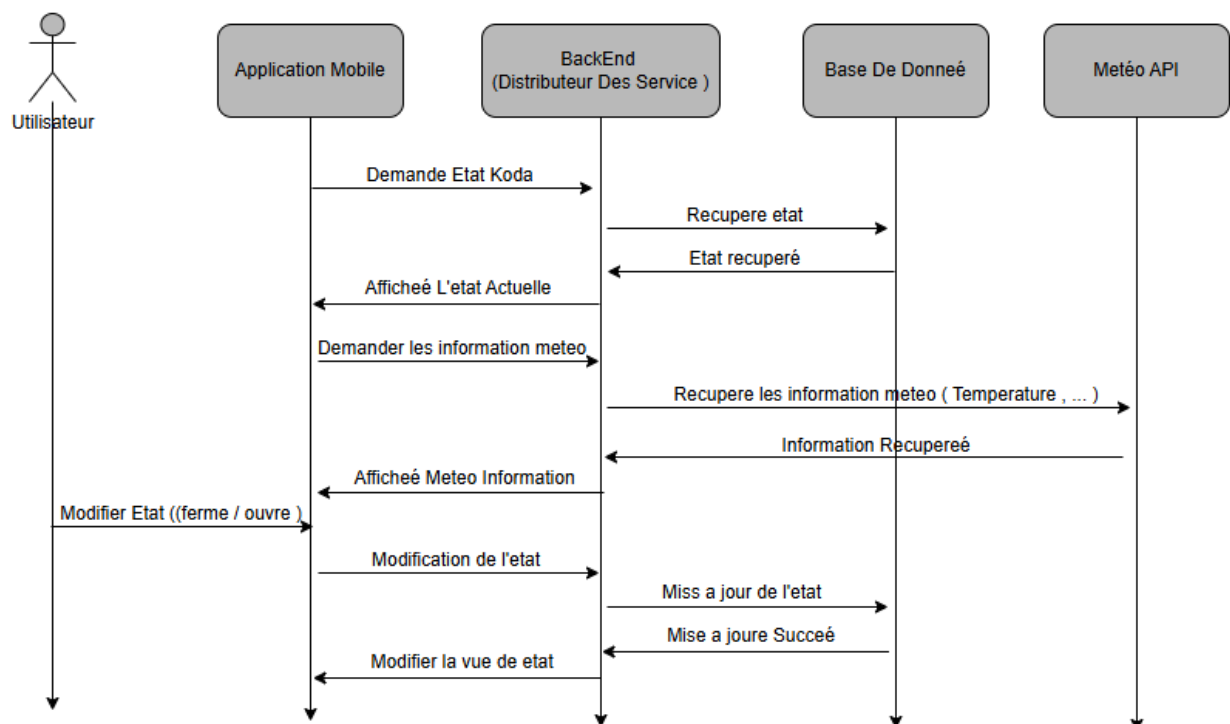


FIGURE VII.10 – Cycle 5 — séquence de supervision : état KODA, météo et marche/arrêt

VII.4.5 Interface de dialogue (Chat)

Rôle. Offrir un canal textuel avec KODA, complémentaire de la voix. L'utilisateur formule une question ; la réponse s'appuie sur le même raisonnement n8n que le Cycle 1, orchestré par le Distributeur.

Fonctionnement. Le message de l'utilisateur apparaît immédiatement dans la conversation ; l'application transmet la question au système central, qui déclenche l'analyse et la réflexion (chapitre III), enregistre l'échange en mémoire, puis renvoie une réponse formulée pour l'affichage. La capture photo prépare l'envoi multimodal ; l'intégration complète avec la reconnaissance faciale reste une évolution prévue.

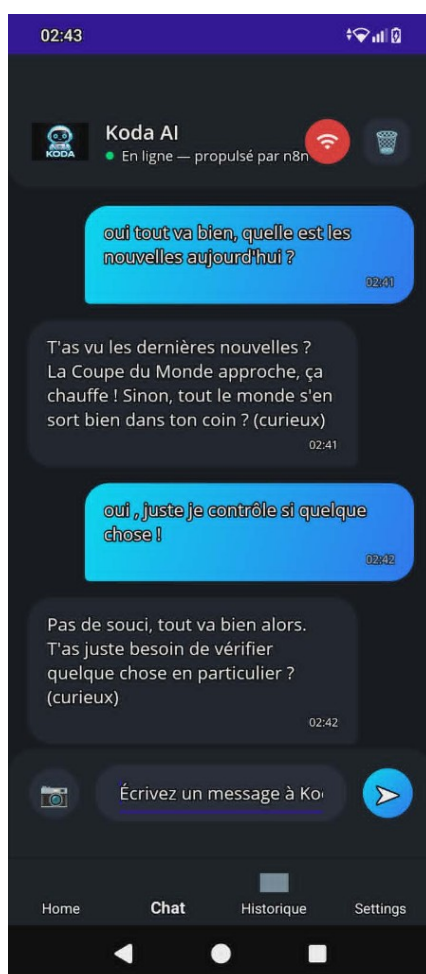


FIGURE VII.11 – Cycle 5 — dialogue texte avec KODA

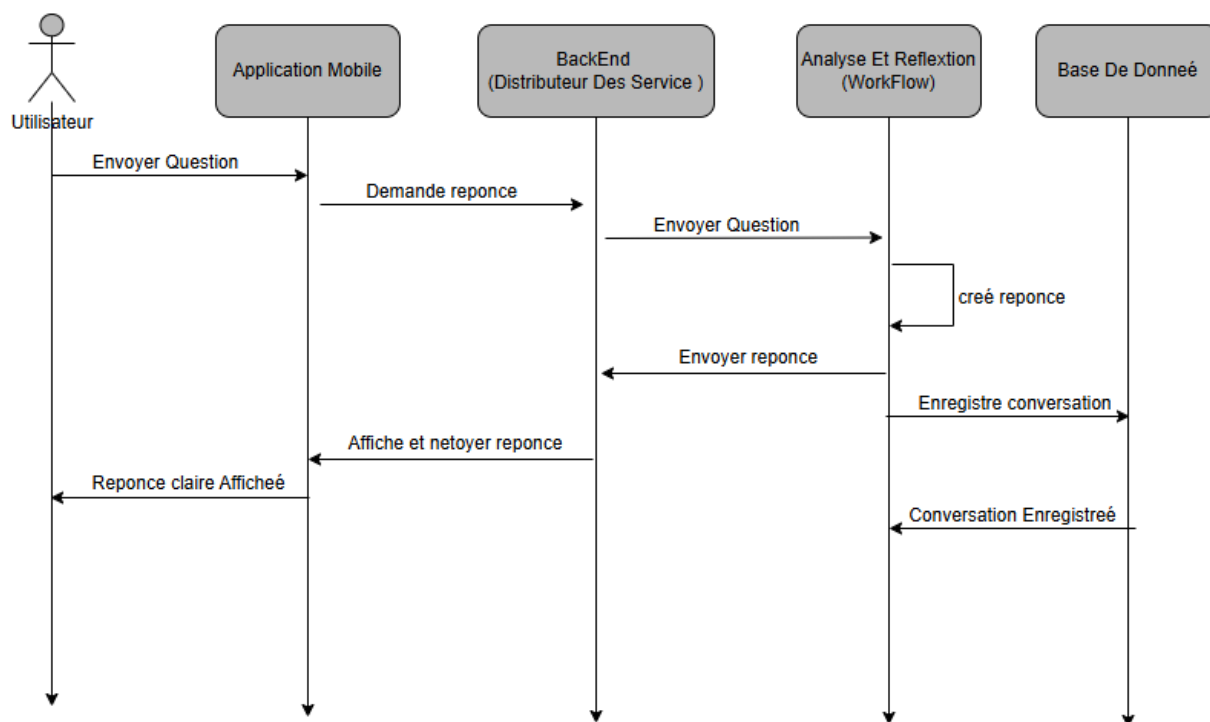


FIGURE VII.12 – Cycle 5 — séquence d’envoi d’une question et affichage de la réponse

VII.4.6 Interface d’historique

Rôle. Donner de la **continuité mémorielle** à l’usage : retrouver ce qui a été dit, filtrer par période, rechercher un mot-clé. Cette fonction s’aligne sur la mémoire PostgreSQL alimentée par le Cycle 1 et exposée par le Distributeur.

Fonctionnement. L’écran charge les dernières conversations depuis le backend, propose des filtres temporels (*aujourd’hui, hier, cette semaine*, date au choix), puis affine localement par recherche textuelle sur la question, la réponse et le type d’échange. Les résultats sont regroupés par jour pour faciliter la lecture.



FIGURE VII.13 – Cycle 5 — historique des conversations avec filtre et recherche

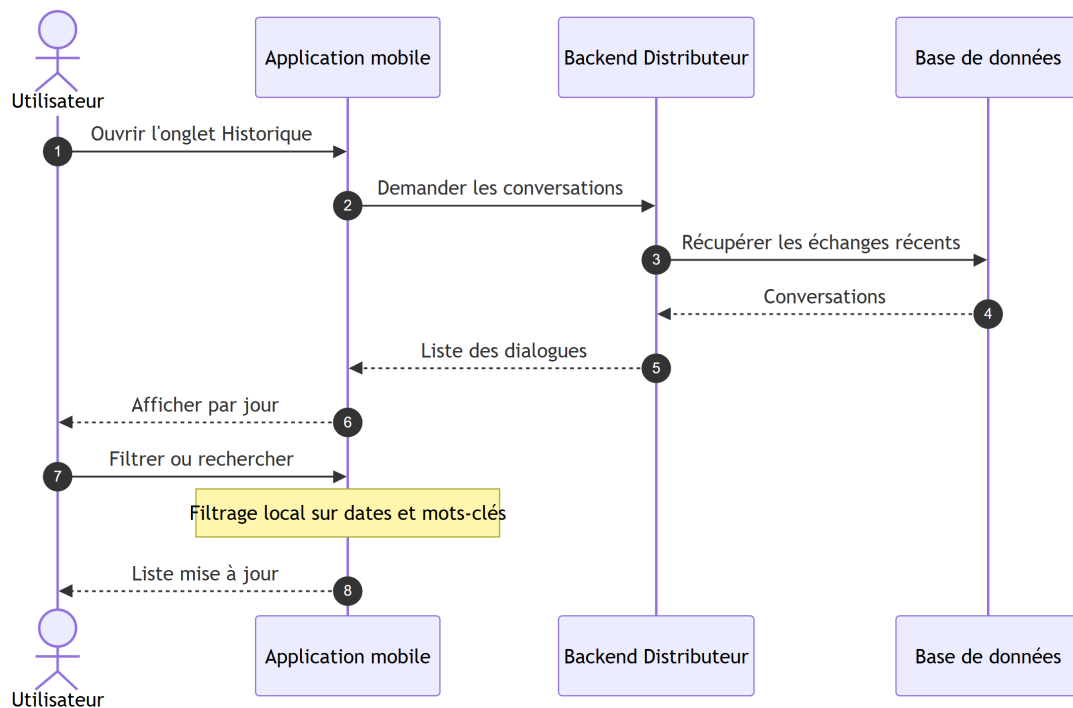


FIGURE VII.14 – Cycle 5 — séquence de consultation de l'historique

VII.4.7 Interface de paramètres

Rôle. Centraliser les réglages durables : profil du robot, préférences d’interface, réseau Wi-Fi (simulation ou scan Android), notifications de démonstration et déconnexion.

Fonctionnement. Les champs liés au robot sont synchronisés avec le backend comme lors de la configuration initiale. Le choix d’un SSID Wi-Fi est enregistré localement pour l’indicateur visuel ; sur Android, un scan peut lister les réseaux visibles après autorisation de localisation. Certaines options (voix, notifications avancées) sont visibles dans l’interface mais encore partiellement déconnectées du backend. La déconnexion efface la session et le SSID mémorisé pour repartir d’un état neutre.

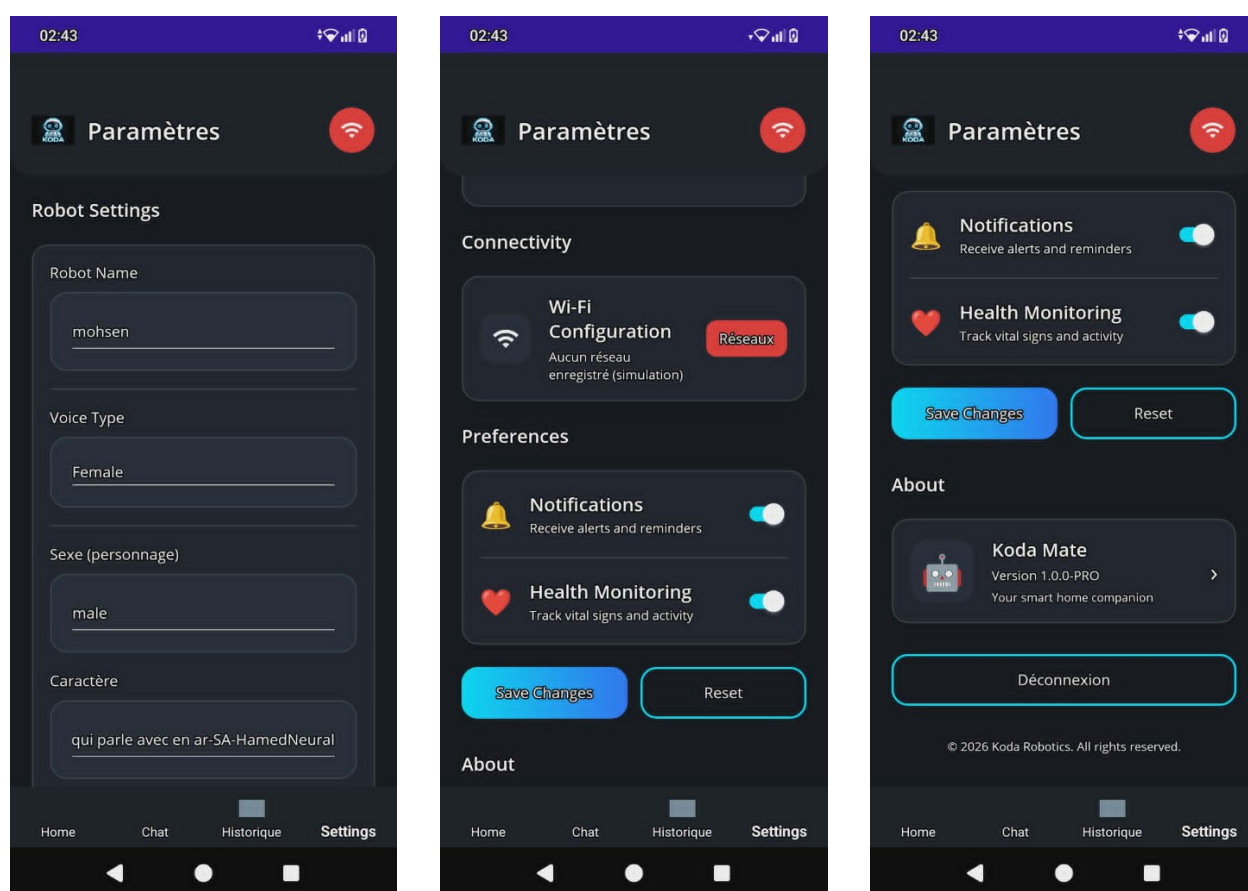


FIGURE VII.15 – Cycle 5 — paramètres : profil robot (gauche), Wi-Fi et préférences (centre), informations et déconnexion (droite)

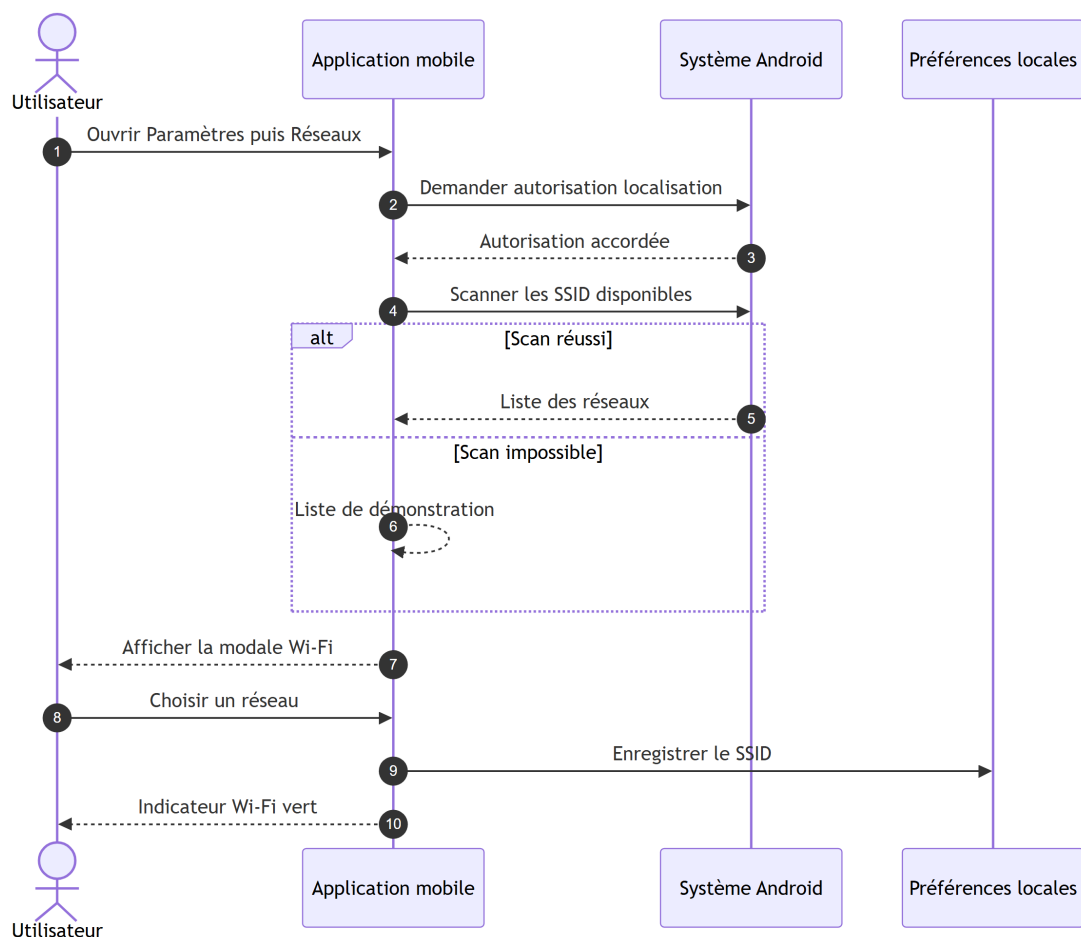


FIGURE VII.16 – Cycle 5 — séquence de sélection d’un réseau Wi-Fi (simulation locale)

VII.5 Organisation technique du projet

Au-delà des écrans, le code respecte une séparation **Views / ViewModels / Services / Models**. Le bootstrap (`App.xaml.cs`) choisit la page initiale ; `AppShell.xaml` porte la navigation par onglets ; `DistributeurApiService` regroupe les appels REST ; `SessionService` et les préférences MAUI conservent un minimum local (session, URL serveur, indicateur de configuration, SSID Wi-Fi).

TABLE VII.2 – Correspondance services principaux et besoins utilisateur

Composant	Usage dans KodaMate
<code>DistributeurApiService</code>	Authentification, paramètres, chat, historique, marche/arrêt.
<code>SessionService</code>	Persistance de la session et du statut « robot configuré ».
<code>WeatherService</code>	Météo d’accueil via Open-Meteo (Alger).
<code>WifiNetworkService</code>	Scan ou liste de démonstration des réseaux.
<code>MockFirebaseService</code>	Données de démonstration pour les notifications.

Sur Android, le manifeste autorise réseau, Wi-Fi, localisation et vibration ; le trafic HTTP clair facilite les tests en LAN et devra être remplacé par HTTPS en déploiement public.

VII.6 Fiabilité et session

La robustesse repose sur des règles simples mais strictes :

- aucun accès aux onglets sans session valide vérifiée au démarrage ;
- effacement automatique de la session si le backend ne répond plus ;
- délais d'attente sur les appels HTTP (backend et météo) ;
- messages d'erreur explicites sur chaque écran ;
- confirmation serveur après une commande marche/arrêt ;
- normalisation de l'URL en environnement de développement.

Les données locales restent volontairement limitées : elles ne remplacent pas la base PostgreSQL, elles permettent seulement de reprendre l'expérience entre deux ouvertures de l'application.

VII.7 Tests et validation

La validation du Cycle 5 porte à la fois sur la compilation multi-plateforme et sur les scénarios utilisateur. Les tests manuels restent importants car une partie du comportement dépend du réseau local, de la plateforme d'exécution et de la disponibilité du backend.

TABLE VII.3 – Tests de validation par interface

Interface	Procédure	Critère OK
Projet MAUI	Compiler sur la plateforme disponible	Build sans erreur
Connexion	Tester l'URL LAN puis se connecter	Backend joignable, session créée
Chargement	Redémarrer avec session existante	Orientation correcte (config. ou onglets)
Configuration	Modifier le profil puis enregistrer	Paramètres visibles après rechargement
Accueil	Ouvrir l'onglet Home	État robot et météo affichés
Accueil	Action marche/arrêt	État confirmé après rafraîchissement
Chat	Envoyer un message à KODA	Réponse affichée ou erreur lisible
Historique	Filtrer et rechercher	Liste cohérente par jour
Paramètres	Choisir un SSID Wi-Fi	Indicateur vert, SSID mémorisé
Paramètres	Déconnexion	Retour à l'écran de connexion

Les commandes minimales de validation côté développement sont :

```
dotnet restore KodaMate/KodaMate/KodaMate.csproj
dotnet build KodaMate/KodaMate/KodaMate.csproj
```

Listing VII.1 – Commandes de validation du Cycle 5

VII.8 Limites et perspectives

L'application KodaMate fournit déjà une base cross-plateforme fonctionnelle. Certaines limites restent toutefois identifiées dans le code actuel :

- remplacer `MockFirebaseService` par un service temps réel pour les notifications ;
- harmoniser définitivement le payload du chat avec le backend si le workflow attend un champ `message` au lieu de `question` ;
- ajouter l'envoi réel de l'image capturée vers un endpoint multimodal ou vers `/api/identify-face` ;
- passer les tests réseau locaux en HTTPS pour une version distribuable ;
- ajouter des tests unitaires sur les `ViewModels` et des tests d'intégration sur les services REST ;
- remplacer l'adresse IP LAN fixe par une découverte ou une configuration plus guidée.

VII.9 Conclusion

Le Cycle 5 achève la boucle utilisateur de l'écosystème KODA : après la cognition (Cycle 1), l'orchestration (Cycle 2) et l'incarnation physique (Cycles 3–4), **KodaMate** offre une présence numérique quotidienne. Chaque interface répond à un besoin précis — entrer, configurer, superviser, dialoguer, se souvenir, régler — sans court-circuiter le Distributeur.

Structurée par écran et fondée sur une base MAUI unique, l'application traduit en expérience tangible la continuité promise par les chapitres précédents : un même compagnon, une même mémoire, plusieurs modalités d'interaction.

Points clés du Cycle 5

- **Compagnon utilisateur** : canal mobile/desktop complémentaire de la voix
- **Sept interfaces** : chargement, connexion, configuration, accueil, chat, historique, paramètres
- **Passage unique** par le Distributeur ; n8n et PostgreSQL restent centralisés
- **Supervision et dialogue** : état du robot, marche/arrêt, chat texte, mémoire consultable
- **Évolutions identifiées** : notifications temps réel, multimodal photo, HTTPS en production

_____ CHAPITRE VIII _____

CONCLUSION GÉNÉRALE ET SYNTHÈSE DU PROJET

VIII.1 Du constat initial à la solution réalisée

Ce rapport a documenté la conception et la réalisation de **KODA**, un compagnon robotique intelligent visant à dépasser les limites des assistants génériques : absence de personnalité durable, froideur des réponses, dépendance à des clouds fermés et faible intégration entre la voix, le corps du robot et le pilotage utilisateur (chapitre I).

La réponse apportée repose sur une **architecture distribuée** articulée autour de cinq briques complémentaires, développées selon un **modèle en spirale** (chapitre II) : à chaque tour, une partie du système est précisée, construite, testée et raccordée au reste, jusqu'à former un écosystème cohérent. Le présent chapitre en donne une **lecture d'ensemble**, pour que le lecteur puisse comprendre, en un seul passage, le rôle de chaque cycle et la manière dont les composants coopèrent.

VIII.2 Vue d'ensemble : les cinq cycles et leurs rôles

Le tableau VIII.1 résume la contribution de chaque cycle. En dessous, chaque pilier est rappelé dans le langage du besoin utilisateur, puis dans celui de la réalisation technique.

TABLE VIII.1 – Synthèse des cycles KODA — besoin couvert et composant principal

N°	Cycle	Besoin adressé	Composant
0	Cadre & préparation (ch. I–II)	Cadrer le projet, formaliser les exigences et la méthode	Besoins, architecture cible, environnements
1	Analyseur et réflexion	Comprendre, mémoriser et prendre l'initiative de parler	Workflows n8n (3 webhooks)
2	Distributeur de services	Unifier les accès données, voix, mobile et robot	Backend FastAPI (pivot central)
3	Embarqué matériel	Donner un corps mobile, expressif et sensoriel au compagnon	Raspberry Pi + Arduino + capteurs/actionneurs
4	Embarqué logiciel	Écouter, parler, afficher, bouger et dialoguer avec le backend	Logiciel Python sur Raspberry Pi
5	Application KodaMate	Configurer, superviser et converser à distance	Application .NET MAUI

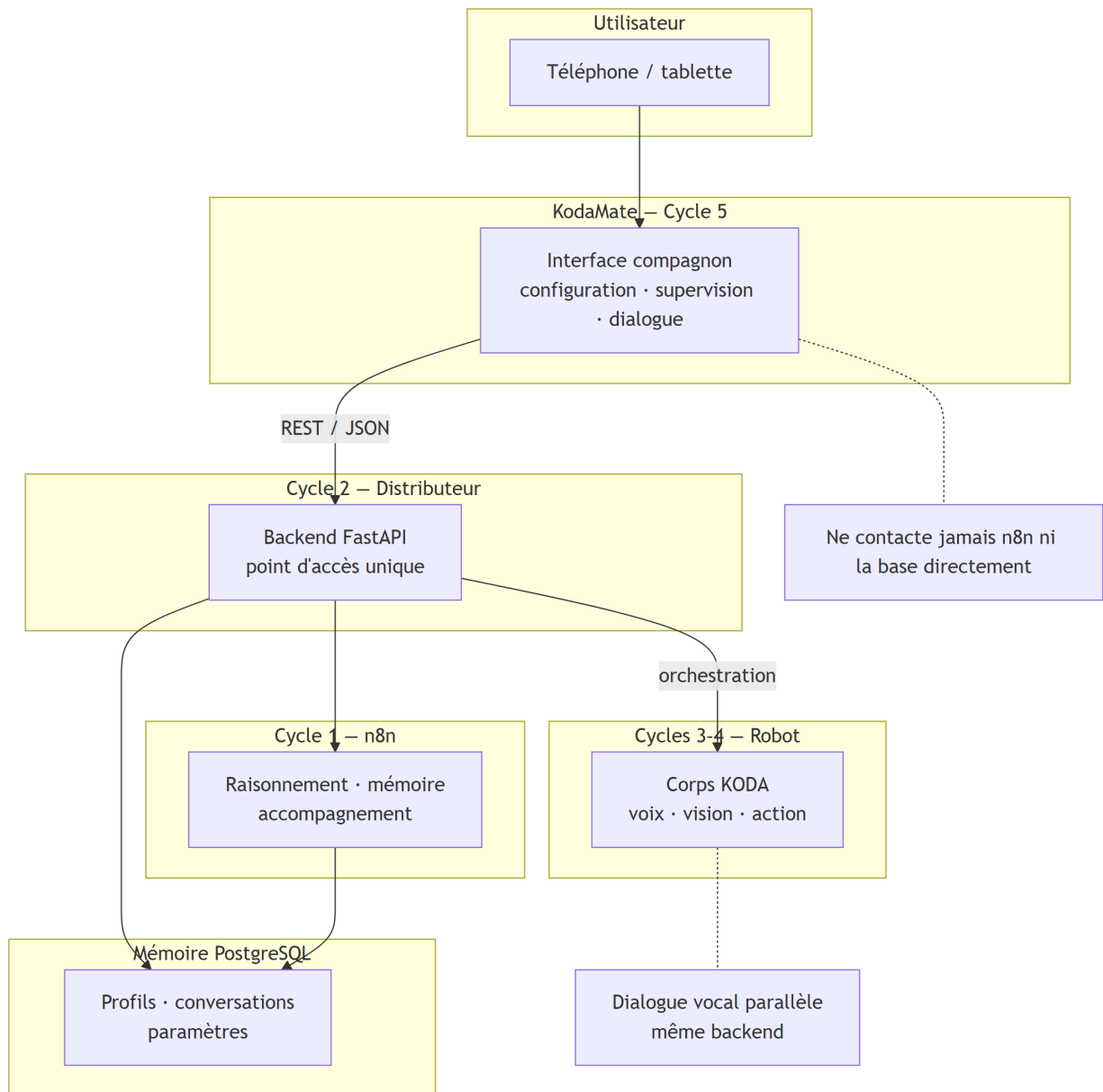


FIGURE VIII.1 – Vue synthétique de l'écosystème KODA au terme du projet

VIII.3 Les cinq piliers du système KODA

VIII.3.1 Le cerveau conversationnel (Cycle 1)

Le **Cycle 1** répond à la question : *comment KODA réfléchit-il et se souvient-il ?* Trois workflows n8n structurent la cognition :

- **Webhook 1** : traitement des questions en temps réel (classification sémantique, confrontation à l'historique, réponse contextualisée, notes) ;
- **Webhook 2** : consolidation mémorielle journalière (synthèses d'accompagnement) ;
- **Webhook 3** : spontanéité — KODA peut *prendre la parole* en s'appuyant sur la mémoire consolidée.

Ce cycle incarne le passage d'un assistant qui « répond » à un compagnon qui « réfléchit », en s'appuyant sur PostgreSQL et des services d'IA (OpenRouter, SerpAPI, etc.). Sans lui, le robot n'aurait ni personnalité évolutive ni continuité dans la relation.

VIII.3.2 Le système nerveux central (Cycle 2)

Le **Cycle 2** répond à la question : *comment tout le monde communique-t-il sans se perdre ?* Le **Distributeur FastAPI** est le **seul pivot** entre le robot, l'application mobile, la base de données, n8n et les services cloud (Azure pour la voix et la vision, Supabase/PostgreSQL pour la mémoire). Il expose un catalogue d'endpoints REST et WebSocket, gère l'authentification, la reconnaissance faciale, le pipeline vocal et le déclenchement des workflows.

À **retenir** : ni le téléphone ni le Raspberry Pi n'appellent n8n directement ; ils passent par le Distributeur, ce qui garantit cohérence des données, sécurité et évolutivité.

VIII.3.3 Le corps du compagnon (Cycles 3 et 4)

Les **Cycles 3 et 4** répondent à la question : *comment KODA est-il présent physiquement ?*

Le **Cycle 3** a posé la plateforme matérielle : architecture biprocesseur (Raspberry Pi pour l'intelligence, Arduino pour le temps réel moteur), mobilité, capture audio/vidéo, écran Nextion pour l'expression émotionnelle, alimentation et tests d'intégration mécanique.

Le **Cycle 4** a animé ce corps par un logiciel Python asynchrone : mot de réveil (Vosk), écoute par détection d'activité vocale, envoi des requêtes au backend, synthèse et lecture audio, commandes de mouvement vers l'Arduino, affichage des émotions sur Nextion, diagnostics matériels et arrêt propre. C'est le cycle qui transforme l'assemblage électronique en **compagnon interactif sur site**.

VIII.3.4 La relation à distance (Cycle 5)

Le **Cycle 5** répond à la question : *comment l'utilisateur habite-t-il sa relation avec KODA au quotidien, même loin du robot ?* L'application **KodaMate** (.NET MAUI) offre sept espaces fonctionnels (chargement, connexion, configuration, accueil, chat, historique, paramètres). Elle prolonge les besoins du chapitre II — pilotage à distance, dialogue par message, personnalisation — sans dupliquer l'intelligence du Cycle 1.

VIII.4 Comment lire l'ensemble : un scénario type

Pour **comprendre le rapport dans sa globalité**, imaginez un utilisateur qui revient chez lui le soir :

1. Sur son téléphone, il ouvre **KodaMate**, consulte l'**accueil** (état du robot, météo) et envoie un

message dans le **chat**.

2. **KodaMate** transmet la demande au **Distributeur**, qui enrichit le contexte depuis **PostgreSQL** et déclenche le **workflow n8n**.
3. La réponse est enregistrée en mémoire et renvoyée à l'application ; elle sera visible plus tard dans l'**historique**.
4. Sur place, le **Raspberry Pi** détecte le mot de réveil, capture la voix, obtient la même logique via le backend, fait parler KODA, affiche une émotion sur **Nextion** et peut actionner les **moteurs** via **Arduino**.

Ce scénario montre que KODA n'est pas une juxtaposition de modules, mais une **chaîne de présence** : cognition centralisée, orchestration unique, corps local, lien mobile — chaque cycle couvre un maillon, aucun ne le remplace.

VIII.5 Apports du projet

Synthèse — ce qu'il faut retenir du rapport

1. **Problème** : concevoir un compagnon personnalisé, mémoriel et supervisable, au-delà des assistants génériques.
2. **Méthode** : développement en spirale, six cycles thématiques, intégration incrémentale et tests à chaque tour.
3. **Cognition (C1)** : trois webhooks n8n — réponse, mémoire, proactivité.
4. **Orchestration (C2)** : Distributeur FastAPI = point d'accès unique (robot, mobile, BDD, n8n, cloud).
5. **Incarnation (C3–C4)** : matériel biprocesseur + logiciel embarqué vocal et expressif.
6. **Relation utilisateur (C5)** : KodaMate pour configurer, superviser et dialoguer à distance.
7. **Fil rouge** : une seule mémoire, une seule personnalité, plusieurs modalités d'interaction (voix, texte, présence physique).

Sur le plan **ingénierie**, le projet démontre la faisabilité d'un système robotique distribué à coût maîtrisé (Raspberry Pi, APIs ouvertes, workflows modifiables) tout en respectant une séparation claire des responsabilités. Sur le plan **humain**, il explore une relation d'accompagnement où l'utilisateur reste maître de la configuration de son compagnon — nom, langue, caractère — et peut suivre son activité sans expertise technique.

VIII.6 Limites et enseignements

Comme tout prototype de fin d'études, KODA présente des limites assumées, déjà identifiées dans les chapitres de cycles :

- **Environnement de démonstration** : tests souvent réalisés en réseau local (HTTP, adresse IP manuelle dans KodaMate), ce qui diffère d'un déploiement grand public sécurisé (HTTPS, découverte automatique des services).
- **Maturité mobile** : certaines fonctions (notifications temps réel, envoi photo vers la reconnaissance faciale) restent partielles ou simulées.
- **Robustesse embarquée** : dépendance au backend pour la cognition ; modes dégradés limités en cas de coupure prolongée.
- **Validation globale** : le **Cycle 6** (tests et validation bout-en-bout formalisés) est présent dans la méthodologie du chapitre II mais reste à renforcer par une campagne de recette utilisateur plus systématique.

Ces limites ne remettent pas en cause la **cohérence architecturale** du projet ; elles définissent une feuille de route réaliste pour une industrialisation future.

VIII.7 Perspectives

Les évolutions prioritaires découlent naturellement de la synthèse ci-dessus :

- **Production** : HTTPS, durcissement sécurité (JWT, rate limiting), déploiement conteneurisé du backend et observabilité (métriques, traces n8n).
- **Expérience utilisateur** : notifications push depuis le Distributeur vers KodaMate, multimodalité complète (photo → identification faciale), parcours de configuration guidé sans saisie d'IP.
- **Compagnon physique** : démarrage automatique au boot du Raspberry, télémétrie embarquée, réponses de secours hors ligne.
- **Cognition** : affinage des prompts n8n, évaluation qualitative des synthèses mémorielles, tests A/B sur la spontanéité (Webhook 3).

VIII.8 Conclusion finale

Ce rapport de stage a présenté **KODA** dans sa dimension complète : une ambition (compagnon de vie intelligent et personnalisé), une méthode (spirale et cycles thématiques), une architecture (n8n, Distributeur, embarqué, mobile, PostgreSQL) et une réalisation incrémentale validée par

prototypes, tests et captures d'interface.

En relisant les chapitres dans l'ordre — du cadre général (I) à la préparation (II), puis des Cycles 1 à 5 — le lecteur suit le cheminement d'une idée à un système opérationnel. **Le Cycle 1** pense et mémorise ; **le Cycle 2** orchestre ; **les Cycles 3 et 4** incarnent ; **le Cycle 5** relie l'utilisateur à son compagnon au quotidien. Ensemble, ils composent une réponse structurée à la problématique initiale : offrir un accompagnement plus humain, plus personnel et plus maîtrisé que les assistants du marché, tout en gardant l'utilisateur au centre de la relation avec son robot.

Le travail réalisé constitue une **base solide** pour poursuivre le projet vers une version démontrable en conditions réelles ; les cycles documentés fournissent à la fois une mémoire technique pour l'équipe et une carte lisible pour toute personne souhaitant comprendre, en une lecture, l'ensemble de l'écosystème KODA.